

Ло Юнь

ВЕКТОРНЫЕ БАЗЫ ДАННЫХ

Разработка и применение

ОМК
ИЗДАТЕЛЬСТВО

Ло Юнь

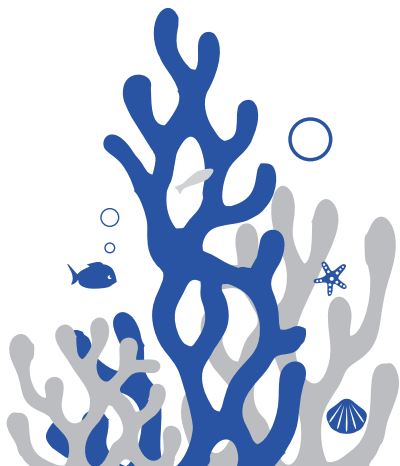
Векторные базы данных

TURING 图灵原创

从零构建 向量数据库

Build a VECTOR DATABASE from Scratch

罗云 © 著



人民邮电出版社
北京

Векторные базы данных

Ло Юнь



Москва, 2026

УДК 004.6:004.272.25

ББК 16.35

Л68

Ло Юнь

Л68 Векторные базы данных / пер. с англ. И. А. Шевкуна. – М.: ДМК Пресс, 2026. – 268 с.: ил.

ISBN 978-5-93700-422-2

Популярность векторных баз данных резко выросла в последние годы как прямое следствие перехода индустрии к моделям представления знаний через эмбединги и задачам, где классические реляционные или даже NoSQL-подходы оказываются неэффективными. Данная книга является практическим руководством по созданию векторных баз данных с нуля, а также раскрывает базовые принципы векторного представления данных.

Издание предназначено всем, кто интересуется передовой технологией работы с данными и хочет научиться применять ее на практике. Для эффективного чтения необходимо знать основы программирования.

УДК 004.6:004.272.25

ББК 16.35

《从零构建向量数据库》(ISBN: 978-7-115-64978-2), 作者: 罗云

Russian translation rights are arranged with Posts & Telecom Press Co., Ltd. through Media Solutions, Tokyo Japan (info@mediasolutions.jp). All rights reserved.

Все права защищены. Любая часть этой книги не может быть воспроизведена в какой бы то ни было форме и какими бы то ни было средствами без письменного разрешения владельцев авторских прав.

ISBN (кит.) 978-7-115-64978-2

ISBN (рус.) 978-5-93700-422-2

Copyright © 2024 Posts and Telecom Press

© Оформление, издание, перевод, ДМК Пресс, 2026

Оглавление

Предисловие от издательства	9
Отзывы экспертов	10
Рекомендация. Все к лучшему	12
Рекомендация. Путеводитель по будущему управления данными	14
Глубокое изучение основ и теории	14
Систематическое объяснение практических приемов	14
Полное расширение в горизонтальном и вертикальном направлениях	15
Введение	16
Для чего написана эта книга	16
Для кого написана эта книга	17
Ключевые особенности	18
Как читать эту книгу	18
Благодарности	22
ЧАСТЬ 1. ЗНАКОМСТВО С ВЕКТОРНЫМИ БАЗАМИ ДАННЫХ	23
Глава 1. Основы векторных баз данных	24
1.1. Вектор	24
1.1.1. Что такое вектор	24
1.1.2. Все может быть вектором	26
1.1.3. Сходство между векторами	28
1.1.4. Примеры использования сходства	30
1.2. База данных	34
1.2.1. Что такое база данных	34
1.2.2. Реляционные базы данных	37
1.2.3. Нереляционные базы данных	38
1.2.4. Ограничения традиционных баз данных	39
1.3. Зачем нужны векторные базы данных	40
1.3.1. Различия между векторными и традиционными данными	40
1.3.2. Появление векторных баз данных	42
1.3.3. Интеллектуальная платформа хранения в эпоху больших моделей	43
1.4. Резюме	45

Глава 2. Краткая история векторных баз данных	46
2.1. Период зарождения (1980–2012)	46
2.1.1. Стремительное развитие глубоких нейронных сетей.....	47
2.1.2. Связь глубоких нейронных сетей и векторных баз данных	49
2.2. Период становления (2012–2017)	50
2.3. Период развития (2017 год – настоящее время)	51
2.3.1. Развитие отрасли	52
2.3.2. Сравнение возможностей ключевых продуктов	53
2.3.3. Технологическая архитектура ключевых продуктов.....	56
2.4. Резюме	60
Глава 3. Основные возможности векторных баз данных.....	62
3.1. Основные возможности	62
3.1.1. Логические уровни.....	63
3.1.2. Индексы	69
3.1.3. Ключевые характеристики	73
3.2. Расширенные возможности	74
3.2.1. Динамическая схема.....	75
3.2.2. Механизм псевдонимов	76
3.2.3. Векторизация	77
3.2.4. Гибридные запросы	78
3.3. Резюме	79
ЧАСТЬ 2. СОЗДАНИЕ ВЕКТОРНОЙ БАЗЫ ДАННЫХ.....	81
Глава 4. Реализация автономной векторной базы данных	82
4.1. Реализация индекса векторных данных	82
4.1.1. Основные функции библиотеки FAISS	83
4.1.2. Реализация плоского индекса.....	90
4.1.3. Основные функции библиотеки HNSWLib	98
4.1.4. Реализация HNSW-индекса	107
4.2. Реализация гибридного индекса данных.....	112
4.2.1. Реализация индекса скалярных данных	112
4.2.2. Единый вход управления	114
4.2.3. Реализация фильтрующего индекса	119
4.3. Реализация восстановления системы после сбоев.....	128
4.3.1. Персистентность данных на основе журнала	129
4.3.2. Персистентность данных на основе моментальных снимков.....	135
4.4. Резюме	142
Глава 5. Реализация распределенной векторной базы данных	144
5.1. Управление данными в кластере	146
5.1.1. Знакомство с NuRaft	148

5.1.2. Установление отношения главный–подчиненный	152
5.1.3. Реализация репликации данных	158
5.2. Управление трафиком кластера	162
5.2.1. Управление метаданными кластера	163
5.2.2. Единый входной поток	167
5.2.3. Разделение чтения и записи	171
5.2.4. Обеспечение согласованности чтения и записи	173
5.3 Управление исключениями в кластере	174
5.3.1. Обнаружение нового главного узла	174
5.3.2. Обнаружение отказа подчиненного узла	176
5.3.3. Реализация переключения при отказе	178
5.4. Сегментирование кластера	180
5.4.1. Настройка стратегии сегментирования кластера	182
5.4.2. Перенаправление запросов в соответствии со стратегией сегментирования	184
5.5. Резюме	194
Глава 6. Оптимизация векторной базы данных	197
6.1. Оптимизация производительности	198
6.1.1. Оптимизация векторных вычислений с использованием набора инструкций	198
6.1.2. Оптимизация алгоритмов запросов	200
6.1.3. Оптимизация протокола связи	203
6.1.4. Настройка инструмента для эталонного тестирования	206
6.2. Оптимизация затрат	213
6.2.1. Гибридное развертывание нескольких модулей	214
6.2.2. Развертывание на одном узле	217
6.3. Оптимизация удобства использования	218
6.3.1. Пакет SDK	218
6.3.2. Аутентификация доступа	222
6.3.3. Резервное копирование данных	232
6.4. Резюме	235
ЧАСТЬ 3. ПРАКТИКА И ПЕРСПЕКТИВЫ РАЗВИТИЯ ВЕКТОРНЫХ БАЗ ДАННЫХ	237
Глава 7. Практические примеры использования векторных баз данных	238
7.1. Создание системы поиска по изображениям	238
7.1.1. Векторизация изображений	239
7.1.2. Загрузка и запрос изображений	241
7.1.3. Обзор системы	244
7.2. Создание персональной базы знаний	245
7.2.1. Предобработка знаний	246
7.2.2. Векторизация знаний	247
7.2.3. Управление базой знаний	248

7.2.4. Система вопросов и ответов	250
7.2.5. Обзор системы	252
7.3. Резюме	253

Глава 8. Перспективы развития

255

8.1. С точки зрения эволюции отрасли	256
8.1.1. Новая парадигма управления данными человеком	256
8.1.2. Векторные данные устраняют различия в форматах данных	259
8.1.3. Ключевые аспекты платформизации векторной базы данных	260
8.2. С точки зрения применения в отрасли	262
8.2.2. Снижение порога использования RAG	264
8.3. Резюме	266

Предисловие от издательства

Отзывы и пожелания

Мы всегда рады отзывам наших читателей. Расскажите нам, что вы думаете об этой книге – что понравилось или, может быть, не понравилось. Отзывы важны для нас, чтобы выпускать книги, которые будут для вас максимально полезны.

Вы можете написать отзыв на нашем сайте www.dmkpress.com, зайдя на страницу книги и оставив комментарий в разделе «Отзывы и рецензии». Также можно послать письмо главному редактору по адресу dmkpress@gmail.com; при этом укажите название книги в теме письма.

Если вы являетесь экспертом в какой-либо области и заинтересованы в написании новой книги, заполните форму на нашем сайте по адресу http://dmkpress.com/authors/publish_book/ или напишите в издательство по адресу dmkpress@gmail.com.

Список опечаток

Хотя мы приняли все возможные меры для того, чтобы обеспечить высокое качество наших текстов, ошибки все равно случаются. Если вы найдете ошибку в одной из наших книг – возможно, ошибку в основном тексте или программном коде, – мы будем очень благодарны, если вы сообщите нам о ней. Сделав это, вы избавите других читателей от недопонимания и поможете нам улучшить последующие издания этой книги.

Если вы найдете какие-либо ошибки в коде, пожалуйста, сообщите о них главному редактору по адресу dmkpress@gmail.com, и мы исправим это в следующих тиражах.

Нарушение авторских прав

Пиратство в интернете по-прежнему остается насущной проблемой. Издательство «ДМК Пресс» очень серьезно относится к вопросам защиты авторских прав и лицензирования. Если вы столкнетесь в интернете с незаконной публикацией какой-либо из наших книг, пожалуйста, пришлите нам ссылку на интернет-ресурс, чтобы мы могли применить санкции.

Ссылку на подозрительные материалы можно прислать по адресу dmkpress@gmail.com.

Мы высоко ценим любую помощь по защите наших авторов, благодаря которой мы можем предоставлять вам качественные материалы.

Отзывы экспертов

В эпоху стремительного развития искусственного интеллекта информационно-коммуникационная сфера глубоко интегрируется с интеллектуальными технологиями. Можно с уверенностью сказать, что векторные базы данных будут играть ключевую роль в обработке данных в этой области. Книга «Векторные базы данных» является плодом творчества Ло Юня и его команды специалистов в области ИИ. В ней не только подробно раскрываются принципы работы векторных баз данных, но и предлагаются богатые примеры, практические приемы и ценные идеи, которые будут полезны исследователям и новаторам в области интеллектуальных технологий.

– Ван Цзяндань (Wang Jiangdan),
член Китайской академии инженерных наук

Ло Юнь – один из первых специалистов в области облачных вычислений и опытный эксперт в области баз данных, сетей и распределенных систем. Эта книга является практическим руководством, в котором доступным языком объясняется, как создать высокопроизводительную векторную базу данных, и я настоятельно рекомендую ее к прочтению.

– Лю Ин (Liu Ying),
вице-президент Tencent Cloud

Развитие ИИ-технологий требует унифицированного представления мультимодальных данных и управления ими. Решение этой задачи привело к появлению новой технологии – векторных баз данных. Книга в ясной и краткой форме знакомит с их основными понятиями, начиная с самых основ и постепенно углубляясь в детали, с акцентом на практическое применение. Это отличное справочное пособие для изучения векторных баз данных!

– Ду Сяюнь (Du Xiaoyong),
профессор Института информатики Китайского народного университета,
директор Ведущей лаборатории по инженерии данных и знаний
Министерства образования КНР

Книга «Векторные базы данных» содержит многолетний опыт Ло Юня и команды Tencent Cloud Database, накопленный при обслуживании группы компаний Tencent и ее внешних клиентов. Содержание книги легко усваивается и идеально подходит для технических специалистов, интересующихся технологиями векторных баз данных.

– Ли Голян (Li Guoliang),
профессор Университета Цинхуа,
IEEE Fellow

В условиях «демократизации ИИ-технологий» векторные базы данных быстро становятся основой интеллектуальных приложений. Эта книга призвана устранить недостаток подробной документации и содержит доступные объяснения и практические руководства. Рекомендуется к прочтению специалистам в области баз данных и искусственного интеллекта.

– Лю Чжиюань (Liu Zhiyuan),
доцент Университета Цинхуа

Автор, обладая глубокими академическими знаниями и богатым практическим опытом, подробно излагает сложные концепции технологии векторных баз данных и ведет читателей от самых базовых сведений к созданию полноценной векторной базы данных. Книга содержит как теорию векторных технологий, так и практический опыт в распределенных базах данных, а также описывает соответствующие области применения, что делает ее полезной как для профессионалов в области баз данных, так и для тех, кто интересуется ИИ-технологиями.

– Ян Чэнху (Yang Chenghu),
соучредитель и технический директор Beijing Fengqing Technology

Рекомендация. Все к лучшему

Гай Гоцян (Gai Guoqiang),
основатель Enmotech,
обладатель титула MVP (самый ценный эксперт) в Kunpeng

Ло Юнь в предисловии выразил особую благодарность своему редактору: «Именно с вашего письма установились наши взаимоотношения». Эта фраза вернула меня в 2004 год, когда я только что прибыл в Пекин. Именно благодаря одному письму я установил взаимоотношения с моим редактором и издательством связи и телекоммуникаций – с тех пор нас связывает двадцатилетняя дружба.

Наши отношения с технологиями часто начинаются случайно, но развиваются только благодаря упорству.

Автор и его команда многие годы исследовали задачи извлечения векторов, движимые потребностями в вычислениях над неструктурированными данными. Они создавали и постоянно совершенствовали инновационные продукты, обслуживая огромное количество пользователей и накапливая ценный опыт первопроходцев. Впоследствии с появлением больших языковых моделей роль векторных баз данных стала очевидна всему миру.

Я полагаю, что достижения в любом технологическом направлении происходят именно так. Благодаря упорной работе и удовлетворению определенных потребностей векторные базы данных достигли момента, когда они смогли проявить себя и произвести большое впечатление.

В июле 2023 года на презентации VectorDB компании Tencent Cloud я стал свидетелем глубокого понимания и проницательности Ло Юня в отношении векторных баз данных. В контексте обслуживания широкого набора индивидуальных сценариев у Tencent появилась возможность удовлетворять самые разнообразные потребности в векторных вычислениях, и именно поэтому их векторная база данных демонстрирует выдающуюся техническую конкурентоспособность.

Теория без практики остается неполной, истинное понимание приходит только через личный опыт. Автор утверждает, что это книга ориентирована на практику и требует активного участия. Только через практическое применение и личный опыт можно понять суть технологии. Самое важное при чтении книги – это понять суть мыслей и опыта автора. Я считаю, что «практика и участие» – это основное знание, которое автор стремится передать.

Конечно, первые две и последняя главы книги взаимосвязаны по содержанию, и их можно рассматривать как научно-популярный материал, который можно прочитать без практических упражнений и получить базовое представление о векторных базах данных. Эти три главы охватывают прошлое, настоящее и будущее векторных баз данных, и в них повсюду виден личный опыт и глубокие размышления автора.

При написании своей новой книги я обращался к автору за советом по разделу о векторных базах данных и получил много полезной информации. С публикацией его собственной работы заполнился пробел в литературе о векторных базах данных, что является важным вкладом в отрасль. Я уверен, что все читатели смогут извлечь пользу из этой книги.

Чтение этой книги имеет и другую практическую значимость: мы можем узнать, как различные продукты Tencent, которые мы используем ежедневно, способны быстро и точно отвечать, эффективно и интеллектуально искать изображения, рекомендовать персонализированный контент и многое другое. За всем этим стоят высокоскоростные векторные базы данных.

Все можно представить в виде векторов, и чтение книги обязательно принесет вам необходимые для этого знания.

Рекомендация. Путеводитель по будущему управления данными

Ван Хаофэн (Wang Haofen),
приглашенный исследователь Университета Тунцзи,
инициатор создания OpenKG (Китайское сообщество открытых знаний)

В эпоху стремительного развития искусственного интеллекта и больших языковых моделей векторные базы данных стали основой новой инфраструктуры. Их важность сопоставима с реляционными базами данных эпохи сетей персональных компьютеров и большими данными эпохи мобильного интернета. Поэтому существует потребность в систематичном учебном пособии, подходящем как для специалистов смежных областей, которые хотят понять суть векторных баз данных, так и для профессионалов отрасли, которые стремятся овладеть этой технологией. В условиях стремительного развития технологий и сильного желания обучающихся получить соответствующие знания я рад стать свидетелем выхода в свет книги «Векторные базы данных».

Эта книга написана Ло Юнем, руководителем направления векторных баз данных в Tencent Cloud, обладающим богатым практическим опытом. Она всесторонне и подробно освещает построение векторных баз данных и их практическое применение в различных сценариях. Целью книги является помощь читателям в овладении теоретическими знаниями и в создании распределенной векторной базы данных с нуля. Кроме того, книга демонстрирует широкие перспективы и мощный потенциал векторных баз данных. При внимательном рассмотрении можно выделить три основные особенности этой книги.

ГЛУБОКОЕ ИЗУЧЕНИЕ ОСНОВ И ТЕОРИИ

Книга начинается с базовых знаний с подробным изложением концепции, принципов и архитектуры векторных баз данных. Ло Юнь простым и понятным языком с использованием доступных аналогий объясняет ключевые идеи и технические аспекты векторных баз данных. Также он углубленно рассматривает такие важные технологии, как векторизация, запросы на сходство и векторные индексы, помогая читателям сформировать полную систему знаний и заложить прочную теоретическую основу для последующей практической работы.

СИСТЕМАТИЧЕСКОЕ ОБЪЯСНЕНИЕ ПРАКТИЧЕСКИХ ПРИЕМОВ

Будучи специалистом в области искусственного интеллекта, я глубоко осознаю важность выбора подходящей векторной базы данных для успеха проекта, особенно в процессе построения генерации, дополненной поиском. Благодаря

систематическому изложению читатели могут полностью освоить процесс построения векторной базы данных, включая импорт данных, построение индексов, оптимизацию запросов и настройку производительности. Ло Юнь делится множеством базовых деталей и практических советов из собственного опыта, что позволяет читателям не только узнать, как что-то делается, но и почему это делается именно так. Это бесценно для любого специалиста, желающего применять векторные базы данных в реальных проектах.

Полное расширение в горизонтальном и вертикальном направлениях

В эпоху искусственного интеллекта векторные базы данных не ограничиваются одним приложением. Их можно расширять горизонтально, добавляя поддержку различных модальностей, включая текст, изображения, аудио и видео, что позволяет гибко переносить их в разные сценарии. Книга объясняет, как построить расширяемую систему векторных баз данных, способную гибко адаптироваться к различным типам данных и приложениям. Также можно расширять в вертикальном направлении: в книге рассматривается, как интегрировать различные универсальные или специализированные большие языковые модели и бесшовно соединять их с производственными системами и системами больших данных, реализуя интеграцию исследований и эксплуатации.

«Векторные базы данных» – это техническая книга, сосредоточенная в равной степени на теории и практике, а также важное руководство, раскрывающее направление будущего управления данными. Искренне рекомендую эту книгу всем, кто интересуется векторными базами данных, и надеюсь, что она принесет вдохновение и реальную помощь в вашем обучении и работе.

Введение

2023 год называют «годом демократизации ИИ». За последние год-два мы вступили в новую эпоху стремительного роста технологий искусственного интеллекта. Непрерывно появляются новые термины, концепции и технологии, такие как трансформер, GPT, большие языковые модели, ИИ-агент, цепочка мышления, векторные базы данных и др. Мы потратили немало времени на изучение и освоение этих новых технологий. В этот период мы заметили частое упоминание некоторых традиционных математических концепций, таких как векторы, матрицы и тензоры. Особенно часто обсуждаются векторы.

Следует упомянуть, что вектор – это не только традиционное математическое понятие. С помощью моделей глубокого обучения можно преобразовывать различные данные, созданные человеческим обществом, в данные, представленные векторами. Это можно выразить фразой «все можно сделать вектором». Данные, представленные векторами, называются «векторными данными». На основе информации, содержащейся в векторных данных, ИИ-технологии могут легко понимать различные данные человеческого общества. Таким образом, векторные данные постепенно становятся основными данными при внедрении интеллектуальных технологий.

Поскольку векторные данные приобретают все большее значение, эффективное управление ими становится все более важным. За десятилетия своего развития технологии баз данных стали важной ветвью компьютерных наук, специально предназначенной для эффективного управления данными. Однако в силу различий подходов традиционные технологии баз данных не способны справиться с современными вызовами, характеризующимися высокой размерностью, высокой точностью и большими масштабами. Векторные базы данных были специально разработаны с учетом особенностей векторных данных, что позволяет им лучше хранить, индексировать и выполнять запросы таких данных. Эта технология появилась относительно недавно, но в ней уже появилось много деталей, которые стоит изучить и обсудить.

Для чего написана эта книга

Будучи профессионалом с более чем десятилетним опытом работы в компьютерной индустрии, я глубоко понимаю силу «обучения через практику». В период обучения в аспирантуре я получил «инсайты» во время работы с исходным кодом из классических книг, и эти моменты до сих пор остаются в моей памяти. К тому времени я уже мог решать множество задач по алгоритмам и программированию, но мое понимание механизмов работы компьютерных систем оставалось поверхностным. В частности, я понимал лишь на теоретическом уровне, как операционная система управляет процессами, распределяет ресурсы памяти и выполняет операции ввода-вывода. В итоге благодаря углубленному изучению и практической работе с богатым набором исходного

кода из книг «Ядро Linux» авторов Д. Бовет и М. Чезати и «Understanding Linux Network Internals: Guided Tour to Networking on Linux» автора Ch. Benvenuti я осознал, что в компьютерной области нет ничего мистического – это, по сути, код, который управляет нулями и единицами.

С тех пор я выработал привычку глубже разбирать новые технологии через практику. Практическая работа с кодом, стоящим за технологией, позволяет изучить ее более основательно, чем просто понять основные концепции. Для многих начинающих специалистов базы данных кажутся сложными, и они считают, что за ними скрываются «космические технологии». На самом деле базы данных – это технология, основанная на операционных системах и теории распределенных систем и реализующая сложное управление данными через программное обеспечение. За годы развития появилось множество теоретических и практических книг, которые могут служить справочником для новичков по этой теме.

Что касается новой технологии векторных баз данных, на рынке пока нет книги, которая бы направляла читателя в создании векторной базы данных с нуля. К счастью, я и моя команда работаем в этой области уже много лет, обслуживая многочисленных клиентов и накапливая ценный опыт первопроходцев этого направления. Основываясь на этом опыте, я надеюсь начать с самого простого кода программы «Hello World!» и провести вас через процесс написания собственной векторной базы данных на уровне исходного кода. Я верю, что благодаря этому процессу «высокотехнологичная» векторная база данных перестанет казаться такой загадочной.

Я понимаю, что из-за ограничений моего личного понимания и навыков писателя эта книга не может охватить все темы и аспекты и не гарантирует абсолютной безошибочности. Но если эта книга поможет вам лучше понять эту технологию и создать собственную упрощенную версию векторной базы данных, то моя цель будет достигнута.

Для кого написана эта книга

Эта книга является практическим руководством, в котором также объясняются базовые принципы. Технические аспекты в книге будут поняты начинающими программистами. Тем не менее для эффективного чтения необходимо знать основы программирования, так как в книге представлено немало исходного кода, с которым вам предстоит работать.

- Если вас интересуют векторные базы данных и вы хотите глубже понять процесс их создания на уровне исходного кода, эта книга научит вас создавать распределенные векторные базы данных с нуля. В книге рассматриваются такие темы, как создание системы индексов в автономной подсистеме базы данных, повышение способности системы к восстановлению после сбоя, а также реализация распределенной и кластерной работы, включая репликацию данных, управление трафиком и метаданными.
- Если вы интересуетесь базами данных и хотите глубже понять процесс их создания на уровне исходного кода, эта книга также будет полезна для вас – полный процесс создания распределенной векторной базы данных охватывает ключевые знания в этой области.

- Если вас интересует разработка ИИ-приложений и вы хотите понять, как создавать и управлять векторными данными, лежащими в их основе, книга расскажет о связи векторных данных с большими языковыми моделями и проведет вас через весь процесс выполнения запросов к векторным базам данных. Это поможет вам более эффективно интегрировать векторные базы данных для оптимизации ИИ-приложений, обновления знаний и более эффективного решения задач, возникающих при внедрении таких приложений.
- Если вы являетесь экспертом в разработке ИИ-приложений или в области баз данных и хотели бы внести вклад в улучшение этой книги и развитие отрасли в целом, эта книга также заслуживает вашего внимания. Чтение книги может вдохновить вас на более ценные размышления. Векторные базы данных – это относительно новая область, и обмен информацией, безусловно, будет способствовать ее прогрессу.

Ключевые особенности

Эта книга исповедует «практический подход». Как упоминалось ранее, этот принцип принес мне много пользы, поэтому я надеюсь через эту книгу поделиться этим методом с вами. Книга не нацелена на глубокое изучение принципов ИИ-технологий, а больше сосредоточена на векторных базах данных, лежащих в их основе. Направляя вас шаг за шагом к созданию собственной векторной базы данных, книга постепенно приводит к пониманию, как работает векторная база данных. Конечно, перед практическими занятиями будут представлены некоторые максимально краткие базовые знания с той технической глубиной, которая необходима для специалистов по базам данных.

Основная часть книги (главы 4–7) состоит преимущественно из фрагментов кода (с подробными комментариями) и анализа принципов его реализации. Весь код написан мной с нуля и охватывает темы от самых простых плоских индексов в векторной базе данных до несколько более сложных HNSW-индексов. В этом процессе я буду использовать существующий открытый исходный код для создания собственного сервиса. Это эффективная практика разработки программного обеспечения, ведь нам не нужно каждый раз изобретать велосипед.

Для еще более эффективного применения этого метода обучения я от начала до конца проведу вас через процесс создания ИИ-приложения на базе созданной ранее векторной базы данных, реализуя принцип «догфудинг». Представьте, что все – от веб-страницы до базового уровня хранения векторов, индексации и выполнения запросов – создано вашими руками. Вы, безусловно, испытаете огромное чувство удовлетворения, и с помощью этой книги вы достигнете такого уровня мастерства.

Как читать эту книгу

Структура

Книга разделена на три основные части, содержание которых организовано от простого к сложному, от основ к практике. Можно читать книгу по порядку

глав, но, если вы уже знакомы с материалом какой-то части, можно пропустить ее. Если вы хотите с нуля создать векторную базу данных, я настоятельно рекомендую читать по порядку.

Первая часть. Знакомство с векторными базами данных (главы 1–3)

Эта вводная часть книги призвана помочь заложить необходимые теоретические основы.

- Глава 1. Введение в базовые знания о векторах и векторных базах данных, определение основных понятий: вектор, база данных, векторная база данных.
- Глава 2. Сведения о развитии векторных баз данных как новой технологии, включая ключевые достижения, рост связанных компаний и стартапов, а также наиболее распространенные архитектурные модели в индустрии. Эта глава поможет понять эволюцию технологий на макроуровне.
- Глава 3. На примере Tencent Cloud VectorDB мы исследуем реализацию основных функций векторных баз данных. Если вы планируете самостоятельно разрабатывать или углубленно использовать векторные базы данных, эта глава познакомит вас с базовыми знаниями.

Вторая часть. Создание векторной базы данных (главы 4–6)

Эта часть является основной и подробно описывает, как создать с нуля и оптимизировать векторную базу данных. Здесь обсуждаются различные технические детали и приводятся примеры.

- Глава 4. Создание автономной векторной базы данных с нуля. Глава основана на концепции модульной архитектуры. Здесь используются некоторые зрелые открытые компоненты, постепенно добавляются общие программные и базовые технологии, применяются технологии персистентности и восстановления после сбоя из индустрии баз данных. В итоге демонстрируется процесс создания автономной векторной базы данных, которая пока мала, но уже полноценна и сопоставима с прототипами некоторых открытых автономных векторных баз данных.
- Глава 5. На основе автономной векторной базы данных продолжается разработка распределенной векторной базы данных, включая эффективное управление метаданными, управление трафиком и обработку исключений. Начиная с этой главы, наша векторная база данных будет обладать прототипным уровнем поддержки управления векторными данными определенного масштаба. Если вы хотите понять механизмы распределенных систем на низком уровне и узнать, как расширить автономную систему до распределенной, не пропускайте эту главу.
- Глава 6. На основе распределенной векторной базы данных, построенной в предыдущих двух главах, демонстрируются приемы оптимизации, включая производительность, издержки и удобство использования. В части оптимизации производительности будет рассмотрено, как использо-

вать вычислительные возможности CPU/GPU для повышения производительности. В части оптимизации издержек будет уделено внимание упрощению модели развертывания бизнеса для снижения операционных затрат. В заключение обсуждается, как сделать продукт векторной базы данных более удобным для разработчиков, например, предоставляя хороший пакет SDK и механизмы резервного копирования данных, что крайне важно для продвижения новой технологии.

Третья часть. Практика и перспективы векторных баз данных (главы 7–8)

Эта часть является заключительной и сосредоточена на практике применения и перспективах. Изучив эту часть, вы сможете не только соединить теоретические знания из книги с практическими навыками, но и заглянуть в будущее технологий, что поможет вам найти направление для дальнейшего обучения и планирования карьеры.

- Глава 7. В этой главе на практике проверяются методы использования разработанной нами векторной базы данных в ИИ-приложениях. Мы реализуем упомянутые в главе 1 сценарии поиска по изображению и базы знаний, которые в последние годы широко используются в ИИ-приложениях.
- Глава 8. Эта глава вернет нас к истории развития компьютерной индустрии, чтобы исследовать будущее векторных баз данных. С одной стороны, здесь обсуждается, почему векторные базы данных могут стать платформенной технологией. С другой стороны, на основе текущего применения векторных баз данных в сценариях генерации, дополненной поиском, развивается обсуждение будущего развития векторных баз данных. В этой главе я бы хотел вместе с вами поразмышлять и спрогнозировать долгосрочные тенденции векторных баз данных.

Ресурсы и соглашения

Весь исходный код можно скачать на домашней странице книги в сообществе Turing (www.ituring.com.cn/book/3305). Код включает десятки тысяч строк кода и позволяет сочетать чтение с практическими упражнениями.

1. Иерархические ментальные карты

В некоторых главах представлены иерархические ментальные карты, которые призваны помочь вам быстро освоить основные функции и состав модулей векторной базы данных. Перед чтением материала я настоятельно рекомендую ознакомиться с соответствующей ментальной картой, чтобы получить общее представление.

2. Итерации и обновления версий

В этой книге мы вместе создадим векторную базу данных, которая пройдет через несколько итераций и обновлений версий от v0.0.1 до v0.6. Переход от версии v0.0.1 к версии v0.0.2 сопровождается незначительными изменениями и в книге называется итерацией версии. А переход от v0.0.2 к v0.1, который вклю-

чает более значительные изменения, называется обновлением версии. Каждая новая версия будет включать добавление или обновление определенных функций. Для удобства реализации функций соответствующих версий, а также для возможности в любой момент ознакомиться с ними или вспомнить их в оглавление книги включены номера итераций и новых версий. Кроме того, для значительных обновлений версий в разделе «Резюме» соответствующей главы предоставлена схема версий, которая поможет вам полностью понять способы реализации и изменения архитектуры векторной базы данных.

3. Список новых и обновленных модулей и функций

В книге в табличной форме приводится список новых и обновленных модулей и функций в каждой версии, что помогает лучше понять основные возможности векторной базы данных и в конечном итоге реализовать код самостоятельно.

Я рекомендую в процессе чтения активно использовать предоставленные в книге исходные коды, ментальные карты, схемы архитектуры, модули информации о версиях и списки функций. Все это – тщательно разработанные мной инструменты, которые помогут вам более эффективно изучить содержание книги.

4. Открытые библиотеки

Существует множество открытых проектов, которые уже инкапсулировали необходимые нам функции, поэтому нет необходимости писать код самостоятельно – достаточно использовать открытые библиотеки. В табл. 0.1 приведены открытые библиотеки, используемые в книге, и типы их лицензий.

Таблица 0.1. Используемые в книге открытые библиотеки и их лицензионные соглашения

Название	Краткое описание	Лицензионное соглашение
FAISS	Библиотека управления векторами	MIT
RapidJSON	Библиотека JSON	BSD
cpp-httpLib	Библиотека HTTP-сервисов	MIT
HNSWLib	Библиотека управления векторами	Apache-2.0
spdlog	Библиотека управления журналами	MIT
RocksDB	База данных типа «ключ – значение»	GPLv2, Apache-2.0
RoaringBitmap	Библиотека управления битовыми картами	Apache-2.0
NuRaft	Реализация протокола Raft	Apache-2.0
Etdcd	Система хранения типа «ключ – значение»	Apache-2.0
gRPC	Подсистема удаленных вызовов	Apache-2.0
jwt-cpp	Библиотека управления аутентификацией	MIT
BGE	Семантическая векторная модель	MIT
Flask	Платформа разработки веб-приложений	BSD-3-Clause

БЛАГОДАРНОСТИ

Я посвящаю эту книгу своей семье, особенно моей супруге Лун Сяоцин (Long Xiaocīng). В течение последнего года, чтобы поддержать меня в написании книги в свободное время, ты вместе со мной отказалась от всех праздников, и часть иллюстраций в книге также была создана тобой. Без твоей поддержки я бы никогда не смог завершить эту книгу. Также благодарю свою мать за помощь супруге в заботе о семье. Благодарю сына и дочь – возможно, вы пока не понимаете значения этой книги для меня, но ваша любознательность к процессу написания тронула меня, и именно вы дали мне наибольшее количество энергии.

Благодарю своих коллег из Tencent Cloud – одного из первых облачных провайдеров в Китае, реализовавших векторные базы данных. Большая часть моего понимания содержания книги пришла ко мне во время обсуждений с вами и из практики обслуживания клиентов. Я верю, что без неустанных усилий нашей команды не было бы такого отличного продукта. А без этого продукта я бы не смог извлечь столько ценного опыта для написания этой книги.

Хочу выразить особую благодарность редактору этой книги Лю Мэйин (Liu Meiyīng). Именно с вашего письма установились наши взаимоотношения, и вы дали мне мотивацию попробовать написать эту книгу. Для программиста, впервые пытающегося написать книгу, этот процесс был непростым. Материал книги прошел множество итераций, включая проектирование структуры и редактирование текста. В течение этого времени ваши наставления и советы принесли мне большую пользу. Можно сказать, что именно благодаря вашему упорству и профессионализму эта книга смогла увидеть свет.

Благодарю всех, кто оказал мне помощь в моей профессиональной карьере, включая учителей, родных, коллег и друзей. Вы стали свидетелями моего пути от юноши, не знавшего ничего, до программиста, имеющего некоторое влияние в отрасли. Без вашей поддержки и понимания я бы не достиг того, что имею сегодня.

В заключение выражаю благодарность всем читателям, купившим эту книгу, – я рад возможности познакомиться с вами через ее страницы. Будучи одним из первых специалистов, начавших работать с векторными базами данных, я не ожидал, что так быстро смогу применить свои знания и поделиться ими столь замечательным образом с более широкой аудиторией. Если вы стремитесь преуспеть в эпоху бурного развития искусственного интеллекта, надеюсь, эта книга станет отправной точкой для вашего великого путешествия. Я всегда придерживаюсь принципа «Think big, do small» (думай масштабно, действуй локально), ведь, только обладая грандиозным видением, мы можем преодолевать трудности и смело двигаться вперед!



Часть 1

Знакомство с векторными базами данных

На мой взгляд, для глубокого понимания концепции эффективным методом является следование пути «снизу вверх»: сначала овладеть ее основными понятиями, а затем постепенно углубляться в связанные приложения и сценарии. Этот метод подчеркивает процесс постепенного построения системы знаний от простого к сложному. При изучении векторных баз данных мы также будем использовать эту стратегию обучения.

В первой части мы начнем с основ векторных баз данных – понятия и свойств векторов. Мы раскроем их принципиальные отличия от традиционных баз данных, а также исследуем необходимость и преимущества векторных баз данных в современной обработке данных. Затем мы пройдем через краткую историю развития векторных баз данных – от зарождения этой технологии до ее нынешнего роста – и изучим ее эволюцию и текущее состояние применения в отрасли. В заключение мы углубимся в основные возможности векторных баз данных, включая их базовые функции и переломные технические характеристики, чтобы представить вам полную картину этой технологической области.

Глава 1

Основы векторных баз данных

Озарение предшествует внедрению.

Макс Планк

В этой главе мы начнем с основного понятия «вектор», дадим его определение, опишем свойства и области применения. Таким образом мы поймем первую часть в термине «векторная база данных». Затем рассмотрим историю развития технологий баз данных и пройдем путь от реляционных до нереляционных баз данных. Таким образом мы поймем вторую часть в термине «векторная база данных». В заключение проанализируем различия между векторными и традиционными данными, обсудим, почему требуется специализированная векторная база данных для обработки векторных данных, и рассмотрим возможную роль векторных баз данных в эпоху больших языковых моделей.

1.1. ВЕКТОР

В повседневной жизни мы часто сталкиваемся с различными неструктурированными данными, такими как текст, изображения, аудио и видео. Для лучшего понимания и анализа этих данных необходимо преобразовать их в форму, которую можно обработать математическими методами. Вектор является такой формой представления. В этом разделе вы узнаете о важной роли векторов в реальном мире, что даст вам полезные ориентиры для исследований и практики в связанных областях. Давайте же приступим к исследованию мира векторов!

1.1.1. Что такое вектор

Представьте, что в приятный субботний день папа везет маму и двоих детей (брата и сестру) в новый торговый центр на шопинг. В этом, казалось бы, обычном жизненном сценарии повсюду можно встретить векторы и скаляры.

1. Вектор

По дороге в торговый центр папа должен следовать указаниям навигатора, чтобы определить направление движения. Навигатор сообщает папе «через 500 метров поверните налево». Эти «500 метров» являются *вектором*, который указывает расстояние до перекрестка и направление движения. В жизни существует множество векторных данных, таких как ветер (вектор ветра, обозначающий скорость и направление ветра), сила и др., которые включают в себя два элемента: величину и направление, как показано на рис. 1.1.

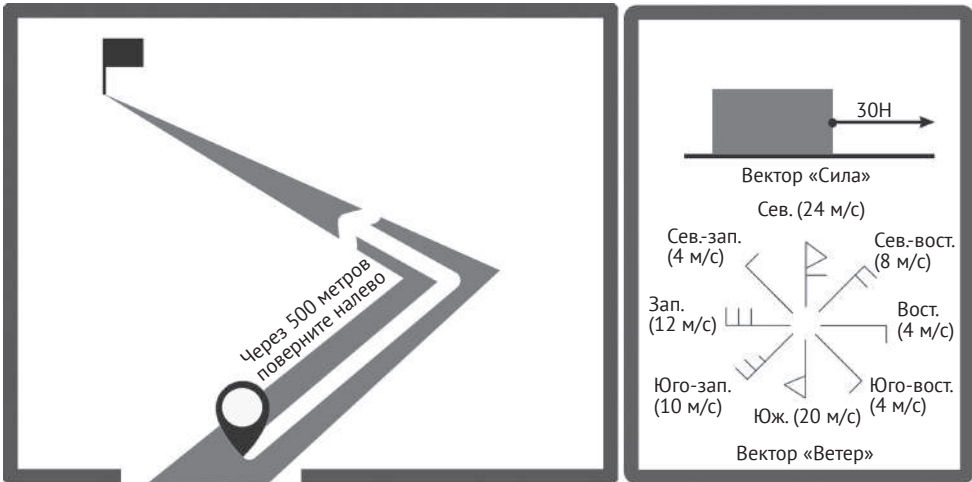


Рис. 1.1. Векторы в повседневной жизни

В математике векторы обычно обозначаются полужирным курсивом, например $\mathbf{a}$, $\mathbf{b}$, $\mathbf{c}$ и т. д. Векторы можно также представить в координатном виде. Например, вектор на плоскости можно выразить как $\mathbf{a} = (x, y)$, где x и y представляют собой компоненты вектора по осям x и y соответственно. Аналогично в трехмерном пространстве можно представить как $\mathbf{a} = (x, y, z)$. На самом деле векторы можно расширить до более высоких размерностей: четырехмерной, пятимерной и т. д. В этом случае вектор можно представить как $\mathbf{a} = (x_1, x_2, x_3, \dots, x_n)$, где n обозначает размерность вектора.

В компьютерах векторы могут быть представлены массивом чисел с плавающей запятой. Например, трехмерный вектор можно представить как $[x, y, z]$, а четырехмерный – как $[x_1, x_2, x_3, x_4]$ и т. д. Такой способ представления позволяет легко выполнять над векторами различные математические операции. В геометрическом смысле сумма двух векторов – это вектор, соединяющий начало первого вектора и конец второго, при этом конец первого вектора совпадает с началом второго. Вектор, равный по величине текущему вектору, но противоположный по направлению, называется обратным к текущему. Разность двух векторов можно представить как сумму первого вектора и обратного ко второму, как показано на рис. 1.2.

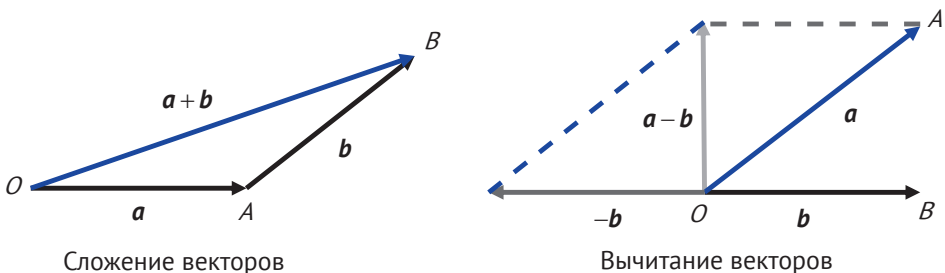


Рис. 1.2. Сложение и вычитание векторов

2. Скаляр

В торговом центре маме приглянулось красивое платье, и она поинтересовалась его ценой. Увидев на ценнике «200 юаней», она задумалась, хорошая ли эта цена. Здесь «200 юаней» – это скаляр, который обозначает только величину, не учитывая направление. В повседневной жизни существует множество скаляров, таких как температура, масса и т. д.

В математике скаляры обычно обозначаются курсивными буквами, такими как a , b , c и т. д.

В компьютерах скаляр обычно представляется отдельным числовым значением, таким как целое число, число с плавающей запятой или комплексное число. Эти типы чисел имеют соответствующие представления в различных языках программирования. Например, в Python скаляр можно представить целым числом, как в выражении `price = 200`. Такой способ представления позволяет легко выполнять над скалярами различные математические операции, такие как сложение, вычитание, умножение и деление.

3. Различия между векторами и скалярами

На основе вышеизложенного можно сформулировать основные различия между векторами и скалярами.

1. Векторы имеют величину и направление, тогда как скаляры имеют только величину.
2. Векторы можно представить с помощью координат, тогда как скаляры обычно выражаются обычными числовыми значениями.
3. Векторы могут участвовать в операциях сложения, вычитания и умножения на число, тогда как скаляры могут участвовать в операциях сложения, вычитания, умножения и деления.

1.1.2. Все может быть вектором

В торговом центре брату с сестрой больше всего нравится магазин игрушек, особенно секция с аниме-игрушками. Родителям сложно понять, какие персонажи представлены среди множества игрушек, но дети обычно могут отличить их по характерным чертам. Можно ли упростить эту задачу с помощью цифрового подхода?

Простой способ – расположить эти игрушки на одномерной координатной оси, упорядочив их по длине тела, нормализованной в диапазоне от 0 до 1. Ультрамен имеет максимальную длину тела 0.999, а мышонок Джерри – минимальную 0.001. Когда мы располагаем этих персонажей на этой оси, Джерри и Ультрамен оказываются далеко друг от друга, что облегчает их различение. Однако у братьев Марио и Луиджи длина тела одинаковая и равна 0.500, то есть на этой оси они неразличимы. Что делать в этом случае? Можно добавить еще одно измерение – массу персонажа. Луиджи немного стройнее, поэтому, поместив Марио и Луиджи в двухмерную систему координат с добавлением оси массы, получим их координаты: (0.500, 0.180) и (0.500, 0.120), которые позволяют лучше их различать.

Однако длина и масса братьев Хайэр совершенно одинаковы, их параметры равны (0.550, 0.180). В такой ситуации мы пытаемся ввести еще одну размерность, нормализуя цвет брюк (соотношение красного, зеленого и синего, прозрачность и насыщенность и т. д.) до числа с плавающей запятой для упрощенного примера. После добавления цвета брюк братьев Хайэр можно будет представить двумя трехмерными наборами координат (0.550, 0.180, 0.100) и (0.550, 0.180, 0.800). Аналогичным образом можно добавить больше размерностей (например, цвет шляпы, длина усов, длина волос, цвет обуви и т. д.), чтобы различать большее количество персонажей аниме и игр. Это эквивалентно помещению этих персонажей в многомерный мир (0.550, 0.180, 0.100, 0.220, 0.833...), в котором каждый персонаж представляет собой одну точку, а направленный отрезок, указывающий от начала координат к его местоположению, является вектором, как показано на рис. 1.3.

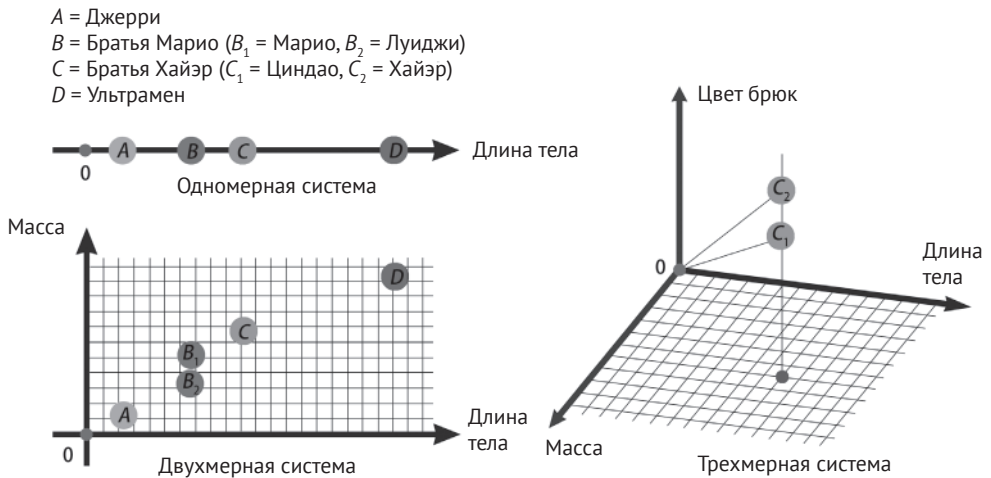


Рис. 1.3. Векторизация персонажей аниме и игр

Более интересно то, что, если мы вычтем вектор Джерри из вектора Тома, мы обнаружим, что разность этих векторов очень близка к разности векторов Ультрамена и Монстра. Как было сказано в разделе 1.1.1, вычитание векторов определяется их относительным положением, а не абсолютным, то есть различия между Ультраменом и Монстром (различия между героем и злодеем, победой и поражением, предпочтениями зрителей и т. д.) очень похожи на различия между Томом и Джерри. Таким образом, математические отношения между векторами можно сопоставить с фактическими отношениями между персонажами. Аналогично с увеличением размерностей некоторые трудно распознаваемые неструктурированные данные после векторизации начинают проявлять определенные связи. Здесь данные о характеристиках персонажей аниме и игр приведены лишь в качестве примера. Для извлечения этих характеристик требуются методы машинного обучения, среди которых особенно эффективными являются модели глубокого обучения.

Модели глубокого обучения¹

Глубокое обучение является подразделом машинного обучения. Оно пытается имитировать работу человеческого мозга, обучаясь абстрактным представлениям данных через многослойные нейронные сети. Модели глубокого обучения могут обрабатывать различные типы данных, такие как изображения, текст, аудио и видео. Автоматически изучая иерархические особенности данных, эти модели способны преобразовывать сложные неструктурированные данные в простые векторные представления.

В основе моделей глубокого обучения лежат нейронные сети, которые представляют собой вычислительные модели, имитирующие структуру нейронов человеческого мозга. Нейронная сеть состоит из узлов, расположенных на множестве уровней. Каждый узел имеет определенный вес и функцию активации. Когда входные данные проходят через узлы, они последовательно обрабатываются и преобразуются в соответствии с весами и функциями активации, в конечном итоге генерируя выходной результат. Путем обучения нейронная сеть может автоматически корректировать веса и функции активации, чтобы модель могла лучше соответствовать входным данным.

Преимущество моделей глубокого обучения заключается в их мощной способности к выражению и высокой адаптивности. В отличие от традиционных методов машинного обучения модели глубокого обучения могут автоматически изучать высокоуровневые абстрактные особенности данных без необходимости в ручном проектировании признаков. Это позволяет этим моделям демонстрировать выдающуюся производительность в различных сложных задачах.

Модели глубокого обучения, обученные на больших объемах данных, могут абстрагировать их математические особенности, которые в совокупности представляют собой векторное выражение этих объектов в цифровом мире.

После векторизации всех объектов через модели глубокого обучения становится важным распознавание отношений между векторами, что включает в себя вычисление сходства между ними.

1.1.3. Сходство между векторами

Сходство между векторами используется для измерения степени близости двух векторов в числовом выражении. Вычисление сходства является одной из ключевых задач векторного анализа. Далее мы рассмотрим распространенные методы вычисления сходства – косинусное сходство, скалярное произведение и евклидово расстояние – и проиллюстрируем их различия и применимость на примере цифровых персонажей аниме и игр.

1. Косинусное сходство

Косинусное сходство – это мера для оценки степени сходства двух векторов. Оно оценивает их сходство путем вычисления косинуса угла между векторами. Та-

¹ Это лишь краткое описание процесса. Для более глубокого понимания принципов моделей глубокого обучения я рекомендую прочитать книгу «Глубокое обучение на Python (2-е издание)» (Издательство связи и телекоммуникаций, 2022).

ким образом, косинусное сходство акцентирует внимание на угле между двумя векторами, измеряя степень их сходства по направлению.

Формула для вычисления косинусного сходства:

$$\text{cosine_similarity}(\mathbf{A}, \mathbf{B}) = \frac{\mathbf{A} \cdot \mathbf{B}}{\|\mathbf{A}\| \|\mathbf{B}\|},$$

где $\mathbf{A}$ и $\mathbf{B}$ обозначают два вектора, угол между ними равен θ . $\text{cosine_similarity}(\mathbf{A}, \mathbf{B})$ можно также записать как $\cos \theta$. $\mathbf{A} \cdot \mathbf{B}$ обозначает скалярное произведение векторов $\mathbf{A}$ и $\mathbf{B}$ (также называемое поточечным произведением, см. далее), а $\|\mathbf{A}\|$ и $\|\mathbf{B}\|$ обозначают модули (или нормы) векторов $\mathbf{A}$ и $\mathbf{B}$ соответственно.

Значение косинусного сходства варьируется от -1 до 1 . Если два вектора очень схожи, угол между ними будет очень мал. Если направления векторов полностью совпадают, косинусное сходство равно 1 . И наоборот, если направления векторов противоположны, косинусное сходство равно -1 . Если векторы перпендикулярны, косинусное сходство равно 0 . Преимущество косинусного сходства заключается в том, что оно не зависит от величины векторов, а только от их направления. Поэтому, когда необходимо сравнить степень сходства в направлении между двумя векторами, можно использовать косинусное сходство.

2. Скалярное произведение

Скалярное произведение измеряет длину проекции одного вектора на другой в том же направлении, отражая степень сходства их проекций. Скалярное произведение вычисляется путем суммирования произведений соответствующих компонент двух векторов. Когда необходимо сравнить абсолютные значения двух векторов по определенным характеристикам (пример будет приведен далее), можно использовать скалярное произведение.

Формула для вычисления скалярного произведения:

$$\text{inner_product}(\mathbf{A}, \mathbf{B}) = \mathbf{A} \cdot \mathbf{B}.$$

Пусть $\mathbf{A} = (a_1, a_2, \dots, a_n)$ и $\mathbf{B} = (b_1, b_2, \dots, b_n)$, тогда их скалярное произведение определяется как:

$$\mathbf{A} \cdot \mathbf{B} = a_1 \times b_1 + a_2 \times b_2 + \dots + a_n \times b_n.$$

В геометрическом смысле скалярное произведение векторов равно произведению их модулей на косинус угла между ними, то есть:

$$\mathbf{A} \cdot \mathbf{B} = \|\mathbf{A}\| \|\mathbf{B}\| \cos \theta.$$

3. Евклидово расстояние

Евклидово расстояние – это расстояние по прямой между двумя точками в евклидовом пространстве. Оно интуитивно представляет степень близости между двумя точками, а его формула проста и применима в двухмерном, трехмерном и даже в пространствах более высокой размерности. Поэтому евклидово расстояние широко используется в обработке изображений, распознавании образов, добыче данных и других областях для измерения сходства или рас-

стояния между векторами, образцами или данными. Например, в алгоритме k ближайших соседей и кластерном анализе.

Евклидово расстояние между двумя векторами вычисляется путем суммирования квадратов разностей соответствующих компонент и извлечения квадратного корня из полученной суммы. Евклидово расстояние акцентирует внимание на абсолютном расстоянии между векторами и подходит для сравнения их относительного положения в пространстве. Когда нас интересует степень различия между двумя векторами, можно использовать евклидово расстояние.

Формула для вычисления евклидова расстояния:

$$\text{euclidean_distance}(\mathbf{A}, \mathbf{B}) = \sqrt{\sum_{i=1}^n (A_i - B_i)^2},$$

где $\mathbf{A}$ и $\mathbf{B}$ обозначают два n -мерных вектора, а A_i и B_i обозначают i -е компоненты векторов $\mathbf{A}$ и $\mathbf{B}$. Евклидово расстояние часто сокращенно называют L2, где L обозначает длину (length), а 2 – это показатель степени в приведенной выше формуле.

Вернемся к примеру с персонажами аниме и игр. Если мы хотим сосредоточиться на таких характеристиках, как сила или принадлежность к положительным или отрицательным персонажам, мы можем использовать косинусное сходство для оценки. Хотя Ультрамен и братья Марио сильно различаются по телосложению и силе, их характерные черты очень похожи. То есть в этом цифровом многомерном мире они имеют высокую степень сходства по направлению. Аналогично отрицательные персонажи также имеют высокую степень сходства по направлению в этом цифровом многомерном мире.

Теперь давайте обратим внимание на сценарий, в котором для оценки сходства можно использовать скалярное произведение векторов. Хотя Ультрамен и братья Марио являются в своих мирах сильными персонажами, очевидно, что сила Ультрамена больше. Как же мы можем отразить это различие, используя их векторы в цифровом мире? Как упоминалось ранее, для сравнения абсолютных значений векторов по определенным признакам можно использовать скалярное произведение. Оно тесно связано с абсолютным расстоянием векторов от начала координат и может непосредственно отражать степень удаленности векторов от начала координат.

Евклидово расстояние оценивает сходство на основе относительного положения векторов в многомерном пространстве, отражая прямое расстояние между двумя векторами. Хотя гриб Тоад меньше и слабее братьев Марио, в абстрактном цифровом мире Тоад находится гораздо ближе к братьям Марио, чем к Ультрамену. Если мы хотим уменьшить значение абсолютного положения векторов относительно начала координат и сосредоточиться на относительном расстоянии между векторами, то сравнение по евклидову расстоянию является более подходящим выбором.

1.1.4. Примеры использования сходства

Поняв, что все в мире может быть подвергнуто извлечению признаков и векторизации с помощью моделей глубокого обучения, и узнав, что сходство

между векторами может отражать взаимосвязь реальных объектов, мы можем рассмотреть несколько более конкретных примеров.

1. Поиск по изображению

Представьте, что у вас в телефоне есть фотография чрезвычайно красивого пейзажа, но вы уже не помните точное место съемки. Вы хотите найти больше похожих фотографий, чтобы вспомнить больше приятных моментов. Традиционный метод заключается в просмотре сотен фотографий в галерее телефона. Если известно примерное время, когда была сделана фотография, поиск будет относительно простым. Однако, если вы не уверены в конкретном времени, просмотр каждой фотографии может очень быть утомительным. Но благодаря векторизации разработчики приложений для галереи могут значительно повысить эффективность поиска связанных изображений. С помощью моделей глубокого обучения сильно похожие изображения можно преобразовать в векторы с большим сходством. Этот процесс предобучения позволяет получить обученную модель векторизации.

Эта модель имеет две основные функции: во-первых, она преобразует все сохраненные в галерее изображения в векторную форму. Во-вторых, когда в приложении галереи мы выполняем поиск по изображению, модель также преобразовывает его в векторную форму, после чего система вычисляет сходство этого вектора с векторами ранее преобразованных изображений. Таким образом, мы можем быстро найти наиболее похожие фотографии пейзажей среди сотен изображений – наши глаза и пальцы наконец-то освобождены от рутины!

Реализация поиска похожих изображений на основе векторизации является типичным примером применения сходства между векторами. Во второй части книги мы с нуля создадим распределенную векторную базу данных, а в третьей части на ее основе построим систему поиска по изображениям, как показано на рис. 1.4.



Рис. 1.4. Поиск по изображению

2. Распознавание музыки

Вы часто сталкиваетесь с ситуацией, когда в голове постоянно звучит мелодия, но вы никак не можете вспомнить название песни и текст? Эта навязчивая мелодия может не давать вам покоя целый день. Обычно вы напеваете мелодию, чтобы кто-то из родных или друзей помог вам определить, что это за песня. Однако они не всегда могут помочь. С векторизацией эта проблема решается гораздо проще.

Для повышения эффективности программного обеспечения для поиска музыки можно использовать модели глубокого обучения, чтобы обработать большое количество музыкальных произведений и преобразовать их в векторные данные. Этот процесс создает уникальные характеристики для каждой песни, и чем выше сходство между характеристиками, тем сильнее схожесть песен. Как только песни в нашей музыкальной библиотеке преобразованы в векторы высокой размерности, мы можем записать мелодию, которая у нас в голове, и загрузить ее в программу. Программа векторизует эту мелодию, сравнит ее с уже векторизованными песнями в библиотеке и быстро найдет совпадение с оригинальной композицией. Кроме того, программа может рекомендовать нам другие произведения, схожие с искомой композицией, что еще больше обогащает наш музыкальный опыт, – технологии снова улучшают качество нашей жизни. На рис. 1.5 изображен интерфейс системы распознавания песен (эта система не реализована в книге, предлагаем вам попробовать создать ее самостоятельно после прочтения).



Рис. 1.5. Распознавание песни

3. Понимающий вас клиентский сервис

Если вы часто совершаете покупки онлайн, вы, вероятно, знакомы с некоторыми проблемами в послепродажном обслуживании на платформах электронной коммерции. Например, когда вы недовольны приобретенным товаром и хотите сообщить об этом службе поддержки, часто приходится повторять одну и ту же проблему нескольким сотрудникам. Или же другие покупатели уже

поднимали схожие вопросы и получили решение, но служба поддержки не может точно понять ваши потребности. Почему бы службе поддержки не просмотреть ваши предыдущие коммуникации (голосовые и письменные), чтобы более эффективно вам помочь? Причина в том, что в традиционной модели обслуживания сотрудники поддержки могут за короткое время обрабатывать запросы множества клиентов, что затрудняет запоминание конкретного содержания диалогов с каждым клиентом.

Однако с помощью векторизации и других ИИ-технологий разработчики могут значительно улучшить этот опыт. После сохранения диалогов клиентов со службой поддержки можно на этих данных обучить модели глубокого обучения, чтобы можно было распознавать семантически схожие вопросы и содержание общений. Когда клиент обращается в службу поддержки, система может автоматически связать вопрос клиента с предыдущими запросами и решениями через запрос сходства векторов, включая связывание информации из нескольких диалогов одного клиента, а также связывание схожих вопросов разных клиентов, как показано на рис. 1.6. На основании полученных связей система может предоставить сотруднику поддержки информацию о контексте и решениях по схожим вопросам. Таким образом, сотрудники смогут более эффективно общаться с клиентами, основываясь на предыдущей истории диалогов, что позволит лучше сгладить негативные эмоции клиентов и значительно улучшить их опыт покупок. В третьей части книги будет построена система персональной базы знаний, в описанном здесь сценарии обслуживания клиентов можно использовать эту базу знаний для внесения улучшений.

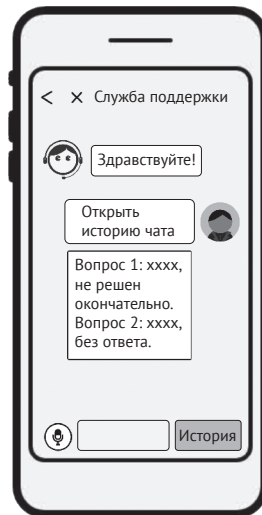


Рис. 1.6. Связывание информации диалогов в системе поддержки клиентов

4. Технологический процесс объединения векторов с моделями глубокого обучения

Мы рассмотрели в качестве примера три различных сценария, все они связаны с объединением векторных данных и моделей глубокого обучения. Несмотря на различия в применении, они имеют сходство с технологической точки зрения.

Во-первых, разработчикам необходимо получить крупномасштабный набор данных, который может быть как общедоступным, так и частным.

Затем необходимо использовать ресурсы GPU и платформы глубокого обучения для обучения модели – этот этап называется предобучением. После нескольких часов или дней обучения разработчик получит векторизованную модель, обладающую способностью извлечения признаков. Эта модель способна преобразовывать часть неструктурированных данных в векторы высокой размерности.

С помощью векторизованной модели разработчик может заранее провести векторизацию изображений, музыки или текстов, которые необходимо использовать для поиска, и сохранить эти данные в виде векторов высокой размерности. Когда пользователь вводит запрос, приложение через векторизованную модель проводит векторизацию и генерирует вектор высокой размерности для запроса. В итоге приложение вычисляет сходство между вектором пользовательского запроса и ранее сохраненными векторами, реализуя интеллектуализированный поиск.

Это типичные сценарии из реальной жизни, в которых мы объединяем векторизацию всего сущего с технологиями искусственного интеллекта. У нас есть основания полагать, что вектор, как наиболее подходящий математический способ выражения на базовом уровне, в сочетании с ИИ-технологиями будет в будущем тесно связан с различными аспектами человеческой жизни.

Проведение высококачественной векторизации неструктурированных данных, а также как эффективное хранение и извлечение векторизованных данных стали ценными навыками, которыми специалисты по хранению данных стремятся освоить.

С момента появления технологий баз данных их основной задачей было решение вышеупомянутых задачи. Далее мы обсудим, как традиционные технологии баз данных решают задачи хранения и запроса данных. Мы оценим, способны ли традиционные базы данных справляться с этими задачами при работе с векторизованными данными.

1.2. База данных

В современном цифровом мире данные стали одним из ключевых активов как для предприятий, так и для частных лиц. Чтобы лучше хранить и использовать эти активы, специалисты в области компьютерных наук разработали множество подходов, таких как системы хранения неструктурированных данных и системы баз данных для хранения структурированных данных.

В чем разница между этими двумя типами систем? Почему необходимо разделять системы баз данных от систем хранения? Какие подкатегории существуют в системах баз данных? С учетом этих вопросов давайте подробнее рассмотрим реальный сценарий работы с данными.

1.2.1. Что такое база данных

В Китае существует традиция, согласно которой у больших и малых семей есть свои генеалогические книги, которые в текстовой форме фиксируют

удобно, так как приходится по отдельности открывать каждый файл и вручную определять, кто из людей соответствует условиям. Основная причина неудобства заключается в том, что наши данные хранятся в неструктурированном виде, и информация о каждом человеке представлена в виде неструктурированного текста. Такой текст записан в форме, понятной человеку, но недоступной для понимания компьютером.

В контексте определения соответствия пожилых людей условиям эта система хранения данных оказывается неэффективной, поскольку требует вмешательства человека для анализа и получения окончательного результата. Специалисты в области компьютерных наук, обнаружив эту проблему, начали размышлять о том, как можно оптимизировать доступ к данным.

Если внимательно проанализировать данные генеалогического древа, можно заметить, что ключевая информация о каждом члене семьи весьма схожа. Мы можем разделить данные на несколько определенных полей, таких как «имя», «дата рождения», «поколение» и т. д. Этот процесс абстракции в компьютере называется структурированием данных. В структурированных данных каждое поле имеет фиксированное значение, и мы можем хранить их в фиксированном формате. Хотя такой формат трудно читается человеком, он значительно упрощает использование данных компьютером.

Однако специалисты в области компьютерных наук не были намерены останавливаться на достигнутом. Как только данные становятся структурированными, система баз данных может дополнительно использовать некоторые данные для ускорения доступа к членам семьи. Например, можно использовать дополнительные данные временной шкалы, чтобы отсортировать и сохранить членов семьи по дате рождения, что позволит быстрее находить нужного человека. Такие дополнительные данные, используемые для ускорения доступа к исходным данным, в технологии баз данных называются «индексами». Индексы в данном контексте выполняют функцию, аналогичную указателям в словарях.

Системы хранения, ориентированные на структурированные данные, обычно используют технологию индексов для ускорения доступа к соответствующим данным. Такие программные системы называются базами данных.

На основе этого примера можно сделать следующие выводы о базах данных. База данных представляет собой крупную категорию систем хранения. По своей сути база данных остается системой хранения, решающей задачи хранения и чтения данных.

В отличие от обычных систем хранения базы данных сосредоточены на структурированных данных, в которых каждое поле имеет свое отдельное значение, что облегчает компьютеру выполнение последующей обработки этих данных.

Поскольку каждое поле имеет фиксированный формат и значение, система баз данных может добавлять к этим фиксированным полям дополнительные данные, чтобы связать или упорядочить исходную информацию. Например, отсортировать по времени, по хешу, по древовидной структуре и т. д. Эти дополнительные данные позволяют ускорить доступ к исходным данным, что является основной важной технологией индексов в базе данных.

1.2.2. Реляционные базы данных

В процессе управления данными генеалогического древа можно заметить некоторые особенности данных. Для каждого члена семьи информация, которую необходимо хранить, в основном фиксирована: у каждого есть дата рождения и имя. Размерность этих данных редко увеличивается или уменьшается, и их значение также остается фиксированным. Это позволяет определить, какие значения являются правильными, а какие – ошибочными.

Таким образом, если мы заранее определим, какие в генеалогическом древе должны быть поля, а также какие данные в каждом поле являются допустимыми, то в дальнейшем управлять этими данными будет проще, и будет возникать меньше ошибок. Если при вводе данных обнаружится отсутствие поля с именем, такие данные не будут записаны. Если в поле пола введено значение «ни мужчина, ни женщина», это также можно будет обнаружить заранее. Это и есть определение структуры таблицы в системе баз данных. Предварительное определение структуры помогает уменьшить вероятность ошибок в данных: только данные, прошедшие проверку на допустимость, будут записаны в базу данных. Нельзя добавить или убрать поле, нельзя ошибиться в значении поля.

Реляционная база данных – это тип базы данных, в которой необходимо заранее определить формат каждой строки и столбца данных. На этой основе была разработана мощная теоретическая база и модели проектирования, включая такие понятия, как атомарность данных, согласованность, изоляция и персистентность. Эти четыре характеристики обеспечивают надежность и целостность структурированных данных в реляционных базах данных. На рис. 1.8 показан типичный реляционный набор данных, в котором для ограничения формата числового столбца используются ограничения атрибутов, а для ограничения конкретных значений в числовом столбце – ограничения внешнего ключа.

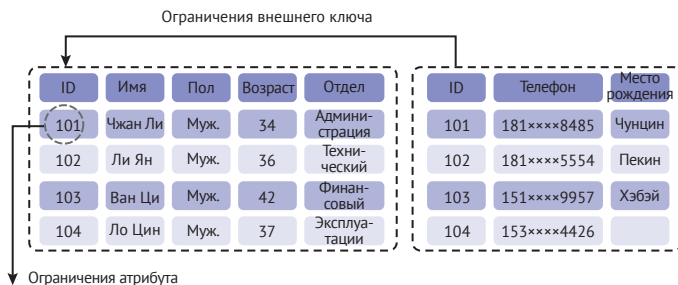


Рис. 1.8. Реляционный набор данных

За десятилетия развития реляционные базы данных достигли высокой степени зрелости, и в этой специализированной области возникло множество выдающихся компаний и продуктов.

Oracle Database, IBM Db2, Microsoft SQL Server – это старейшие представители реляционных баз данных, и все они являются коммерческими продуктами. Когда нашей компании требуется система баз данных, мы можем заплатить определенную сумму этим производителям баз данных и использовать их системы, не заботясь о том, каков фактический код, лежащий в ос-

нове этих систем. Однако с быстрым развитием индустрии персональных компьютеров (ПК) и интернета в сфере баз данных появились новые конкуренты, среди которых наиболее известными примерами являются MySQL и PostgreSQL. Они являются лидерами нового поколения реляционных баз данных и используют модель конкуренции с открытым исходным кодом. Открытость позволяет разработчикам просматривать код, оценивать стабильность и надежность системы баз данных и более обоснованно решать, применять ли ее в своем бизнесе.

Если описывать реляционные базы данных с точки зрения человеческих качеств, я бы выбрал «строгость и надежность». Реляционные базы данных стабильно поддерживают цифровую инфраструктуру общества во всех аспектах жизни.

1.2.3. Нереляционные базы данных

Примерно в 2012 году с быстрым развитием технологий мобильного интернета требования общества к системам баз данных также начали эволюционировать. По сравнению с эпохой ПК и интернета разработчики мобильных приложений больше акцентируют внимание на быстром обновлении версий приложений и улучшении пользовательского опыта за счет снижения задержки доступа. Типичный пример: раньше приложения обычно обновлялись ежемесячно или ежеквартально, тогда как мобильные интернет-приложения требуют ежедневных или еженедельных обновлений. Особенно в игровой среде обновления контента становятся крайне непредсказуемыми, а количество игроков в игре также значительно увеличивается, что предъявляет новые требования к системам баз данных.

В реляционных базах данных формат данных каждой строки и столбца заранее определен, и, если впоследствии потребуется изменить формат, этот процесс окажется достаточно трудоемким. Кроме того, перед записью данных в базу система проверяет их допустимость, что также влечет за собой определенные издержки.

Основываясь на этих двух типичных болевых точках, специалисты в области компьютерных наук, работающие с базами данных, предложили новое решение, которое ослабляет требования к предварительному определению формата данных. Разработчики могут динамически добавлять строки и столбцы данных, не проектируя заранее структуру таблицы. Это концептуальное изменение значительно облегчило работу в условиях изменчивых бизнес-сценариев мобильного интернета. Разработчики смогли с помощью более частых итераций улучшать свои продукты, не ограничиваясь фиксированной структурой таблиц базы данных. Кроме того, упрощение предварительного определения таблиц сделало проверку данных при записи проще, что повысило общую производительность использования базы данных и снизило задержки доступа. На рис. 1.9 показан типичный нереляционный набор данных, в котором каждая запись может иметь собственный формат, а содержание и формат данных являются относительно гибкими, без таких ограничений, как внешние ключи и атрибуты, характерные для реляционных наборов данных.


```

Без ограничений внешнего ключа  Без ограничений атрибутов
{"name": "John Doe", "age": 30, "isStudent": false}
{"name": "Anna", "age": 36, "courses": ["history", "chemistry"]}
{"name": "Bella", "address": {"street": "123 Main St", "city": "Anytown"}}

```

Рис. 1.9 Нереляционный набор данных

Redis является типичным примером нереляционной системы баз данных, поддерживающей богатые структуры данных и не требующей предварительного определения структуры таблиц. Кроме того, Redis не акцентирует внимание на персистентности данных, используя память как основной носитель для хранения данных, что делает его продуктом с крайне низкой задержкой доступа, помогая разработчикам в эпоху мобильного интернета выдерживать многочисленные пиковые нагрузки.

MongoDB также является примером нереляционной базы данных, не требующей предварительного определения структуры таблиц. Разработчики могут в любой момент добавлять или удалять поля данных в зависимости от бизнес-требований. Благодаря этой особенности MongoDB получила широкое признание в игровой индустрии и активно используется при разработке игр.

Если описывать нереляционные базы данных с точки зрения человеческих качеств, я бы выбрал «энергичность». Для некоторых быстро меняющихся приложений нереляционные базы данных могут предложить более гибкие и адаптивные механизмы. Возможно, они могут не удовлетворить все потребности в краткосрочной перспективе, но в определенных областях способны стать незаменимыми.

1.2.4. Ограничения традиционных баз данных

Из предыдущего обзора мы можем заключить, что традиционные технологии баз данных, будь то реляционные или нереляционные, в основном предназначены для хранения и эффективного доступа к структурированным данным. Однако в нашем человеческом обществе большую часть данных нельзя абстрагировать и структурировать так просто, как генеалогическое древо. Например, текстовые данные из многочисленных книг, аудиоданные из музыки, изображения и видеоданные. К этим данным нельзя напрямую получить доступ через базы данных. Пропорция структурированных и неструктурированных данных примерно соответствует правилу Парето: структурированные данные составляют около 20 % от общего объема данных человечества, а неструктурированные – около 80 %. Традиционные базы данных могут обрабатывать только 20 % структурированных данных, как показано на рис. 1.10.

Оставшиеся 80 % неструктурированных данных, в свою очередь, хранятся в многочисленных компьютерных системах хранения, разбросанных по всему цифровому миру. Поскольку эти данные сами по себе не структурированы, их трудно анализировать непосредственно с помощью компьютеров. Часто требуется вмешательство человека, что делает обработку неструктурированных данных довольно неэффективной. Можно сказать, что огромные сокровища данных ждут, чтобы их раскопало наше человеческое общество.

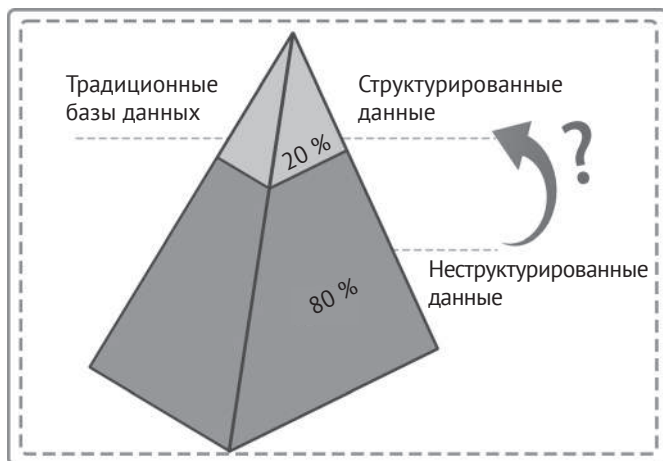


Рис. 1.10. Традиционные базы данных могут обрабатывать только 20 % структурированных данных

В разделе 1.1, изучая векторы, мы обнаружили, что с развитием технологий глубокого обучения специалисты в области компьютерных наук постепенно научились преобразовывать неструктурированные данные в векторные данные с помощью моделей глубокого обучения. Векторные данные представляют собой структурированное выражение неструктурированных данных. Как только данные становятся структурированными, компьютер может обрабатывать их автоматически, без необходимости вмешательства человека, что значительно повышает эффективность работы с данными.

Далее мы попытаемся приоткрыть завесу над «векторными базами данных».

1.3. ЗАЧЕМ НУЖНЫ ВЕКТОРНЫЕ БАЗЫ ДАННЫХ

Индустрия баз данных накопила значительный опыт в хранении и использовании данных. Можно ли применить этот опыт к векторным данным так же, как и к структурированным данным?

Что произойдет, когда база данных столкнется с векторами – новой формой данных? Пospособствует ли это дальнейшему развитию индустрии баз данных в новом направлении? Вооружившись этими вопросами, нам нужно критически оценить традиционные системы баз данных.

1.3.1. Различия между векторными и традиционными данными

Векторные данные в компьютере обычно представлены в виде массива, например [0.450, 0.180, 0.100, 0.220, 0.883...], который может достигать тысяч измерений. Тогда как традиционные данные представлены в виде нескольких пар «ключ–значение», например `id=001, name="Сяомин", city="Пекин"`.

Существует четыре характерных различия между векторными и традиционными структурированными данными.

1. Различие в размерности

Формат векторных данных имеет большее количество измерений, а традиционные форматы данных имеют меньшее количество измерений. Нельзя просто разделить каждое измерение векторных данных и хранить их в строках и столбцах традиционной базы данных, так как текущая модель проектирования не позволяет напрямую хранить такое количество измерений, иначе это приведет к «проклятию размерности». Это связано с тем, что многомерные данные нельзя разделить на строки и столбцы, векторные данные хранятся как единое целое, занимая основное пространство хранения. Учитывая это различие, при проектировании структуры хранения векторной базы данных можно использовать более эффективную блочную систему хранения, что оптимизирует эффективность хранения.

2. Различие в корреляции внутри полей

Между различными измерениями векторных данных существует сильная корреляция, так как эти измерения извлекаются нейронной сетью после обучения. Поэтому для более точного измерения сходства двух векторов часто требуется вычисление комбинации нескольких полей. Векторные данные редко сравниваются по отдельным измерениям, множество измерений действует совместно и взаимосвязано. В то время как традиционные форматы данных, состоящие из нескольких пар «ключ–значение», являются полностью независимыми и представляют собой отдельные аспекты данных. Это различие напрямую приводит к тому, что при доступе к векторным данным необходимо рассматривать многомерные данные как единое целое, и, следовательно, архитектура индексов векторной базы данных будет использовать иной подход.

3. Различие в методах оптимизации

Формат векторных данных однороден, и данные каждого измерения часто имеют фиксированный формат (числа с плавающей запятой, двоичные целые числа и т. д.), что позволяет использовать некоторые методы оптимизации производительности. Например, для математических операций можно более эффективно использовать механизм кеширования процессора для ускорения выполнения, а также лучше использовать аппаратные преимущества CPU/GPU. Основываясь на этой особенности векторных данных, можно глубже заниматься алгоритмической оптимизацией векторной базы данных и относительно легко добиться хороших результатов.

4. Различие в методах использования

Помимо различий в формате этих двух типов данных на низком уровне, более значительные различия проявляются в фактическом использовании.

Например, для текстовых данных традиционные форматы данных основываются на точном совпадении ключевых слов без понимания семантического значения слов, в то время как сопоставление векторных данных базируется на семантическом понимании, и два слова, которые на первый взгляд кажутся очень разными, на самом деле могут иметь близкое семантическое значение.

Возьмем, к примеру, выражения «море» и «море по колено». Если ориентироваться на совпадение ключевых слов, их сходство будет высоким, но при семантическом сопоставлении после векторизации мы обнаружим, что их значения сильно различаются. Или, например, фразы «хорошая погода» и «безоблачное небо». На первый взгляд они не связаны друг с другом, и по ключевым словам их невозможно соотнести. Однако их семантическое сходство очень высоко.

Аналогично для таких неструктурированных данных, как аудио, изображения и видео, традиционные реляционные данные сложнее индексировать, и обычно для этого применяются метки, добавляемые вручную. Когда пользователь использует ключевые слова для поиска по неструктурированным данным, по сути, происходит сопоставление по ключевым словам, а не используется изначально семантический способ запроса – это не гарантирует точности полученных результатов, в то время как сопоставление, основанное на векторизации данных, изначально является семантическим, и результаты будут более естественными и предсказуемыми.

1.3.2. Появление векторных баз данных

Именно из-за столь значительных различий между векторными и традиционными данными появилась необходимость в разработке специализированных *векторных баз данных*, которые будут способны более эффективно решать задачи хранения, индексирования и запроса векторных данных с учетом их особенностей. Векторная база данных может предоставлять возможности семантического запроса, что является ключевой функцией, отличающей ее от традиционных баз данных. Векторные данные могут лучше организовывать отношения между векторами, создавая структуры данных, более подходящие для сопоставления по сходству векторов, такие как индексы, основанные на кластеризации или графах, что значительно повышает эффективность запросов к векторным данным. С точки зрения структуры хранения специализированные векторные базы данных могут лучше учитывать особенности векторных данных при разработке алгоритмов распределения и сжатия данных, что повышает эффективность хранения и снижает затраты.

Конечно, существует множество традиционных технологий баз данных, которые можно применить для обработки векторных данных. Например, с помощью распределенной архитектуры можно обеспечить наличие нескольких реплик данных и гарантировать согласованность данных между ними, чтобы в случае потери одной из реплик можно было восстановить данные с помощью других. Также благодаря технологии сегментирования данных можно разбивать крупномасштабные данные на несколько блоков и использовать распределенные вычислительные мощности для параллельной обработки данных, что повышает степень параллелизма векторных вычислений и общую производительность системы.

Кроме того, стремительно развиваются ИИ-технологии, и специализированные векторные базы данных, не имея исторического наследия традиционных баз данных, могут быстрее адаптироваться и развиваться в сочетании с ними. С развитием ИИ-технологий и увеличением разнообразия сценариев

их применения потребность в векторных базах данных будет все больше расти, а скорость их развития будет иметь решающее значение.

Таким образом, построение профессиональной векторной базы данных опирается на следующие три аспекта.

1. Разработка соответствующих компонентов для хранения, индексирования и запроса, учитывая формат векторных данных, с акцентом на уникальную «векторную» часть, чтобы довести до совершенства управление векторными данными.
2. Уделение внимания традиционным возможностям, накопленным за многие годы в технологиях баз данных, и их применению в векторных базах данных. Ведь по своей сути векторная база данных все еще остается системой баз данных.
3. Не обременять систему избыточным функционалом и не стремиться немедленно перенимать уникальные возможности традиционных баз данных, а постепенно, эволюционно наращивать функциональность векторной базы данных. Небольшими, но быстрыми шагами успевая за стремительным развитием ИИ-технологий.

1.3.3. Интеллектуальная платформа хранения в эпоху больших моделей

Я уверен, что в период с конца 2022 до 2024 года вы неоднократно слышали термин «большая языковая модель» (или просто «большая модель») в различных контекстах. В обсуждениях постоянно подчеркивается потенциал больших моделей, и кажется, что их применение охватывает все сферы. На самом деле мы уже начали использовать большие модели в повседневной жизни и работе, например для консультаций с чат-ботами на основе больших моделей или для написания кода с помощью инструментов программирования, основанных на больших моделях, что демонстрирует их практическую ценность.

Однако задумывались ли вы о том, что такое большая модель по своей сути? Это более совершенный чат-бот? Или более мощная поисковая система?

Прежде чем ответить на этот вопрос, давайте проанализируем, как человечество управляло цифровым миром до появления больших языковых моделей. Предположим, что вы проголодались в полдень и хотите заказать с помощью телефона доставку еды. Для этого достаточно использовать графический интерфейс пользователя телефона. Однако за кажущейся простотой нажатий на экран скрывается сложная работа программного обеспечения, написанного программистами. Клиентское программное обеспечение преобразует каждое ваше касание в структурированные данные. Эти данные формируют набор инструкций, которые направляются к серверному программному обеспечению. Серверное программное обеспечение способно распознавать эти наборы инструкций и преобразовывать их в вычисления на процессоре. После выполнения вычислений серверное программное обеспечение возвращает результаты вам и одновременно сохраняет важные данные в базах данных и других системах хранения. Весь этот сложный процесс основан на значительном объеме программного кода, написанного программистами. Ключевой момент здесь в том, что для управления ресурсами компьютера (процессор, хранение и т. д.)

вводимые данные должны быть структурированными, так как программы могут распознавать только такие данные. Эти программы недостаточно универсальны, и для каждого приложения требуется работа программистов. Тысячи программ, написанных программистами, тихо работают, поддерживая функционирование цифрового общества.

Возвращаясь к сценарию с большими языковыми моделями, мы обнаруживаем, что после предобучения на огромных объемах данных они могут напрямую принимать инструкции на обычном человеческом языке, в определенной степени обладая способностью понимать неструктурированные данные. С помощью естественного языка человека мы можем «попросить» приложения на основе больших языковых моделей написать код, обработать таблицы, выполнить команды и т. д. Это фактически способствует значительному изменению парадигмы распределения ресурсов в человеческом обществе. Возможно, нам больше не потребуется так много программистов для перевода этих инструкций, а потери информации, вызванные таким переводом, значительно уменьшатся, что приведет к значительному повышению производительности.

По мере того как огромные объемы данных продолжают улучшать возможности больших языковых моделей через обучение, они становятся все более интеллектуальными и могут помочь человечеству в большем количестве областей. Но смогут ли большие модели постепенно запомнить все знания и данные мира в своих внутренних нейронных сетях?

Ответ отрицательный. Большая языковая модель, подобно человеческому мозгу, становится более разумной через обучение, однако она не стремится запомнить каждую единицу информации, а скорее изучает правила и методы, чтобы понимать знания обобщенно. Когда поступают новые данные, мы выводим новые заключения на их основе, а не запоминаем все данные, что не является наиболее эффективным способом их использования. Аналогично большая языковая модель обрабатывает информацию через обучение и вывод, а не запоминает все данные без разбора.

После предобучения на огромных объемах данных большая языковая модель постепенно эволюционирует в интеллектуальную вычислительную платформу. В отличие от традиционных компьютерных систем, где для управления вычислительными ресурсами программисты пишут код, большая модель может напрямую реагировать на команды, сформулированные на естественном языке.

В то же время человеческое общество продолжает генерировать разнообразные данные, часть которых используется для повышения обобщающей способности больших моделей, делая их более «умными». Остальные данные сохраняются в традиционной форме и извлекаются, когда требуется вывод модели. Поскольку управлять вычислительной платформой уже можно с помощью естественного языка, естественно, что и способ управления данными должен эволюционировать в сторону использования естественного языка.

Таким образом, мы подходим к ключевому вопросу: в эпоху ИИ способы управления вычислительными ресурсами и данными человеком будут эволюционировать в сторону использования естественного языка. В этом контексте базы данных должны лучше адаптироваться к такому подходу. Векторная база данных удовлетворяет этой потребности в двух аспектах. Во-первых, мы можем преобразовать различные традиционные данные, созданные человеком,

в векторные данные для хранения в векторной базе данных. Во-вторых, команды для запроса данных также могут быть преобразованы в векторные данные, и через сопоставление сходства между этими двумя типами векторных данных можно вернуть целевые данные, которые мы ищем. Поскольку процесс преобразования традиционных данных и команд в векторные данные является высокообобщенным и поддерживает неструктурированный формат естественного языка, это позволяет управлять данными с помощью естественного языка.

В итоге мы можем сделать следующий прогноз для эпохи ИИ: если определить большую языковую модель как интеллектуальную вычислительную платформу, то векторная база данных, использующая естественный язык, несомненно, станет представителем интеллектуальной платформы хранения. Она будет работать совместно с интеллектуальной вычислительной платформой, становясь ее лучшим помощником в обработке данных, и вместе с ней строить более интеллектуализированный цифровой мир. В главе 8 мы более подробно и разнообразно рассмотрим перспективы развития векторных баз данных.

1.4. РЕЗЮМЕ

В этой главе мы начали с формата векторных данных и с помощью наглядных примеров изучили, как преобразовать сложных персонажей аниме и игр в векторные данные, удобные для выполнения математических операций. Мы также изучили, как использовать математические понятия «косинусное сходство», «скалярное произведение» и «евклидово расстояние» для измерения сходства между векторными данными. Затем на основе практических примеров мы узнали о конкретных сценариях применения векторизованных данных в жизни.

Далее мы перешли к теме баз данных, подчеркнув, что их основная цель – обеспечить надежное хранение данных и эффективный доступ к ним. Для достижения этих целей появились различные типы баз данных, каждая из которых имеет свои специфические области применения. Мы также подробно проанализировали различия между векторными и традиционными форматами данных и предложили стратегии оптимизации векторной базы данных на основе традиционных технологий данных, которые позволят создать профессиональную векторную базу данных.

В заключение мы обсудили важность векторных баз данных в эпоху больших языковых моделей. По мере совершенствования они постепенно станут интеллектуальной вычислительной платформой эпохи ИИ, тогда как векторные базы данных, вероятно, превратятся в платформу интеллектуального хранения. Их сочетание станет важной инфраструктурой будущего мира, помогая человечеству исследовать будущее.

В следующей главе мы познакомимся с краткой историей векторных баз данных.

Глава 2

Краткая история векторных баз данных

История человечества – это лишь миг в масштабе Вселенной, и первый урок истории заключается в необходимости учиться смирению.

Уилл Дюрант, Ариэль Дюрант. Уроки истории

С развитием компьютерных технологий традиционные реляционные базы данных достигли зрелости в эпоху интернета нового тысячелетия. В то же время начиная с 2012 года внимание стали привлекать нереляционные базы данных благодаря способности адаптироваться к изменяющимся формам программных продуктов и удовлетворять высокие требования к производительности. С развитием технологий баз данных на горизонте стали постоянно появляться новые игроки.

Понятие «векторные базы данных» появилось не в последние годы. Векторные базы данных начали постепенно развиваться вместе с быстрым развитием глубоких нейронных сетей и необходимостью хранения, индексирования и выполнения запросов к высокоразмерным векторным данным, создаваемым моделями глубокого обучения. В этой книге мы рассматриваем 2017 год как ключевой в развитии векторных баз данных. Почему именно он? Какие важные вехи можно выделить в процессе их развития? Какие современные продукты векторных баз данных являются наиболее представительными и в чем их различия?

Изучив содержание этой главы, вы получите общие ответы на эти вопросы.

2.1. ПЕРИОД ЗАРОЖДЕНИЯ (1980–2012)

Многие путешествия в мир новых технологий начинаются с исследований в лабораториях. В научных кругах существует широко известная поговорка: «Смело предполагай, тщательно проверяй». Следуя этому принципу, ученые отважно выдвигают инновационные гипотезы в неизведанных областях и подтверждают их с помощью строгих экспериментов. Когда научное достижение получает признание в академической среде, в дело вступают первопроходцы в промышленности. Эти дальновидные компании соединяют научные достижения и реальные потребности бизнеса, способствуя переходу техноло-

гий от теории к практике. Как только научное достижение успешно переходит в практическое применение, оно оказывает мощный демонстрационный эффект, стимулируя дальнейшие инновации.

Под совместным влиянием дальновидных ученых и предпринимателей вся отрасль постепенно развивается и созревает. Развитие векторных баз данных следует по тому же пути. На самом деле ключевой технологией, способствующей развитию векторных баз данных, являются глубокие нейронные сети. Далее мы кратко рассмотрим процесс их развития.

2.1.1. Стремительное развитие глубоких нейронных сетей

Истоки глубоких нейронных сетей восходят к 1980-м годам, когда исследователи пытались использовать многослойные нейронные сети для моделирования связей между нейронами человеческого мозга. *Глубокие нейронные сети* получили свое название благодаря более глубокой структуре по сравнению с традиционными мелкими нейронными сетями. Конкретно свойство «глубины» здесь относится к количеству слоев в структуре сети, а также к числу нейронов и способу их соединения в каждом слое. Нейронная сеть обычно включает входной слой, выходной и несколько скрытых слоев. Многослойная структура позволяет изучать более сложные представления признаков, что повышает точность модели.

Хотя концепция глубоких нейронных сетей появилась еще в 1980-х годах, в течение первых 20 лет она не получила широкого применения по следующим причинам.

- Трудности обучения. Обучение глубоких нейронных сетей требует значительных вычислительных ресурсов, а в то время вычислительные возможности были ограничены и не могли удовлетворить этим потребностям.
- Исчезновение градиента. При обучении глубоких нейронных сетей значения градиента в алгоритме обратного распространения могут становиться очень малыми по мере продвижения к более глубоким слоям сети, что приводит к медленному обновлению весов и, следовательно, значительно замедляет сходимость модели.
- Переобучение. Глубокие нейронные сети имеют большое количество параметров, что может привести к переобучению. То есть ситуации, когда модель хорошо работает на обучающем наборе данных, но имеет слабую способность к обобщению на тестовом наборе.

В 2012 году глубокая нейронная сеть AlexNet, разработанная совместно Алексом Крижевским (Alex Krizhevsky), Ильей Суцкевером (Ilya Sutskever) и Джефффри Хинтоном (Geoffrey Hinton), одержала победу в конкурсе ImageNet по масштабному распознаванию изображений, значительно опередив другие традиционные методы. Это событие не только ознаменовало прорыв технологий глубокого обучения в академической среде, но и стало поворотным моментом для начала применения и продвижения глубоких нейронных сетей в промышленности.

Конкурс ImageNet по масштабному распознаванию изображений

Конкурс ImageNet по масштабному распознаванию изображений (ImageNet Large Scale Visual Recognition Challenge, ILSVRC) проводится с 2010 года и направлен на развитие области компьютерного зрения. Основная задача конкурса заключается в классификации и детекции большого количества изображений. Набор данных включает 1000 категорий и более 14 млн размеченных изображений. Этот конкурс имеет важное значение для оценки и сравнения различных алгоритмов компьютерного зрения и считается Олимпиадой в данной области.

В ILSVRC 2012 года, помимо AlexNet, участвовали и другие модели, такие как методы извлечения признаков SIFT и HOG, основанные на традиционных технологиях компьютерного зрения, а также мелкие нейронные сети. Однако благодаря глубокой структуре и инновационным технологиям (таким как использование функции активации ReLU и техники dropout для уменьшения переобучения, а также использование высокопроизводительных GPU для ускорения процесса обучения и т. д.) AlexNet достигла 15.3 % показателя «топ-5 ошибок» в задаче классификации, что значительно превзошло второй результат в 26.2 %, обеспечив себе тем самым убедительную победу.

«Топ-5 ошибок» указывает на долю случаев, когда правильная категория не появляется среди первых пяти предсказанных моделью категорий. Этот показатель предназначен для оценки способности модели к обобщению на сложных задачах. Результаты AlexNet по этому показателю подчеркнули ее выдающуюся производительность в задачах крупномасштабной классификации изображений.

После выдающихся результатов AlexNet такие технологические гиганты, как Google, Microsoft и Facebook, осознали огромный потенциал глубоких нейронных сетей в области компьютерного зрения. Для дальнейшего повышения технической производительности и применения их к реальным задачам компании предприняли следующие шаги.

- Разработка новых моделей глубокого обучения. Создание специализированных исследовательских команд по глубокому обучению, занимающихся разработкой передовых моделей глубокого обучения, таких как Inception (также известная как GoogLeNet) от Google и ResNet от Microsoft. Эти модели показывали выдающиеся результаты в конкурсах, постоянно обновляя рекорды.
- Инвестиции в платформы глубокого обучения. Google разработала TensorFlow, Microsoft – CNTK, Facebook – PyTorch. Эти важные инструменты облегчают исследователям и инженерам создание, обучение и развертывание моделей глубокого обучения.
- Создание крупномасштабной вычислительной инфраструктуры. Для ускорения процесса обучения и вывода моделей глубокого обучения инвестировались средства в создание крупномасштабной вычислительной инфраструктуры. Например, Google разработала специализированное оборудование для глубокого обучения – TPU (тензорный процессор), NVIDIA представила платформу параллельных вычислений CUDA, которая в сочетании с ее GPU предоставляет более универсальные возможности параллельных вычислений.

Под влиянием технологических гигантов технологии глубокого обучения развивались стремительно, и глубокие нейронные сети постепенно находили широкое применение в таких областях, как распознавание изображений, распознавание речи и обработка естественного языка, что заложило прочный фундамент для развития искусственного интеллекта.

2.1.2. Связь глубоких нейронных сетей и векторных баз данных

Мы уже затрагивали эту тему в главе 1, когда обсуждали вопрос, зачем нужны векторные базы данных. Когда мы говорим о векторных данных или о векторных базах данных, глубокие нейронные сети остаются неотъемлемой частью обсуждения. В этом разделе мы кратко подведем итоги, разделив их на два аспекта. Во-первых, стремительное развитие глубоких нейронных сетей стимулирует появление векторных баз данных. Во-вторых, развитие векторных баз данных невозможно без глубоких нейронных сетей.

1. Развитие глубоких нейронных сетей стимулирует появление векторных баз данных

С новым витком взрывного роста ИИ резко увеличивается объем неструктурированных данных, таких как текст, изображения, аудио и видео, которые необходимо обрабатывать. Традиционные реляционные и нереляционные базы данных оказываются неэффективными при обработке таких данных, и возникает острая необходимость в новом способе хранения, индексации и выполнения запросов.

- Выходные данные моделей глубокого обучения: модели глубокого обучения, ранее представленные сверточными нейронными сетями и рекуррентными нейронными сетями, в настоящее время представлены такими большими языковыми моделями, как GPT. Они способны преобразовывать неструктурированные данные в высокоразмерные векторы. Эти векторы содержат богатую семантическую информацию и требуют специализированных баз данных для эффективного хранения, индексации и выполнения запросов.
- Необходимость в запросах сходства. В таких приложениях, как рекомендательные системы, распознавание изображений и текстовые запросы, необходимо быстро находить векторы, наиболее схожие с запросом. Это требует от системы баз данных поддержки эффективных запросов сходства векторов, что сложно реализовать в традиционных базах данных.
- Новая волна внедрения ИИ-приложений. С быстрым развитием и широким применением новых технологий ИИ, таких как GPT и GLM, возрастает потребность в обработке большого объема векторных данных и поддержке инфраструктуры, связанной с предобучением и выводом больших языковых моделей. Векторные базы данных являются одной из ключевых технологий для удовлетворения этой потребности.

2. Развитие векторных баз данных требует глубоких нейронных сетей

Модели глубокого обучения способны эффективно извлекать признаки из неструктурированных данных и генерировать векторные представления, которые могут захватывать глубокую семантическую информацию данных, что позволяет векторным базам данных выполнять эффективные запросы сходства и сложный анализ данных.

- Технология векторизации. Векторные данные, сгенерированные моделями глубокого обучения, способны захватывать глубокие признаки данных. Благодаря векторизации векторные базы данных могут лучше выявлять внутреннюю сложность информации данных.
- Обработка мультимодальных данных. Модели глубокого обучения способны обрабатывать различные типы неструктурированных данных и преобразовывать их в единый векторный формат. Векторные базы данных могут хранить, индексировать и выполнять запросы по этим мультимодальным данным, а также поддерживать межмодальный поиск и анализ.
- Требования к реальному времени. Во многих приложениях, таких как системы рекомендаций и системы управления финансовыми рисками, требуется быстрое реагирование на запросы. На основе технологий глубоких нейронных сетей векторные базы данных могут оптимизировать структуру индексов, снижать задержку доступа и удовлетворять требованиям реального времени.
- Базы данных с интеллектуальными возможностями. Интегрированные с моделями глубокого обучения системы баз данных могут поддерживать ИИ-функции, снижая порог входа для разработчиков, что является важной способностью векторных баз данных.

2.2. ПЕРИОД СТАНОВЛЕНИЯ (2012–2017)

Как уже упоминалось, после 2012 года с быстрым развитием глубоких нейронных сетей их применение в различных приложениях становится все более распространенным. Учитывая эту тенденцию, обработка вычислений сходства для больших объемов векторных данных и данных постоянного индексирования стала ключевой задачей. Разработчики остро нуждались в универсальных технологических компонентах для поддержки быстрого развития бизнеса. Следуя временной шкале развития технологий, мы подходим к 2017 году. В это время промышленность начала накапливать и внедрять некоторые открытые решения для устранения вышеуказанных проблем.

В 2017 году была выпущена библиотека с открытым исходным кодом FAISS (Facebook AI Similarity Search), что стало беспрецедентным событием в области обработки векторных данных. FAISS, разработанный подразделением FAIR (Facebook AI Research), является высокопроизводительной библиотекой управления векторами, цель которой – предоставление решений для эффективных запросов сходства на больших наборах данных. Важность FAISS заключается в том, что до ее выпуска методы обработки большого объема векторных данных были неэффективными и неудобными. Появление FAISS

заполнило этот пробел, предоставив беспрецедентное решение для данной конкретной задачи.

FAISS предоставляет множество типов индексов и вариантов конфигурации, чтобы соответствовать различным характеристикам наборов данных и сценариям применения. FAISS поддерживает такие типы индексов, как Flat (плоский), IVF Flat (инвертированный файл с плоским индексом), IVF PQ (инвертированный файл с квантизацией произведения) и др., которые позволяют удовлетворить потребности в различных масштабах векторных данных и точности запросов. Плоский индекс подходит для случаев, когда количество векторов для запроса составляет менее 100 тыс. строк, обеспечивая наивысшую точность запроса. Индексы серии IVF (инвертированный файл) используют методы кластеризации для повышения эффективности запроса, поддерживая масштабы векторных данных на уровне миллиардов строк, но при этом снижая точность запроса.

Кроме того, FAISS поддерживает ускорение с помощью GPU для еще большего повышения производительности запросов. Для удобства разработчиков FAISS можно бесшовно интегрировать с различными платформами глубокого обучения (такими как TensorFlow, PyTorch и др.) и инструментами обработки данных (такими как Pandas, NumPy и др.). Это позволяет легко сочетать FAISS с существующими рабочими процессами обработки данных и глубокого обучения.

Еще одной выдающейся библиотекой для работы с крупномасштабными векторами является HNSWLib. Это открытая высокопроизводительная библиотека для приближенного поиска ближайших соседей, основанная на алгоритме HNSW поиска ближайших соседей в графах. В отличие от FAISS, HNSWLib обменивает более высокое использование памяти на более высокую эффективность запроса, поддерживая масштабы векторов на уровне десятков миллионов строк. Хотя официальная дата открытия HNSWLib неизвестна, ее репозиторий на GitHub был создан в июле 2017 года, что указывает на то, что проект был активен по крайней мере с этой даты. Таким образом, можно обоснованно предположить, что HNSWLib оказала значительное влияние на область векторных баз данных в 2017 году.

2017 год стал вехой для векторных баз данных в основном благодаря двум важным проектам – FAISS и HNSWLib, поэтому его можно назвать «годом векторных баз данных». Открытые библиотеки FAISS и HNSWLib заложили основу для оптимизации технологий векторных баз данных и дальнейших исследований. Многие разработчики в области баз данных начали экспериментировать с распределенной архитектурой, высокой доступностью и другими технологиями, чтобы улучшить возможности хранения, индексации и выполнения запросов для крупномасштабных векторных данных.

2.3. ПЕРИОД РАЗВИТИЯ (2017 ГОД – НАСТОЯЩЕЕ ВРЕМЯ)

В «год векторных баз данных» мир стал свидетелем появления двух высококачественных библиотек для управления векторами – FAISS и HNSWLib. На основе этих библиотек стремительно развивались векторные базы данных, появлялись многочисленные инновации и достижения.

Во-первых, начали появляться стартапы, сосредоточенные на разработке векторных баз данных. Эти компании с нуля разрабатывали векторные базы данных, глубоко изучая особенности векторных данных, и создавали выдающиеся продукты, такие как Zilliz (Milvus), Pinecone, Qdrant и Chroma.

Во-вторых, многие зрелые компании также начали разрабатывать специализированные векторные базы данных, стремясь глубоко интегрировать их с существующим бизнесом, чтобы улучшить пользовательский опыт существующих продуктов. Например, такие компании, как Microsoft, Google, Baidu, Alibaba и Tencent, уже сделали шаги в этом направлении.

Наконец, команды по разработке традиционных баз данных также начали интегрировать функции управления векторными данными в свои системы. Они обнаружили, что все больше клиентов хотят использовать векторные данные для развития нового бизнеса. Например, PostgreSQL pgvector и Redis Vector Search делают шаги в этом направлении. Эти усилия значительно способствовали быстрому развитию технологий векторных баз данных и сформировали здоровую конкурентную среду.

В этом разделе мы сначала представим некоторые компании, участвующие в конкуренции в области векторных баз данных, и их историю развития. Затем мы представим некоторые из наиболее знаковых продуктов и приведем их функции, преимущества и обсудим архитектуру этих продуктов.

2.3.1. Развитие отрасли

В Китае как традиционные интернет-компании, так и новые стартапы активно занимаются разработкой векторных баз данных. Ниже приводится краткий обзор развития векторных баз данных в таких компаниях, как Baidu, Alibaba, Tencent и Zilliz.

- Векторная система управления данными Puck разработана компанией Baidu. С момента первого развертывания в 2017 году она обслуживает задачи поиска в библиотеке изображений, насчитывающей десятки миллиардов объектов. К 2019 году система была открыта для внутреннего использования в Baidu и затем широко применялась в поисковых, рекомендательных и облачных сервисах.
- Внутренняя векторная система управления данными OLAMA, запущенная компанией Tencent в 2019 году, обслуживает такие платформы, как Tencent News и Tencent Video, способствуя улучшению пользовательского опыта в этих направлениях бизнеса.
- Векторная система управления данными Proxima разработана в лаборатории ИИ-систем Alibaba DAMO Academy, официальная дата выпуска неизвестна. В настоящее время ее основные функции широко применяются в различных направлениях бизнеса компаний Alibaba и Ant Group, таких как поиск и рекомендации на маркетплейсе Taobao, оплата лицом в платежной системе Alipay, поиск на видеохостинге Youku и поиск рекламы на платформе Alimama.
- Стартап Zilliz был основан группой новаторов, увлеченных технологиями векторных баз данных, во главе с Чарльзом Си (Charles Xie). С момента основания компания сосредоточена на разработке передовых

решений для эффективной обработки векторных данных, предоставляя мощную поддержку управления данными для приложений в области искусственного интеллекта и машинного обучения.

На международной арене Microsoft Cognitive Search и Google Vertex AI Vector Search предоставляют услуги в рамках ИИ-подразделений бизнеса и комплексных ИИ-решений. На зарубежных рынках конкуренция в области независимых векторных баз данных ведется несколькими быстроразвивающимися стартапами, такими как Pinecone, Qdrant и Chroma, которые являются ключевыми игроками в этой области.

- Компания Pinecone основана Эдо Либерти (Edo Liberty), который ранее занимал руководящую должность в ИИ-лаборатории Amazon и участвовал в создании сервиса SageMaker. Pinecone является одной из первых компаний, занимающихся независимой разработкой векторных баз данных за рубежом, и обладает значительным влиянием в отрасли.
- Компания Qdrant основана в 2021 году и располагается в Берлине. Она быстро завоевала признание в ИИ-сфере благодаря своей открытой векторной базе данных и поисковой подсистеме. Технология Qdrant использует язык Rust и предназначена для поддержки приложений следующего поколения в области искусственного интеллекта.
- Компания Chroma основана Джеффом Хубером (Jeff Huber) и Антоном Тройниковым (Anton Troynikov). В компании считают, что с развитием больших языковых моделей требуется новая вычислительная архитектура, включая векторные базы данных, чтобы обеспечить долговременную память для приложений больших моделей.

Команды по разработке традиционных баз данных также постепенно развивают свои возможности управления векторными данными, среди которых широкое применение получили расширения для PostgreSQL и Redis.

- pgvector – это расширение векторных баз данных для PostgreSQL, выпущенное в 2021 году. Оно поддерживает не только базовое хранение векторных данных, но и предоставляет эффективные функции извлечения, а в 2022 году благодаря техническим улучшениям оно было усилено, став ключевым компонентом экосистемы PostgreSQL.
- Redis Vector Search – это тщательно разработанное расширение Redis Labs, предназначенное для обработки неструктурированных данных в эпоху ИИ. Оно значительно оптимизирует производительность машинного обучения и запросов на сходство благодаря мощным функциям векторного поиска. Redis Vector Search выделилось в 2022 году благодаря инновациям и быстро стало важной частью экосистемы Redis.

Далее мы проведем сравнительный анализ нескольких представительных решений векторных баз данных и подключаемых модулей с точки зрения их функциональных возможностей.

2.3.2. Сравнение возможностей ключевых продуктов

С 2017 года индустрия векторных баз данных активно развивается, и различные производители создают свои продукты с учетом собственных потребно-

стей. Далее мы кратко рассмотрим несколько ключевых продуктов с точки зрения основных возможностей, преимуществ и форм поставки, которые представлены в табл. 2.1. Для получения подробных сведений посетите официальные сайты соответствующих продуктов.

Таблица 2.1. Сравнение возможностей ключевых продуктов векторных баз данных

Название	Основные возможности	Преимущества	Форма поставки
Redis Vector Search	1. Поддержка плоских HNSW-индексов. 2. Поддержка смешанных индексов для скаляров и векторов. 3. Поддержка измерений евклидова расстояния, сходства скалярного произведения и косинусного сходства. 4. Поддержка размерности до 32 768, масштаб векторов до десятков миллионов строк в одном кластере	1. Богатые структуры данных экосистемы Redis и многоязычные SDK, высокая простота использования. 2. Основано на зрелых решениях Redis, что обеспечивает высокую доступность сервиса	Открытый исходный код, поддержка разработчиками и производителем
Tencent Cloud	1. Поддержка плоских, HNSW-, IVF- и BI-HNSW-индексов разработки Tencent. 2. Поддержка смешанных индексов для скаляров и векторов. 3. Поддержка измерений евклидова расстояния, сходства скалярного произведения и косинусного сходства. 4. Поддержка размерности до 4096, масштаб векторов до миллиардов строк в одном кластере	1. Поддержка динамической схемы, не требует предопределенной модели данных. 2. Поддержка механизма псевдонимов, координация различных способов онлайн-расширения, высокая масштабируемость. 3. Поддержка визуального управления данными, снижение порога использования для пользователей	Закрытый исходный код, поддержка производителем
Milvus	1. Поддержка плоских, HNSW-, IVF- и DiskANN-индексов. 2. Поддержка смешанных индексов для скаляров и векторов. 3. Поддержка измерений расстояния Хэмминга, евклидова расстояния и сходства скалярного произведения. 4. Поддержка размерности до 32 768, масштаб векторов до миллиардов строк в одном кластере	1. В корпоративной версии имеется возможность автоматического индексирования, помогающая разработчикам в создании индексов. 2. Поддержка ускорения запросов векторов с помощью GPU. 3. Корпоративная версия поддерживает бессерверный режим развертывания, низкий порог для пробного использования. 4. Поддержка богатых инструментов миграции данных и визуального управления	Открытый исходный код, поддержка разработчиками и производителем

Окончание табл. 2.1

Название	Основные возможности	Преимущества	Форма поставки
Chroma	1. Поддержка HNSW-индексов. 2. Поддержка смешанных индексов для скаляров и векторов. 3. Поддержка измерений евклидова расстояния, сходства скалярного произведения и косинусного сходства. 4. Поддержка размерности до 1536, масштаб векторов до десятков миллионов строк в одном кластере	1. Возможность выделенного развертывания на клиенте, низкие требования к ресурсам для развертывания, низкий порог для освоения разработчиками. 2. Встроенные функции векторизации, готовность к использованию разработчиками	Открытый исходный код, поддержка разработчиком
Pinecone	1. Поддержка собственных алгоритмов векторного индексирования, что позволяет пользователям не знать параметров конфигурации индексов. 2. Поддержка смешанных индексов для скаляров и векторов. 3. Поддержка измерений евклидова расстояния, сходства скалярного произведения и косинусного сходства. 4. Поддержка размерности до 20 000, масштаб векторов до миллиардов строк в одном кластере	1. Возможность автоматического индексирования, помогающая разработчикам в создании индексов. 2. Поддержка бессерверного режима развертывания, низкий порог для пробного использования разработчиками. 3. Интеграция с богатым набором ИИ-инструментов и сопутствующей документацией	Закрытый исходный код, поддержка производителем

Из табл. 2.1 видно, что продукты векторных баз данных в основном делятся на две категории.

- Открытые векторные базы данных. Redis Vector Search использует преимущества экосистемы Redis, быстро обслуживая существующих пользователей благодаря простоте использования. Chroma стремится создать предельно простую векторную базу данных, отличающуюся быстрой и легкой установкой, со встроенными функциями векторизации, чтобы снизить порог вхождения для пользователей. Milvus, как независимая векторная база данных с длительной историей развития, обладает более полным функционалом, предлагая не только открытую версию, но и полностью управляемую корпоративную версию, которая имеет сильные конкурентные преимущества.
- Закрытые векторные базы данных. Tencent Cloud, благодаря богатому опыту обслуживания своих внутренних бизнес-процессов, обладает полноценным функционалом, уделяет особое внимание возможностям он-

лайн расширения и сжатия и обеспечивает высокую доступность сервиса. Pinecone является одним из ведущих мировых продуктов. Благодаря автоматизации индексации и бессерверной функции эффективно снижает порог использования для пользователей. Богатый набор ИИ-инструментов и подробная документация заслужили высокую оценку пользователей.

В процессе развития функционала этих продуктов они применили уникальные архитектурные подходы. Некоторые продукты сосредоточены на простоте, скорости и легкости освоения. Другие строятся с нуля, ориентируясь на предоставление превосходной производительности в сценариях управления векторными данными. Далее мы рассмотрим различные технологические архитектуры, используемые в этих продуктах.

2.3.3. Технологическая архитектура ключевых продуктов

1. Модульные векторные базы данных

Мы рассмотрим *модульные векторные базы данных* на примере Redis. Благодаря зрелой модульной архитектуре ядра Redis расширение его функциональности выполнять относительно просто. Для добавления функции поиска векторов в Redis достаточно интегрировать векторный модуль. Упрощенная техническая архитектура Redis Vector Search представлена на рис. 2.1.

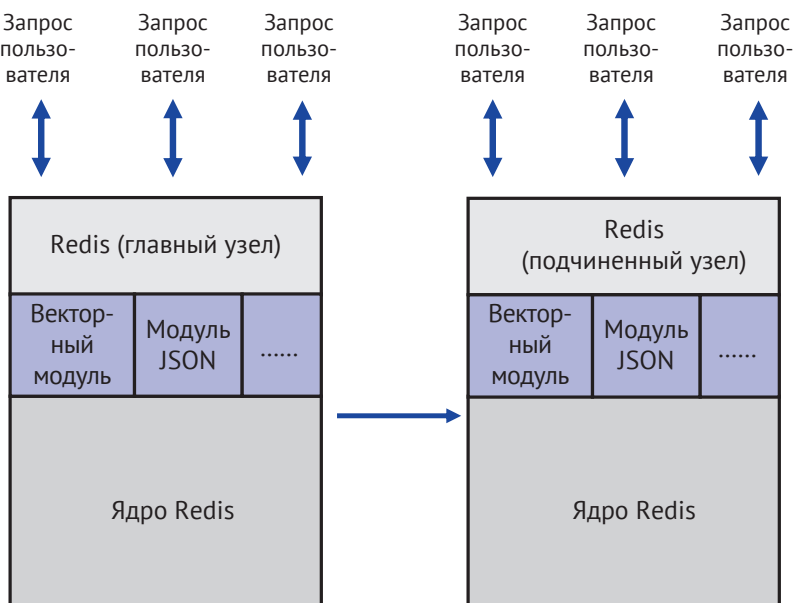


Рис. 2.1. Упрощенная техническая архитектура Redis Vector Search

Векторный модуль реализует различные структуры данных индексов, включая плоские, HNSW и IVF. С помощью этих индексов можно записывать векторные данные в главный узел Redis. На уровне протокола интерфейса записи пользовательских данных векторный модуль должен использоваться совместно с моду-

лем JSON, так как все связанные входные и выходные параметры обмениваются в формате JSON. После записи данных в главный узел они синхронизируются с подчиненными узлами через развитый механизм репликации данных Redis.

Модульный подход Redis Vector Search является типичным примером расширения возможностей управления векторами в традиционных базах данных. Он расширяет существующую технологическую базу, полностью сохраняя текущие функции базы данных. Для существующих пользователей Redis это не только помогает сохранить привычные методы разработки, но и позволяет быстро опробовать передовые функции управления векторными данными. Redis Vector Search предоставляет пользователям традиционных баз данных возможность познакомиться и быстро освоить возможности управления векторными данными.

2. Автономные векторные базы данных

Рассмотрим автономные векторные базы данных на примере Chroma. Эта система ориентирована на легковесное развертывание, ее общая техническая архитектура проста и сосредоточена на решении ключевых задач управления векторными данными. Упрощенная техническая архитектура Chroma представлена на рис. 2.2.

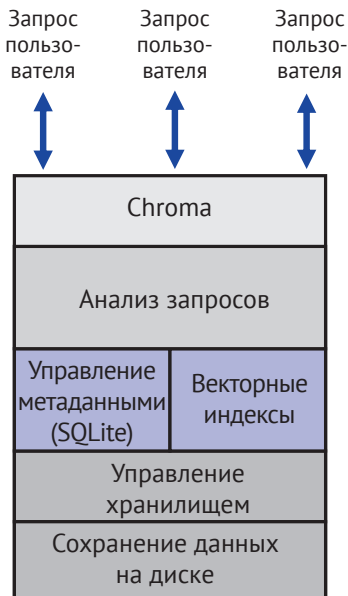


Рис. 2.2. Упрощенная техническая архитектура Chroma

Chroma сосредоточена на функциях хранения, индексирования и запроса векторных данных. Модуль анализа запросов отвечает за прием пользовательских запросов и их разбиение на соответствующие команды выполнения. Для команд управления базой данных модуль анализа запросов генерирует метаданные, связанные с операциями, и сохраняет их через модуль управления метаданными. Операции с векторными данными выполняются через модуль векторных индексов. При чтении данных после сопоставления с векторным

индексом пользователю возвращается наилучший результат. При записи данных обновляются индексы в памяти и активируется модуль управления хранением для сохранения данных на жесткий диск.

Chroma использует подход к проектированию технической архитектуры, ориентированный на оптимизацию под конкретные задачи. Она не предоставляет сложные функции традиционных баз данных, такие как репликация данных между главным и подчиненными узлами, распределенная обработка, механизмы транзакций, сложные запросы и агрегатные вычисления. Благодаря отсутствию этих сложностей Chroma можно быстро развернуть на клиентских устройствах, позволяя пользователям быстро тестировать бизнес-прототипы в условиях ограниченных ресурсов. Кроме того, Chroma поддерживает развертывание на сервере и предоставляет более высокую производительность за счет вертикального масштабирования.

3. Распределенные векторные базы данных с интеграцией хранения и вычислений

Рассмотрим *распределенные векторные базы данных с интеграцией хранения и вычислений* на примере Tencent Cloud. Tencent Cloud – это независимая векторная база данных, специально разработанная для управления векторными данными. Для лучшего обслуживания онлайн-подразделений внутри группы Tencent ее архитектура уделяет особое внимание непрерывности предоставления сервиса. Tencent Cloud имеет распределенную архитектуру и интегрирует в себе функции хранения и вычислений, что снижает сложность системы. Упрощенная техническая архитектура представлена на рис. 2.3.

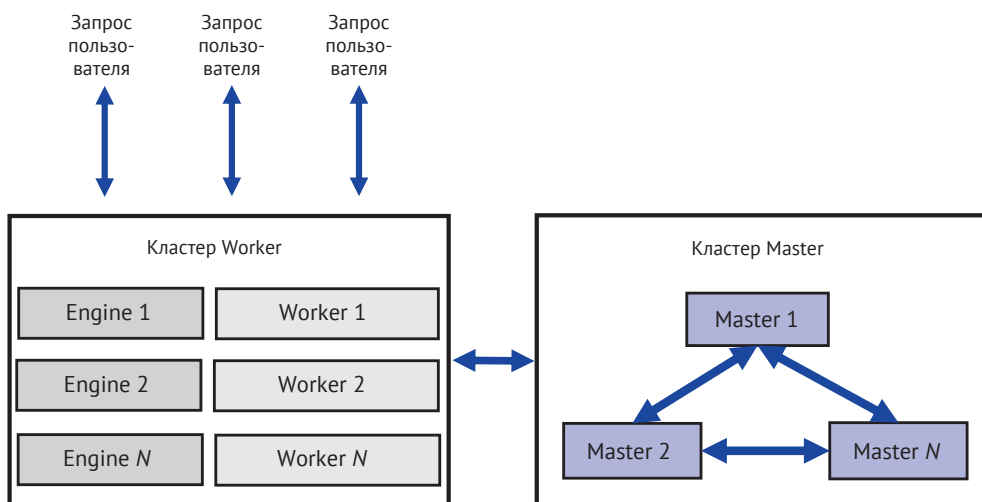


Рис. 2.3. Упрощенная техническая архитектура Tencent Cloud

Общая архитектура Tencent Cloud делится на две основные части: кластер Master и кластер Worker. Кластер Master состоит из нескольких узлов и отвечает за управление метаданными векторной базы данных, включая все операции, связанные с определением данных. Кластер Worker использует архитектуру

интеграции хранения и вычислений, в которой каждый узел самостоятельно отвечает за хранение векторных данных, индексацию и выполнение запросов на своем узле. Это и есть то, что мы называем «распределенная архитектура с интеграцией хранения и вычислений». Узлы Engine принимают пользовательские запросы, анализируют их и распределяют задачи для выполнения узлами Worker. Поскольку вычисления и хранение осуществляются на одном узле, такая архитектура зависит от меньшего количества внешних модулей, что придает ей определенное преимущество в стабильности. Однако, из-за того что ресурсы хранения и вычислений интегрированы в одном узле, при расширении необходимо перемещать локальные данные на другие узлы, что снижает эффективность расширения.

4. Распределенные векторные базы данных с разделением хранения и вычислений

Pinecone и Milvus – это векторные базы данных с относительно длительным периодом развития и определенным сходством в архитектуре. Мы классифицируем их как *распределенные векторные базы данных с разделением хранения и вычислений* и представляем их в одной группе. Их упрощенная техническая архитектура показана на рис. 2.4.

Из рис. 2.4 видно, что ключевой особенностью распределенных векторных баз данных с разделением хранения и вычислений является использование объектного хранилища для хранения файлов индекса. Когда пользовательский запрос данных поступает на уровень доступа, запросы на запись обычно помещаются в очередь сообщений. Запросы на запись из этой очереди обрабатываются двумя различными модулями. С одной стороны, кластер индекса читает эти данные и строит временный векторный индекс в памяти. Поскольку индексные данные обычно имеют небольшой объем, они могут храниться в памяти с помощью плоской структуры индекса. С другой стороны, кластер хранения также обрабатывает эти данные с целью построения более масштабного индекса векторных данных и сохранения данных в объектное хранилище после завершения построения. Эти индексные файлы сразу после сохранения могут быть загружены в память кластером индекса для использования. Это означает, что при расширении или добавлении новых узлов другие узлы индекса могут напрямую загружать данные из объектного хранилища, что и называется распределенной архитектурой с разделением хранения и вычислений. В этой архитектуре кластер индекса отвечает за предоставление вычислительных мощностей, а кластер хранения отвечает за построение индекса, одновременно используя объектное хранилище для сохранения файлов индекса.

Для запросов на чтение система сочетает локальные обновленные данные в памяти кластера индекса с фиксированными индексными файлами в объектном хранилище для выполнения запроса и нахождения конечного результата. По сравнению с распределенной архитектурой с интеграцией хранения и вычислений эта архитектура имеет некоторую сложность в управлении из-за наличия дополнительных модулей. Однако благодаря использованию единого объектного хранилища для сохранения данных при миграции или добавлении узлов нет необходимости перемещать данные на узлах, что ускоряет процесс расширения.

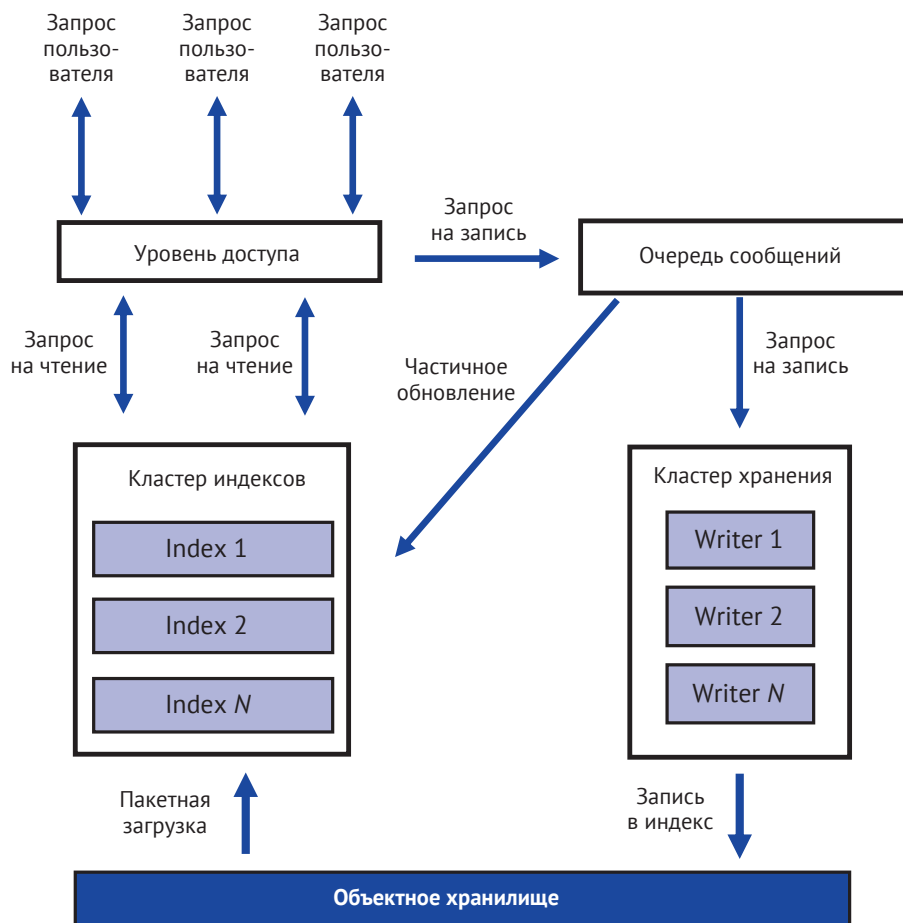


Рис. 2.4. Упрощенная техническая архитектура Pinecone и Milvus

2.4. РЕЗЮМЕ

В этой главе мы разделили развитие векторных баз данных на три основных этапа.

- Первый этап – период зарождения векторных баз данных (1980–2012 годы). В этот период с распространением и применением в различных отраслях глубоких нейронных сетей большое количество неструктурированных данных начало преобразовываться в векторные данные. В 2012 году проект AlexNet добился убедительной победы в конкурсе по масштабному распознаванию изображений ImageNet, что ознаменовало начало внедрения технологий глубоких нейронных сетей.
- Второй этап – период становления векторных баз данных (2012–2017 годы). На этом этапе промышленность начала уделять внимание вопросам эффективного хранения, индексирования и выполнения запросов векторных данных. Благодаря этому векторные библиотеки по-

степенно достигли зрелости. В 2017 году и позже с появлением таких библиотек, как FAISS и HNSWLib, были представлены стандартизированные программные компоненты для реализации векторных баз данных, что способствовало накоплению ключевых технологий.

- Третий этап – период развития векторных баз данных (с 2017 года по настоящее время). С 2017 года индустрия векторных баз данных переживает бурный рост, появляются многочисленные новые игроки. К 2019 году такие компании, как Pinecone, Zilliz и Tencent Cloud, выпустили свои продукты. Большинство этих независимых векторных баз данных построены на распределенной архитектуре и предназначены для предоставления услуг по управлению векторными данными в крупном масштабе для корпоративных пользователей. В период роста эти продукты опираются на потребности предприятий, выбирая подходящую архитектуру. Модульные векторные базы данных акцентируют внимание на использовании существующих возможностей и экосистемы, чтобы быстро обслуживать текущую клиентскую базу и снижать затраты на обучение клиентов, в то время как автономные векторные базы данных сосредоточены на легковесной доставке и развертывании, позволяя заинтересованным клиентам быстро и с минимальными затратами опробовать векторные базы данных. Интеграция хранения и вычислений и разделение хранения и вычислений представляют собой два различных технических подхода. Первый обычно имеет простую и стабильную архитектуру, тогда как второй более гибок в плане расширения.

Несмотря на то что развитие продолжается всего лишь чуть более десяти лет (начиная с периода становления), векторные базы данных как новое направление в области технологий баз данных быстро растут вместе с бурным развитием искусственного интеллекта. С увеличением спроса на векторные базы данных на рынке мы, разработчики на передовой, через постоянные исследования реально способствуем прогрессу в этой области технологий.

В первых двух главах мы изучили историю и развитие векторных баз данных, а в следующей, третьей главе, мы углубимся в изучение их основных возможностей, чтобы подготовиться к практическим занятиям, которые начнутся в четвертой главе.

Глава 3

Основные возможности векторных баз данных

Лучше один раз увидеть, чем сто раз услышать.
Трудно издали оценить военную обстановку.
Я желаю спешно прибыть в Цзиньчэн
и лично составить план для Вашего величества.

Бань Гу. Книга династии Хань

Из предыдущих двух глав мы уже знаем, что векторные данные по своей сути представляют собой понятную для компьютера структурированную форму данных, в которую преобразуются неструктурированные данные. Векторная база данных, как следует из названия, – это система управления базами данных, предназначенная для управления векторными данными.

Если вы имеете некоторое представление о системах управления базами данных, то знаете, что в этой области уже существуют зрелые подходы и системы их реализации. Как векторные базы данных сочетаются с существующими знаниями? Какие возможности они имеют? На какие ключевые показатели следует обращать внимание при использовании векторных баз данных? Какие важные высокоуровневые функции необходимы векторным базам данных для более эффективного предоставления услуг? Изучение главы мы начнем с ответов на эти вопросы. Для удобства изложения возможности векторных баз данных мы будем объяснять на примере Tencent Cloud.

3.1. Основные возможности

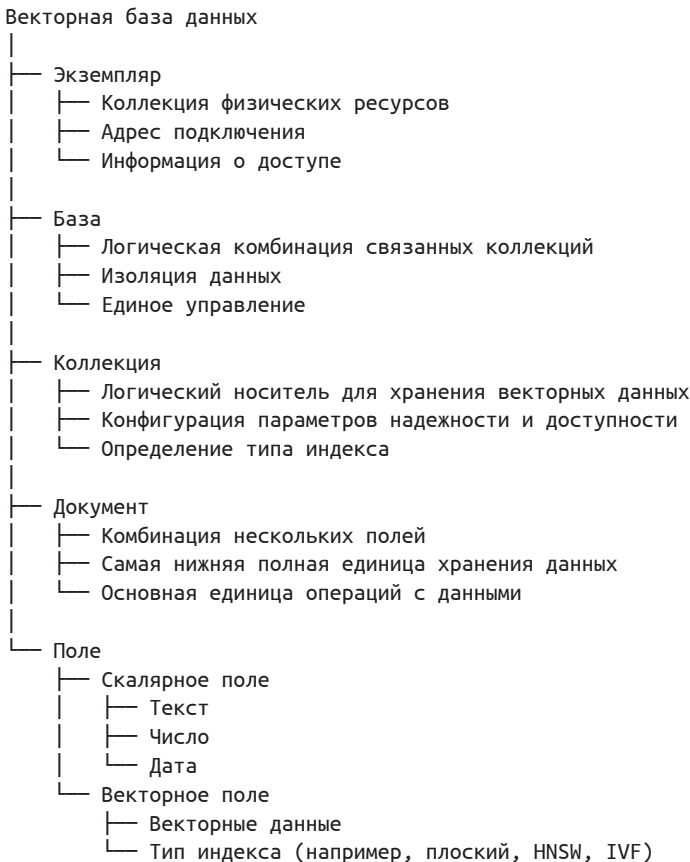
Для эффективной организации и управления данными в базе данных крайне важна хорошо продуманная структура данных, особенно при обработке больших наборов данных. Простое и плоское накопление всех данных приведет к хаосу в управлении и неспособности полностью раскрыть ценность всего объема данных. Поэтому удобная векторная база данных обычно предлагает тщательно разработанную иерархическую структуру данных. Разработчикам следует разумно планировать иерархию данных в соответствии со своими бизнес-сценариями, так как хорошая структура иерархии данных способствует долгосрочному развитию и эксплуатации приложений на основе этой базы данных.

По требованиям к производительности и масштабируемости использование векторных баз данных ближе к традиционным нереляционным базам данных. Поэтому в общей концепции проектирования векторных баз данных используется множество идей из нереляционных баз данных. В этом разделе подробно рассматриваются основные возможности векторных баз данных, чтобы, с точки зрения пользователя, получить более наглядное и конкретное представление о них.

3.1.1. Логические уровни

Векторная база данных обычно включает пять логических уровней: экземпляр, базу, коллекцию, документ и поле. Экземпляр находится на самом верхнем уровне, а поле – на самом нижнем. Все пять уровней последовательно включаются друг в друга, образуя единую логическую структуру.

Я собрал пять основных логических уровней векторной базы данных, а также ключевые характеристики и компоненты каждого уровня в следующей ментальной карте, чтобы наглядно представить организационную структуру векторной базы данных и способы управления и работы с данными на различных уровнях.



1. Экземпляр

Экземпляр (инстанс) является носителем управления ресурсами векторной базы данных. Он представляет собой коллекцию физических ресурсов и реализует отображение между логической и физической концепциями векторной базы данных. Экземпляр находится на самом верхнем уровне базы данных, один экземпляр может содержать несколько баз. Например, в Tencent Cloud ресурс, управляемый разработчиком в консоли, является экземпляром. На рис. 3.1 представлено окно информации об экземпляре.



Рис. 3.1. Информация об экземпляре

После создания экземпляра генерируется ряд связанной информации. Эта информация включает основную информацию (например, ID экземпляра, имя экземпляра и т. д.), информацию о спецификациях (например, спецификации узлов, количество узлов и емкость жесткого диска) и сетевую информацию (например, внутренний IP-адрес, номер порта и адрес доступа). Используя эту важную основную информацию, можно более удобно выполнять разработку и эксплуатацию векторной базы данных.

Наиболее важной частью информации об экземпляре является строка подключения к базе данных и пароль для управления доступом. Например, 10.0.1.14 здесь является адресом для доступа к векторной базе данных. Ниже приведен пример вызова. Команды и код в примерах могут быть разбиты на несколько строк из-за ограничений форматирования, поэтому необходимо внимательно различать фактические переносы строк в командах и коде и переносы строк из-за форматирования. Прямое копирование части команды может привести к ошибкам выполнения. Если это происходит, попробуйте удалить дополнительные переносы строк и повторно выполнить команду.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer account=root&api_key=*' http://$url/$action -d $data
```

Этот вызов является наглядным примером доступа к векторной базе данных. С его помощью мы отправляем запрос на адрес доступа экземпляра. Система векторной базы данных, получив запрос, обрабатывает его в соответ-

ствии с соответствующими параметрами и возвращает ответ. В табл. 3.1 приведено описание параметров запроса.

Таблица 3.1. Параметры запроса на доступ к векторным данным

Параметр	Описание
account	Имя учетной записи разработчика для доступа к экземпляру; на основе имени можно контролировать права доступа
api_key	Ключ доступа разработчика к экземпляру, используется для проверки учетной записи
\$url/\$action	Строка подключения для доступа к экземпляру, где url можно получить в консоли. Например, на рис. 3.1 это 10.0.1.14:80. Поле action задается в зависимости от последующих операций, например /database/create, /document/upsert и т. д.
-d \$data	Поле data передает допустимые данные JSON, соответствующие полю action

2. База

База данных представляет собой логический контейнер, объединяющий коллекции, логически связанные между собой, что облегчает последующее разделение данных и их централизованное управление. Один экземпляр может содержать несколько баз данных, каждая из которых объединяет ряд коллекций.

Разработчики могут использовать концепцию базы данных для реализации разделения данных и их централизованного управления на уровне бизнеса, что повышает эффективность управления данными. Основные операции с базой данных включают создание, удаление и получение списка, которые являются интерфейсами управления метаданными, связанными с базой данных. В табл. 3.2 приведены важные интерфейсы операций на уровне базы данных в векторной базе данных.

Таблица 3.2. Основные интерфейсы операций на уровне базы данных в векторной базе данных

Интерфейс	Описание
/database/create	Создание базы данных в рамках экземпляра
/database/drop	Удаление существующей базы данных вместе со всеми коллекциями и документами в ней
/database/list	Вывод списка существующих баз данных в текущем экземпляре

Ниже приведен пример вызова интерфейса для создания базы данных.

```
Запрос:
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer account=root&api_key=*' http://10.0.1.84:80/database/create -d '{"database": "db-test"}'
Ответ:
{"code":0,"msg":"operation success","affectedCount":1}
```

В этом примере мы создали базу данных с именем `db-test` с помощью интерфейса `/database/create`. На основе созданной базы данных `db-test` мы продолжим объяснение концепции коллекции и примеров интерфейсов.

3. Коллекция

Коллекция является логическим носителем для хранения векторных данных. В векторной базе данных концепция коллекции аналогична таблице в реляционной базе данных. Одна база данных объединяет ряд коллекций, каждая из которых состоит из множества документов с векторными данными. При создании коллекции существует несколько важных параметров конфигурации, таких как тип индекса, метод вычисления сходства, а также количество сегментов и реплик. Тип индекса коллекции указывает, какую структуру данных использовать для организации векторных данных. Этот параметр может варьироваться в зависимости от масштаба векторных данных и значительно влияет на последующую производительность доступа к ним. Методы вычисления сходства были рассмотрены в главе 1, этот параметр указывает, как сравнивать сходство между векторами. Количество сегментов и реплик указывает на максимальную способность коллекции предоставлять услуги и является параметром конфигурации для определения нагрузки системы. Эти конфигурационные данные коллекции применяются ко всем векторным данным, находящимся в ней. Правильная настройка системных параметров является ключевым фактором для долгосрочного управления большими объемами векторных данных. В табл. 3.3 приведены важные интерфейсы операций на уровне коллекции в векторной базе данных.

Таблица 3.3. Важные интерфейсы операций на уровне коллекции в векторной базе данных

Интерфейс	Описание функции
<code>/collection/create</code>	Создание коллекции в существующей базе данных
<code>/collection/drop</code>	Удаление существующей коллекции вместе со всеми документами в ней
<code>/collection/list</code>	Вывод списка существующих коллекций в текущей базе данных
<code>/collection/describe</code>	Запрос параметров конфигурации указанной коллекции
<code>/collection/truncate</code>	Очистка всех данных и индексов в указанной коллекции с сохранением только параметров конфигурации, таких как тип индекса и количество сегментов и реплик; это облегчает обслуживание коллекций

Ниже приведен пример вызова интерфейса для создания коллекции.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer account=root&api_key=*' http://10.0.1.84:80/collection/create -d '{"database": "db-test", "collection": "book-vector", "replicaNum": 2, "shardNum": 1, "description": "this is the collection description",
```

```
"indexes": [{ "fieldName":
"id", "fieldType": "string", "indexType": "primaryKey"}, {"fieldName": "vector",
"fieldType": "vector", "indexType": "HNSW", "dimension": 3,
"metricType": "COSINE", "params": {"M": 16, "efConstruction": 200}},
{"fieldName": "bookName", "fieldType": "string", "indexType": "filter"}]}'
Ответ:
{"code":0,"msg":"operation success","affectedCount":1}
```

В этом примере мы создали коллекцию под названием `book-vector` в базе данных `db-test` с помощью интерфейса `/collection/create`.
Интерфейс создания коллекции включает множество параметров, которые необходимо передать при вызове интерфейса создания. В табл. 3.4 приведено несколько важных параметров с пояснениями.

Таблица 3.4. Важные параметры интерфейса создания коллекции

Параметр	Описание
collection	Имя создаваемой коллекции, носитель для последующего управления коллекцией и векторными данными
shardNum	Количество сегментов данных. Разработчики разбивают крупные наборы данных на несколько поднаборов, чтобы решить проблему ограничения емкости хранения одного узла; каждый поднабор является сегментом
replicaNum	Количество реплик данных, указывает количество реплик для каждого сегмента. Обычно используется для повышения способности к восстановлению после сбоев и решения проблемы ограничения производительности чтения одного узла
индексы	Конфигурация индексов коллекции. Индексы являются важными полями, используемыми для ускорения запросов данных. В разделе 3.1.2 мы рассмотрим индексы более подробно

4. Документы и поля

В векторной базе данных документ аналогичен строке записи в реляционной базе данных. Каждая коллекция состоит из множества документов, каждый из которых, в свою очередь, состоит из нескольких полей. Документ является основной единицей хранения и обработки данных в векторной базе данных, все операции с данными осуществляются на уровне документов.
Поле представляет собой отдельный элемент данных в документе, отражающий одно из его свойств или характеристик. Хорошее планирование полей документа является ключом к эффективному использованию векторной базы данных. Поля могут быть скалярными (строки, целые числа, числа с плавающей запятой) или векторными, то есть точками в многомерном пространстве, полученными в результате векторизации неструктурированных данных соответствующими моделями. Векторные поля являются основными в векторных базах данных.
Векторные базы данных обычно предоставляют интерфейсы для пакетной обработки документов, чтобы повысить эффективность операций, а также поддерживают гибридные запросы, объединяющие векторы и скаляры, для

удовлетворения потребностей сложных бизнес-сценариев. В табл. 3.5 приведены важные интерфейсы для операций на уровне документов.

Таблица 3.5. Важные интерфейсы для операций на уровне документов в векторной базе данных

Интерфейс	Описание
<code>/document/upsert</code>	Запись документов в коллекцию с перезаписью
<code>/document/query</code>	Запрос конкретных документов из коллекции на основе скалярных условий фильтрации
<code>/document/search</code>	Запрос k ближайших документов из коллекции на основе векторных условий
<code>/document/delete</code>	Удаление указанных документов
<code>/document/update</code>	Изменение данных документа по указанному полю, можно вносить отдельные изменения в векторные или скалярные данные

Интерфейс `/document/upsert` используется для записи векторных данных в уже созданную коллекцию. Ниже приведен пример вызова для записи данных в коллекцию.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer account=root&api_key=*' http://10.0.1.84:80/document/upsert -d '{"database": "db-test", "collection": "book-vector", "documents": [{"id": "0001", "vector": [0.2123, 0.23, 0.213], "author": "Ло Гуаньчжун", "bookName": "Троецарствие", "page": 21}]}'
```

Ответ:

```
{"code":0,"msg":"operation success","affectedCount":1}
```

С помощью этого интерфейса мы записали в коллекцию `book-vector` документ, содержащий несколько полей, включая метаданные и массив векторных чисел с плавающей запятой.

Интерфейс `/document/search` используется для поиска k векторов, наиболее схожих с заданным вектором запроса. Этот интерфейс поддерживает запросы сходства на основе указанного ID или текстовых/векторных значений и возвращает первые k наиболее схожих документов.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer account=root&api_key=*' http://10.0.1.84:80/document/search -d '{"database": "db-test", "collection": "book-vector", "search": {"vectors": [[0.3123, 0.43, 0.213]], "params": {"ef": 200, "limit": 3}}}'
```

Ответ:

```
{"code":0,"msg":"operation success","documents":[{"id":"0001", "vector":[0.21230000257492066, 0.23000000417232514,0.21299999952316285],
```

```
"score":0.9714228510856628,"page":21,"author":"Ло Гуаньчжун",
"bookName":"Троецарствие"}]]}]}
```

С помощью этого интерфейса мы нашли, что вектор с `"id": "0001"` является ближайшим соседом к вектору `[0.3123, 0.43, 0.213]`. В возвращаемом значении содержится значение `"score": 0.9714228510856628`, которое рассчитывается на основе метода расчета сходства векторов, заданного разработчиком при создании индекса коллекции. В разделе 1.1.3 мы подробно рассмотрели распространенные методы расчета.

В этой секции мы рассмотрели пять логических уровней базы данных: экземпляр, базу, коллекцию, документ и поле. В практическом применении, чтобы обеспечить изоляцию данных, можно разделять разные уровни экземпляров, баз, коллекций и документов в зависимости от организационной структуры и бизнес-требований.

- Если между различными отделами требуется полная независимость данных, следует выделить независимые экземпляры для каждого отдела.
- В пределах одного отдела различные бизнес-системы можно изолировать с помощью различных баз, чтобы обеспечить независимость данных.
- Для различных модулей в одной бизнес-системе можно использовать коллекции для разделения данных, что облегчает последующую интеграцию и использование данных.
- В одном модуле данные разных пользователей обычно могут храниться в разных документах одной коллекции, различаясь по ID пользователя или другим идентификаторам.

Чтобы эффективно управлять данными в векторной базе данных, необходимо создавать подходящие индексы для соответствующих данных. Какие индексы существуют в векторной базе данных? Как эти индексы влияют на эффективность управления данными? Далее мы ответим на эти и другие вопросы об индексах.

3.1.2. Индексы

Векторные базы данных обычно поддерживают несколько типов индексов, включая первичные ключи, векторные индексы и фильтрующие индексы. Кратко: первичный ключ используется для быстрого поиска конкретного документа, векторный индекс – для быстрого запроса схожих векторов, а фильтрующий индекс – для фильтрации данных. Эти три типа индексов применяются в различных сценариях запросов, помогая эффективно выполнять разнообразные операции запросов.

1. Первичный ключ

Первичный ключ представляет собой тип индекса, предназначенный для быстрого поиска конкретного документа. В первичном ключе каждый документ имеет уникальный идентификатор, называемый первичным ключом. Используя эти ключи, первичный ключ может быстро находить конкретные документы без необходимости сканирования всей коллекции. В Tencent Cloud по умолчанию используется идентификатор документа для построения первичного ключа. В табл. 3.6 приведены важные параметры для настройки первичного ключа.

Таблица 3.6. Важные параметры для настройки первичного ключа

Параметр	Описание
<code>fieldName</code>	Имя поля документа, соответствующее первичному ключу; в Tencent Cloud в качестве первичного ключа по умолчанию используется идентификатор документа
<code>fieldType</code>	Тип поля первичного ключа, например <code>string</code>
<code>indexType</code>	Тип индекса, первичный ключ настроен как <code>primaryKey</code>

2. Векторные индексы

Векторный индекс аналогичен понятию индекса в реляционных базах данных и представляет собой структуру данных, специально разработанную для векторных типов данных с целью повышения доступа и эффективности запросов к векторным данным. Он основан на математических моделях и алгоритмах и обеспечивает высокую производительность по временной и пространственной сложности. Однако, в отличие от традиционных индексов, обрабатывающих структурированные данные, векторные индексы применяются для обработки векторов высокой размерности, которые обычно получаются путем векторизации неструктурированных текстов, изображений и других подобных данных.

Ведущие векторные базы данных обычно поддерживают плоские индексы, HNSW-индексы и IVF-индексы. В табл. 3.7 приведены ключевые особенности этих векторных индексов.

Таблица 3.7. Ключевые особенности векторных индексов

Векторный индекс	Описание	Преимущества	Недостатки	Применение	Настройка параметров
Плоский	Все векторные данные хранятся в плоской структуре	1. Простота реализации. 2. Высокая скорость выполнения запросов для небольших наборов данных. 3. Высокая точность запросов	1. Низкая эффективность запросов для больших наборов данных, требуется линейное сканирование всех данных. 2. Большие затраты памяти, необходимо загружать в память все векторные данные	1. Небольшие наборы векторных данных (обычно менее миллиона векторов). 2. Высокие требования к точности и полноте запросов. 3. Низкие требования к времени и эффективности запросов	Без дополнительных параметров

Окончание табл. 3.7

Векторный индекс	Описание	Преимущества	Недостатки	Применение	Настройка параметров
HNSW	Организация данных через многоуровневую нерекурсивную графовую структуру, поддерживающую быстрые запросы на одном уровне и между уровнями	1. Хорошая масштабируемость, подходит для больших наборов данных (один индекс может достигать десятков миллионов строк). 2. Отличная производительность запросов, обычно требуется сканирование небольшого количества данных. 3. Поддержка динамической записи данных	1. Высокие временные затраты на построение индекса. 2. Точность запросов не достигает 100 %, существует небольшая вероятность ошибки. 3. Большие затраты памяти	1. Большие наборы векторных данных. 2. Высокие требования к времени отклика и эффективности запросов. 3. Допустимость небольшой вероятности ошибки результатов запроса, использование приближенных результатов	1. Параметр efConstruction используется для указания диапазона поиска соседей узлов при запросе, чем больше значение, тем дольше время построения. 2. Параметр M используется для указания количества соседей, которые каждый узел может соединить в односвязном графе, что влияет на объем занимаемой памяти и эффективность запроса
IVF	Сначала производится кластеризация векторных данных, затем внутри кластеров создается инвертированный индекс, имеется возможность наложения алгоритмов квантизации и сжатия	1. Относительно небольшое использование памяти, в сочетании с технологией кодирования квантизации позволяет дополнительно экономить пространство памяти. 2. Хорошая масштабируемость, поддержка сверхкрупных наборов данных	1. Относительно низкая точность запроса. 2. Необходимость предварительного построения алгоритма кластеризации, что снижает оперативность	1. Крупные и сверхкрупные наборы векторных данных (но с ограничением на использование памяти). 2. Большой акцент на затраты по объему хранения, а не на абсолютное время запроса. 3. Возможность применять компромисс между точностью и временем запроса	1. Параметр nlist обозначает количество центров кластеров в индексе, используемых для разделения векторного пространства. 2. Параметр M обозначает размерность каждого подвектора в квантизации произведения, влияет на точность квантизации и объем занимаемого индексом пространства

Плоские индексы, HNSW-индексы и IVF-индексы применяются в различных сценариях. Перед созданием коллекции разработчик должен заранее оценить масштаб векторных данных и требования бизнеса к точности запросов, чтобы выбрать подходящий тип индекса, который обеспечит оптимизацию последующих запросов к векторным данным. Ниже представлены рекомендации по применению, основанные на практическом использовании.

Плоский индекс может обеспечить 100%-ную точность запроса, подходит для сценариев с объемом векторных данных в одном индексе не более 1 млн строк.

HNSW-индекс хотя и не может обеспечить 100% точности выполнения запросов, но все-таки обеспечивает довольно высокий уровень вплоть до 99 % точности. Подходит для сценариев с объемом векторных данных в одном индексе на уровне десятков миллионов строк.

IVF-индекс подходит для сценариев с низкими требованиями к полноте (обычно до 90 %), объем векторных данных в одном индексе может достигать 100 млн строк.

3. Фильтрующие индексы

Фильтрующий индекс строится на основе скалярных полей. В процессе векторного запроса система фильтрует диапазон запроса в соответствии с условиями выражения, заданными фильтром на скалярном поле, для сопоставления схожих векторных документов. Условное выражение представляет собой условие запроса для отбора векторных документов, которое может основываться на значениях полей в векторных документах. Обычно условное выражение состоит из одного или нескольких условий, каждое из которых включает имя поля, оператор сравнения и значение. Конкретный пример использования будет подробно рассмотрен в разделе 3.2.4.

Ниже приведен пример параметров для настройки фильтрующего индекса.

```
{"fieldName": "bookName", "fieldType": "string", "indexType": "filter"}
```

Здесь мы создаем фильтрующий индекс для скалярного поля строкового типа с именем `bookName` в векторной коллекции. Создание фильтрующего индекса позволяет при последующих запросах на сходство векторов сначала использовать значение поля `bookName` для фильтрации подмножества векторных документов, удовлетворяющих условиям, а затем выполнить запрос на сходство векторов на этом подмножестве, тем самым сужая диапазон запроса и повышая его эффективность.

4. Перестроение индексов

Из методов настройки индексов, описанных выше, видно, что разные типы индексов требуют настройки различных параметров, а выбор значений параметров тесно связан с реальным масштабом векторных данных. Например, при увеличении числа строк векторных данных необходимо корректировать в HNSW-индексе параметры `M` и `efConstruction` для обеспечения высокой полноты. Поэтому векторная база данных должна поддерживать динамическое *перестроение индекса*.

Мы используем интерфейс `/index/rebuild` для перестроения индекса, который предназначен для перестроения всех индексов в указанной коллекции, включая удаление ненужных данных индекса, восстановление поврежденных данных индекса и оптимизацию структуры индекса, что повышает производительность.

Ниже приведен пример вызова этого интерфейса.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization:
  Bearer account=root&api_key=*' http://10.0.1.84:80/index/rebuild
  -d '{"database": "db-test", "collection": "book-vector",
    "dropBeforeRebuild": true}'
```

Ответ:

```
{"code":0,"msg":"operation success"}
```

Этот интерфейс уведомляет систему управления базой данных о необходимости продолжить перестроение индекса на основе текущих данных. Особое внимание следует уделить тому, что перестроение индекса является операцией с высоким риском и может негативно повлиять на работу онлайн-сервисов. Обычно интерфейс предоставляет параметры для настройки определенных действий. В вышеуказанном примере параметр `dropBeforeRebuild` указывает, следует ли удалить старый индекс перед созданием нового. Если ресурсов памяти недостаточно, можно сначала удалить старый индекс. Однако режим удаления старого индекса до завершения создания нового имеет серьезные последствия – в процессе перестроения коллекция не может нормально читать и записывать данные. Если ресурсов памяти достаточно, старый индекс можно не удалять. Режим сохранения старого индекса оказывает относительно меньшее влияние – до завершения создания нового индекса коллекция может читать данные, но запись данных запрещена.

3.1.3. Ключевые характеристики

После ознакомления с основными операциями с векторной базой данных необходимо обратить внимание на то, как оценивать эффективность ее работы. Задержка доступа, пропускная способность и полнота являются основными показателями оценки производительности векторной базы данных, с помощью которых измеряют производительность с точки зрения скорости отклика, способности обработки и точности запросов.

Задержка доступа – это время, прошедшее с момента получения системой запроса до возвращения ответа. В векторной базе данных низкая задержка означает, что приложение может быстрее получить результаты запроса, что крайне важно для улучшения пользовательского опыта и скорости отклика системы.

Пропускная способность экземпляра – это максимальная нагрузка, которую может непрерывно обрабатывать один экземпляр сервиса при заданных тестовых условиях, когда ресурсы (например, CPU, память, устройства ввода/вывода и т. д.) достигают предела своей производительности. Обычно она измеряется в количестве запросов, обрабатываемых в секунду (QPS). Высокая

пропускная способность указывает на то, что база данных может эффективно обрабатывать большое количество параллельных запросов, что является ключевым показателем расширяемости и производительности системы.

В векторной базе данных обычно оценивается полнота результатов запроса, то есть доля релевантных целевых документов, найденных системой, от общего числа реально существующих релевантных целевых документов. Высокая полнота означает, что система может вернуть больше ожидаемых результатов запроса, что важно для обеспечения точности результатов. Формула для расчета полноты:

Полнота = Найдено релевантных документов / Всего релевантных документов.

Значение полноты находится в диапазоне от 0 до 1. Чем ближе к 1, тем выше точность системы в поиске целевых векторов, схожих с вектором запроса. В реальных приложениях обычно необходимо достичь баланса между полнотой и скоростью выполнения запросов, чтобы обеспечить как эффективность, так и точность поиска.

Для измерения ключевых показателей системы векторной базы данных обычно используются некоторые стандартные инструменты. ANN-Benchmarks является одним из широко используемых в отрасли инструментов тестирования производительности. Он не только предоставляет клиентскую программу для нагрузочного тестирования и измерения задержки, QPS и других ключевых показателей, но и стандартные наборы данных. С помощью этих стандартных наборов данных можно точно определить, правильные ли возвращаются результаты запроса, и таким образом оценить общую полноту.

Согласно результатам тестирования векторной базы данных Tencent Cloud и открытых векторных баз данных, при одинаковых тестовых условиях и аппаратных ресурсах для выполнения запроса Top10 на данных с 10 млн строк и размерностью 768, с требованием полноты в 95 % средняя задержка составляет менее 10 мс, а задержка p99 – менее 20 мс (то есть 99 % времени отклика на запросы составляет менее 20 мс).

При полной загрузке ресурсов мы обращаем внимание на общую пропускную способность векторной базы данных. При одинаковых тестовых условиях достижение 120 QPS на одном CPU является относительно приемлемым уровнем.

Кроме того, система управления векторной базой данных должна предоставлять показатели мониторинга в реальном времени, чтобы разработчики могли оперативно отслеживать состояние работы всей системы. Например, на рис. 3.2 показан интерфейс мониторинга показателей, аналогичный тому, что представлен в консоли Tencent Cloud.

3.2. РАСШИРЕННЫЕ ВОЗМОЖНОСТИ

Векторная база данных, помимо базовых функций традиционных баз данных, обладает также рядом расширенных возможностей, тесно связанных с экосистемой векторных данных. В этом разделе мы сосредоточимся на описании этих расширенных возможностей.

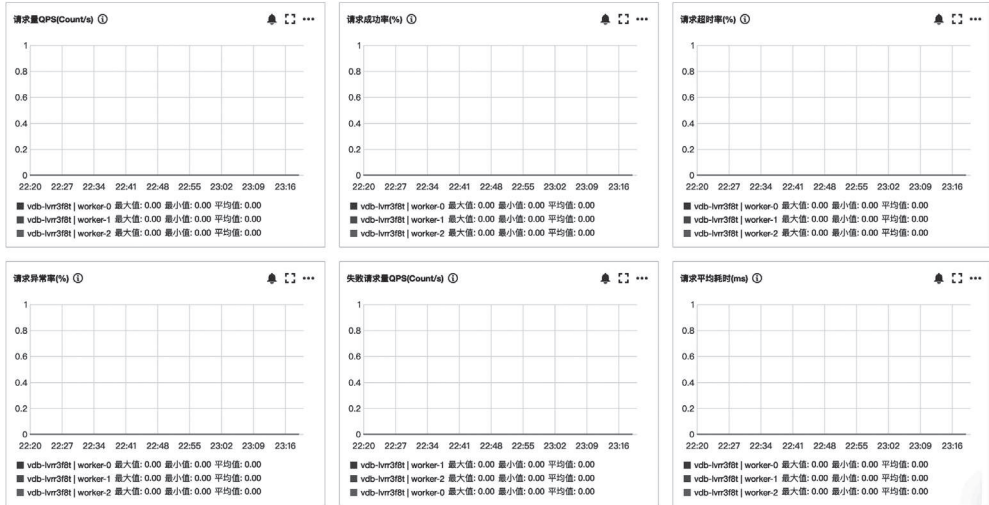


Рис. 3.2. Мониторинг показателей работы векторной базы данных

3.2.1. Динамическая схема

В разделе 3.1.1 мы рассмотрели понятие коллекции в векторной базе данных. При создании коллекции необходимо указать соответствующие параметры, такие как количество сегментов и реплик, которые влияют на надежность и доступность данных коллекции. Кроме того, мы узнали, что выбор подходящих параметров индексов является важным для коллекции. Однако вы могли заметить, что, кроме заранее определенных полей индекса, мы не определяли другие поля коллекции. Это и есть способность векторной базы данных к использованию *динамической схемы*, что отличает ее от традиционных реляционных баз данных. Рассмотрим это на конкретном примере.

Возьмем, к примеру, базу данных MySQL. Прежде чем начать запись данных в таблицу, необходимо сначала определить структуру таблицы.

Например, создадим таблицу с информацией о книгах с помощью следующей команды:

```
CREATE TABLE books (
  book_id INT(11) NOT NULL AUTO_INCREMENT,
  title VARCHAR(255) NOT NULL,
  author VARCHAR(100) NOT NULL,
  category VARCHAR(50) NOT NULL,
  PRIMARY KEY (book_id)
);
```

В базе данных MySQL все поля должны быть определены заранее. В процессе последующей записи данных будет проводиться строгая проверка наличия и типа полей. Если необходимо добавить новое поле или изменить тип поля, потребуется выполнить изменение структуры таблицы в режиме онлайн. Такая операция обычно сопряжена с высоким риском, так как в процессе изме-

нения может быть затронута возможность чтения и записи данных в соответствующую таблицу.

Таким образом, наличие динамической схемы в векторной базе данных значительно облегчает работу разработчиков. При записи и обновлении данных с использованием интерфейсов `upsert/update` можно динамически добавлять и удалять поля в зависимости от изменяющихся потребностей бизнеса, что повышает гибкость использования базы данных в приложениях.

Например, в разделе 3.1.1 при создании коллекции мы указали только `book-Name` в качестве поля индекса, но не определили заранее типы полей `author` и `page`. Когда приложение использует интерфейс `upsert` для записи данных, поля `author` и `page` можно указать динамически в процессе записи, а их типы также определяются динамически при записи данных приложением. Таким образом, при необходимости добавления новых полей в коллекцию или изменения типов полей не требуется выполнять изменение структуры таблицы в режиме онлайн, как в реляционных базах данных. Динамическая схема, являясь стандартной возможностью нереляционных баз данных, также является важной характеристикой векторной базы данных.

3.2.2. Механизм псевдонимов

Из предыдущих глав мы узнали, что в коллекции векторной базы данных каждая строка документа хранит соответствующие векторные данные. Эти векторные данные генерируются с помощью предобученной модели векторизации. При этом все данные должны быть сгенерированы одной и той же моделью векторизации. Если использовать векторные данные, сгенерированные разными версиями моделей векторизации, это негативно скажется на полноте.

Предположим, что мы уже векторизовали 10 млн строк данных на основе модели А, и эти векторные данные поддерживают запросы онлайн-пользователей. Затем мы создали улучшенную модель векторизации В и хотим обновить 10 млн строк векторных данных в базе с помощью новой модели.

Текущий интерфейс для операций с данными позволяет обновлять документы пакетами, и нам необходимо продумать, как осуществить согласованную замену модели векторизации для всех документов в коллекции.

Одним из решений является создание новой коллекции и после обновления данных переключение трафика клиентской программы на новую коллекцию. Однако такой подход требует значительных изменений на стороне клиента и имеет высокий порог внедрения, что затрудняет его реализацию.

Более удачным решением является использование механизма *псевдонимов*.

Интерфейс `/alias/set` используется для назначения псевдонима коллекции. Псевдоним может быть короткой строкой, что облегчает идентификацию и доступ к соответствующей коллекции. Одна коллекция может иметь один или несколько псевдонимов.

Предположим, что в настоящее время клиентский код обращается к коллекции с именем А. Мы можем создать новую коллекцию В в фоновом режиме и записать в нее новые векторные данные, используя новую модель векторизации. После завершения записи коллекция В будет содержать полные векторные данные. Затем мы вызываем интерфейс `/alias/set`, чтобы установить для коллекции В псевдоним А. Клиент продолжает использовать А для доступа к данным,

но, поскольку А теперь является псевдонимом для В, запросы к А фактически будут обращаться к данным в В. Таким образом, мы можем заменить модель векторизации в режиме онлайн без изменений на стороне клиента.

Ниже приведен пример вызова интерфейса псевдонимов.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization:
  Bearer account=root&api_key=*' http://10.0.1.84:80/alias/set
  -d '{"database": "db-test", "collection": "B", "alias": "A"}'
```

Ответ:

```
{"code":0,"msg":"operation success","affectedCount":1}
```

Если после переключения на коллекцию В бизнес-логика не соответствует ожиданиям, можно использовать интерфейс удаления псевдонима [/alias/delete](#), чтобы отменить его.

3.2.3. Векторизация

При записи данных в векторную базу данных разработчику обычно необходимо сначала преобразовать исходные неструктурированные данные в векторные данные с помощью модели *векторизации*. При вызове интерфейсов записи или запроса векторной базы данных входные параметры должны быть уже обработанными векторными данными.

Как мы узнали в разделе 3.1.1, при записи данных с использованием интерфейса [/document/upsert](#) в поле **vector** вводятся заранее сгенерированные через модель векторизации векторные данные.

Чтобы вызвать этот интерфейс, необходимо выбрать подходящую модель векторизации для преобразования исходных данных в векторный вид. Для запуска модели требуется среда с GPU, что повышает порог входа для разработчиков, и многие из них отказываются от попыток использовать векторные базы данных.

Чтобы упростить процесс, векторная база данных может предоставлять механизм, позволяющий разработчикам напрямую взаимодействовать с векторной базой данных, используя исходные данные (текст/изображение/аудио). Некоторые векторные базы данных предлагают такую функцию векторизации.

Ниже приведен пример использования интерфейса [/document/upsert](#) с расширенной способностью векторизации.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization:
  Bearer account=root&api_key=*' http://10.0.1.84:80/document/upsert
  -d '{"database": "db-test", "collection": "book-emb",
    "buildIndex": true, "documents": [{"id": "0001", "text":
      "Говорят, что мир после длительного периода разделения стремится
      к объединению, а после длительного периода единства - к разделению.",
      "author": "Ло Гуаньчжун", "bookName": "Троецарствие", "page": 21}]}'
```

Ответ:

```
{"code":0,"msg":"operation success","affectedCount":1}
```


Из запроса видно, что разработчику достаточно ввести исходные текстовые параметры для записи данных, что значительно снижает порог использования по сравнению с предыдущим способом ввода поля `vector`.

Однако следует отметить, что при создании коллекции мы должны указать, что эта коллекция должна поддерживать векторизацию, и четко обозначить, из какого поля следует извлекать исходный текст для последующих операций с данными. Поэтому в списке параметров интерфейса `/collection/create` необходимо добавить поле `embedding`.

```
"embedding": {"field": "text", "vectorField": "vector", "model": "bge-base-zh"}
```

Этот объект параметров задает команды системы данных для последующих операций с данными: извлечение исходного содержимого из поля `text`, векторизованные данные будут храниться в поле `vector`, модель векторизации – `bge-base-zh`.

3.2.4. Гибридные запросы

Гибридный запрос к векторной базе данных сочетает скалярные и векторные поля, используя настраиваемые скалярные поля (независимые числовые поля с атрибутами документа, такие как ID, текст, числовые значения или даты) и выражения условий фильтрации в качестве условий запроса для выполнения комплексных операций с векторными данными (запись, обновление, запрос и т. д.). Этот способ запроса подходит для сценариев, требующих комплексных операций с несколькими атрибутами.

Например, после векторизации изображений из пользовательской галереи можно сохранить данные о времени и месте съемки вместе с векторными данными в одном документе. При выполнении поиска по сходству это позволяет эффективно отфильтровывать изображения, не соответствующие требованиям по времени и месту, что значительно повышает эффективность и точность запроса.

Для вызова гибридного запроса через интерфейс обычно используется выражение условий фильтрации в формате `<field_name><operator><value>`. Здесь можно соединять несколько выражений с помощью логических операторов `and` (и), `or` (или) и `not` (не). Здесь `<field_name>` обозначает имя фильтруемого поля, `<operator>` – используемый оператор, а `<value>` – значение для сопоставления. Разные типы значений поддерживают различные операторы. Например, для строк доступны операции равенства, неравенства, включения и исключения, а для `uint64` – больше, больше или равно, равно, меньше, меньше или равно.

Ниже приведен пример корректного выражения условий фильтрации в виде логического выражения, комбинирующего числовые и строковые значения.

```
score=90 and (video_tag="dance" or video_tag="music")
```

Далее приведем пример использования гибридного запроса для вызова интерфейса `/document/search`.

Запрос:

```
curl -i -X POST -H 'Content-Type: application/json' -H 'Authorization: Bearer
account=root&api_key=*' http://10.0.1.84:80/document/search -d '{"database":
"db-test", "collection": "book-emb", "search": {"embeddingItems":
["Мир после длительного периода разделения стремится к объединению,
а после длительного периода единства - к разделению"], "limit": 3,
"params": {"ef": 200}, "retrieveVector": false, "filter": "bookName in
(\\\"Троецарствие\\\", \\\"Путешествие на Запад\\\")", "outputFields": ["id",
"author", "text", "bookName"]}]}'
```

Ответ:

```
{"code":0,"msg":"operation success","documents":[[{"id":"0001","score":0.979274
1537094116,"bookName":"Троецарствие","author":"Ло Гуаньчжун","text":"Говорят,
что мир после длительного периода разделения стремится к объединению,
а после длительного периода единства - к разделению."}]]}
```

В этом примере показано, как реализовать гибридный запрос, сочетающий векторные и скалярные поля. В запросе мы указали поле векторного запроса `search.embeddingItems` как "Мир после длительного периода разделения стремится к объединению, а после длительного периода единства - к разделению" и задали выражение условий фильтрации в поле `filter` как `bookName in ("Троецарствие", "Путешествие на Запад")`, что позволило выполнить комплексный запрос для нахождения трех наиболее схожих документов.

В реальных бизнес-приложениях такая модель гибридных запросов, объединяющая векторные и скалярные поля, получила широкое признание и применение среди разработчиков.

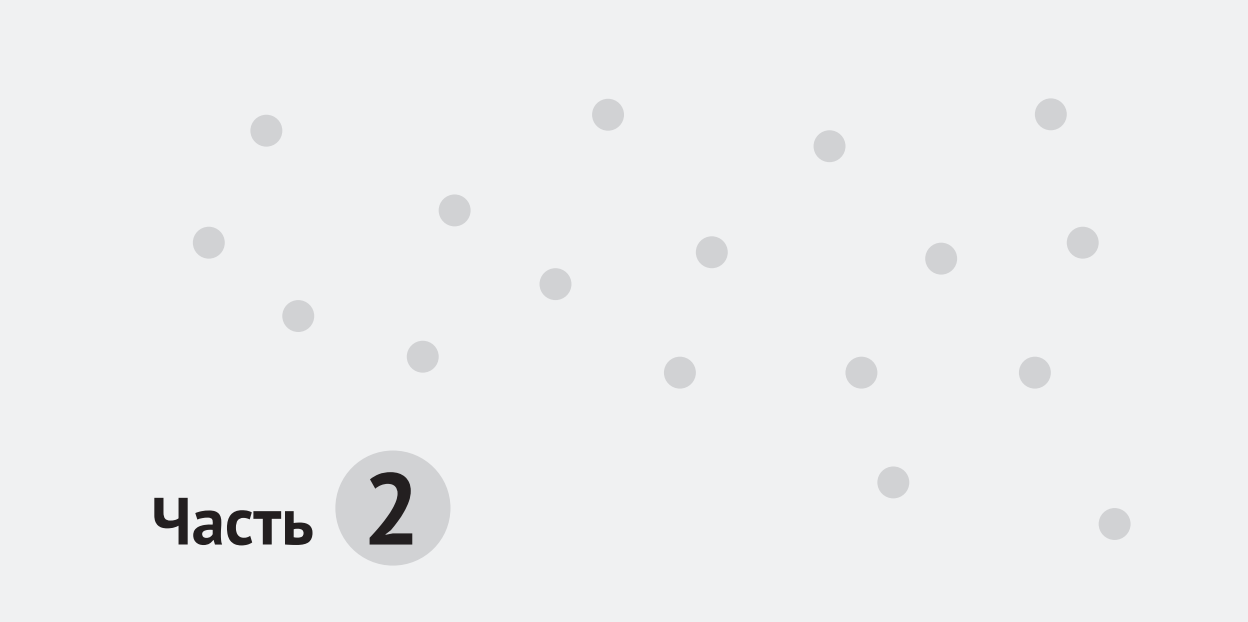
3.3. РЕЗЮМЕ

В этой главе мы начали с базовых понятий и постепенно углубились в изучение основных и продвинутых возможностей векторных баз данных. После изучения этой главы надеемся, что вы получили определенное представление о логической структуре и способах использования векторных баз данных, что облегчит изучение последующих глав. Ниже приведены основные моменты главы.

- Освоив пять логических уровней векторной базы данных (экземпляр, база, коллекция, документ и поле), мы теперь можем более эффективно управлять данными и индексами. Как говорится, «высокое здание начинается с фундамента», и разумное планирование структуры базы данных и стратегии индексации имеет решающее значение для последующего администрирования.
- Для более эффективного поиска документов необходимо выбирать подходящий тип индекса. Плоский индекс, HNSW-индекс и IVF-индекс подходят для различных сценариев применения, и разработчикам нужно делать выбор в соответствии с реальными условиями эксплуатации.
- Ключевые характеристики традиционных баз данных (задержка доступа, пропускная способность экземпляра) также важны и для векторных баз данных, так как они влияют на долгосрочные эксплуатационные рас-

ходы и являются системными показателями, требующими постоянного внимания. Кроме того, в векторных базах данных полнота запроса также имеет первостепенное значение, и обсуждение ее производительности без учета полноты является недопустимым.

- Динамическая схема обеспечивает расширяемость и гибкость, а также позволяет избежать частых изменений структуры таблиц. Следует помнить, что онлайн-изменения структуры таблиц при добавлении данных – это кошмар для администраторов баз данных.
- Механизм псевдонимов позволяет нам бесшовно обновлять модели векторизации, снижая вмешательство в код бизнес-приложений и предоставляя пространство для развития бизнеса.
- Встроенная функция векторизации в векторных базах данных снижает порог использования для разработчиков, позволяя быстро освоиться даже новичкам, что значительно способствует распространению технологии векторных баз данных.
- В практическом применении векторные и скалярные данные часто используются совместно. Векторная база данных поддерживает не только запросы, основанные на векторах, но и гибридные запросы, сочетающие векторные и скалярные условия, что позволяет ускорить выполнение запросов и повысить их точность посредством выражений фильтрации.



Часть 2

Создание векторной базы данных

По сравнению с другими биологическими видами люди обладают преимуществом в решении сложных задач, главным образом благодаря способности разлагать сложные задачи на более мелкие и решать их поэтапно. Разбиение задач приносит две выгоды. Во-первых, «разделяй и властвуй» – разбиение задач снижает их сложность, превращая сложные вызовы в более доступные. Во-вторых, «каждый должен заниматься своим делом» – мелкие задачи в различных областях выполняются соответствующими специалистами. Каждый специалист сосредотачивается на своей области, а результаты работы одних экспертов используются другими. Это позволяет избежать дублирования работы и накапливать достижения, подобно строительным блокам, особенно в сфере баз данных, в которой десятилетия развития привели к накоплению множества компонентов, пригодных для повторного использования.

В процессе создания векторной базы данных мы применим аналогичную стратегию. Для создания распределенной векторной базы данных мы сначала разработаем автономную векторную базу данных.

Глава 4

Реализация автономной векторной базы данных

Без малых шагов не одолеть пути в тысячу ли,
без малых ручьев не наполнить океан.

Сюнь-цзы. Наставление к учению

В этой главе мы реализуем автономную векторную базу данных в три этапа.

Сначала реализуем индекс векторных данных, который поддерживает операции записи и запроса векторных данных. Затем на его основе разработаем гибридный индекс для поддержки записи и запроса скалярных и векторных данных. В завершение мы обеспечим способность системы к восстановлению после сбоя для гарантии надежности данных.

Наша цель – реализовать функции записи и запроса базовых векторных и скалярных данных и обеспечить быстрое восстановление всей системы в случае сбоя.

Начиная с этой главы будет приводиться много сопутствующего исходного кода, который вы можете скачать с домашней страницы книги на сайте сообщества Turing. Рекомендуется вместе с чтением книги одновременно практиковаться с соответствующими исходными кодами, постепенно создавая распределенную векторную базу данных с нуля.

4.1. РЕАЛИЗАЦИЯ ИНДЕКСА ВЕКТОРНЫХ ДАННЫХ

Наше основное требование для векторной базы данных – возможность записи и запроса определенного количества векторных данных. После успешной записи данных мы можем найти среди записанных данных k ближайших векторов к заданному вектору.

Реализацию индекса мы начнем с изучения основных функций и соответствующего исходного кода широко используемых в индустрии открытых векторных библиотек FAISS и HNSWLib.

4.1.1. Основные функции библиотеки FAISS

Основные функции FAISS

- └ Структура данных
 - └ Index (основная структура данных)
 - └ Члены (d, metric_type, ntotal)
 - └ Функции-члены (add, search и remove_ids)
 - └ IndexFlatCodes (наследуется от Index, отвечает за хранение векторов)
 - └ IndexFlat (наследуется от IndexFlatCodes, реализует функцию search)
 - └ IndexIDMap (отвечает за хранение отображения ID и векторов)
- └ Основные функции
 - └ Запись векторных данных
 - └ Использование add или add_with_ids
 - └ Запрос векторных данных
 - └ Использование search
 - └ Удаление векторных данных
 - └ Удаление заданного вектора по ID
- └ Реализация функций
 - └ IndexFlatCodes::add (реализация записи векторных данных)
 - └ IndexFlat::search (реализация запроса векторных данных)
 - └ IndexIDMap::add_with_ids (реализация записи векторных данных по ID)

Стандартный исходный код FAISS можно получить с GitHub с помощью следующей команды:

```
git clone https://github.com/facebookresearch/faiss faiss_xx
```

1. Структуры данных

Основная часть исходного кода FAISS находится в папке /faiss. Основной структурой данных является `struct faiss::Index` (далее будем называть ее `Index` или структура `Index`). `Index` является важным носителем для последующих операций со структурой индексов. Мы выбрали из исходного кода FAISS основную часть кода `Index` (назовем ее «упрощенной версией», аналогично обработаны и другие модули) для более наглядного изучения деталей реализации FAISS. Понимание этих деталей позволит нам в дальнейшем быстро и удобно реализовать собственный функционал плоских индексов на основе библиотеки FAISS. Упрощенный код выглядит следующим образом:

```
struct Index {
    int d; // Размерность вектора.
    MetricType metric_type; // Тип метода для вычисления сходства векторов.
    idx_t ntotal; // Всего векторов в индексе.
    // Виртуальная функция для записи в индекс n векторов,
    // x - указатель на данные векторов.
    virtual void add(idx_t n, const float* x) = 0;
    // Виртуальная функция для записи в индекс n векторов по ID,
    // x - указатель на данные векторов, xids - массив соответствующих ID.
    virtual void add_with_ids(idx_t n, const float* x, const idx_t* xids);
};
```

```
// Виртуальная функция для выполнения запроса, находящего k ближайших
// векторов для n запрашиваемых векторов x.
// Результаты возвращаются в distances (расстояния) и labels
// (информация об ID векторов), params - необязательные параметры запроса.
virtual void search(
    idx_t n,
    const float* x,
    idx_t k,
    float* distances,
    idx_t* labels,
    const SearchParameters* params = nullptr) const = 0;
// Виртуальная функция для удаления векторов, удовлетворяющих условиям
// селектора ID sel, возвращает количество удаленных векторов.
virtual size_t remove_ids(const IDSelector& sel);
};
```

2. Основные функции

В табл. 4.1 приведены важные члены структуры `Index`, которые мы будем использовать при реализации основных функций плоского индекса на основе библиотеки FAISS.

Таблица 4.1. Важные члены структуры `Index`

Имя	Категория	Описание
<code>d</code>	Переменная	Размерность вектора
<code>metric_type</code>	Переменная	Тип метрики для сравнения сходства между векторами (например, евклидово расстояние, скалярное произведение)
<code>ntotal</code>	Переменная	Общее количество векторов, хранящихся в индексе
<code>add</code>	Функция	Запись n векторов, данные векторов передаются через <code>const float* x</code>
<code>add_with_ids</code>	Функция	Запись n векторов, данные векторов передаются через <code>const float* x</code> , дополнительно передается массив ID для n векторов через <code>const idx_t* xids</code>
<code>search</code>	Функция	Пакетный запрос n векторов, данные векторов передаются через <code>const float* x</code> , возвращает расстояния до k ближайших векторов (через <code>float* distances</code>) и массив ID (через <code>idx_t* labels</code>). В <code>SearchParameters</code> можно дополнительно передать функцию сравнения для фильтрации ID
<code>remove_ids</code>	Функция	Удаление векторов из индекса, возвращает количество удаленных векторов. В параметре <code>IDSelector</code> можно дополнительно передать функцию сравнения для сравнения ID при удалении

Изучив структуру `Index`, мы видим, что члены-переменные `d`, `metric_type` и `ntotal` хранят важные метаданные векторных данных. Члены-функции `add`

и `add_with_ids` используются для записи векторных данных внутрь структуры для хранения. Эти данные впоследствии используются для поиска ближайших соседей с помощью функции `search` или для удаления с помощью функции `remove_ids`. Функции `search` и `remove_ids` могут принимать дополнительные функции сравнения, которые можно понимать как фильтры на основе ID данных.

Также стоит отметить, что перечисленные здесь функции объявлены как виртуальные, и соответствующие подклассы могут перегружать свои функции в зависимости от конкретных сценариев. Изучая эти подклассы, мы можем глубже понять, как реализовать перегрузку функций в структуре `Index`.

3. Реализация функций

В табл. 4.2 приведены подструктуры `Index`, на которые следует обратить особое внимание при реализации функций плоского индекса.

Таблица 4.2. Подструктуры `Index`, на которые следует обратить особое внимание при реализации плоского индекса

Имя	Описание
<code>IndexFlatCodes</code>	Наследуется от <code>Index</code> , кодирует все векторы в структуру фиксированного размера, не предоставляет реализацию функции запроса
<code>IndexFlat</code>	Наследуется от <code>IndexFlatCodes</code> , реализует функцию <code>search</code> , используя алгоритм последовательного перебора всех векторов и сравнения
<code>IndexIDMap</code>	Наследуется от <code>Index</code> , предоставляет динамический массив <code>vector</code> , который хранит векторы и соответствующие им ID, облегчая последующие операции на основе ID

`IndexFlatCodes` и `IndexFlat` являются подструктурами, реализующими с помощью перегрузки конкретные функции родительской структуры `Index`, которая определяет общее направление и рамки. `IndexFlatCodes` выполняет функцию хранения векторных данных, каждый новый записанный вектор хранится в памяти последовательно в порядке записи. Ниже приведена его упрощенная реализация.

```
struct IndexFlatCodes : Index {
    size_t code_size; // Размер закодированных данных одного вектора в байтах.
    std::vector<uint8_t> codes; // Непрерывная область памяти для хранения
                               // данных векторов.
    // Перегрузка функции add из Index для записи n векторов в индекс.
    void add(idx_t n, const float* x) override;
    // Перегрузка функции remove_ids из Index для удаления указанных
    // векторов по селектору ID.
    size_t remove_ids(const IDSelector& sel) override;
};
```

Базовая реализация функции `add` также довольно проста: достаточно записать введенные пользователем векторные данные в `codes`, как показано ниже.

```
void IndexFlatCodes::add(idx_t n, const float* x) {
    // Изменение размера динамического массива codes для размещения
    // кодированных данных новых n векторов.
    codes.resize((ntotal + n) * code_size);
    // Кодирование новых n векторов с плавающей запятой x и сохранение
    // закодированных данных в codes, начиная с текущей позиции ntotal векторов.
    sa_encode(n, x, codes.data() + (ntotal * code_size));
    ntotal += n; // Обновление общего количества векторов в индексе.
}
```

Функция `sa_encode` по умолчанию реализована как простая функция `memcpy`, то есть по умолчанию функция `add` будет напрямую копировать векторные данные в соответствующую область памяти `codes`. Изучив функцию `add`, можно заметить, что в члене `codes` хранятся все векторные данные, общий размер которых составляет `ntotal * code_size`. `ntotal` обозначает количество векторов, уже хранящихся в `Index`, а `code_size` обозначает размер каждого вектора после кодирования. Таким образом, мы можем сделать вывод о том, как используется расположение в памяти члена `codes` при фактическом хранении векторных данных. На рис. 4.1 демонстрируется расположение в памяти члена `codes` при хранении векторных данных с размерностью 4 в виде чисел с плавающей запятой.

V1F1	V1F2	V1F3	V1F4
V2F1	V2F2	V2F3	V2F4
V3F1	V3F2	V3F3	V3F4
...	...	...	...

Рис. 4.1. Хранение векторов в плоском индексе: расположение в памяти члена `codes` (на примере векторов с плавающей запятой)

В этом фрагменте V1, V2 и V3 обозначают различные векторы, а F1, F2 и F3 представляют компоненты векторов в различных размерностях, расположенные в памяти, как показано на рис. 4.1. Обычно число с плавающей запятой одинарной точности занимает 4 байта, соответственно, один компонент вектора V1 занимает 4 байта. Компоненты от F1 до F4 в сумме составляют четыре компонента, что соответствует $4 \times 4 = 16$ байтам. Последующие векторы V2 и V3 также занимают по 16 байт, и все компоненты векторов хранятся рядом без дополнительных промежутков или разделителей. Такая организация позволяет более эффективно обращаться к вектору в памяти. В реальных приложениях `IndexFlatCodes` выступает в роли родительской структуры, реализуя часть функционала, главным образом управление пространством хранения векторных данных и реализацию функции `add`. Функция `search` реализуется в дочерних структурах. Например, структура `IndexFlat`, реализующая запрос с последовательным обходом, наследуется от `IndexFlatCodes` и не определяет дополнительных переменных-членов, а содержит перегрузку соответствующих функций-членов. Приведем упрощенный код реализации.

```

struct IndexFlat : IndexFlatCodes {
    // Перегрузка функции поиска для нахождения k ближайших векторов
    // к запрашиваемому вектору x в индексе.
    void search(
        idx_t n, // Количество запрашиваемых векторов.
        const float* x, // Указатель на массив запрашиваемых векторов.
        idx_t k, // Количество ближайших векторов, которые нужно найти.
        float* distances, // Массив расстояний до ближайших векторов.
        idx_t* labels, // Массив ID ближайших векторов.
        // Необязательные параметры для предоставления функции фильтрации
        // при запросе.
        const SearchParameters* params = nullptr) const override;
    ... // При необходимости здесь можно определить другие реализации
        // виртуальных функций.
};

```

Базовая реализация функции-члена `search` структуры `IndexFlat` представлена ниже.

```

void IndexFlat::search(
    idx_t n,
    const float* x,
    idx_t k,
    float* distances,
    idx_t* labels,
    const SearchParameters* params) const {
    // Если переданы SearchParameters, получить из них IDSelector,
    // иначе использовать nullptr.
    IDSelector* sel = params ? params->sel : nullptr;
    // Проверка, что k больше 0, если нет, то вызывается исключение.
    FAISS_THROW_IF_NOT(k > 0);
    // Выполнение различных операций запроса в зависимости от типа метрики.
    if (metric_type == METRIC_INNER_PRODUCT) {
        // При метрике скалярного произведения для хранения результатов
        // запроса используется минимальная куча.
        float_minheap_array_t res = {size_t(n), size_t(k), labels, distances};
        // Вызов функции запроса по скалярному произведению.
        knn_inner_product(x, get_xb(), d, n, ntotal, &res, sel);
    } else if (metric_type == METRIC_L2) {
        // При метрике L2 для хранения результатов запроса используется
        // максимальная куча.
        float_maxheap_array_t res = {size_t(n), size_t(k), labels, distances};
        // Вызов функции запроса по расстоянию L2.
        knn_L2sqg(x, get_xb(), d, n, ntotal, &res, nullptr, sel);
    } else {
        ... // Если тип метрики не относится к вышеуказанным,
            // здесь можно добавить обработку других типов метрик.
    }
}

```


Функция `search` сначала проверяет, является ли параметр `params` пустым. Если нет, то значение `params->sel` присваивается локальной переменной `sel`, в противном случае `sel` устанавливается в нулевой указатель. `sel` в итоге указывает на член `sel` в `params` или является нулевым указателем. `sel` используется для последующих функций сравнения, реализуя механизм фильтрации. Затем функция выбирает соответствующую функцию поиска ближайших соседей на основе указанного при создании индекса типа метрики и выполняет окончательный запрос, записывая результаты в `labels` и `distances`. В `labels` хранятся ID k ближайших соседей, а в `distances` – расстояния между этими соседями и вектором запроса.

Теперь перейдем к определению структуры `IndexIDMap` и разберемся, как она взаимодействует с `IndexFlat` для выполнения функций записи и запроса векторных данных. `IndexIDMap` реализуется с помощью шаблонных функций. Приведем упрощенное определение `IndexIDMap`.

```
struct IndexIDMapTemplate : IndexT {
    // Указатель на индекс родительского класса для базовых операций
    индексации.
    IndexT* index;
    // Динамический массив ID векторов для поддержания соответствия
    // внутреннему индексу.
    std::vector<idx_t> id_map;
    // Конструктор, принимающий указатель на индекс родительского класса.
    explicit IndexIDMapTemplate(IndexT* index);
    // Перегрузка функции add_with_ids родительского класса для записи
    // по ID в индекс.
    void add_with_ids(idx_t n, const component_t* x, const idx_t* xids) override;
    // Перегрузка функции search родительского класса, выполняет запрос
    // и возвращает информацию о k ближайших векторах к запрашиваемому.
    void search(
        idx_t n,
        const component_t* x,
        idx_t k,
        distance_t* distances,
        idx_t* labels,
        const SearchParameters* params = nullptr) const override;
    // Перегрузка функции remove_ids родительского класса, удаляет
    // указанные вектора по селектору ID.
    size_t remove_ids(const IDSelector& sel) override;
};
// Создание нового псевдонима типа IndexIDMap с использованием
IndexIDMapTemplate и Index.
using IndexIDMap = IndexIDMapTemplate<Index>;
```

Как видно, в `IndexIDMap` есть дополнительные переменные-члены `index` и `id_map`. Мы можем проанализировать конкретную реализацию функций-членов `add_with_ids` и `search`, чтобы понять, как функционируют эти переменные-члены. Реализация функции-члена `add_with_ids` представлена ниже.

```

template <typename IndexT>
void IndexIDMapTemplate<IndexT>::add_with_ids(
    idx_t n,
    const typename IndexT::component_t* x,
    const idx_t* xids) {
    // Вызов функции add родительского класса для записи векторов x в индекс.
    index->add(n, x);
    // Перебор всех векторов, которые нужно записать.
    for (idx_t i = 0; i < n; i++)
        // Запись ID каждого вектора в динамический массив id_map.
        id_map.push_back(xids[i]);
    // Обновление переменной ntotal текущего класса для соответствия общему
    // количеству векторов в индексе родительского класса.
    this->ntotal = index->ntotal;
}

```

Мы видим, что при выполнении операции записи вектора класс `IndexIDMap` вызывает функцию-член `add` переменной-члена `index` для хранения векторных данных в `index`, одновременно записывая ID векторов в `id_map` в порядке их записи. Этот порядок записи соответствует позиции записи вектора, то есть в дальнейшем можно получить ID вектора из `id_map` по его позиции записи.

`IndexIDMap` для выполнения запроса вызывает функцию `search`. Приведем упрощенный код реализации функции `search`.

```

template <typename IndexT>
void IndexIDMapTemplate<IndexT>::search(
    idx_t n,
    const typename IndexT::component_t* x,
    idx_t k,
    typename IndexT::distance_t* distances,
    idx_t* labels,
    const SearchParameters* params) const {
    index->search(n, x, k, distances, labels, params);
    // Перебор результатов запроса и преобразование меток в исходные ID
    // (если необходимо).
    idx_t* li = labels;
    for (idx_t i = 0; i < n * k; i++) {
        // Если метка отрицательная (возможно, потому что она отсутствует
        // в исходном наборе данных), возвращается метка,
        // в противном случае используется ID из id_map, соответствующий
        // этой метке.
        li[i] = li[i] < 0 ? li[i] : id_map[li[i]];
    }
}

```

`IndexIDMap` также вызывает функцию `search` переменной-члена `index` для выполнения окончательной операции запроса. Функция `search` возвращает данные меток (`labels`) ближайших соседей, которые соответствуют позиции хранения вектора в `id_map`. На основе этой позиции можно получить ID соответствующего вектора из `id_map` и вернуть его вызывающей стороне.

Мы изучили детали реализации четырех структур: `Index`, `IndexFlatCodes`, `IndexFlat` и `IndexIDMap`. В дальнейшем мы можем на их основе реализовать собственные функции записи и запроса в плоский индекс.

`IndexFlatCodes` отвечает за хранение векторов, `IndexFlat` – за поиск сходства векторов, а `IndexIDMap` – за отображение ID и позиции хранения векторов. В конечном итоге `IndexIDMap` используется как обертка для предоставления внешнего интерфейса. Следующий код можно использовать для инициализации объекта структуры FAISS, на его основе можно выполнять последующие операции записи и запроса векторных данных.

```
faiss::Index* index = new faiss::IndexIDMap(new faiss::IndexFlat(dim, faiss_
metric))
```

4.1.2. Реализация плоского индекса

Реализация плоского индекса

- └─ Класс `FaissIndex`
 - └─ Скрывает внутреннюю реализацию библиотеки FAISS
 - └─ Предоставляет простой интерфейс: `insert_vectors` и `search_vectors`
- └─ Класс `IndexFactory`
 - └─ Инициализирует индекс: `IndexType` и `MetricType`
 - └─ Получает индекс: возвращает соответствующий объект индекса по типу индекса
- └─ Класс `HttpServer`
 - └─ Реализует HTTP-сервис на основе `cpp-httplib`
 - └─ Обрабатывает запросы `/search` и `/insert`
- └─ Функция `main`
 - └─ Инициализирует журналирование
 - └─ Запускает `HttpServer` для прослушивания порта

На основе базовых возможностей, предоставляемых библиотекой FAISS, мы приступим к реализации автономной подсистемы записи и запроса векторов на основе плоского индекса. Чтобы в дальнейшем реализовать больше возможностей, при проектировании автономной версии мы стремимся приблизиться к требованиям, предъявляемым к предоставлению услуг во внешней среде, и максимально организовать модульную архитектуру. Общая концепция проектирования кода следует известному в экосистеме Linux принципу «предоставлять механизмы, а не стратегии», предлагая различные механизмы модульным способом для удобства постепенного дополнения функционала.

1. Класс `FaissIndex`

Начав с инкапсуляции библиотеки FAISS, определим собственный класс `FaissIndex`. Цель этого класса – максимально скрыть внутренние концепции FAISS, такие как `IndexIDMap` и `IndexFlat`, и предоставлять только операции, связанные с функциями плоского индекса. Это также способствует неинвазивному расширению функций плоского индекса в будущем. Определение класса `FaissIndex` представлено ниже.

```

class FaissIndex {
public:
    // Конструктор, принимающий указатель на объект индекса FAISS.
    FaissIndex(faiss::Index* index);
    // Публичная функция-член для записи в индекс данных векторов
    // и соответствующих меток.
    void insert_vectors(const std::vector<float>& data, uint64_t label);
    // Публичная функция-член для выполнения запроса в индексе
    // и нахождения k ближайших векторов к запрашиваемому.
    // Возвращает пару, содержащую два динамических массива: метки найденных
    // векторов и соответствующие расстояния.
    std::pair<std::vector<long>, std::vector<float>> search_vectors(const
std::vector<float>& query, int k);
private:
    // Приватная переменная-член, хранящая указатель на объект индекса FAISS.
    faiss::Index* index;
};

```

Класс `FaissIndex` инкапсулирует операции с объектом индекса FAISS. В классе содержатся функции-члены (конструктор, функция записи вектора и функция запроса вектора), а также приватная переменная-член для хранения объекта индекса FAISS. В дальнейшем мы будем выполнять операции с векторными данными на основе переменной-члена `faiss::Index* index`. Реализация функции-члена `insert_vectors` представлена ниже.

```

void FaissIndex::insert_vectors(const std::vector<float>& data, uint64_t label)
{
    // Преобразование метки в тип long для соответствия требованиям
    // к ID индекса FAISS.
    long id = static_cast<long>(label);
    // 1 обозначает запись одного вектора, data.data() предоставляет указатель
    // на данные вектора, &id предоставляет ID вектора.
    index->add_with_ids(1, data.data(), &id);
}

```

Функция `insert_vectors` принимает два параметра: `data` – это векторные данные, которые необходимо записать в плоский индекс, `label` – ID вектора, соответствующий `data`. Эта функция затем использует член `index`, чтобы вызвать метод `add_with_ids` с входными параметрами `data` и `label`, завершая фактическую операцию записи. В текущей версии за один раз записывается только одна запись.

Реализация метода `search_vectors` представлена ниже.

```

std::pair<std::vector<long>, std::vector<float>> FaissIndex::search_
vectors(const std::vector<float>& query, int k) {
    // Получение размерности вектора запроса из атрибута размерности индекса.
    int dim = index->d;
    // Вычисление количества векторов запроса путем деления длины динамического
    // массива на размерность вектора.
    int num_queries = query.size() / dim;
}

```

```

// Создание динамического массива для хранения всех результатов запроса,
// размер массива равен количеству векторов запроса, умноженному на k.
std::vector<long> indices(num_queries * k);
// Создание динамического массива для хранения всех расстояний результатов
// запроса, размер которого также равен количеству векторов запроса,
// умноженному на k.
std::vector<float> distances(num_queries * k);
// Выполнение операции запроса: передача количества векторов запроса,
// данных, значения k, указателей на результаты расстояний
// и идентификаторов векторов.
index->search(num_queries, query.data(), k, distances.data(), indices.data());
// Возврат объекта pair, содержащего идентификаторы ближайших соседей
// и соответствующие расстояния.
return {indices, distances};
}

```

Функция `search_vectors` принимает в качестве параметра `query` векторные данные, которые необходимо запросить. Затем она сначала получает через переменную-член `index` соответствующие параметры инициализации. Затем на основе входных параметров формирует параметры, необходимые для вызова функции запроса библиотеки FAISS (`index->search`), значение которых мы уже рассмотрели в разделе 4.1.1. В завершение вызывается функция запроса для выполнения запроса векторов, полученные результаты возвращаются вызывающему.

2. Класс `IndexFactory`

Мы определили класс `FaissIndex`, который уже можно использовать извне. Однако, учитывая, что в будущем в систему могут быть добавлены другие типы векторных индексов, мы создаем единый класс `IndexFactory` для управления различными типами векторных индексов в системе. Ниже представлена реализация класса `IndexFactory`.

```

class IndexFactory {
public:
    void init(IndexType type, int dim, MetricType metric = MetricType::L2);
    void* getIndex(IndexType type) const;
private:
    std::map<IndexType, void*> index_map;
};
IndexFactory* getGlobalIndexFactory();

```

Член `index_map` класса `IndexFactory` хранит векторные индексы, уже инициализированные в системе, и мы определяем его как `void*`, чтобы он был совместимым с возможными новыми типами индексов в будущем. Функция `getGlobalIndexFactory` – это глобальная функция, определенная в заголовочном файле, которая позволяет другим модулям системы получать объект класса `IndexFactory`, уникальный для системы.

Реализация функции `init`, определенной в `IndexFactory`, представлена ниже.

```

void IndexFactory::init(IndexType type, int dim, MetricType metric) {
    switch (type) {
        case IndexType::FLAT:
            index_map[type] = new FaissIndex(
                new faiss::IndexIDMap(
                    new faiss::IndexFlat(dim, faiss_metric)
                )
            );
            break;
        default:
            break;
    }
}

```

Функция `init` является функцией-членом класса `IndexFactory` и используется для инициализации фабрики индексов. Она определяет, какой тип индекса создавать, в зависимости от значения параметра `type`. Здесь обрабатывается только случай `IndexType::FLAT`, соответствующий плоскому индексу. Особое внимание следует обратить на то, что этот плоский индекс инициализируется с использованием `faiss::IndexIDMap`, чтобы можно было хранить ID каждого вектора. Инициализированный объект индекса хранится в экземпляре класса `FaissIndex`, а указатель на экземпляр `FaissIndex` хранится в `index_map`, чтобы его можно было получить позже через функцию `getIndex`. Реализация функции `getIndex` представлена ниже.

```

void* IndexFactory::getIndex(IndexType type) const {
    // Использование функции find для поиска объекта индекса с ключом type
    // в таблице отображения индексов index_map.
    auto it = index_map.find(type);
    if (it != index_map.end()) {
        // Если найден объект индекса, возвращается указатель на этот объект.
        return it->second;
    }
    // Если объект индекса не найден, возвращается пустой указатель nullptr.
    return nullptr;
}

```

Функция `getIndex` возвращает соответствующий объект индекса, основываясь на переданном в вызове типе `type`. Следует отметить, что, поскольку возвращается тип `void*`, вызывающий должен преобразовать его в фактический объект в зависимости от значения `indexType`. В настоящее время система поддерживает только объекты `FaissIndex`, и текущий способ реализации обеспечит удобство для расширения системы в будущем.

Мы объявляем глобальный объект фабрики индексов `globalIndexFactory` в анонимном пространстве имен и получаем его указатель через функцию `getGlobalIndexFactory`, тем самым реализуя глобально уникальную одиночную фабрику.

```

namespace {
    IndexFactory globalIndexFactory;
}

```

```
IndexFactory* getGlobalIndexFactory() {
    return &globalIndexFactory;
}
```

Любой, кто получает доступ к этой фабрике, может использовать соответствующий объект индекса. Это сочетание одиночного и фабричного методов часто применяется в программировании и обладает хорошей инкапсуляцией и расширяемостью. Добавление новых типов индексов в `IndexFactory` становится весьма удобным. Однако при использовании этого шаблона одиночки в многопоточной среде необходимо учитывать использование мьютексов для предотвращения проблем многопоточного доступа. Текущая реализация упрощает обработку многопоточности.

3. Класс `HttpServer`

Далее, чтобы наш класс `FaissIndex` мог предоставлять сервисы, необходимо выбрать протокол для внешнего взаимодействия. Мы будем использовать для этого протокол HTTP, так как он обладает удобством использования и простотой отладки. Вызывающий может выполнять запись и запросы векторных данных через стандартный протокол HTTP.

Для реализации этого функционала определим класс `HttpServer`, реализующий прослушивание HTTP-сервиса и обработку соответствующих запросов. Эта функция реализована на основе открытой библиотеки `cpp-httplib`, которая очень легка и проста в использовании, требуется лишь подключение файла `httplib.h`. Кроме того, для удобства обработки данных мы будем использовать библиотеку `RapidJSON` для разбора команд, чтобы вызывающий мог взаимодействовать с сервером в формате JSON.

При запуске сервис `HttpServer` регистрирует функции обработки для двух HTTP-путей: `/insert` и `/search`. Когда на сервер поступает запрос пользователя, выполняются функции `insertHandler` и `searchHandler`. Пример кода представлен ниже.

```
// Конструктор класса HttpServer, используется для инициализации HTTP-сервера.
HttpServer::HttpServer(const std::string& host, int port)
    : host(host), port(port) {
    // Создание объекта HTTP-сервера с использованием библиотеки cpp-httplib
    // и установка адреса хоста и номера порта для прослушивания.
    // Когда путь HTTP-запроса равен /search, вызывается обработчик searchHandler.
    server.Post("/search",
        [this](const httplib::Request& req, httplib::Response& res) {
            // Вызов функции-члена searchHandler для обработки
            // запроса на поиск.
            // searchHandler(req, res);
        });
    // Когда путь HTTP-запроса равен /insert, вызывается обработчик insertHandler.
    server.Post("/insert",
        [this](const httplib::Request& req, httplib::Response& res) {
            // Вызов функции-члена insertHandler для обработки
            // запроса на запись.
            insertHandler(req, res);
        });
}
```

Далее приведен упрощенный код реализации обработчика `insertHandler`.

```
// Функция-член класса HttpServer, используется для обработки операции записи
// в HTTP-запросе.
void HttpServer::insertHandler(const httplib::Request& req,
                               httplib::Response& res) {
    // Извлечение и разбор типа индекса из HTTP-запроса.
    IndexFactory::IndexType indexType = getIndexTypeFromRequest(json_request);

    // Получение объекта индекса соответствующего типа через глобальную
    // фабрику индексов.
    void* index = getGlobalIndexFactory()->getIndex(indexType);

    // Преобразование указателя на полученный объект индекса в указатель
    // типа FaissIndex.
    FaissIndex* faissIndex = static_cast<FaissIndex*>(index);

    // Использование функции insert_vectors объекта FaissIndex для записи
    // данных вектора, data - данные вектора, label - метка.
    faissIndex->insert_vectors(data, label);

    // Установка содержимого ответа HTTP в формате JSON,
    // json_response - объект json, содержащий данные ответа,
    // res - объект HTTP-ответа.
    setJsonResponse(json_response, res);
}
```

Функция `insertHandler` извлекает параметры, необходимые для записи данных, включая векторные данные и соответствующую информацию об ID, из запроса пользователя. Затем она находит соответствующий индекс в глобальной фабрике индексов и вызывает функцию `insert_vectors` для записи векторных данных.

Ниже приведен упрощенный код обработчика `searchHandler` (часть кода аналогична `insertHandler`).

```
void HttpServer::searchHandler(const httplib::Request& req,
                               httplib::Response& res) {
    IndexFactory::IndexType indexType = getIndexTypeFromRequest(json_request);
    void* index = getGlobalIndexFactory()->getIndex(indexType);
    FaissIndex* faissIndex = static_cast<FaissIndex*>(index);

    // Использование функции search_vectors объекта FaissIndex для выполнения
    // запроса вектора, query - вектор запроса, k - количество
    // результатов запроса.
    results = faissIndex->search_vectors(query, k);

    setJsonResponse(json_response, res);
}
```


Функция `searchHandler` извлекает параметры, необходимые для выполнения запроса, включая вектор для поиска и значение `k`, из запроса пользователя. Затем она находит соответствующий объект индекса в глобальной фабрике индексов по типу индекса и выполняет поиск `k` ближайших соседей для векторных данных.

Выше представлены только основные фрагменты кода двух функций, их полная реализация также включает дополнительные проверки параметров, обработку исключений и другую логику. Для упрощения понимания эти аспекты здесь опущены. Однако в реальном применении важно учитывать исключительные ситуации, чтобы система могла справляться с различными аномалиями.

4. Функция `main`

После реализации класса `HttpServer` нужно определить точку входа программы – функцию `main`, которая отвечает за инициализацию системы и содержит вспомогательный код. Инициализация системы включает инициализацию журналирования и фабрики индексов. Система журналирования помогает фиксировать ключевую информацию, а фабрика индексов упрощает управление объектами индексов в системе.

```
int main() {
    init_global_logger();
    set_log_level(spdlog::level::debug);
    GlobalLogger->info("Global logger initialized");
    IndexFactory* globalIndexFactory = getGlobalIndexFactory();
    globalIndexFactory->init(IndexFactory::IndexType::FLAT, dim);
    GlobalLogger->info("Global IndexFactory initialized");
    HttpServer server("localhost", 8080);
    GlobalLogger->info("HttpServer created");
    server.start();
    return 0;
}
```

Ниже приведено объяснение основных моментов этого кода.

- Инициализация глобальной системы журналирования и установка уровня логирования на `debug` для записи подробной информации.
- Получение экземпляра глобальной фабрики индексов с помощью функции `getGlobalIndexFactory` и инициализация индекса типа FLAT (плоский), где `dim` обозначает размерность вектора.
- Создание экземпляра `HttpServer`, прослушивающего порт 8080 на локальном компьютере ("localhost").
- Запуск `HttpServer` для начала приема и обработки HTTP-запросов.
- Нормальное завершение работы программы и возврат значения 0.

Для удобства отладки системы мы используем `spdlog` – легковесную библиотеку журналирования на C++, которая позволяет постоянно отслеживать и диагностировать состояние системы.

Ниже приведены примеры запросов и ответов при тестировании команд доступа.

Запрос и ответ на запись вектора

Запрос:

```
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.8], "id": 2, "indexType": "FLAT"}' http://localhost:8080/insert
```

Ответ:

```
{"retCode":0}
```

Запрос и ответ на поиск вектора

Запрос:

```
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.5], "k": 2, "indexType": "FLAT"}' http://localhost:8080/search
```

Ответ:

```
{"vectors": [2], "distances": [0.09000000357627869], "retCode": 0}
```

Ошибочный запрос и ответ

Запрос:

```
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.5], "k": 2, "indexType": "FLAT1"}' http://localhost:8080/search
```

Ответ:

```
{"retCode": -1, "errorMsg": "Invalid indexType parameter in the request"}
```

Поскольку мы с самого начала внедрили систему журналирования, мы можем в любой момент просматривать журналы в стандартизированном формате (как показано ниже), что способствует эффективной эксплуатации всей системы. Это позволяет системному администратору быстро локализовать и проанализировать проблему, если запросы и ответы не соответствуют ожиданиям.

```
[2023-10-06 00:18:23.909] [GlobalLogger] [info] Global logger initialized
[2023-10-06 00:18:30.017] [GlobalLogger] [info] Insert request parameters:
{"vectors": [0.8], "id": 2, "indexType": "FLAT"}
[2023-10-06 00:18:56.844] [GlobalLogger] [info] Search request parameters:
{"vectors": [0.5], "k": 2, "indexType": "FLAT"}
[2023-10-06 00:21:52.551] [GlobalLogger] [error] Invalid indexType parameter in the request
```

► Начальная версия v0.0.1

На текущий момент мы выполнили разработку векторной базы данных с базовыми функциями записи и запроса векторных данных, обозначим эту версию как v0.0.1. Это первый шаг на долгом пути, и мы должны всегда помнить, что хорошая архитектура и модульный дизайн создают прочную основу для последующего расширения.

В табл. 4.3 приведен перечень модулей и функций, добавленных в версии v0.0.1.

Таблица 4.3. Модули и функции, добавленные в версии v0.0.1

Название модуля	Задействованные файлы	Описание
<code>FaissIndex</code>	<code>faiss_index.h</code> , <code>faiss_index.cpp</code>	Инкапсуляция объекта плоского индекса в системе, скрывающая детали реализации FAISS
<code>IndexFactory</code>	<code>index_factory.h</code> , <code>index_factory.cpp</code>	Класс фабрики индексов векторов, осуществляющий генерацию и получение векторных индексов, поддерживающий плоский индекс
<code>HttpServer</code>	<code>http_server.h</code> , <code>http_server.cpp</code>	Реализация на основе <code>cpp-httplib</code> , разбирает HTTP-запросы, вызывает соответствующий индекс для выполнения конкретных операций, поддерживает команды <code>/insert</code> и <code>/search</code>
<code>GlobalLogger</code>	<code>logger.h</code> , <code>logger.cpp</code>	Реализация на основе <code>spdlog</code> , обеспечивает возможность ведения журналов, повышая способность системы к непрерывной эксплуатации
<code>VDBServer</code>	<code>vdb_server.cpp</code>	Реализация функции <code>main</code> , точка инициализации и запуска сервиса

4.1.3. Основные функции библиотеки HNSWLib

Основные функции библиотеки HNSWLib

- ├ Структуры данных
 - ├ Класс `AlgorithmInterface`
 - └ Виртуальные функции `addPoint` и `searchKnn`
 - └ Класс `HierarchicalNSW`
 - └ Наследуется от `AlgorithmInterface`
 - └ Члены (`max_elements`, `maxM` и т.д.)
 - └ Функции-члены (`addPoint` и `searchKnn`)
- ├ Расположение в памяти
 - ├ Общая память нулевого уровня
 - ├ Расположение в памяти индексов соседей для ненулевых уровней
 - └ Анализ затрат памяти
- └ Реализация функций
 - ├ Функция инициализации: выделение памяти, инициализация структуры данных
 - ├ Функция `addPoint`: логика записи векторных данных
 - └ Функция `searchKnn`: логика запроса векторных данных

На основе FAISS мы создали векторную базу данных версии v0.0.1, поддерживающую функции записи и запроса. Однако v0.0.1 реализует только плоский индекс. Такой метод запроса гарантирует 100%-ную полноту запроса и достигает высокой эффективности запросов в условиях небольших объемов данных, но при увеличении объема данных задержка доступа быстро возрастает. В дальнейшем мы планируем реализовать HNSW-индекс. HNSW – это много-слойный графовый алгоритм индексации, который обеспечивает лучший баланс между эффективностью запроса и полнотой, поддерживая эффективное чтение и запись больших объемов векторных данных.

Перед реализацией HNSW мы изучим основные функции библиотеки HNSWLib, специально предназначенной для крупномасштабного приближенного поиска ближайших соседей, которая предоставляет эффективный метод запросов через алгоритм HNSW. Хотя этот метод слегка теряет точность полностью, он хорошо подходит для сценариев, требующих обработки большого количества векторных данных и высококонкурентных запросов.

Стандартный исходный код HNSWLib можно получить с GitHub с помощью следующей команды.

```
git clone https://github.com/nmslib/hnswlib hnswlib
```

Использовать HNSWLib довольно просто, она не требует дополнительной компиляции, достаточно просто включить соответствующие файлы в проект. Далее мы изучим важные части исходного кода HNSWLib, чтобы подготовиться к написанию собственной высокопроизводительной подсистемы для записи и запроса векторов.

1. Структуры данных

Класс `AlgorithmInterface` является самой верхней структурой данных, он определяет стандартный интерфейс доступа к классам индексов HNSWLib. Упрощенное определение класса `AlgorithmInterface` приведено ниже.

```
template<typename dist_t>
class AlgorithmInterface {
public:
    virtual void addPoint(
        const void* datapoint, labeltype label,
        bool replace_deleted = false
    ) = 0;

    virtual std::priority_queue<std::pair<dist_t, labeltype>>
    searchKnn(
        const void* query_data, size_t k,
        BaseFilterFunctor* isIdAllowed = nullptr
    ) const = 0;
};
```

Ниже приведено описание основных функций кода.

- `AlgorithmInterface` – это шаблонный класс, использующий параметр шаблона `dist_t` для обозначения типа расстояния.
- В классе определены две виртуальные функции, и любой класс-наследник от `AlgorithmInterface` должен предоставить конкретную реализацию этих функций. `HierarchicalNSW` – это подкласс, который мы будем использовать в дальнейшем.
- Функция `addPoint`:
 - ◆ назначение: записывает новые векторные данные в структуру индекса;
 - ◆ параметры:

- ◊ `const void* datapoint`: указатель на векторные данные;
 - ◊ `labeltype label`: метка векторных данных, обычно хранит ID вектора;
 - ◊ `bool replace_deleted`: булево значение, указывающее, следует ли заменять удаленные векторные данные, по умолчанию `false`.
- Функция `searchKnn`:
- ◆ назначение: выполняет поиск ближайших соседей, возвращает метки и расстояния k векторных данных, ближайших к точке запроса;
 - ◆ параметры:
 - ◊ `const void* query_data`: указатель на данные вектора запроса;
 - ◊ `size_t k`: количество ближайших соседей, которые нужно найти;
 - ◊ `BaseFilterFuncor* isIdAllowed`: указатель на функцию фильтрации (необязательный), используемый для определения, какие идентификаторы разрешены для запроса; по умолчанию равен `nullptr` (означает отсутствие фильтрации);
 - ◆ возвращаемое значение: очередь с приоритетом, элементы которой представляют собой пары «расстояние–метка», отсортированные по возрастанию расстояния.

Ключевая функция класса `AlgorithmInterface` заключается в предоставлении унифицированного интерфейса для реализации алгоритма индексации многослойного графа векторов. Этот интерфейс позволяет различным алгоритмам обрабатывать запись и запрос векторных данных согласованным образом, сохраняя при этом гибкость в реализации конкретных деталей алгоритма.

В табл. 4.4 приведена информация по двум важным функциям-членам класса `AlgorithmInterface`.

Таблица 4.4. Важные функции-члены класса `AlgorithmInterface`

Название	Категория	Описание
<code>addPoint</code>	Функция	Запись вектора, векторные данные передаются через <code>const void* datapoint</code> , дополнительно через <code>label</code> передается идентификатор вектора, <code>replace_deleted</code> поддерживает замену ранее помеченного для удаления вектора новым записанным вектором
<code>searchKnn</code>	Функция	Запрос вектора, векторные данные передаются через <code>const void* query_data</code> , запрос ближайших k векторов, <code>std::priority_queue<std::pair<dist_t, labeltype>></code> возвращает массив ID k результатов, <code>BaseFilterFuncor* isIdAllowed</code> может дополнительно передать функцию сравнения для фильтрации ID при запросе

Класс `HierarchicalNSW` наследуется от шаблонного класса `AlgorithmInterface` и реализует алгоритм послойного поиска ближайших соседей, который может эффективно обрабатывать задачи поиска сходства в больших наборах данных. Ниже представлена его базовая реализация.

```

template<typename dist_t>
class HierarchicalNSW : public AlgorithmInterface<dist_t> {
public:
    size_t max_elements_{0};
    size_t maxM_{0};
    size_t maxM0_{0};
    size_t ef_construction_{0};
    size_t ef_{0};
    int maxlevel_{0};
    tableint enterpoint_node_{0};
    char *data_level0_memory_{nullptr};
    char **linkLists_{nullptr};
    std::vector<int> element_levels_;
    std::unordered_map<labeltype, tableint> label_lookup_;

    void addPoint(
        const void* data_point, labeltype label,
        bool replace_deleted = false
    ) override;

    std::priority_queue<std::pair<dist_t, labeltype>>
    searchKnn(const void* query_data, size_t k,
        BaseFilterFunctor* isIdAllowed = nullptr)
        const override;
}

```

В классе `HierarchicalNSW` задействовано множество переменных и функций-членов, в табл. 4.5 приведена сводная информация по важным переменным и функциям-членам.

Таблица 4.5. Важные переменные и функции класса `HierarchicalNSW`

Название	Категория	Описание
<code>max_elements_</code>	Переменная	Максимальное количество векторов в индексе
<code>maxM_</code>	Переменная	Максимальное количество ближайших соседей для узла индекса
<code>maxM0_</code>	Переменная	Максимальное количество ближайших соседей для векторов на уровне 0
<code>ef_construction_</code>	Переменная	Максимальное количество кандидатов на ближайших соседей при построении
<code>ef_</code>	Переменная	Максимальное количество кандидатов на ближайших соседей при запросе k ближайших соседей
<code>maxlevel_</code>	Переменная	Максимальное количество уровней в текущей структуре индекса
<code>enterpoint_node_</code>	Переменная	Номер узла входа, используемый при запросе на уровне, отличном от 0

Окончание табл. 4.5

Название	Категория	Описание
<code>data_level0_memory_</code>	Переменная	Указатель типа <code>char*</code> , указывающий на начало памяти индекса на уровне 0, где хранятся все данные уровня 0
<code>linkLists_</code>	Переменная	Указатель типа <code>char**</code> , хранящий индексы ближайших соседей для каждого узла на уровнях, отличных от 0
<code>element_levels_</code>	Переменная	Вектор типа <code>std::vector<int></code> , отображающий номер узла в номер уровня, на котором находится узел
<code>addPoint</code>	Функция	Через <code>const void* data_point</code> передаются векторные данные, через <code>labeltype label</code> передается ID узла, реализует функцию записи вектора в индекс
<code>searchKnn</code>	Функция	Через <code>const void* query_data</code> передаются данные запроса вектора, через <code>size_t k</code> передается количество ближайших соседей, через <code>BaseFilterFuncor* isIdAllowed</code> можно дополнительно передать функцию сравнения для фильтрации идентификаторов при запросе

2. Расположение в памяти

Для более полного понимания реализации функций-членов `addPoint` и `searchKnn` необходимо подробно проанализировать, как класс `HierarchicalNSW` организует хранение всех векторных данных в памяти. Наиболее важными членами являются `data_level0_memory_` и `linkLists_`. `data_level0_memory_` – это указатель на начальный адрес памяти (то есть входной адрес хранения всех векторных данных уровня 0). Смещение на позицию определенного вектора на этом адресе указателя позволяет получить его местоположение в памяти (таким образом, можно получить исходные данные вектора и информацию об индексе ближайших соседей каждого вектора на уровне 0). `linkLists` хранит информацию об индексе ближайших соседей всех векторов на каждом уровне, кроме уровня 0. На рис. 4.2 и 4.3 демонстрируется для класса `HierarchicalNSW` структура расположения в памяти данных векторов на уровне 0 и расположение в памяти ближайших соседей на уровнях, отличных от 0.

Каждый элемент вектора имеет две основные области памяти. Первая область обусловлена исходными данными вектора и информацией об индексе ближайших соседей в пространстве уровня 0 (см. рис. 4.2). Эта часть памяти состоит из следующих трех блоков.

Первый блок памяти хранит метаданные вектора:

размер (`size`) + флаг (`flag`) + зарезервировано (`reserved`) + соседи (`neighbors`).

- Размер указывает на общую величину блока памяти, занимаемого элементами вектора с данным номером.
- Флаг используется для хранения информации о конфигурации, такой как метки и удаление векторных данных.
- Зарезервированное поле облегчает последующее расширение.
- Массив ближайших соседей хранит ID `maxM0` ближайших векторов к текущему вектору (ID одного вектора занимает 4 байта, `maxM0` можно задать через параметры инициализации).

Вектор № 0						Вектор № 1						
Размер (2 байта)	Бит флага (1 байт)	Заре- зерви- ровано (1 байт)	Массив ближай- ших соседей	Данные вектора	ID вектора	Размер (2 байта)	Бит флага (1 байт)	Заре- зерви- ровано (2 байта)	Массив ближай- ших соседей	Данные вектора	ID вектора	

Рис. 4.2. Расположение в памяти данных векторов на уровне 0 в классе `HierarchicalNSW`

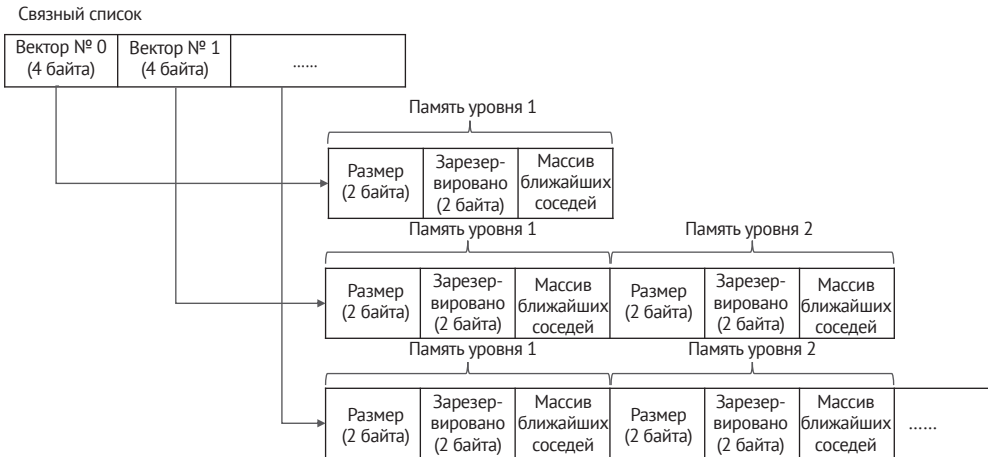


Рис. 4.3 Расположение в памяти ближайших соседей на уровнях, отличных от 0, в классе `HierarchicalNSW`

Общий размер первой части памяти составляет:

$$4 + \text{maxM0} \times 4 \text{ байт.}$$

Вторая часть памяти хранит исходные данные вектора, выполняя функцию фактического хранения пользовательских векторных данных, соответствует блоку данных вектора (`data`) на рис. 4.2, размер составляет:

$$\text{размерность вектора} \times \text{байт на измерение.}$$

Третья часть памяти содержит метку ID вектора, соответствует блоку ID вектора (`label`) на рис. 4.2. В HNSWLib для хранения метки ID используется 4-байтовое целое число, поэтому размер блока ID вектора составляет 4 байта.

Общая память первой части составляет:

$$(4 + \text{maxM0} \times 4) + (\text{размерность вектора} \times \text{байт на измерение}) + 4 \text{ байт.}$$

Второй блок памяти обусловлен тем, что каждому элементу вектора в системе требуется отдельное пространство для хранения информации о соседях на ненулевом уровне (см. рис. 4.3).

Каждый элемент в списке (`linkLists`) на рис. 4.3 соответствует информации о всех соседних векторах на ненулевом уровне для вектора с данным номером. Например, если вектор был выбран для записи на уровне N , его номер будет 0, и указатель на 0-ю позицию списка указывает на место в памяти, которое тре-

буется для этого вектора. Чтобы вектор мог находить информацию о соседях на каждом уровне сверху вниз, это пространство должно хранить информацию о ближайших соседях вектора на каждом уровне от 1 до N . Для хранения этой информации на каждом уровне предусмотрено 4 байта для описания заголовка, включающего размер блока соседей и зарезервированное поле. Кроме того, необходимо хранить данные ID maxM соседних векторов. ID каждого вектора хранится в 4-байтовом целочисленном поле, поэтому размер памяти для хранения информации о соседних векторах на уровне составляет: $4 + \text{maxM} \times 4$ байт. Если вектор был размещен на уровне N при записи, то требуется $N \times (4 + \text{maxM} \times 4)$ байт памяти. Если начальный адрес указателя в списке для этого вектора занимает 4 байта, то общий размер составляет:

$$4 + N \times (4 + \text{maxM} \times 4) \text{ байт,}$$

где maxM можно задать при инициализации структуры HNSW-индекса.

В стандартной реализации алгоритма HNSW, чтобы улучшить эффективность запросов, на уровне 0 будет $\text{maxM} \times 2$ ближайших соседей. Предположим, что наше значение M равно 10, размерность вектора равна 1, он хранится на уровне 4, и размерность использует формат с одинарной точностью (то есть каждый элемент размерности занимает 4 байта), тогда размер первого блока памяти составит:

$$(4 + 10 \times 2 \times 4) + (1 \times 4) + 4 = 92 \text{ байта.}$$

Размер второго блока памяти составит:

$$4 + 4 \times (4 + 10 \times 4) = 180 \text{ байт.}$$

Оба блока занимают в общей сложности $92 + 180 = 272$ байта.

Из этих расчетов видно, что **HierarchicalNSW** использует значительное количество памяти для хранения каждого узла и информации о его ближайших соседях. Причем память для хранения информации о соседях превышает часть с исходными векторами. Этот подход подходит для управления большими объемами векторов с высокой размерностью, так как доля памяти, используемой для хранения данных о соседях, будет меньше, и, соответственно, будет меньше потеря памяти.

3. Реализация функций

Изучив структуру памяти, далее мы рассмотрим, как параметры и память организованы в функции инициализации. Приведем важный код в функции инициализации **HierarchicalNSW**.

```
size_links_level0_ = maxM0_ * sizeof(tableint) + sizeof(linklistsizeint);
size_data_per_element_ = size_links_level0_ + data_size_ + sizeof(labeltype);
data_level0_memory_ = (char *) malloc(max_elements_ * size_data_per_element_);
linkLists_ = (char **) malloc(sizeof(void*) * max_elements_);
```

В соответствии с логикой инициализации `HierarchicalNSW` видно, что `HierarchicalNSW` инициализирует память в зависимости от максимального количества векторов, которое может вместить система. `data_level0_memory_` указывает на память, необходимую для хранения данных векторов на уровне 0, а `linkLists_` указывает на память для хранения информации о ближайших соседях векторов на уровнях, отличных от 0. В текущей версии библиотеки HNSWLib акцент сделан на эффективность запросов и стабильность системы, и вся память организована в виде массива, что позволяет быстро находить данные вектора по его номеру. Однако такая организация памяти затрудняет ее освобождение и перемещение данных. Поэтому в текущей версии HNSWLib основная память выделяется статически и одновременно при запуске системы, а последующие операции удаления используют метод маркировки, не освобождая память фактически. Это типичный пример подхода «использования пространства для экономии времени».

После изучения кода инициализации памяти класса `HierarchicalNSW` становится понятной логика работы функций `addPoint` и `searchKnn` становится относительно простым.

Упрощенная реализация функции `addPoint` представлена ниже.

```
addPoint(const void* data_point, labeltype label) {
    // Инициализация номера позиции вектора.
    tableint cur_c = cur_element_count;
    cur_element_count++;
    label_lookup_[label] = cur_c;

    int curlevel = getRandomLevel(mult_);
    element_levels_[cur_c] = curlevel;
    tableint currObj = enterpoint_node_;

    // Инициализация информации памяти уровня 0.
    memset(data_level0_memory_ + cur_c * size_data_per_element_ + offsetLevel0_,
           0, size_data_per_element_);
    memcpy(getExternalLabelP(cur_c), &label, sizeof(labeltype));
    memcpy(getDataByInternalId(cur_c), data_point, data_size_);

    // Инициализация информации памяти уровня, отличного от 0.
    linkLists_[cur_c] = (char *) malloc(size_links_per_element_ * curlevel + 1);
    memset(linkLists_[cur_c], 0, size_links_per_element_ * curlevel + 1);

    // Обход от текущей точки входа, выбор M ближайших векторов на каждом уровне,
    // затем обновление.
    for (int level = std::min(curlevel, maxlevelcopy); level >= 0; level--) {
        std::priority_queue<std::pair<dist_t, tableint>,
                           std::vector<std::pair<dist_t, tableint>>,
                           CompareByFirst> top_candidates =
            searchBaseLayer(currObj, data_point, level);

        currObj = mutuallyConnectNewElement(data_point, cur_c,
                                             top_candidates, level, false);
    }
}
```

```

// Обновление максимального уровня в соответствии с текущим уровнем.
if (curlevel > maxlevelcopy) {
    enterpoint_node_ = cur_c;
    maxlevel_ = curlevel;
}
}

```

Эта часть кода предназначена для записи новых векторных данных в структуру данных HNSW-индекса.

Сначала функция выделяет уникальный номер позиции для новых векторных данных и сохраняет его вместе с меткой (фактически ID вектора) в таблице поиска меток. Затем на основе коэффициента умножения текущего уровня случайным образом определяется уровень новых векторных данных, и эта информация записывается в массив уровней. Далее функция инициализирует информацию о памяти уровня 0, выделяет пространство для новых векторных данных и копирует их метку и содержимое данных. Для уровней, отличных от 0, функция также выделяет и инициализирует память.

Затем функция, проходя по каждому уровню, использует функцию `searchBaseLayer` для запроса и получения верхних кандидатов, и с помощью функции `mutuallyConnectNewElement` соединяет новый векторный элемент с этими кандидатами. Этот процесс является ядром построения структуры данных HNSW-индекса, обеспечивая эффективное соединение между векторными данными.

Наконец, если уровень новых векторных данных превышает текущий максимальный уровень, функция обновляет максимальный уровень и входной узел, чтобы последующие векторные данные могли записываться и запрашиваться на более глубоких уровнях. Такая архитектура позволяет структуре данных HNSW динамически адаптироваться к распределению и количеству векторных данных, эффективно поддерживая поиск ближайших соседей в больших наборах данных. На этом запись векторных данных завершается.

Далее мы подробно изучим реализацию функции `searchKnn`.

```

std::priority_queue<std::pair<dist_t, labeltype >> searchKnn(
    const void* query_data,
    size_t k,
    BaseFilterFuncor* isIdAllowed = nullptr) const
{
    std::priority_queue<std::pair<dist_t, labeltype >> result;
    std::priority_queue<std::pair<dist_t, tableint>,
                      std::vector<std::pair<dist_t, tableint>>,
                      CompareByFirst> top_candidates;

    top_candidates = searchBaseLayerST<false, true>(
        currObj, query_data, std::max(ef_, k), isIdAllowed);

    while (top_candidates.size() > k) {
        top_candidates.pop();
    }
}

```

```

while (top_candidates.size() > 0) {
    std::pair<dist_t, tableint> rez = top_candidates.top();
    result.push(std::pair<dist_t, labeltype>(
        rez.first, getExternalLabel(rez.second)));
    top_candidates.pop();
}

return result;
}

```

Опишем основную логику функции `searchKnn`.

Сначала `searchKnn` создает две очереди с приоритетом: `result` для хранения окончательных результатов запроса и `top_candidates` для хранения кандидатов в процессе запроса.

Затем вызывается функция `searchBaseLayerST`, которая является вспомогательной функцией запроса, используемой для получения кандидатов на определенном уровне через запрос структуры данных HNSW-индекса. Запрос начинается с входного вектора. Сначала находятся несколько ближайших кандидатов, затем из соседей этих кандидатов ищется более близкий вектор и т. д., и применяется жадный алгоритм для нахождения окончательных кандидатов. Параметры запроса включают текущий входной узел `currObj` и вектор запроса `query_data`, количество кандидатов для запроса – это большее значение из `ef` и `k`, что позволяет обеспечить достаточное количество кандидатов для запроса. Если предоставлена функция фильтрации `isIdAllowed`, она будет использоваться для определения, узлы с какими определенными ID разрешены для запроса.

После выполнения запроса, если количество кандидатов превышает необходимое количество ближайших соседей `k`, лишние кандидаты удаляются. Затем функция преобразует кандидатов в очередь результатов, извлекая ID соответствующих узлов с помощью функции `getExternalLabel` и помещая их вместе с расстоянием в очередь `result`.

В конце функция возвращает очередь результатов, содержащую `k` ближайших соседей.

4.1.4. Реализация HNSW-индекса

Реализация HNSW-индекса

- └─ Класс `HNSWLibIndex`
 - └─ Конструктор: `dim`, `int num_data`, `MetricType` и т.д.
- └─ Функции `insert_vectors` и `search_vectors`
 - └─ `insert_vectors`
 - └─ Вызов `index->addPoint` для записи векторов
 - └─ `search_vectors`
 - └─ Установка параметра `ef_search`
 - └─ Вызов `index->searchKnn` для выполнения запроса
 - └─ Обработка и возврат результатов
- └─ Функция `insertHandler`
 - └─ Шаг 1: Интеграция в `IndexFactory`
 - └─ Шаг 2: Распознавание операций HNSW в `HttpServer`
 - └─ Шаг 3: Инициализация `HNSWLibIndex` в `vdb_server`

После изучения основных функций библиотеки HNSWLib приступим к построению возможностей записи и запросов HNSW-индекса в векторной базе данных на его основе.

1. Класс HNSWLibIndex

На основе системы и модулей, построенных в разделе 4.1.2, сначала определим класс `HNSWLibIndex`, как показано ниже.

```
class HNSWLibIndex {
public:
    HNSWLibIndex(int dim, int num_data,
                  IndexFactory::MetricType metric,
                  int M = 16, int ef_construction = 200); // Конструктор.
    void insert_vectors(const std::vector<float>& data, uint64_t label);
                                                    // Функция записи вектора.
    std::pair<std::vector<long>, std::vector<float>>
        search_vectors(const std::vector<float>& query,
                      int k, int ef_search = 50); // Функция запроса вектора.
private:
    int dim;
    hnsplib::SpaceInterface<float>* space;
    hnsplib::HierarchicalNSW<float>* index;
}
```

Класс `HNSWLibIndex` инкапсулирует интерфейс алгоритма HNSW, предоставляя функции для создания, управления и использования HNSW-индекса. Далее приведем описание ключевых фрагментов кода.

Конструктор

`HNSWLibIndex`: создает экземпляр `HNSWLibIndex`, при инициализации необходимо указать размерность векторных данных `dim`, количество векторных элементов `num_data`, которые может содержать индекс, тип метрики расстояния `metric`, максимальное количество ближайших соседей для каждого узла `M` и размер очереди с приоритетом при построении индекса `ef_construction`.

Методы

`insert_vectors`: используется для записи новых векторных данных в индекс. Принимает параметр `data`, содержащий векторные данные, и параметр `label`, представляющий метку векторных данных, и записывает их в индекс.

`search_vectors`: используется для запроса `k` ближайших векторов в индексе. Принимает динамический массив `query`, содержащий вектор для запроса, а также количество ближайших соседей `k` и размер очереди с приоритетом `ef_search` в качестве параметров. Возвращает пару, содержащую метки ближайших соседей и соответствующие расстояния. Эти результаты хранятся в виде динамических массивов и возвращаются в качестве результата функции.

Приватные члены

`dim`: размерность векторных данных.

`space`: указатель на объект типа `hnsplib::SpaceInterface<float>`, используемый для вычисления сходства между векторными данными.

`index`: указатель на объект типа `hswlib::HierarchicalNSW<float>`, представляющий сам HNSW-индекс, который отвечает за хранение векторных данных и выполнение операций запроса.

2. Функции `insert_vectors` и `search_vectors`

Два метода `insert_vectors` и `search_vectors` класса `HNSWLibIndex` реализуют возможности записи и запроса HNSW-индекса.

```
void HNSWLibIndex::insert_vectors(
    const std::vector<float>& data,
    uint64_t label
) {
    index->addPoint(data.data(), label);
}

std::pair<std::vector<long>, std::vector<float>>
HNSWLibIndex::search_vectors(
    const std::vector<float>& query,
    int k,
    int ef_search
) { // Изменение типа возвращаемого значения
    index->setEf(ef_search);
    auto result = index->searchKnn(query.data(), k);

    std::vector<long> indices(k);
    std::vector<float> distances(k);
    for (int j = 0; j < k; j++) {
        auto item = result.top();
        indices[j] = item.second;
        distances[j] = item.first;
        result.pop();
    }

    return {indices, distances};
}
```

Этот код представляет реализацию двух методов класса `HNSWLibIndex`, приведем описание логики его работы.

Функция `insert_vectors` используется для записи новых векторных данных в HNSW-индекс. Она выполняет запись векторных данных: вызывает функцию `addPoint` объекта индекса и передает векторные данные `data` и метку векторных данных `label`.

Назначение функции `search_vectors` – вернуть k ближайших векторных данных в HNSW-индексе. Процесс работы функции следующий.

- Сначала функция настраивает размер очереди с приоритетом, используемой при запросе, вызывая функцию `setEf` объекта HNSW-индекса. Параметр `ef_search` определяет точность и производительность запроса. Увеличение значения `ef_search` может повысить полноту запроса, то есть увеличить вероятность нахождения более близких соседей к запросу.

- Затем функция выполняет запрос k ближайших соседей, используя функцию `searchKnn` объекта индекса. Она принимает вектор запроса `query` и ожидаемое количество ближайших соседей `k` в качестве входных параметров.
- Операция запроса возвращает отсортированную очередь с приоритетом, которая упорядочена по степени сходства с вектором запроса и содержит информацию о k ближайших векторах.
- Функция затем проходит по этой очереди с приоритетом, извлекая информацию об ID каждого ближайшего вектора и соответствующем расстоянии, которые сохраняются в динамических массивах `indices` и `distances` соответственно.
- В итоге функция `search_vectors` возвращает пару, содержащую информацию об ID и расстоянии k ближайших векторов, в качестве результата операции запроса.

3. Функция `insertHandler`

После определения класса `HNSWLibIndex` нам нужно выполнить следующие три шага, чтобы наша векторная база данных могла предоставлять возможности записи и запроса HNSW-индекса.

Первый шаг: интеграция `HNSWLibIndex` в `IndexFactory` путем добавления условия `case` в функцию `init IndexFactory`.

```
case IndexFactory::IndexType::HNSW:
    index_map[type] = new HNSWLibIndex(dim, num_data, metric, 16, 200);
    break;
```

Второй шаг: научить `HttpServer` распознавать операции HNSW-индекса и связывать их с функциями `HNSWLibIndex`.

- Добавить условие `else` в функцию `getIndexTypeFromRequest HttpServer`:

```
else if (index_type_str == INDEX_TYPE_HNSW) { // Добавление поддержки HNSW.
    return IndexFactory::IndexType::HNSW;
}
```

- Добавить условие `case` в функцию `insertHandler HttpServer`:

```
case IndexFactory::IndexType::HNSW: { // Добавление логики HNSW-индекса.
    HNSWLibIndex* hnswIndex = static_cast<HNSWLibIndex*>(index);
    hnswIndex->insert_vectors(data, label);
    break;
}
```

- Добавить условие `case` в функцию `searchHandler`:

```
case IndexFactory::IndexType::HNSW: {
    HNSWLibIndex* hnswIndex = static_cast<HNSWLibIndex*>(index);
    results = hnswIndex->search_vectors(query, k);
    break;
}
```

Третий шаг: инициализировать `HNSWLibIndex` через `IndexFactory` в процессе инициализации функции `main` в `VDBServer`.

```
globalIndexFactory->init(IndexFactory::IndexType::HNSW, dim, num_data);
```

Итерация версии v0.0.2

С помощью трех простых шагов мы добавили поддержку индексов типа HNSW. Обозначим новую версию как v0.0.2. Благодаря модульному и абстрактному подходу объем дополнительного кода оказался минимальным. Ниже приведены примеры запросов и ответов для соответствующих тестов.

Запись векторных данных типа HNSW:

Запрос:

```
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.2], "id": 3, "indexType": "HNSW"}' http://localhost:8080/insert
```

Ответ:

```
{"retCode":0}
```

Запрос векторных данных типа HNSW:

Запрос:

```
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.5], "k": 2, "indexType": "HNSW"}' http://localhost:8080/search
```

Ответ:

```
{"vectors": [3], "distances": [0.090000000357627869], "retCode": 0}
```

В табл. 4.6 приведен перечень новых и обновленных модулей, а также добавленного функционала в версии v0.0.2.

Таблица 4.6. Новые и обновленные модули, добавленные функции версии v0.0.2

Модуль	Задействованные файлы	Описание
<code>HNSWLibIndex</code>	<code>hnsplib_index.h</code> , <code>hnsplib_index.cpp</code>	Реализует инкапсуляцию объекта HNSW-индекса, скрывая детали реализации библиотеки <code>HNSWLib</code>
<code>IndexFactory</code>	<code>index_factory.h</code> , <code>index_factory.cpp</code>	Расширяет класс фабрики векторных индексов, добавляя поддержку индекса типа HNSW
<code>HttpServer</code>	<code>http_server.h</code> , <code>http_server.cpp</code>	Расширяет команды <code>/insert</code> и <code>/search</code> для поддержки операций с запросами типа HNSW
<code>VDBServer</code>	<code>vdb_server.cpp</code>	Расширяет инициализацию <code>HNSWLibIndex</code> при запуске

4.2. РЕАЛИЗАЦИЯ ГИБРИДНОГО ИНДЕКСА ДАННЫХ

В реальных сценариях использования векторных баз данных, помимо записи векторных данных, разработчики обычно добавляют запросы и фильтрацию на основе скалярных данных. Например, после записи векторных данных можно сначала выполнить запрос на основе скалярных условий, чтобы получить определенные векторные данные, а затем использовать эти данные как отправную точку для поиска ближайших соседей. Кроме того, можно добавлять к векторным данным скалярные метки, что позволяет при выполнении запросов ближайших соседей фильтровать данные, не удовлетворяющие условиям. Гибридные запросы на основе векторов и скаляров являются довольно часто используемой функцией в векторных базах данных.

4.2.1. Реализация индекса скалярных данных

Реализация индекса скалярных данных

- └─ Класс `ScalarStorage`
 - └─ Инкапсуляция функций работы с базой данных RocksDB
- └─ Функции `insert_scalar` и `get_scalar`
 - └─ `insert_scalar`
 - └─ Принимает скалярные данные и записывает их в RocksDB
 - └─ `get_scalar`
 - └─ Принимает скалярные данные и выполняет запрос к RocksDB

Для реализации функции запроса векторных данных на основе скалярных меток необходимо ввести в систему удобную подсистему хранения, чтобы сохранять скалярные данные и индексировать их. База данных RocksDB является идеальным выбором, так как она отлично справляется с высокопроизводительными операциями записи и выполнения запросов и широко используется в инфраструктуре хранения многих реляционных и нереляционных баз данных, что подтверждено масштабной эксплуатацией в промышленности.

1. Класс `ScalarStorage`

Чтобы интегрировать подсистему хранения скаляров с нашей существующей системой, сначала необходимо определить класс `ScalarStorage`.

```
class ScalarStorage {
public:
    ScalarStorage(const std::string& db_path);
    ~ScalarStorage();
    void insert_scalar(uint64_t id, const std::vector<float>& data);
    std::vector<float> get_scalar(uint64_t id);
private:
    rocksdb::DB* db_;
};
```



```

rapidjson::Document ScalarStorage::get_scalar(uint64_t id) {
    rapidjson::Document
    std::string value;
    rocksdb::Status status = db_->Get(rocksdb::ReadOptions(),
                                      std::to_string(id), &value);

    rapidjson::Document data;
    data.Parse(value.c_str());
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
    data.Accept(writer);
    return data;
}

```

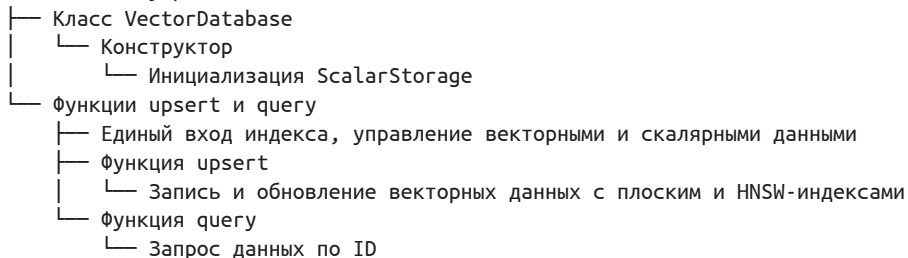
Этот фрагмент кода представляет собой реализацию двух функций-членов класса `ScalarStorage`.

- Функция `insert_scalar` используется для хранения данных в формате JSON в базе данных RocksDB. Она принимает параметр `id` в качестве уникального идентификатора данных и параметр `data` типа `rapidjson::Document`, который содержит скалярные данные для хранения. Функция сначала преобразует параметр `data` в строку в формате JSON, а затем с помощью функции `Put` библиотеки RocksDB сохраняет ее в базу данных.
- Функция `get_scalar` предназначена для запроса скалярных данных по указанному `id` из базы данных RocksDB. Она возвращает объект `rapidjson::Document`, содержащий запрошенные данные JSON. Функция использует функцию `Get` библиотеки RocksDB для запроса соответствующей JSON-строки из базы данных по `id`. Если запрос успешен, она с помощью функции `Parse` библиотеки `rapidjson` преобразует JSON-строку в объект `rapidjson::Document` и возвращает его. Если запрос неудачен или в базе данных не найден соответствующий `id`, она возвращает пустой объект `rapidjson::Document`.

Используя библиотеку RapidJSON для обработки JSON-данных, функции могут гибко работать с различными сложными структурами данных и сохранять их в базе данных RocksDB в легкочитаемом формате.

4.2.2. Единый вход управления

Единый вход управления



В предыдущих разделах этой главы мы реализовали функции записи и запроса векторных данных на основе библиотек FAISS и HNSWLib, а также функции записи и запроса скалярных данных на основе базы данных RocksDB. Эти две части могут работать независимо. Однако в реальных приложениях часто требуется комбинировать операции с векторами и скалярами.

Рассмотрим следующий сценарий использования: нам необходимо поддерживать интерфейс `upsert`, который позволяет пользователю одновременно передавать векторные и скалярные данные. Предположим, что мы используем поле ID для идентификации этих данных. Сначала необходимо проверить, существует ли данный ID в системе скалярных данных. Если ID существует, мы применяем логику обновления для обновления скалярных и векторных данных. Если ID не существует, то применяем логику создания новой записи для записи скалярных и векторных данных. В этом случае необходимо объединить в одной программной логике два типа данных – скалярные и векторные.

1. Класс `VectorDatabase`

Для удовлетворения потребностей ключевого сценария работы с векторными базами данных мы разработаем класс комбинированных операций под названием `VectorDatabase`, определение которого представлено ниже.

```
class VectorDatabase {
public:
    // Конструктор.
    VectorDatabase(const std::string& db_path);

    // Запись и обновление данных.
    void upsert(uint64_t id, const rapidjson::Document& data,
               IndexFactory::IndexType index_type);

    // Добавление интерфейса функции запроса.
    rapidjson::Document query(uint64_t id);

private:
    ScalarStorage scalar_storage_;
}
```

Из названия класса и кода реализации очевидно, что класс `VectorDatabase` является единым входом для всех последующих операций с данными. Совершая архитектуру системы, мы на данном этапе ввели это ключевое определение класса для удовлетворения потребностей в функционале векторной базы данных.

Класс `VectorDatabase` инкапсулирует объект скалярного хранения `ScalarStorage` (`scalar_storage_`), который отвечает за хранение информации о скалярных метках, связанных с векторными данными. Эти метки могут быть любыми метаданными, связанными с вектором, такими как текстовые описания, метки классификации и другие атрибуты. Сами векторные данные хранятся и управляются в виде массивов с плавающей запятой через `ScalarStorage`.

С помощью `upsert` класс `VectorDatabase` позволяет пользователю записывать и обновлять векторные данные и их связанные метки, а `query` предоставляет возможность запроса уже записанных данных в системе по конкретному ID. Такая архитектура делает класс `VectorDatabase` полноценным решением для комплексного управления векторными и скалярными данными, которое поддерживает не только запись и запрос данных, но и эффективно обрабатывает различные операции, связанные с векторами.

2. Функции `upsert` и `query`

После написания функции `upsert` вы получите более глубокое понимание единого класса `VectorDatabase`. Ниже представлена ключевая часть кода функции `upsert` и соответствующие пояснения.

На первом этапе мы сначала проверяем, существуют ли в хранилище скаляров данные, соответствующие данному ID, используя интерфейс `get_scalar(id)`.

```
rapidjson::Document existingData;
try {
    existingData = scalar_storage_.get_scalar(id);
} catch (const std::runtime_error& e) {
    // Вектор не существует, игнорируем эту ошибку и продолжаем выполнение.
}
```

Более конкретно – мы пытаемся получить из `scalar_storage_` векторные данные и связанные метки, ассоциированные с заданным ID. Если данные существуют, `existingData` будет заполнен представлением этих данных в виде `rapidjson::Document`. Если при попытке получения данных возникает исключение `std::runtime_error` (например, данные, соответствующие ID, не существуют), мы можем записать информацию в журнал или выполнить другие необходимые операции по обработке ошибок, а затем продолжить операцию записи или обновления данных.

На втором этапе для уже существующего ID мы используем глобальную фабрику индексов для получения объекта индекса указанного типа и удаляем существующий ID через подсистему векторного индекса. Обратите внимание, что здесь мы реализовали новый интерфейс `remove_vectors`. Этот интерфейс сам по себе несложен, он основан на существующих функциях библиотек FAISS и HNSWLib и представляет собой простую инкапсуляцию. Базовый код вызова интерфейса удаления представлен ниже.

```
void* index = getGlobalIndexFactory()->getIndex(index_type);
// Выполнение различных операций в зависимости от типа индекса.
switch (index_type) {
    case IndexFactory::IndexType::FLAT: {
        // Если индекс плоский, преобразуем указатель индекса в указатель
        // типа FaissIndex.
        FaissIndex* faiss_index = static_cast<FaissIndex*>(index);
```

```

        // Вызов функции remove_vectors объекта FaissIndex для удаления данных
        // с указанным ID из индекса.
        faiss_index->remove_vectors({static_cast<long>(id)});
        break;
    }
    case IndexFactory::IndexType::HNSW: {
        // Если индекс HNSW, преобразуем указатель индекса в указатель
        // типа HNSWLibIndex.
        HNSWLibIndex* hnsw_index = static_cast<HNSWLibIndex*>(index);
        hnsw_index->remove_vectors({id});
        break;
    }
}

```

На третьем этапе мы записываем новые данные в системы хранения векторов и скаляров, базовый код представлен ниже.

```

void* index = getGlobalIndexFactory()->getIndex(index_type);
switch (index_type) {
    case IndexFactory::IndexType::FLAT: {
        FaissIndex* faiss_index = static_cast<FaissIndex*>(index);
        // Если индекс плоский, преобразуем объект индекса в объект FaissIndex
        // и вызываем функцию записи вектора.
        faiss_index->insert_vectors(newVector, id);
        break;
    }
    case IndexFactory::IndexType::HNSW: {
        HNSWLibIndex* hnsw_index = static_cast<HNSWLibIndex*>(index);
        // Если индекс HNSW, преобразуем объект индекса в объект HNSWLibIndex
        // и вызываем функцию записи вектора.
        hnsw_index->insert_vectors(newVector, id);
        break;
    }
    default:
        break;
}
// Запись скалярных данных в скалярное хранилище.
scalar_storage.insert_scalar(id, data);

```

Этот код выполняет различные операции в зависимости от типа индекса. Сначала из глобальной фабрики индексов извлекается объект индекса указанного типа. Затем в зависимости от типа индекса вызывается функция записи вектора. В завершение скалярные данные записываются в хранилище скаляров.

Изучив эти этапы, можно заметить, что, хотя база данных называется «векторной», на самом деле она требует сочетания систем хранения скаляров и векторов для предоставления сложных функций, подходящих для сценариев использования векторной базы данных.

Помимо функции `upsert`, мы также предоставили классу `VectorDatabase` более удобную функцию запроса `query`. Реализация `query` довольно проста: она

быстро возвращает скалярные данные, соответствующие ID, из локальной базы данных RocksDB, поэтому здесь мы не будем подробно разбирать код реализации.

После реализации функций `upsert` и `query` рассмотрим простой пример использования.

```
Запрос:
curl -X POST -H "Content-Type: application/json" \
  -d '{"vectors": [0.555555], "id": 3, "indexType": "FLAT", "Name": "hello",
    "Ci":1111}' \
    http://localhost:8080/upsert

Ответ:
{"retCode":0}

Запрос:
curl -X POST -H "Content-Type: application/json" \
  -d '{"id": 3}' \
    http://localhost:8080/query

Ответ:
{"vectors":[0.555555],"id":3,"indexType":"FLAT","Name":"hello","Ci":1111,"retCode":0}
```

Первый запрос отправляет данные, содержащие вектор, ID, тип индекса, имя и поле `Ci`, и указывает маршрут запроса как `/upsert`. Цель этого запроса – записать векторные данные с определенным ID в индекс типа `FLAT`, одновременно предоставляя дополнительную информацию с именем `hello` и значением поля `Ci`, равным 1111. После обработки запроса векторная база данных возвращает ответ в формате JSON, где значение поля `retCode` равно 0, что указывает на успешное выполнение запроса.

Второй запрос отправляет данные, содержащие поле ID, и указывает маршрут запроса как `/query`. Цель этого запроса – получить векторные данные, соответствующие определенному ID. После получения запроса векторная база данных возвращает векторные данные, связанные с этим ID, а также дополнительную информацию, такую как тип индекса, имя и поле `Ci`. Аналогично значение поля `retCode` равно 0, что указывает на успешное выполнение запроса.

Благодаря инкапсуляции интерфейсов в классе `VectorDatabase` нашу систему можно представить в более приближенном к реальным сценариям использования виде, что делает всю систему еще более удобной в использовании.

► Обновление версии v0.1

На этом этапе наша векторная база данных достигла версии v0.1, которая является важной вехой в создании векторной базы данных с нуля. В новую версию была добавлена система хранения скаляров поверх векторного хранилища, определен ключевой компонент класса `VectorDatabase`.

В табл. 4.7 приведены новые и обновленные модули, добавленные функции версии v0.1.

Таблица 4.7. Новые и обновленные модули, добавленные функции версии v0.1

Модуль	Задействованные файлы	Описание
ScalarStorage	scalar_storage.h, scalar_storage.cpp	Реализует инкапсуляцию базы данных RocksDB, предоставляет поддержку хранения скалярных данных
VectorDatabase	vector_database.h, vector_database.cpp	Объединяет операции с векторными и скалярными данными, все последующие интерфейсы векторной базы данных предоставляются этим классом
HttpServer	http_server.h, http_server.cpp	Расширяет /upsert и /query, принимает операции перезаписи с векторами и скалярами, поддерживает запросы векторных данных и данных меток скаляров

4.2.3. Реализация фильтрующего индекса

- Реализация фильтрующего индекса
 - Схема запроса с фильтрующим выражением
 - | — Использование команды cURL для записи данных в векторную базу данных
 - | — Выполнение гибридного запроса с фильтрующим выражением
 - | — Оптимизация структуры хранения
 - | — Комбинирование документов с одинаковыми значениями полей для уменьшения избыточности хранения
 - | — Использование RoaringBitmap для повышения эффективности использования пространства
 - Функции записи и запроса фильтрующего индекса
 - | — addIntFieldFilter: создание битовой карты и добавление ID
 - | — updateIntFieldFilter: обновление ID в битовой карте
 - | — getIntFieldFilterBitmap: получение битовой карты на основе операции
 - Запрос с учетом фильтрующих условий
 - | — Наследование от faiss::IDSelector
 - | — Реализация функции is_member для определения наличия ID
 - | — Использование функции search библиотеки FAISS для фильтрации ID

В приложениях векторных баз данных, помимо двух ключевых команд /upsert и /query, которые мы уже реализовали, пользователи чаще всего выполняют гибридные запросы, комбинируя скалярные и векторные данные. Это означает, что сначала выполняется фильтрация на основе скалярных данных, а затем в пределах отфильтрованного набора данных выполняется векторный запрос. Далее мы поэтапно реализуем эту ключевую возможность на основе предыдущей системы.

1. Схема запроса с фильтрующим выражением

Сначала кратко рассмотрим суть этой функции. Мы использовали следующую команду для записи строки данных в векторную базу данных.

Запрос:

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"id": 1, "vectors": [0.1], "int_field": 42, "indexType": "FLAT"}' \
  http://localhost:8080/upsert
```

После записи данных мы попробуем выполнить гибридный запрос с фильтрующим выражением, объединяющий скалярные и векторные данные.

Запрос:

```
curl -X POST \
  -H "Content-Type: application/json" \
  -d '{"vectors": [0.9], "k": 5, "indexType": "FLAT", \
    "filter":{"fieldName":"int_field","value":43, "op":"="}}' \
  http://localhost:8080/search
```

При выполнении запроса с этим фильтрующим условием мы можем выбрать только 5 ближайших соседних векторов из коллекции, где `int_field` равно 43. Хотя другие векторы могут иметь большее сходство с нашим запросом, они будут отброшены фильтром.

Для поддержки отбора ID по заданным фильтрующим условиям наша система должна построить индекс данных для этих условий. Ключ индекса – это название поля документа, которое можно отфильтровывать, например `int_field`, а значение должно содержать ID документа и значение этого поля в документе, например 1 и 42. Простое и прямое решение – использовать структуру данных `map` для хранения их соответствий.

```
std::map<std::string, std::vector<std::pair<std::int, int>>>
```

Когда необходимо выполнить сопоставление с фильтрующим условием, мы получаем динамический массив по ключу, затем поочередно сравниваем элементы массива, чтобы найти коллекцию ID, удовлетворяющих условию, и продолжить дальнейшие действия.

Однако при детальном анализе этого динамического массива выявляются следующие проблемы.

Во-первых, значения фильтрующих полей для разных документов с одинаковыми ID в динамическом массиве могут совпадать, поскольку одноименные поля в разных документах могут иметь одинаковое значение. Это снижает эффективность хранения и сопоставления меток данных.

Во-вторых, эффективность хранения ID низкая, так как каждый целочисленный элемент занимает 4 байта. Здесь можно использовать структуру данных битовой карты для более эффективного хранения. Битовая карта – это структура данных, предназначенная для эффективного управления состоянием наличия данных, она представляет наличие определенного ID в наборе данных с помощью одного бита. Например, если мы используем битовую карту для отслеживания, какие ID являются действительными, то каждый бит в битовой

карте соответствует уникальному ID. Если второй бит (начиная с 1) установлен в 1, это означает, что данные с ID 1 присутствуют в наборе данных; если этот бит равен 0, это означает, что данные с ID 1 отсутствуют в наборе данных. Таким образом, битовая карта позволяет в очень компактной форме хранить информацию о большом количестве ID.

Учитывая эти два аспекта, мы можем объединить документы с одинаковыми значениями фильтрующих полей в одну коллекцию, чтобы управлять ими как единым целым. Эти объединенные документы будут соответствовать одной битовой карте, которая хранит коллекцию всех ID объединенных документов, что позволяет эффективно решать вышеупомянутые проблемы. Таким образом, мы приходим к следующей структуре хранения:

```
std::map<std::string, std::map<long, roaring_bitmap_t*>>
```

Ключом этой структуры по-прежнему является имя поля, по которому необходимо фильтровать данный документ, а значение хранит объект отображения. Ключи вложенного объекта отображения представляют собой фактические значения фильтруемого поля, которые объединены для различных документов. С помощью этого значения мы можем снова индексировать битовую карту, соответствующую значению. На рис. 4.4 изображена схема хранения на основе отображения и битовой карты.

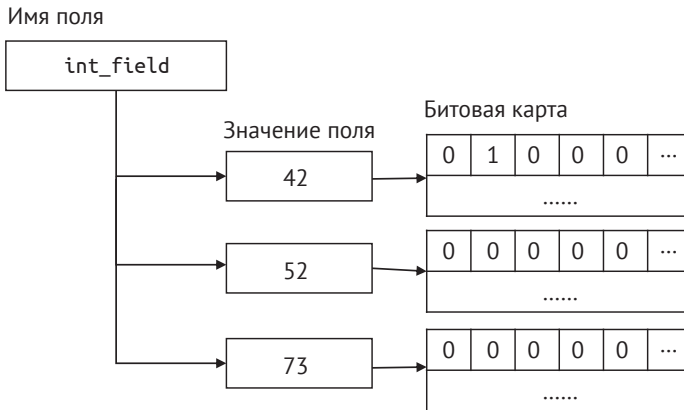


Рис. 4.4. Схема хранения на основе отображения и битовой карты

Конкретные данные на рис. 4.4 получены на основе предыдущих тестовых команд. Здесь ключом является `int_field`, а ключом вложенного объекта отображения – значение 42. Значение вложенного объекта отображения – это битовая карта, где каждая позиция слева направо представляет собой ID. Самая левая позиция обозначает ID 0, с каждым шагом вправо ID увеличивается на 1. В данном примере вторая позиция установлена в 1, то есть значение `int_field` для документа с ID 1 равно 42. Здесь мы используем формат `RoaringBitmap`, который обладает высокой эффективностью использования пространства, для хранения информации битовой карты.

Поскольку значение условия фильтрации используется в качестве ключа вложенного объекта отображения, мы можем очень удобно использовать команды структуры данных отображения (`map`) для операций больше, меньше и т. д., чтобы быстро вернуть все битовые карты, соответствующие целевому значению в фильтрационном условии. Затем, объединив эти битовые карты, можно быстро вернуть коллекцию ID, удовлетворяющих условиям.

2. Функции записи и запроса фильтрующего индекса

После определения формата хранения данных введем новый объект индекса `FilterIndex`, который мы называем фильтрующим индексом. Ниже приведено его определение.

```
class FilterIndex {
public:
    FilterIndex();
    // Добавление условия фильтрации для целочисленного поля и связывание
    // с соответствующим ID.
    void addIntFieldFilter(
        const std::string& fieldname,
        int64_t value,
        uint64_t id
    );

    // Обновление условия фильтрации для целочисленного поля, замена старого
    // значения на новое и обновление связанного ID.
    void updateIntFieldFilter(
        const std::string& fieldname,
        int64_t old_value,
        int64_t new_value,
        uint64_t id
    );

    // Получение битовой карты, соответствующей условию фильтрации
    // целочисленного поля, для хранения совпадающих ID.
    void getIntFieldFilterBitmap(
        const std::string& fieldname,
        Operation op,
        int64_t value,
        roaring_bitmap_t* result_bitmap
    );
private:
    // Структура хранения условий фильтрации для целочисленных полей,
    // отображение битовой карты по имени поля и значению.
    std::map<std::string, std::map<long, roaring_bitmap_t*>> intFieldFilter;
}
```

Класс `FilterIndex` предоставляет функции записи, обновления и запроса условий фильтрации для целочисленных полей. Условия фильтрации позволяют отбирать данные на основе целочисленных значений конкретного поля. Внутри используется тип данных `std::map` для хранения отношений отображения

между именами полей и условиями фильтрации, а также между целочисленными значениями и битовыми картами (`roaring_bitmap_t`).

Новый объект фильтрующего индекса будет выполнять функцию хранения всех условий фильтрации в нашей системе. В текущей версии мы упростили значения и поддерживаем только индексы для полей типа `int`. Другие типы можно добавить по мере необходимости.

Ниже приведена реализация кода функции `addIntFieldFilter`.

```
void FilterIndex::addIntFieldFilter(
    const std::string& fieldname,
    int64_t value,
    uint64_t id
) {
    // Создание нового объекта битовой карты для хранения ID,
    // удовлетворяющих условию фильтрации.
    roaring_bitmap_t* bitmap = roaring_bitmap_create();

    // Запись указанного ID в битовую карту, указывая,
    // что этот ID удовлетворяет условию фильтрации.
    roaring_bitmap_add(bitmap, id);

    // Связывание объекта битовой карты с именем поля и целым значением,
    // хранение в отображении intFieldFilter.
    intFieldFilter[fieldname][value] = bitmap;
}
```

Этот код реализует функцию `addIntFieldFilter` в классе `FilterIndex`. Она отвечает за создание новой битовой карты, добавление ID и связывание его с конкретным именем поля и целочисленным значением для последующих операций фильтрации.

Функция `updateIntFieldFilter` обновляет связанные данные, и ее реализация схожа с логикой `addIntFieldFilter`, поэтому здесь она не приводится.

Рассмотрим часть, связанную с сопоставлением. Эта часть выполняется с помощью функции-члена `getIntFieldFilterBitmap` класса `FilterIndex`, которая используется для получения битовой карты, соответствующей условиям фильтрации для целочисленного поля. Ее реализация выглядит следующим образом:

```
void FilterIndex::getIntFieldFilterBitmap(const std::string& fieldname,
                                          Operation op,
                                          int64_t value,
                                          roaring_bitmap_t* result_bitmap) {
    auto it = intFieldFilter.find(fieldname);
    if (it != intFieldFilter.end()) {
        // Получение отображения значений на битовую карту, связанного с именем поля.
        auto& value_map = it->second;

        // Если тип операции - равенство, то поиск битовой карты
        // для конкретного значения.
```

```

    if (op == Operation::EQUAL) {
        auto bitmap_it = value_map.find(value);
        if (bitmap_it != value_map.end()) {
            // Выполнение побитовой операции ИЛИ с найденной битовой
            // картой и существующей result_bitmap, объединение результатов.
            roaring_bitmap_or_inplace(result_bitmap, bitmap_it->second);
        }

        // Если тип операции - неравенство, то поиск битовых карт
        // для всех значений, не равных заданному.
    } else if (op == Operation::NOT_EQUAL) {
        for (const auto& entry : value_map) {
            if (entry.first != value) {
                // Выполнение побитовой операции ИЛИ с битовой картой
                // текущего значения и result_bitmap, объединение результатов.
                roaring_bitmap_or_inplace(result_bitmap, entry.second);
            }
        }
    }
}

```

Из реализации функции `getIntFieldFilterBitmap` видно, что на текущий момент поддерживаются только условия равенства и неравенства. В случае равенства достаточно найти и вернуть битовую карту, соответствующую `value`. В случае неравенства после исключения битовой карты, соответствующей `value`, объединяются и возвращаются остальные битовые карты.

Сравнивая начальный способ фильтрации путем поочередного перебора и реализацию на основе битовой карты, очевидно, что реализация на основе битовой карты значительно проще. Здесь есть один совет: если процесс написания кода кажется вам очень сложным и запутанным, я советую не спешить, а потратить некоторое время на организацию и обдумывание – хорошо спроектированные алгоритмы и архитектура программного обеспечения значительно упрощают работу программиста. Обдумайте все перед началом работы – это, как правило, сделает вашу работу более эффективной.

3. Объединение с условиями фильтрации в запросах

Однако возникает вопрос: как полученные здесь битовые карты можно объединить с ранее разработанными функциями векторного запроса `FaissIndex` и `HNSWLibIndex`? Вернемся к функции фильтрации в процессе векторного запроса, которую мы ранее не рассматривали подробно.

Функция `search` библиотеки FAISS поддерживает передачу указателя на структуру типа `IDSelector` в качестве параметра вызова. Эта структура в процессе запроса определяет, соответствует ли текущий ID условиям фильтрации. С помощью этого параметра можно задать фильтрующую функцию в процессе запроса. Здесь мы совместили структуру `RoaringBitmap` для реализации, подходящей для нашей системы.

```

struct RoaringBitmapIDSelector : faiss::IDSelector {
    // Конструктор, принимает указатель на RoaringBitmap и инициализирует
    // член bitmap_.
    RoaringBitmapIDSelector(const roaring_bitmap_t* bitmap)
        : bitmap_(bitmap) {}

    // Перегрузка функции is_member в интерфейсе IDSelector для проверки,
    // существует ли указанный ID в битовой карте.
    bool is_member(int64_t id) const final {
        return roaring_bitmap_contains(
            bitmap_, static_cast<uint32_t>(id)
        );
    }

    // Деструктор для освобождения ресурсов (хотя здесь нет выделения новых
    // ресурсов, но поддерживается согласованность интерфейса).
    ~RoaringBitmapIDSelector() override {}

    // Переменная, хранящая указатель на RoaringBitmap.
    const roaring_bitmap_t* bitmap_;
}

```

Объект структуры `RoaringBitmapIDSelector` можно инициализировать в процессе запроса, принимая битовую карту, удовлетворяющую условиям. Во время запроса функция `is_member` вызывается многократно для определения, существует ли текущий ID в битовой карте. Реализация в библиотеке HNSWLib практически идентична, поэтому здесь не будем подробно останавливаться на этом.

После реализации фильтрующего индекса и расширения фильтрующей функции необходимо расширить `indexFactory` для поддержки фильтрующего индекса, чтобы он также управлялся как категория фабрики индексов. Это изменение требует добавления следующего условия `case` в функцию `init` фабрики индексов.

```

case IndexFactory::IndexType::FILTER:
    index_map[type] = new FilterIndex();
    break;

```

Наконец, мы объединим вышеописанные функции, чтобы переопределить интерфейс `/search` через объект `VectorDatabase`. Ниже приведен базовый код реализации интерфейса `/search`.

```

std::pair<std::vector<long>, std::vector<float>> VectorDatabase::search(
    const rapidjson::Document& json_request) {
    // Код для получения параметров запроса из JSON-запроса опущен.
    ...
    // Проверка, содержит ли запрос параметр filter.
    roaring_bitmap_t* filter_bitmap = nullptr;
    if (json_request.HasMember("filter") && json_request["filter"].IsObject()) {
        const auto& filter = json_request["filter"];
        std::string fieldName = filter["fieldName"].GetString();
    }
}

```

```

std::string op_str = filter["op"].GetString();
int64_t value = filter["value"].GetInt64();
FilterIndex::Operation op = (op_str == "=")
    ? FilterIndex::Operation::EQUAL
    : FilterIndex::Operation::NOT_EQUAL;

// Получение FilterIndex через функцию getIndex из getGlobalIndexFactory.
FilterIndex* filter_index = static_cast<FilterIndex*>(
    getGlobalIndexFactory()->getIndex(IndexFactory::IndexType::FILTER));

// Вызов функции getIntFieldFilterBitmap из FilterIndex.
filter_bitmap = roaring_bitmap_create();
filter_index->getIntFieldFilterBitmap(fieldName, op, value, filter_bitmap);
}
void* index = getGlobalIndexFactory()->getIndex(indexType);
std::pair<std::vector<long>, std::vector<float>> results;
switch (indexType) {
    case IndexFactory::IndexType::FLAT: {
        FaissIndex* faissIndex = static_cast<FaissIndex*>(index);
        // Передача filter_bitmap в функцию search_vectors.
        results = faissIndex->search_vectors(query, k, filter_bitmap);
        break;
    }
    case IndexFactory::IndexType::HNSW: {
        HNSWLibIndex* hnswIndex = static_cast<HNSWLibIndex*>(index);
        // Передача filter_bitmap в функцию search_vectors.
        results = hnswIndex->search_vectors(query, k, filter_bitmap);
        break;
    }
    default:
        break;
}
if (filter_bitmap != nullptr) {
    delete filter_bitmap;
}
return results;
}

```

В этом новом интерфейсе `/search` мы должны реализовать получение параметров фильтрации, выполнить запрос векторного индекса с учетом фильтрации и затем вернуть результаты. Ниже приводится описание логики выполнения этой функции.

Обработка условий фильтрации

Если в запросе содержится параметр `filter`, функция создаст `filter_bitmap`, который будет использоваться для хранения ID документов, удовлетворяющих условиям фильтрации. Он получает битовую карту, соответствующую имени поля, оператору и значению, через фильтрующий индекс.

Получение объекта индекса

Функция получает через `IndexFactory` объект индекса, соответствующий типу индекса запроса.

Выполнение запроса

В зависимости от типа индекса функция выполняет различную логику запроса. Если это плоский индекс, объект индекса преобразуется в тип `FaissIndex`, и для выполнения запроса будет вызвана его функция `search_vectors`. Если это HNSW-индекс, объект индекса преобразуется в тип `HNSWLibIndex`, и для выполнения запроса будет вызвана его функция `search_vectors`.

Результаты запроса

Результаты запроса сохраняются в переменную `results`, которая представляет собой пару, содержащую ID ближайших соседей и расстояния до них.

После реализации интерфейса `/search` в `VectorDatabase` мы можем протестировать его с помощью команды с параметрами фильтрации, чтобы проверить его эффективность. Ниже приведены примеры запросов и ответов для тестирования.

```
Запрос:
curl -X POST -H "Content-Type: application/json" -d '{"id": 6, "vectors":
  [0.9], "int_field": 47, "indexType": "FLAT"}' http://localhost:8080/upsert
Ответ:
{"retCode":0}

Запрос:
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.9], "k": 5,
  "indexType": "FLAT", "filter":{"fieldName":"int_field","value":47, "op":"="}}'
  http://localhost:8080/search
Ответ:
{"vectors":[6],"distances":[0.0],"retCode":0}

Запрос:
curl -X POST -H "Content-Type: application/json" -d '{"vectors": [0.9], "k": 5,
  "indexType": "FLAT", "filter":{"fieldName":"int_field","value":47, "op":"!="}}'
  http://localhost:8080/search
Ответ:
{"retCode":0}
```

Из приведенных выше примеров запросов и ответов видно, что будут найдены только ближайшие векторы, удовлетворяющие условиям фильтрации. Такой сценарий гибридного запроса весьма распространен в практике применения векторных баз данных, а реализация на основе битовой карты является решением, сочетающим эффективность по времени и пространству. Эту версию мы обозначаем как v0.1.1.

► Итерация версии v0.1.1

В табл. 4.8 приведены новые и обновленные модули, добавленные функции версии v0.1.1 после реализации гибридного запроса скаляров и векторов.

Таблица 4.8. Новые и обновленные модули, добавленные функции версии v0.1.1

Модуль	Затрагиваемые файлы	Описание
<code>FilterIndex</code>	<code>filter_index.h</code> , <code>filter_index.cpp</code>	С помощью инвертированного индекса скаляров хранит битовую карту всех ID документов с одинаковым значением поля
<code>VectorDatabase</code>	<code>vector_database.h</code> , <code>vector_database.cpp</code>	Переопределен интерфейс <code>/search</code> с фильтрующим выражением на основе фильтрующего индекса
<code>FaissIndex</code>	<code>faiss_index.h</code> , <code>faiss_index.cpp</code>	Реализован класс <code>RoaringBitmapIDSelector</code> , который может фильтровать допустимые ID в процессе векторного запроса на основе битовой карты
<code>HNSWLibIndex</code>	<code>hnswlib_index.h</code> , <code>hnswlib_index.cpp</code>	Реализован класс <code>RoaringBitmapIDFilter</code> , который может фильтровать допустимые ID в процессе векторного запроса на основе битовой карты

4.3. РЕАЛИЗАЦИЯ ВОССТАНОВЛЕНИЯ

СИСТЕМЫ ПОСЛЕ СБОЕВ

Система эволюционировала до такой степени, что уже поддерживает функции чтения и записи векторной базы данных для основных сценариев. С внешней точки зрения она уже может функционировать. Однако для нас, как разработчиков этой системы, остается более важная задача. Поскольку база данных является системой хранения данных, мы должны гарантировать, что данные, записанные пользователем в нашу систему, не будут утеряны (свойство *персистентности*). В текущей системе существует две основные части данных: одна часть – это индексные данные, другая – скалярные данные. Индексные данные состоят из векторного индекса и фильтрующего индекса, и в настоящее время они хранятся только в памяти, что приводит к их потере при перезапуске системы. Скалярные данные, включающие фактические массивы векторов с плавающей запятой и связанную с ними информацию о метках, хранятся на жестком диске с использованием механизма базы данных RocksDB (при вызове интерфейса записи в RocksDB данные автоматически сохраняются), что позволяет восстановить их после перезапуска системы.

Для индексных данных нам необходимо разработать механизм, который обеспечит их сохранность после перезапуска системы, то есть необходимо значительно повысить способность этих данных к персистентности.


```
private:
    uint64_t increaseID_; // Автоинкрементная переменная для генерации уникального ID.

    std::fstream wal_log_file_; // Поток файла для записи журнала операций.
}
```

Наиболее важными переменными-членами в классе `Persistence` являются `increaseID_` и `wal_log_file_`. `increaseID_` представляет собой текущее значение ID журнала, используемое системой хранения, которое увеличивается автоматически, обеспечивая уникальность и упорядоченность ID. При запуске системы воспроизведение операций происходит в порядке записи в журнал. `wal_log_file_` – это дескриптор для чтения и записи файлов в системе персистентности, с помощью которого можно записывать журналы в файл, тем самым реализуя сохранность данных.

2. Функция `writeWALLog`

Теперь рассмотрим реализацию функции `writeWALLog`, которая является одной из ключевых функций системы персистентности, обеспечивающей запись всех изменений в журнал.

```
void Persistence::writeWALLog(const std::string& operation_type,
                             const rapidjson::Document& json_data,
                             const std::string& version) {
    // Добавлен параметр version.
    uint64_t log_id = increaseID();
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
    json_data.Accept(writer);
    wal_log_file_ << log_id << "|" << version << "|" << operation_type << "|"
        << buffer.GetString() << std::endl;
    // Добавление version в формат журнала.
    if (wal_log_file_.fail()) { // Проверка на наличие ошибок.
        GlobalLogger->error(
            "An error occurred while writing the WAL log entry. Reason: {}",
            std::strerror(errno)); // Вывод сообщения в журнал.
    } else {
        GlobalLogger->debug(
            "Wrote WAL log entry: log_id={}, version={}, operation_type={}, “
            “json_data_str={}",
            log_id, version, operation_type, buffer.GetString());
        // Вывод журнала.
        wal_log_file_.flush(); // Принудительная персистентность.
    }
}
```

Этот код реализует функцию предзаписи в журнал. Функция `writeWALLog` принимает три параметра: `operation_type` (тип операции), `json_data` (JSON-данные для записи) и `version` (версия журнала). В функции сначала вызывается функция `increaseID` для получения уникального ID журнала, затем данные JSON

преобразуются в строковый формат и записываются в файл журнала. Если в процессе записи возникает ошибка, фиксируется сообщение об ошибке, если нет – в журнале фиксируется успешное выполнение и данные принудительно сохраняются на жесткий диск.

Из этого фрагмента кода мы видим, что главная цель системы персистентности заключается в точном сохранении данных, записанных пользователем, и их восстановлении при перезапуске системы. Для достижения этой цели необходимо тщательно разработать формат предзаписи в журнал при записи данных пользователем – использовать фиксированный формат для записи и тот же формат для чтения при перезапуске. Это позволит обеспечить персистентность и восстановление данных. В настоящее время формат предзаписи в журнал включает четыре поля, представленные в табл. 4.9.

Таблица 4.9. Четыре поля формата предзаписи в журнал

Имя поля	Описание
log_id	log_id увеличивается при каждой записи, обеспечивая уникальность и упорядоченность предзаписи в журнал
version	Указывает версию журнала, чтобы в будущем можно было обеспечить обратную совместимость при изменении формата журнала
operation_type	Фактический тип операции пользователя, например <code>upsert</code>
Buffer.GetString()	Полная строка JSON при запросе данных пользователем

Чтобы обеспечить быстрое восстановление системы в процессе чтения и записи, поля формата предзаписи в журнал должны быть максимально компактными. Кроме того, можно рассмотреть возможность использования сжатия для экономии пространства. Стандартные методы сжатия обычно хорошо справляются с этой задачей, поэтому здесь они не рассматриваются.

3. Функция `readNextWALLog`

Функция `readNextWALLog` предназначена для чтения следующей записи журнала при перезапуске, ее реализация представлена ниже.

```
void Persistence::readNextWALLog(
    std::string* operation_type,
    rapidjson::Document* json_data
) {
    // Запись отладочной информации, указывающей на начало чтения предзаписи в журнал.
    GlobalLogger->debug("Reading next WAL log entry");

    std::string line; // Чтение следующей строки из файла журнала.
    if (std::getline(wal_log_file_, line)) {
        // Разбор строки журнала с использованием строкового потока.
        std::istringstream iss(line);
        std::string log_id_str, version, json_data_str;
```

```

// Разделение строки по символу '|', последовательное чтение полей журнала.
std::getline(iss, log_id_str, '|');
std::getline(iss, version, '|');
// Указатель типа операции для возврата.
std::getline(iss, *operation_type, '|');
std::getline(iss, json_data_str, '|');

// Преобразование строки ID журнала в тип uint64_t.
uint64_t log_id = std::stoull(log_id_str);

// Если ID журнала больше текущего increaseID_, обновить increaseID_.
if (log_id > increaseID_) {
    increaseID_ = log_id;
}

// Указатель для возврата JSON-данных.
json_data->Parse(json_data_str.c_str());

// Запись в журнал отладочной информации с содержимым прочитанной предзаписи.
GlobalLogger->debug(
    "Read WAL log entry: log_id={}, operation_type={}, json_data_str={}",
    log_id_str, *operation_type, json_data_str
);
} else {
    wal_log_file_.clear(); // Если достигнут конец файла,
                          // сбросить флаг конца файла.

// Запись в журнал отладочной информации, указывающей на отсутствие
// дополнительных предзаписей для чтения.
GlobalLogger->debug("No more WAL log entries to read");
}
}

```

Функция `readNextWALLog` предназначена для чтения следующей записи журнала из файла. Она использует тип `std::getline` для чтения строки из файла, а затем с помощью `std::istream` разбивает эту строку по разделителю '|'. Разобранный тип операции и данные JSON возвращаются вызывающей стороне через указатели. В процессе выполнения после считывания `log_id` мы обновляем `increaseID_` до максимального значения среди записанных ID. Кроме того, после чтения последней строки данных вызывается функция `wal_log_file_.clear()`, чтобы сбросить флаг конца файла, обеспечивая возможность последующих записей в журнал, сохраняя персистентность и согласованность данных.

4. Функция `reloadDatabase`

После реализации класса `Persistence` необходимо интегрировать его в существующую систему. Класс `VectorDatabase` является единым объектом-входом для работы с векторной базой данных, и нам нужно расширить его для поддержки персистентности системы. Ниже приведены новые члены и объявления функций, добавленные в `VectorDatabase` для поддержки персистентности.

```
class VectorDatabase {
public:
    // Добавление объявления функции-члена reloadDatabase.
    void reloadDatabase();
    // Добавление объявления функции-члена writeWALLog.
    void writeWALLog(const std::string& operation_type,
                    const rapidjson::Document& json_data);
private:
    Persistence persistence_; // Добавление объекта Persistence.
}
```

Функции `writeWALLog` и `reloadDatabase` используют объект `persistence_` для выполнения записи и чтения данных журнала, их реализация представлена ниже.

```
void VectorDatabase::writeWALLog(const std::string& operation_type,
                                const rapidjson::Document& json_data) {
    // Устанавливаем фактический номер версии.
    std::string version = "1.0";
    // Передача version в функцию writeWALLog.
    persistence_.writeWALLog(operation_type, json_data, version);
}
```

Как видно, `writeWALLog` определяет версию системного журнала по умолчанию, а затем вызывает объект `persistence_` для выполнения записи в журнал.

```
void VectorDatabase::reloadDatabase() {
    GlobalLogger->info("Entering VectorDatabase::reloadDatabase()");
    std::string operation_type;
    rapidjson::Document json_data;
    persistence_.readNextWALLog(&operation_type, &json_data);

    while (!operation_type.empty()) {
        GlobalLogger->info("Operation Type: {}", operation_type);

        // Вывод прочитанной строки.
        rapidjson::StringBuffer buffer;
        rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
        json_data.Accept(writer);
        GlobalLogger->info("Read Line: {}", buffer.GetString());

        if (operation_type == "upsert") {
            uint64_t id = json_data[REQUEST_ID].GetUint64();
            IndexFactory::IndexType index_type = getIndexTypeFromRequest(json_data);

            // Вызов интерфейса VectorDatabase::upsert для восстановления данных.
            upsert(id, json_data, index_type);
        }

        rapidjson::Document().Swap(json_data); // Очистка json_data.
    }
}
```

```

        operation_type.clear(); // Чтение следующей предзаписи в журнале.
        persistence_.readNextWALLog(&operation_type, &json_data);
    }
}

```

Функция `reloadDatabase` посредством циклического вызова функции `readNextWALLog` объекта `persistence_` построчно читает файл журнала и выполняет соответствующие операции восстановления данных в зависимости от типа операции, записанного в журнале. В настоящее время система поддерживает только операции типа `upsert`, и при обнаружении такой операции используется функция `VectorDatabase::upsert` для восстановления пользовательских данных на основе журнала. Эта функция также используется системой для обработки запросов на обновление данных от пользователей. Благодаря такому унифицированному подходу мы можем гарантировать, что данные будут восстановлены до состояния, в котором они находились при первоначальной записи пользователем, обеспечивая после восстановления согласованность данных с состоянием системы на момент ее завершения. После каждой записи журнала система очищает объект `json_data`, чтобы подготовить его к приему данных из следующей записи журнала, пока не останется больше данных для чтения.

Наконец, при приеме сервером `HttpServer` команды обновления от пользователя мы используем функцию `writeWALLog` класса `VectorDatabase` для записи в журнал, а при перезапуске системы вызываем функцию `reloadDatabase` для повторного чтения журнала. Мы обозначим эту небольшую версию как v0.1.2.

► Итерация версии v0.1.2

В табл. 4.10 содержатся новые и обновленные модули, добавленные функции версии v0.1.2 для поддержки персистентности системы.

Таблица 4.10. Новые и обновленные модули, добавленные функции версии v0.1.2

Модуль	Затронутые файлы	Описание
<code>Persistence</code>	<code>persistence.h</code> , <code>persistence.cpp</code>	Реализован механизм предзаписи в журнал, обеспечена персистентность и восстановление данных
<code>VectorDatabase</code>	<code>vector_database.h</code> , <code>vector_database.cpp</code>	Интегрированы возможности <code>Persistence</code> , обеспечена персистентность данных и восстановление системы на критическом пути системы

На текущем этапе вы, возможно, заметили, что по мере увеличения объема данных в нашем журнале время, необходимое для восстановления данных из начального состояния, также увеличивается. Когда система работает продолжительное время, время восстановления данных становится неприемлемым. Поэтому мы продолжаем оптимизацию.

4.3.2. Персистентность данных на основе моментальных снимков

```

Персистентность данных на основе моментальных снимков
├── Моментальные снимки индексов данных
│   ├── Реализация FaissIndex, HNSWLibIndex и FilterIndex
│   └── Поддержка моментальных снимков в IndexFactory
├── Расширение возможностей персистентности
│   ├── Функции takeSnapshot и loadSnapshot
│   └── Предоставление интерфейса моментальных снимков через HttpServer по
        адресу /admin/snapshot
├── Процесс восстановления системы
│   └── Загрузка данных моментального снимка с использованием
        Persistence::loadSnapshot
└── Воспроизведение инкрементальных данных на основе предзаписи в журнал
    └── Обеспечение согласованности данных
  
```

Технология *моментальных снимков* позволяет зафиксировать данные в определенный момент времени и затем сохранить их. Это похоже на фотографирование, когда данные «снимаются» с помощью технологии моментальных снимков. Если мы будем регулярно создавать моментальные снимки данных и использовать их в качестве базовой линии для восстановления, а на основе этой базовой линии записывать последующие предзаписи в журнал, то количество наших предзаписей в журнал будет контролируемым, и время восстановления системы не будет бесконечно увеличиваться. Моментальные снимки и предзапись в журнал являются отличными партнерами в технологиях персистентности баз данных, совместно повышая общую эффективность и производительность системы персистентности.

1. Моментальные снимки индексов данных

Мы постепенно проанализируем и разработаем технологическое решение для моментальных снимков индексов данных. В настоящее время в нашей системе необходимо сохранять три типа индексов данных: плоский индекс, HNSW-индекс и фильтрующий индекс. Библиотека FAISS, на которой основан плоский индекс, предоставляет функции `faiss::write_index` и `faiss::read_index`, с помощью которых мы можем быстро создать моментальный снимок плоского индекса. Аналогично библиотека HNSWLib, на которой основан HNSW-индекс, также предоставляет две функции – `hnswlib::saveIndex` и `hnswlib::loadIndex`, с помощью которых мы можем быстро создать моментальный снимок HNSW-индекса.

Далее мы сосредоточимся на том, как реализовать поддержку моментальных снимков для фильтрующего индекса.

Фильтрующий индекс представляет собой структуру данных, в которой в памяти комбинируются отображение и битовая карта. Для реализации его моментального снимка необходимо сначала реализовать функции сериализации и десериализации. Сериализация преобразует структуру данных из памяти в удобную для хранения форму. Десериализация, напротив, загружает

сохраненные данные обратно в память, организовав их в удобные для быстрого чтения и записи отображение и битовую карту. Ниже приведен код реализации функции сериализации фильтрованного индекса `serializeIntFieldFilter`.

```
std::string FilterIndex::serializeIntFieldFilter() {
    std::ostringstream oss;
    for (const auto& field_entry : intFieldFilter) {
        const std::string& field_name = field_entry.first;
        const std::map<long, roaring_bitmap_t*>& value_map = field_entry.second;

        for (const auto& value_entry : value_map) {
            long value = value_entry.first;
            const roaring_bitmap_t* bitmap = value_entry.second;

            // Сериализация битовой карты в массив байтов.
            uint32_t size = roaring_bitmap_portable_size_in_bytes(bitmap);
            char* serialized_bitmap = new char[size];
            roaring_bitmap_portable_serialize(bitmap, serialized_bitmap);

            // Запись имени поля, значения и сериализованной битовой карты
            // в выходной поток.
            oss << field_name << "|" << value << "|";
            oss.write(serialized_bitmap, size);
            oss << std::endl;

            delete[] serialized_bitmap;
        }
    }
    return oss.str();
}
```

Функция `serializeIntFieldFilter` предназначена для сериализации данных битовой карты в фильтрующем индексе в массив байтов и создания строки, содержащей все сериализованные данные.

Сначала функция создает объект `std::ostringstream oss` для построения итоговой строки сериализованных данных. Затем она перебирает каждый элемент, хранящийся в `intFieldFilter`, для каждого элемента получает его имя и подотображение значений к битовой карте. Для каждой пары значений и битовой карты в подотображении функция вычисляет размер сериализации битовой карты и создает массив байтов `serialized_bitmap` для хранения сериализованной битовой карты. С помощью функции `roaring_bitmap_portable_serialize` битовая карта сериализуется в массив байтов. Затем функция записывает имя поля, значение и сериализованную битовую карту в объект `oss`. После записи данных функция освобождает память, выделенную для сериализованной битовой карты. Наконец, функция преобразует содержимое объекта `oss` в строку и возвращает ее в качестве результата.

Далее мы реализуем функцию десериализации `deserializeIntFieldFilter`.

```

void FilterIndex::deserializeIntFieldFilter(
    const std::string& serialized_data
) {
    std::istringstream iss(serialized_data);
    std::string line;
    while (std::getline(iss, line)) {
        std::istringstream line_iss(line);
        // Чтение имени поля, значения и сериализованной битовой карты
        // из входного потока.
        std::string field_name;
        std::getline(line_iss, field_name, '|');
        std::string value_str;
        std::getline(line_iss, value_str, '|');
        long value = std::stol(value_str);
        // Чтение сериализованной битовой карты.
        std::string serialized_bitmap(std::istreambuf_iterator<char>(line_iss), {});
        // Десериализация битовой карты.
        roaring_bitmap_t* bitmap = roaring_bitmap_portable_deserialize(
            serialized_bitmap.data()
        );
        // Запись десериализованной битовой карты в intFieldFilter.
        intFieldFilter[field_name][value] = bitmap;
    }
}

```

Этот фрагмент кода определяет функцию `deserializeIntFieldFilter`, которая предназначена для десериализации данных фильтрующего индекса, ранее сериализованных, в объект битовой карты в памяти. Функция принимает строку `serialized_data`, содержащую сериализованные данные, в качестве входного параметра и затем построчно анализирует эти данные. Для каждой строки она сначала извлекает имя поля и целочисленное значение, затем десериализует строковое представление данных битовой карты в объект типа `roaring_bitmap_t`. В завершение этот объект битовой карты связывается с именем поля и целочисленным значением и сохраняется в отображении `intFieldFilter` для последующих операций фильтрации. Этот процесс позволяет данным, изначально хранящимся во внешнем формате, стать доступными для непосредственного использования в программе, работающей в памяти.

Однако реализация только сериализации и десериализации недостаточна, поскольку эти две функции не решают проблему персистентности. Поэтому необходимо перейти к реализации функций `saveIndex` и `loadIndex` в классе `FilterIndex`, чтобы сохранить данные на подходящем носителе. Здесь мы выбрали реализацию на основе базы данных RocksDB.

```

void FilterIndex::saveIndex(
    ScalarStorage& scalar_storage,
    const std::string& key
) {
    std::string serialized_data = serializeIntFieldFilter();
}

```

```

    // Сохранение сериализованных данных в ScalarStorage.
    scalar_storage.put(key, serialized_data);
}

void FilterIndex::loadIndex(
    ScalarStorage& scalar_storage,
    const std::string& key
) {
    std::string serialized_data = scalar_storage.get(key);
    // Десериализация intFieldFilter из сериализованных данных.
    deserializeIntFieldFilter(serialized_data);
}

```

Этот фрагмент кода определяет два члена функции в классе `FilterIndex`, ниже приведена логика их реализации.

- Функция `saveIndex` сначала вызывает функцию `serializeIntFieldFilter`, чтобы сериализовать данные битовой карты фильтрующего индекса в строковую форму `serialized_data`. Затем использует функцию `put` объекта `ScalarStorage`, чтобы сохранить эти сериализованные данные.
- Функция `loadIndex` выполняет противоположную операцию. Она сначала использует функцию `get` объекта `ScalarStorage`, чтобы выполнить запрос сериализованных данных. Затем вызывает функцию `deserializeIntFieldFilter`, чтобы десериализовать эти данные и восстановить фильтрующий индекс в его исходное состояние.

На данном этапе все три типа данных индекса в памяти имеют свои собственные функции `saveIndex` и `loadIndex`. Чтобы сделать систему моментальных снимков более унифицированной и универсальной, добавим соответствующие функции `saveIndex` и `loadIndex` в класс `IndexFactory`. Эти две функции инкапсулируют операции хранения и загрузки всех объектов индексов в системе. Для внешнего использования достаточно воспользоваться функциями `saveIndex` и `loadIndex`, предоставляемыми `IndexFactory`, которые будут предоставлять сервис как единое целое.

```

void IndexFactory::saveIndex(const std::string& folder_path,
                             ScalarStorage& scalar_storage) {
    for (const auto& index_entry : index_map) {
        IndexType index_type = index_entry.first;
        void* index = index_entry.second;
        // Генерация имени файла для каждого типа индекса.
        std::string file_path = folder_path +
                                std::to_string(static_cast<int>(index_type)) +
                                ".index";
        // Вызов соответствующей функции saveIndex в зависимости от типа индекса.
        if (index_type == IndexType::FLAT) {
            static_cast<FaissIndex*>(index)->saveIndex(file_path);
        } else if (index_type == IndexType::HNSW) {
            static_cast<HNSWLibIndex*>(index)->saveIndex(file_path);
        } else if (index_type == IndexType::FILTER) {

```

```

        // Сохранение индекса типа FilterIndex.
        static_cast<FilterIndex*>(index)->saveIndex(scalar_storage, file_path);
    }
}
}

```

Как видно, функция `saveIndex` класса `IndexFactory` циклически перебирает объекты индекса, хранящиеся в его отображении, и вызывает функцию `saveIndex` для объектов индекса различных типов, завершая персистентное хранение данных индекса. Логика реализации функции `loadIndex` класса `IndexFactory` аналогична и приведена ниже.

```

// Добавление реализации функции loadIndex.
void IndexFactory::loadIndex(const std::string& folder_path,
                             ScalarStorage& scalar_storage) {
    for (const auto& index_entry : index_map) {
        IndexType index_type = index_entry.first;
        void* index = index_entry.second;
        std::string file_path = folder_path
            + std::to_string(static_cast<int>(index_type)) + ".index";
        // Вызов соответствующей функции loadIndex в зависимости от типа индекса.
        if (index_type == IndexType::FLAT) {
            static_cast<FaissIndex*>(index)->loadIndex(file_path);
        } else if (index_type == IndexType::HNSW) {
            static_cast<HNSWLibIndex*>(index)->loadIndex(file_path);
        } else if (index_type == IndexType::FILTER) {
            // Загрузка индекса типа FilterIndex.
            static_cast<FilterIndex*>(index)->loadIndex(scalar_storage, file_path);
        }
    }
}
}

```

2. Расширение возможностей персистентности

После реализации соответствующих возможностей `saveIndex` и `loadIndex` в `IndexFactory` мы можем интегрировать эти две возможности в класс `Persistence`, поскольку именно последний модуль в конечном итоге предоставляет унифицированные возможности персистентности. Добавим в `Persistence` два новых метода.

```

void takeSnapshot(ScalarStorage& scalar_storage);
void loadSnapshot(ScalarStorage& scalar_storage);

```

На этом этапе необходимо уделить особое внимание проектированию механизма, который позволит фиксировать текущую позицию моментального снимка в момент его создания. Это требуется для того, чтобы впоследствии можно было найти в журнале предзаписи точку восстановления, с которой начнется применение инкрементальных изменений. Благодаря этому достигается бесшовная интеграция данных из предзаписи и файлов моментального снимка при восстановлении.

Ключом к реализации этого механизма является запись максимального ID предзаписи в журнал в момент создания моментального снимка. Мы считаем, что предзаписи с ID до этого уже были сохранены с помощью моментального снимка, и при перезапуске системы нужно будет воспроизводить только предзаписи после этого ID.

Таким образом, мы добавляем в `Persistence` две функции.

```
void saveLastSnapshotID();
void loadLastSnapshotID();
```

Теперь вернемся к реализации функции `takeSnapshot`.

```
void Persistence::takeSnapshot(ScalarStorage& scalar_storage) {
    GlobalLogger->debug("Taking snapshot");
    lastSnapshotID_ = increaseID_;
    std::string snapshot_folder_path = "snapshots_";
    IndexFactory* index_factory = getGlobalIndexFactory();
    index_factory->saveIndex(snapshot_folder_path, scalar_storage);
    saveLastSnapshotID();
}
```

Функция `takeSnapshot` предназначена для создания и хранения моментального снимка текущих данных индекса.

Сначала функция сохраняет текущее значение автоинкрементного ID предзаписи в переменной-члене `lastSnapshotID_`, так что при перезапуске системы только записи в журнал с ID, превышающим это значение, нуждаются в воспроизведении.

Затем функция вызывает метод `saveIndex` класса `IndexFactory`, передавая префикс пути к папке для хранения моментального снимка и ссылку на объект `ScalarStorage`, чтобы сохранить текущие данные индекса системы в указанной папке. Таким образом, текущие данные индекса сохраняются в виде файла моментального снимка.

В завершение вызывается функция `saveLastSnapshotID`, чтобы сохранить в хранилище максимальный автоинкрементный ID предзаписи, зафиксированный до выполнения операции моментального снимка, для использования при перезапуске системы.

3. Процесс восстановления системы

Теперь рассмотрим процесс загрузки файла моментального снимка из персистентного хранилища. Для реализации этой функции предназначена член-функция `loadSnapshot` класса `Persistence`.

```
void Persistence::loadSnapshot(ScalarStorage& scalar_storage) {
    // Запись отладочной информации, указывающей на начало операции загрузки
    // моментального снимка.
    GlobalLogger->debug("Loading snapshot");
    IndexFactory* index_factory = getGlobalIndexFactory();
```

```
// Вызов функции loadIndex из IndexFactory для загрузки состояния индекса
// из указанного пути к папке.
// Здесь используется путь к папке snapshots, который является местом
// хранения данных моментального снимка.
index_factory->loadIndex("snapshots_", scalar_storage);
}
```

Функция использует интерфейс `loadIndex` объекта `IndexFactory` для выполнения восстановления данных. Однако, чтобы поддерживать связь между предзаписью и данными моментального снимка, нам необходимо внести небольшое изменение в функцию `Persistence::readNextWALLog`.

```
if (log_id > lastSnapshotID){
    // Указатель для возврата json_data.
    json_data->Parse(json_data_str.c_str());
    GlobalLogger->debug(
        "Read WAL log entry: log_id={}, operation_type={},
        json_data_str={}",
        log_id_str, *operation_type, json_data_str
    );
    return;
}else {
    GlobalLogger->debug(
        "Skip Read WAL log entry: log_id={}, operation_type={},
        json_data_str={}",
        log_id_str, *operation_type, json_data_str);
}
```

В процессе фактической загрузки предзаписи необходимо определить, был ли текущий `log_id` сохранен в данных моментального снимка. Только данные журнала, записанные после последнего моментального снимка, нуждаются в воспроизведении.

► Обновление версии v0.2

На текущий момент наш объект `Persistence` полностью поддерживает функции `takeSnapshot` и `loadSnapshot`. Учитывая, что выполнение `takeSnapshot` вызывает определенные системные затраты, мы добавили интерфейс `/admin/snapshot` в `HttpServer`, чтобы администратор мог выбрать подходящий момент для выполнения операции моментального снимка. Функция `loadSnapshot` вызывается в процессе запуска системы, комбинируя файл моментального снимка и предзапись для восстановления данных системы. На данный момент наша автономная векторная база данных обладает достаточно полным функционалом, и мы обозначаем эту версию как v0.2. В табл. 4.11 приведены новые и обновленные модули, добавленные функции версии v0.2 для поддержки персистентности системы.

Таблица 4.11. Новые и обновленные модули, добавленные функции версии v0.2

Модуль	Задействованные файлы	Описание
FilterIndex	filter_index.h, filter_index.cpp	Реализует сериализацию и десериализацию фильтрующего индекса, поддерживает возможности saveIndex и loadIndex
FaissIndex	faiss_index.h, faiss_index.cpp	Поддерживает возможности saveIndex и loadIndex для структуры плоского индекса
HNSWLibIndex	hnswlib_index.h, hnswlib_index.cpp	Поддерживает возможности saveIndex и loadIndex для структуры HNSW-индекса
Persistence	persistence.h, persistence.cpp	Поддерживает возможности takeSnapshot и load-Snapshot
VectorDatabase	vector_database.h, vector_database.cpp	Комбинирует механизм моментальных снимков и предзапись, позволяя системе восстанавливаться на основе времени моментального снимка
HttpServer	http_server.cpp	Добавляет интерфейс <code>/admin/snapshot</code> , поддерживающий выполнение операции моментального снимка через команду

4.4. РЕЗЮМЕ

В этой главе, опираясь на некоторые базовые функции, уже существующие в отрасли, мы с нуля реализовали автономную векторную базу данных. Текущая версия обладает относительно простыми функциями, но благодаря модульной архитектуре будущие версии можно обогатить функциями и усовершенствованиями. Процесс реализации автономной векторной базы данных кратко изложен ниже.

Управление векторными данными

Мы начали с изучения основных функций открытой библиотеки FAISS, чтобы понять, как она использует структуры данных в памяти для индексации векторных данных. На основе FAISS мы реализовали функции записи и запроса для плоского индекса, который часто используется в сценариях с небольшими объемами векторных данных. Чтобы добавить поддержку управления векторными данными большего масштаба, мы изучили исходный код библиотеки HNSWLib, чтобы понять, как она использует многослойные графовые структуры для индексации векторных данных. На основе HNSWLib мы реализовали функции записи и запроса для HNSW-индекса, который часто используется в сценариях со средними и большими объемами векторных данных.

Управление смешанными данными

Для поддержки наиболее часто используемого в реальных бизнес-сценариях векторных баз данных режима гибридного запроса векторов и скаляров мы внедрили компонент скалярного хранилища базы данных RocksDB. С помощью RocksDB мы можем одновременно хранить данные векторов с плавающей запятой и данные фильтрации меток, которые затем можно легко извлечь из RocksDB по ID вектора. Далее, мы на основе отображений и структур данных битовой карты построили в памяти индексную структуру, поддерживающую гибридные

запросы. На основе этой индексной структуры мы реализовали фильтрующий индекс, что позволило удовлетворить базовые потребности в записи и запросе векторных данных в реальных бизнес-сценариях. В разделе 4.2.2 мы представили комбинированный класс `VectorDatabase`, который является единым входом для управления пользовательскими запросами. Определение класса `VectorDatabase` является одним из знаковых шагов в реализации векторной базы данных.

Восстановление после сбоев системы

В завершение мы сосредоточили внимание на «подводной части» – способности системы базы данных к восстановлению после сбоев. В конце концов, суть системы базы данных заключается в хранении данных и поддержке эффективных запросов, а поддержка персистентности данных делает систему базы данных полноценной. Для обеспечения персистентности мы сначала реализовали модуль предзаписи. С помощью предзаписи мы фиксируем каждое изменение, запрашиваемое пользователем, и при перезапуске системы можем восстановить данные на основе этой предзаписи. Однако такой метод восстановления с увеличением объема данных приводит к увеличению времени восстановления системы. Поэтому мы внедрили в систему механизм моментальных снимков, который опирается как на возможности хранения и загрузки индексов библиотек FAISS и HNSWLib, так и на возможности хранения и загрузки нашего собственного фильтрующего индекса. Мы эффективно скоординировали данные предзаписи и данные моментальных снимков, чтобы обеспечить быстрое восстановление системы.

В итоге наша векторная база данных была обновлена до версии v0.2, которая представляет собой автономную векторную базу данных с базовыми функциями. С точки зрения пользователя, эта версия уже является предварительно пригодной для использования – она поддерживает запись и запросы векторных данных, а также функции персистентности данных. Архитектура автономной векторной базы данных версии v0.2 представлена на рис. 4.5.

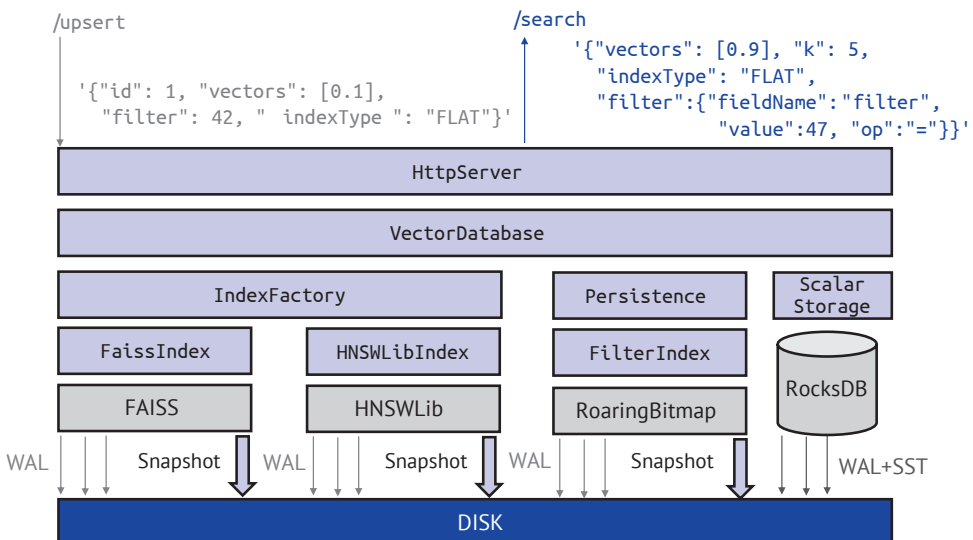


Рис. 4.5. Архитектура автономной векторной базы данных версии v0.2

Глава 5

Реализация распределенной векторной базы данных

Рыба – это то, чего я желаю; медвежья лапа – тоже то, чего я желаю.
Но если нельзя получить и то и другое, я откажусь от рыбы
и возьму медвежью лапу. Жизнь – это то, чего я желаю,
справедливость – тоже то, чего я желаю.
Если нельзя сохранить и то и другое, я откажусь от жизни
и выберу справедливость.

Мэн-цзы. Гао-цзы

В главе 4 мы реализовали автономную векторную базу данных, применив стратегию разложения сложных задач на более простые. Эта стратегия представляет собой алгоритмический подход «разделяй и властвуй», применяемый для решения сложных проблем. Однако автономная векторная база данных сталкивается с двумя ключевыми проблемами: во-первых, ограниченность ресурсов автономной системы, когда ресурсы (CPU, память, жесткий диск и т. д.) достигают предела, векторная база данных не может расширяться. Во-вторых, из-за вероятности отказа аппаратного обеспечения в случае сбоя автономной системы векторная база данных не может продолжать предоставлять сервисы. К счастью, благодаря десятилетиям развития компьютерных наук индустрия накопила богатый опыт «объединения частей в целое», который может помочь нам объединить несколько автономных систем в единое целое для предоставления единых услуг. Это по своей сути и является технологией распределенных баз данных.

Для преобразования в этой главе автономной векторной базы данных в распределенную мы сосредоточимся на решении следующих трех ключевых вопросов:

- как обеспечить согласованность данных на нескольких автономных узлах, то есть управление данными кластера;
- как управлять потоком доступа к кластеру, чтобы каждый узел в кластере выполнял свою роль, то есть управление потоком кластера;
- как обеспечить непрерывное предоставление услуг кластером в случае сбоя автономной системы, то есть управление исключениями кластера.

Кластер

Кластер обычно состоит из нескольких соединенных по сети компьютеров, которые совместно работают для выполнения определенной задачи. В распределенных системах кластер может использоваться для повышения производительности, доступности и масштабируемости.

Кластер обычно включает один или несколько главных узлов (также называемых лидерами, начальниками или главными серверами) и несколько подчиненных узлов (также называемых репликами, резервными или подчиненными серверами). Главный узел принимает от клиентов запросы на запись данных, подчиненные узлы синхронизируются с главным узлом, чтобы обеспечить согласованность данных и надежность кластера. Главный и подчиненные узлы обычно также принимают от клиентов запросы на чтение данных.

Теорема CAP

В проектировании распределенных систем существует основополагающий и ключевой теоретический результат, называемый *теоремой CAP* (также известной как теорема Брюера). Она утверждает, что в распределенной системе невозможно одновременно достичь всех трех свойств: C, A и P. C (consistency, согласованность) означает, что все реплики данных в системе остаются согласованными в любой момент времени. A (availability, доступность) – это способность системы непрерывно предоставлять услуги. P (partition tolerance, устойчивость к разделению) означает, что система может продолжать работать в условиях сетевого разделения. Согласно теореме распределенная система может одновременно удовлетворять только двум из трех свойств.

На практике из-за сетевых сбоев, аппаратных отказов, программных ошибок и других факторов сетевое разделение является распространенной и неизбежной ситуацией. При невозможности игнорировать устойчивость к разделению разработчики систем обычно должны делать выбор между согласованностью и доступностью в зависимости от конкретных требований.

Выбор в пользу согласованности означает, что система в любой момент времени гарантирует согласованность всех реплик данных. Например, когда система получает запрос на сохранение данных, она применяет определенные механизмы для обеспечения обновления всех реплик данных, чтобы гарантировать их согласованность. В режиме строгой согласованности система считает операцию записи успешной только после успешного обновления всех реплик. Однако такие строгие требования к согласованности могут привести к увеличению задержки операций записи и снижению доступности системы.

Выбор доступности может привести к определенному снижению согласованности. В этой модели система может применять некоторые стратегии для приоритизации обеспечения доступности сервиса. Например, сообщать пользователю об успешной записи сразу после успешной записи на главный узел, даже если другие реплики еще не завершили запись. Это позволяет системе продолжать предоставлять услуги, даже если часть узлов выходит из строя.

Однако такой подход может увеличить риск потери данных; например, если другие узлы находятся в процессе восстановления, а текущий обслуживающий узел снова выходит из строя, данные могут быть утеряны из-за отсутствия избыточного хранения.

В целом согласованность и доступность в распределенных системах взаимно ограничивают друг друга, и не существует идеального решения. Разработчики должны на этапе проектирования взвешивать эти аспекты в зависимости от конкретных требований и реальных условий работы системы.

5.1. УПРАВЛЕНИЕ ДАННЫМИ В КЛАСТЕРЕ

Первый шаг в реализации распределенной векторной базы данных – это организация управления данными в распределенной среде, что мы сокращенно называем управлением данными в кластере. Эффективное управление данными в кластере включает два ключевых аспекта: во-первых, репликация данных между множеством узлов, обеспечивающая точную передачу и согласованное хранение данных на разных узлах. Во-вторых, выбор ролей, то есть определение, какой узел имеет право на запись данных, в то время как другие узлы выполняют функции приема и репликации данных. Этот выбор ролей должен динамически корректироваться в зависимости от реальной ситуации в системе, особенно в случае отказа узла, когда необходимо перераспределить роли для обеспечения стабильной работы системы и согласованности данных. В распределенных системах для этого обычно используются протоколы согласования. Протокол согласования отвечает за обеспечение согласованности операций с данными между различными узлами, что позволяет системе даже в условиях сетевого разделения или отказа узлов правильно выбирать роли и поддерживать согласованность данных.

При реализации управления данными в кластере часто используются такие протоколы согласования, как Paxos, Raft и Gossip. Эти протоколы различаются по архитектуре, сложности реализации и применяемым сценариям. Мы можем выбрать подходящий протокол в зависимости от характеристик и требований системы.

Протокол Paxos

- Архитектура: протокол Paxos был предложен Лесли Лампортом (Leslie Lamport) и предназначен для решения проблемы согласованности в распределенных системах. Он основан на идее выборов главного узла и принятия решений большинством, обеспечивая согласование узлов системы по какому-либо значению через поэтапные предложения и механизмы голосования.
- Сложность реализации: реализация протокола Paxos относительно сложна, так как она включает в себя несколько этапов передачи сообщений и механизмов голосования, требующих обработки различных возможных сценариев и аномальных ситуаций.
- Применяемые сценарии: протокол Paxos обычно используется в системах, требующих строгой согласованности, таких как распределенные базы данных, распределенные транзакции и т. д.

Протокол Raft

- Архитектура: протокол Raft был предложен Диего Онгаро (Diego Ongaro) и Джоном Оустерхутом (John Ousterhout) с целью разработки более понятного и легко реализуемого алгоритма распределенной согласованности. Он вводит такие концепции, как выборы главного узла, репликация данных на основе журналов и безопасность, снижая сложность алгоритма за счет упрощения архитектуры и четких переходов в конечном автомате.
- Сложность реализации: по сравнению с протоколом Paxos реализация протокола Raft более интуитивна и понятна, так как он разделяет проблему согласованности на два основных вопроса: выборы главного узла и репликация данных на основе журналов, предоставляя четкую схему переходов в конечном автомате.
- Применяемые сценарии: протокол Raft подходит для систем, требующих высокой доступности и достаточно сильной согласованности, таких как распределенные системы хранения, распределенные вычислительные фреймворки и т. д.

Протокол Gossip

- Архитектура: протокол Gossip представляет собой децентрализованный распределенный протокол связи, вдохновленный феноменом слухов (английское слово *gossip* переводится как «слухи») в распространении информации. Он достигает согласованности данных во всей системе посредством случайной связи и обмена информацией между узлами, постепенно распространяя и синхронизируя информацию в системе.
- Сложность реализации: реализация протокола Gossip относительно проста, поскольку он не требует централизованного управляющего узла. Узлы могут самостоятельно осуществлять связь и обмен информацией. Однако из-за децентрализованной природы это может привести к задержкам в распространении сообщений и неопределенности.
- Сценарии применения: протокол Gossip обычно используется в крупных распределенных системах, таких как одноранговые сети, распределенные базы данных для выбора ролей и другие подобные сценарии.

В индустрии многие разработчики также используют функции репликации данных и выбора ролей собственной разработки, что позволяет сосредоточиться на основных функциях и обеспечить более высокую настраиваемость. Например, в открытой СУБД MySQL реализована функция репликации данных между множеством узлов, однако переключение главный–подчиненный требует поддержки внешних систем. В системе Redis, хотя и используется протокол Gossip для выбора ролей в кластере, репликация данных между узлами разработана самостоятельно.

С увеличением распространенности протокола Raft в открытых решениях все больше профессиональных систем баз данных начинают строиться на его основе. Использование существующей инфраструктуры для разработки новых систем стало основной концепцией архитектуры в индустрии. В нашей работе для управления данными в кластере векторной базы данных было выбрано открытое решение NuRaft.

5.1.1. Знакомство с NuRaft

Знакомство с NuRaft	
├ Краткое введение	
├ ─ Оптимизация и улучшение на основе протокола Raft	
├ Выборы главного узла	
├ ─ Механизм приоритетов	
├ ─ Проверка доступности и процесс выборов	
├ Конечный автомат и интерфейс NuRaft	
├ ─ Внешний API-интерфейс взаимодействия	
├ ─ Модуль пользовательской разработки	
├ Режимы репликации данных	
├ ─ Строгая согласованность	
├ ─ Асинхронная репликация	

1. Краткое введение

NuRaft – это легковесная версия протокола Raft, реализованная на C++ и открытая командой eBay. Она унаследовала два основных принципа и механизма работы протокола Raft: во-первых, автоматические выборы главного узла для эффективной координации ролей множества узлов в кластере (далее мы будем называть это выборами главного узла). Во-вторых, репликация данных между множеством узлов на основе журнала (далее мы будем называть это репликацией данных), но с некоторыми оптимизациями и улучшениями для реальных сценариев применения, особенно подходящими для крупных распределенных систем.

В NuRaft добавлены некоторые новые функции на основе протокола Raft, чтобы удовлетворить специфические потребности крупных компаний, таких как eBay, в реальных приложениях. Далее мы кратко рассмотрим некоторые из новых функций.

- Протокол предварительного голосования: проведение одного раунда предварительного голосования перед официальными выборами для сокращения ненужных выборов и повышения их эффективности.
- Выборы главного узла на основе приоритетов: влияние на результаты выборов путем установки приоритетов, что делает процесс выборов более контролируемым и предсказуемым.
- Тайм-аут главного узла: введение концепции истечения срока действия главного узла (тайм-аут), чтобы избежать застоя кластера из-за длительной неактивности главного узла.
- Логические моментальные снимки на основе объектов: оптимизация процесса хранения и восстановления журнала с помощью механизма моментальных снимков на основе логических блоков объектов.
- Поддержка протокола SSL/TLS: обеспечение безопасного механизма связи, повышающего безопасность передачи данных.

Здесь мы сосредоточим внимание на двух основных функциях – выборах главного узла и репликации данных.

2. Выборы главного узла

В механизме выборов главного узла NuRaft использует внутренний механизм постоянной проверки доступности множества узлов, чтобы в случае аномалии

одного узла другие узлы могли выбрать новый главный узел с помощью метода большинства голосов. NuRaft реализует механизм выборов главного узла на основе приоритетов, описание которого приводится ниже.

На рис. 5.1 изображена схема выборов в системе NuRaft с несколькими узлами. В системе, состоящей из пяти узлов, каждому узлу присваивается разный приоритет выборов. S1 является начальным главным узлом. Tn обозначает целевой приоритет текущего узла Sn , а Pn – собственный приоритет текущего узла Sn . Например, в момент времени 0 $T1$ для S1 равен 100, и $P1$ также равен 100. В то время как для S3 $T3$ равен 100, а $P3$ равен 80. X указывает на то, что в текущий момент времени в узле Sn происходит событие (например, таймаут узла, голосование и т. д.). Горизонтальные стрелки указывают направление передачи событий между узлами.

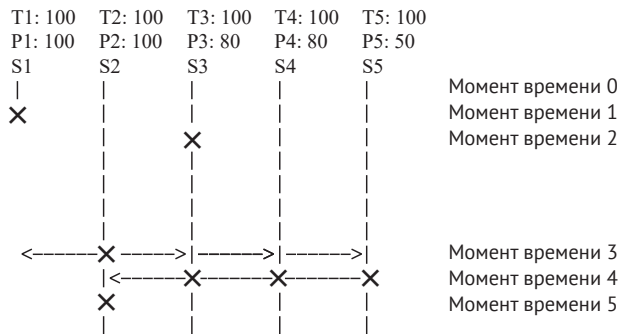


Рис. 5.1. Схема выборов в системе NuRaft с несколькими узлами

Процесс выборов выглядит следующим образом.

- В момент времени 0 инициализируются $T1$ – $T5$ значением 100, $P1$ и $P2$ – значением 100, $P3$ – значением 80, $P4$ – значением 80, $P5$ – значением 50, S1 становится главным узлом.
- В момент времени 1 узел S1 выходит из строя и отключается.
- В момент времени 2 узел S3 обнаруживает, что узел S1 отключен, но из-за того, что $P3$ меньше $T3$, не инициирует голосование за выборы главного узла.
- В момент времени 3 узел S2 обнаруживает, что узел S1 отключен, и так как $P2$ больше или равно $T2$, инициирует голосование за выборы главного узла.
- В момент времени 4 узлы S3, S4 и S5 обнаруживают, что $P2$ больше или равно $T3$, $T4$, $T5$, и выбирают S2 в качестве нового главного узла.
- В момент времени 5 S2 становится новым главным узлом.

3. Конечный автомат и интерфейсы NuRaft

Помимо механизма выборов главного узла, NuRaft также предоставляет полный набор интерфейсов для преобразования конечного автомата и репликации данных, поддерживая разработчиков в создании пользовательской логики преобразования конечного автомата и поведения репликации данных между узлами. Официальная документация NuRaft подробно описывает основные

возможности интерфейсов, предоставляемых в рамках NuRaft, часть из которых представлена на рис. 5.2.

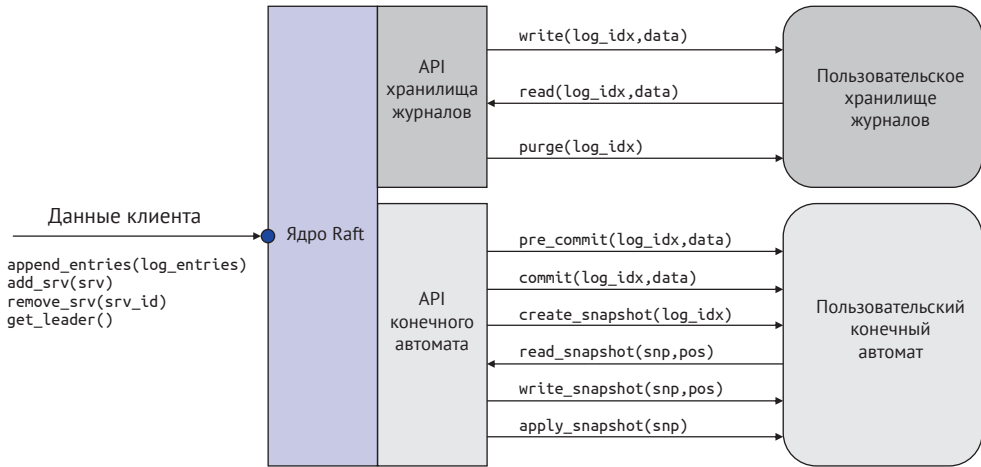


Рис. 5.2. Схема конечного автомата и интерфейсов NuRaft

NuRaft предоставляет несколько основных внешних интерфейсов, таких как `append_entries`, `add_srv`, `remove_srv` и др., которые позволяют клиентским данным взаимодействовать с подсистемой NuRaft через сетевые протоколы (TCP/IP).

Что касается внутренних интерфейсов, функциональность NuRaft можно разделить на два логических модуля: API-интерфейс хранилища журналов и API-интерфейс конечного автомата. Модуль API-интерфейса хранилища журналов предоставляет интерфейсы операций, связанные с возможностями журналов, включая интерфейс записи (`write`, предоставляет возможность записи в журнал), интерфейс чтения (`read`, предоставляет возможность чтения из журнала), интерфейс очистки (`purge`, предоставляет возможность очистки журнала). Модуль API-интерфейса конечного автомата предоставляет интерфейсы операций преобразования конечного автомата в процессе фиксации журналов Raft, включая интерфейсы фиксации данных (`pre_commit`, `commit`) и интерфейсы, связанные с моментальными снимками (`create_snapshot`, `read_snapshot`, `write_snapshot`, `apply_snapshot`). Оба модуля предоставляют разработчикам возможность настройки, позволяя им интегрировать бизнес-логику в рамках подсистемы NuRaft и разрабатывать пользовательский код для обработки данных.

На рис. 5.3 изображена диаграмма процесса смены состояния основных API-интерфейсов NuRaft, которая дополнительно объясняет, как главный и подчиненные узлы вызывают основные интерфейсы NuRaft в процессе записи данных. Эта диаграмма последовательности помогает глубже понять, как узлы взаимодействуют друг с другом в реальных операциях и как в этом процессе вызываются API-интерфейсы.

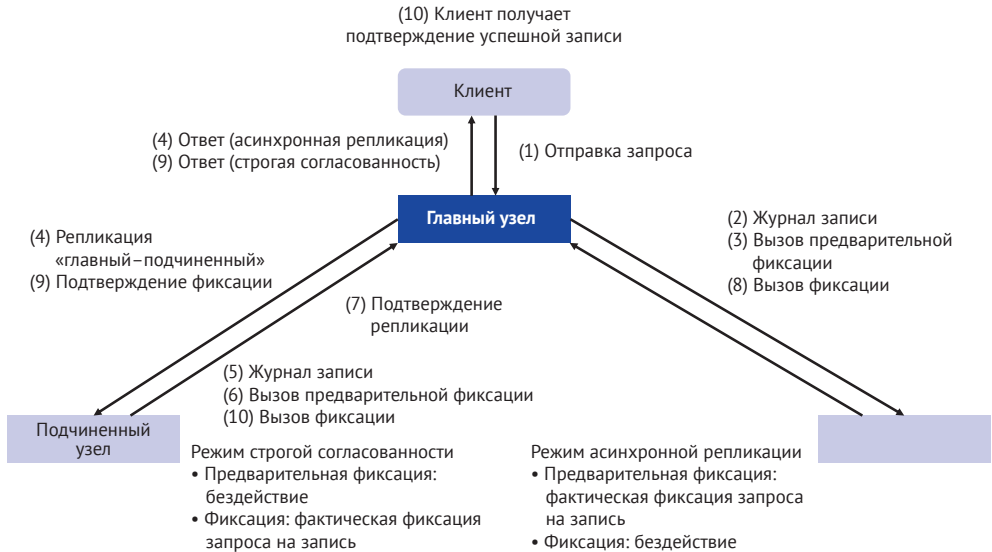


Рис. 5.3. Диаграмма процесса смены состояния основных API-интерфейсов NuRaft

Рисунок 5.3 подробно демонстрирует полный процесс от отправки клиентом запроса до получения клиентом ответа об успешной записи, а также использование интерфейсов, связанных с преобразованием конечного автомата и репликацией данных на всем пути. Полный процесс записи выглядит следующим образом.

1. Клиент отправляет запрос на запись главному узлу.
2. Главный узел, получив запрос на запись, вызывает интерфейс записи в журнал (`append_entries`) и записывает его в локальный журнал.
3. После записи в локальный журнал вызывается интерфейс предварительной фиксации (`pre_commit`). Здесь следует обратить особое внимание: в режиме строгой согласованности предварительная фиксация фактически ничего не делает. В то время как в режиме асинхронной репликации предварительная фиксация выполняет операцию вступления в силу запроса на запись, делая данные доступными для других запросов.
4. После предварительной фиксации данные в форме журнала Raft копируются с главного узла на подчиненные. В режиме асинхронной репликации главный узел на этом этапе возвращает клиенту ответ об успешной записи.
5. Подчиненные узлы записывают данные в локальный журнал.
6. Подчиненные узлы вызывают интерфейс предварительной фиксации.
7. Подчиненные узлы сообщают главному узлу, что данные успешно скопированы на подчиненные узлы.
8. Главный узел вызывает интерфейс фиксации (`commit`). Здесь следует обратить особое внимание: в режиме строгой согласованности фиксация действительно завершает операцию вступления в силу запроса на запись. В режиме асинхронной репликации фиксация фактически ничего не делает.
9. После того как главный узел завершает операцию вступления в силу данных запроса на запись, он уведомляет подчиненные узлы о необходимости также завершить эту операцию. Одновременно в режиме строгой согласованности главный узел возвращает клиенту ответ об успешной записи.

10. Подчиненный узел, получив запрос на подтверждение фиксации от главного узла, завершает операцию вступления в силу данных локально. На этом процесс записи данных завершается.

4. Режимы репликации данных

Когда главный узел получает запрос на запись от клиента, подсистема NuRaft координирует операции между главным и подчиненными узлами, вызывая соответствующие API-интерфейсы в определенном порядке для завершения репликации данных. NuRaft предоставляет два режима репликации: режим строгой согласованности и режим асинхронной репликации.

Режим строгой согласованности

Главный узел ожидает, пока большинство подчиненных узлов подтвердят успешное получение и репликацию записи журнала, прежде чем зафиксировать эту запись. Это означает, что до фиксации записи журнала главным узлом большинство подчиненных узлов должны завершить репликацию этой записи, что обеспечит согласованность данных. Этот режим предоставляет наивысший уровень согласованности, но может увеличить задержку, так как главный узел должен ожидать ответа от подчиненных узлов.

Режим асинхронной репликации

Главный узел может продолжать обрабатывать следующий запрос после отправки записи журнала подчиненным узлам, не дожидаясь их ответа. Этот режим может повысить пропускную способность и производительность системы, так как главный узел может продолжать обработку запросов без ожидания ответа от подчиненных узлов. Однако это может привести к тому, что данные на подчиненных узлах будут «отставать» от данных на главном узле, снижая согласованность.

Эти два режима предоставляют различные уровни гарантии согласованности данных. Если разработчик больше ценит производительность, например в интернет-торговле, можно выбрать режим асинхронной репликации. Если разработчик больше ценит согласованность данных, например в финансовом бизнесе, можно выбрать режим строгой согласованности.

5.1.2. Установление отношения главный–подчиненный

Установление отношения главный-подчиненный

- ├ Пользовательская реализация конечного автомата и логики репликации данных
 - ├ inmem_state_mgr: менеджер состояний
 - ├ log_state_machine: логика конечного автомата для обработки журналов
- ├ Унифицированное управление информацией Raft
 - ├ RaftStuff: унифицированный класс узлов Raft
 - ├ Важные функции
 - ├ init: инициализация узла Raft
 - ├ addSrv: добавление узла
 - ├ isLeader: определение главного узла
- ├ Интеграция функций главный-подчиненный в HttpServer
 - ├ Добавление указателя RaftStuff
 - ├ Реализация addFollowerHandler для обработки запросов новых подчиненных узлов

Чтобы интегрировать подсистему NuRaft в автономную векторную базу данных, описанную в главе 4, необходимо сосредоточиться на двух ключевых классах NuRaft: `state_mgr` (менеджер состояний) и `state_machine` (конечный автомат).

- `state_mgr` отвечает за управление информацией о состоянии узлов, включая включение и выключение узлов, выборы главного узла из-за тайм-аута и изменения конфигурации. `state_mgr` синхронизирует информацию о состоянии между узлами и обрабатывает преобразования и обновления состояний.
- `state_machine` является абстрактным классом, представляющим логику преобразования конечного автомата системы NuRaft. Он определяет, как обрабатывать и фиксировать записи журнала после их получения для изменения состояния системы. `state_machine` определяет бизнес-логику и поведение системы и является ключевым классом для реализации прикладной логики в NuRaft.

1. Пользовательская реализация конечного автомата и логики репликации данных

На основе примера `inmem_state_mgr` в исходном коде NuRaft мы реализуем менеджер состояний `inmem_state_mgr`, подходящий для нашей системы. Базовый код реализации приведен ниже.

```
class inmem_state_mgr: public state_mgr {
public:
    // Конструктор, инициализация информации об узле и хранилища журналов.
    inmem_state_mgr(int srv_id, const std::string& endpoint,
                    VectorDatabase* vector_database)
        : my_id_(srv_id)
        , my_endpoint_(endpoint)
        , cur_log_store_( cs_new<inmem_log_store>(vector_database) ) {
        // Инициализация информации о конфигурации сервера и кластера.
        my_srv_config_ = cs_new<srv_config>( srv_id, endpoint );
        saved_config_ = cs_new<cluster_config>();
        saved_config_ ->get_servers().push_back(my_srv_config_);
    }
    // Деструктор.
    ~inmem_state_mgr() {}
    ptr<cluster_config> load_config();// Загрузка информации о конфигурации кластера.
    void save_config(const cluster_config& config); // Сохранение информации
                                                // о конфигурации кластера.
    void save_state(const srv_state& state); // Сохранение информации
                                                // о состоянии узла.
    ptr<srv_state> read_state(); // Чтение информации о состоянии узла.
    ptr<log_store> load_log_store(); // Загрузка хранилища журналов.
    int32 server_id(); // Получение ID сервера.
    ptr<srv_config> get_srv_config(); // Получение информации о конфигурации сервера.
private:
    int my_id_;
    std::string my_endpoint_;
    ptr<inmem_log_store> cur_log_store_;
```

```
ptr<srv_config> my_srv_config_;
ptr<cluster_config> saved_config_;
ptr<srv_state> saved_state_;
};
```

Этот фрагмент кода реализует класс `inmem_state_mgr`, предназначенный для управления информацией о состоянии узла. Класс предоставляет функции управления и доступа к информации о состоянии узла, конфигурации кластера и хранилищу журналов через несколько методов, поддерживая тем самым работу протокола NuRaft.

После завершения реализации пользовательского класса `inmem_state_mgr`, NuRaft может работать на его основе в сочетании с другими встроенными классами, чтобы выполнять такие основные функции, как подключение и отключение узлов, изменение конфигурации и выбор главного узла.

Конечный автомат, будучи носителем преобразований состояний, играет ключевую роль при обработке журналов, отправленных пользователем. Он предоставляет настраиваемую точку входа для определения ключевых функций в процессе фиксации журналов, таких как `commit`, `pre_commit` и др. Здесь мы можем сначала определить простой класс-заглушку, чтобы заложить основу для последующей реализации конкретных функций. Например, класс `log_state_machine` определяет логику преобразования конечного автомата для обработки журналов.

```
class log_state_machine : public state_machine {
public:
    ptr<buffer> commit(const ulong log_idx, buffer& data); // Фиксация журнала
                                                         // и возврат результата.
    void pre_commit(const ulong log_idx); // Действия на этапе предварительной
                                         // фиксации.
    void rollback(const ulong log_idx); // Откат к указанному индексу журнала.
    ulong last_commit_index(); // Получение индекса последней фиксации журнала.
};
```

2. Унифицированное управление информацией Raft

После определения классов `inmem_state_mgr` и `log_state_machine` мы готовы интегрировать подсистему NuRaft в нашу векторную базу данных. Для этого мы введем новый класс `RaftStuff`. Этот объект не только служит унифицированным интерфейсом для всех функций, связанных с Raft, но и является базовой управляющей единицей узла Raft в нашей системе. Базовый код реализации `RaftStuff` представлен ниже.

```
class RaftStuff {
public:
    // Конструктор, инициализирующий объект RaftStuff.
    RaftStuff(int node_id, const std::string& endpoint, int port);
    void init(); // Инициализация узла Raft.
    // Добавление нового узла в кластер Raft.
```

```

ptr< cmd_result< ptr<buffer> > > addSrv(
    int srv_id, const std::string& srv_endpoint);
// Добавление объявления функции isLeader для определения, является ли
// текущий узел главным.
bool isLeader() const;
private:
    ptr<state_mgr> smgr_; // Указатель на объект менеджера состояний.
    ptr<state_machine> sm_; // Указатель на объект конечного автомата.
    raft_launcher launcher_; // Объект запуска Raft.
    ptr<raft_server> raft_instance_; // Указатель на объект сервера Raft.
};

```

В табл. 5.1 приведено описание важных переменных и функций-членов структуры **RaftStuff**.

Таблица 5.1. Важные члены структуры **RaftStuff**

Имя	Категория	Описание
smgr_	Переменная	Менеджер состояний NuRaft, соответствующий данному узлу Raft
sm_	Переменная	Конечный автомат NuRaft, соответствующий данному узлу Raft
launcher_	Переменная	Запускающий модуль узла Raft
raft_instance	Переменная	Объект времени выполнения узла Raft
init	Функция	Функция инициализации объекта RaftStuff
addSrv	Функция	Добавление других узлов Raft на основе уже инициализированного raft_instance , например добавление подчиненного узла
isLeader	Функция	Функция, определяющая, является ли текущий узел главным, на основе информации о состоянии из smgr_

Функция **init** в **RaftStuff** реализует запуск узла протокола Raft, ниже представлен ее код.

```

void RaftStuff::init() {
    // Инициализация менеджера состояний и конечного автомата.
    smgr_ = cs_new<inmem_state_mgr>(node_id, endpoint);
    sm_ = cs_new<log_state_machine>();

    // Настройка параметров службы ASIO.
    asio_service::options asio_opt;
    asio_opt.thread_pool_size_ = 1;

    // Настройка параметров Raft.
    raft_params params;

```

```

params.election_timeout_lower_bound_ = 100000;
params.election_timeout_upper_bound_ = 200000;
// Инициализация экземпляра Raft.
raft_instance_ = launcher_.init(sm_, smgr_, nullptr, port_, asio_opt, params);
GlobalLogger->debug(
    "RaftStuff initialized with node_id: {}, endpoint: {}, port: {}", node_id,
    endpoint, port_); // Добавление записи в журнал.
}

```

Из кода видно, что функция `init` инициализирует `smgr_` и `sm_` через `inmem_state_mgr` и `log_state_machine`. Эти два члена автоматически вызываются в процессе работы подсистемы NuRaft при преобразовании конечного автомата. Затем с помощью функции `init` объекта `launcher_` запускается локально работающий узел Raft и возвращается объект `raft_instance_`, выполняющийся после запуска. Этот объект будет служить унифицированной точкой входа для последующих операций с данным узлом Raft.

Функция `addSrv` отвечает за добавление других узлов в текущую группу Raft. Ее код представлен ниже.

```

ptr< cmd_result< ptr<buffer> > > RaftStuff::addSrv(
    int srv_id, const std::string& srv_endpoint) {
    ptr<srv_config> peer_srv_conf = cs_new<srv_config>(
        srv_id, srv_endpoint);
    // Добавление записи в журнал.
    GlobalLogger->debug(
        "Adding server with srv_id: {}, srv_endpoint: {}",
        srv_id, srv_endpoint);
    return raft_instance_->add_srv(*peer_srv_conf);
}

```

Функция-член `addSrv` класса `RaftStuff` добавляет новый серверный узел в существующий кластер Raft для его расширения. Функция принимает два параметра: ID нового серверного узла (`srv_id`) и адрес точки доступа (`srv_endpoint`). В функции сначала создается новый объект конфигурации сервера `peer_srv_conf`, на основе которого вызывается функция `add_srv` экземпляра Raft. В нее передается новый объект конфигурации сервера, а возвращается результат добавления сервера. Конечно, для реализации этого процесса необходимо убедиться, что целевой узел также запустил соответствующий экземпляр `raft_instance_`, ID и адрес точки доступа которого – это информация о добавляемом узле, что является основным условием построения кластера Raft.

3. Интеграция функций главного и подчиненного узлов

в класс `HttpServer`

Для того чтобы класс `RaftStuff` мог предоставлять услуги, мы интегрируем его в качестве члена-переменной в `HttpServer`. Поэтому в определении `HttpServer` нужно добавить новую переменную.

```
RaftStuff* raft_stuff_; // Изменено на указатель RaftStuff.
```

С помощью этого указателя `raft_stuff` мы можем предоставлять в `HttpServer` команды, связанные с Raft. Ниже приведена реализация функции добавления подчиненного узла в `HttpServer`.

```
void HttpServer::addFollowerHandler(const httplib::Request& req,
httplib::Response& res) {
    GlobalLogger->debug("Получен запрос addFollower");
    // Извлечение информации о подчиненном узле из JSON-запроса.
    int node_id = json_request["nodeId"].GetInt();
    std::string endpoint = json_request["endpoint"].GetString();
    // Вызов функции addSrv из RaftStuff для добавления нового подчиненного
    // узла в кластер.
    raft_stuff->addSrv(node_id, endpoint);
}
```

Далее мы регистрируем обработчик команды `addFollower` в конструкторе `HttpServer`.

```
server.Post("/admin/addFollower", [this](const httplib::Request& req,
httplib::Response& res) {
    addFollowerHandler(req, res);
}));
```

На этом этапе мы уже можем запускать два различных экземпляра `vdb_server` и объединять их в кластер Raft. Эти два экземпляра могут работать на одной машине, достаточно настроить их на прослушивание разных портов. Затем мы можем выбрать один из узлов в качестве главного узла и с помощью команды `addFollower` добавить другой узел в качестве подчиненного в кластер Raft. Таким образом, два узла Raft смогут установить отношение главный–подчиненный. Ниже приведен пример соответствующего запроса.

```
Запрос:
curl -X POST -H "Content-Type: application/json" -d '{"nodeId": 2, "endpoint":
"127.0.0.1:9091"}' http://localhost:8080/admin/addFollower
Ответ:
{"retCode":0}
```

Этот код использует команду cURL для отправки запроса HTTP POST по адресу `http://localhost:8080/admin/addFollower`. Содержимое запроса – данные в формате JSON: `{"nodeId": 2, "endpoint": "127.0.0.1:9091"}`. Ожидается, что этот запрос добавит узел с ID 2 и адресом доступа на порту 9091 на текущем узле в качестве подчиненного. Сервер, получив запрос, успешно обработал его и вернул данные в формате JSON `{"retCode":0}`. Здесь значение `retCode` равно 0, что означает успешное добавление этого узла в текущий кластер, и целевой узел стал подчиненным узлом текущего узла.

5.1.3. Реализация репликации данных

Реализация репликации данных

- ├─ Класс `inmem_log_store`
 - ├─ Выполнение репликации данных между главным и подчиненным узлами
- ├─ Функция `append`
 - ├─ Запись журнала пользовательских запросов на запись
- ├─ Функция `commit`
 - ├─ Фиксация данных журнала, запись и обновление вступают в силу
- ├─ Функция `upsertHandler`
 - ├─ Обработка запросов на запись в `HttpServer` с записью в журнал

Следующая задача после установления отношения главный–подчиненный между двумя узлами с помощью протокола Raft – усовершенствовать систему, реализовав репликацию данных между главным и подчиненным узлами. Этот шаг крайне важен, так как он не только обеспечивает хранение на подчиненном узле данных, записанных через главный узел, но и усиливает распределенные возможности системы, предоставляя определенную степень защиты от сбоев.

Подсистема NuRaft предоставляет родительский класс `log_store`, который открывает разработчикам возможность для собственной реализации. Через этот интерфейс мы можем объединить функцию репликации данных NuRaft с нашей существующей системой.

1. Класс `inmem_log_store`

Мы воспользовались примером `inmem_log_store` из исходного кода NuRaft и реализовали в `inmem_log_store` функцию репликации данных между главным и подчиненным узлами. Базовое определение класса `inmem_log_store` выглядит следующим образом:

```
class inmem_log_store : public log_store {
public:
    // Добавление указателя VectorDatabase для инициализации.
    inmem_log_store(VectorDatabase* vector_database);
    ~inmem_log_store(); // Деструктор
    __nocopy__(inmem_log_store); // Запрет конструктора копирования.
public:
    ulong next_slot() const; // Получение следующего слота журнала.
    ulong start_index() const; // Получение начального индекса.
    ptr<log_entry> last_entry() const; // Получение последней записи журнала.
    ulong append(ptr<log_entry>& entry); // Добавление записи журнала.
    // Запись записи журнала по указанному индексу.
    void write_at(ulong index, ptr<log_entry>& entry);
private:
    // Статическая функция создания копии записи журнала.
    static ptr<log_entry> make_clone(const ptr<log_entry>& entry);
    // Контейнер записей журнала, индекс в качестве ключа.
    std::map<ulong, ptr<log_entry>> logs_;
    mutable std::mutex logs_lock_; // Мьютекс для защиты контейнера записей журнала.
```

```

// Начальный индекс журнала, атомарный тип для конкурентных операций.
std::atomic<ulong> start_idx_;
// Указатель на VectorDatabase для операций с векторной базой данных.
VectorDatabase* vector_database_;
};

```

В `inmem_log_store`, основанном на родительском классе `log_store`, мы реализовали ключевую функцию `append`. Эта функция используется для записи пользовательских запросов на запись, а также вызывает функцию персистентности для сохранения данных. Для удобства взаимодействия с текущим классом `vector_database` мы передаем указатель `vector_database` в конструктор `inmem_log_store` и сохраняем его в качестве члена-переменной.

2. Функция `append`

Ниже приведена реализация функции `append`.

```

ulong inmem_log_store::append(ptr<log_entry>& entry) {
    // Клонирование записи журнала для обеспечения потокобезопасности.
    ptr<log_entry> clone = make_clone(entry);
    // Блокировка контейнера записей журнала для предотвращения конкурентной записи.
    std::lock_guard<std::mutex> l(logs_lock_);
    // Вычисление индекса текущей записи журнала.
    size_t idx = start_idx_ + logs_.size() - 1;
    // Добавление клонированной записи журнала в контейнер записей журнала.
    logs_[idx] = clone;
    // Печать содержимого журнала в зависимости от типа записи.
    if (entry->get_val_type() == log_val_type::app_log) {
        buffer& data = clone->get_buf();
        std::string content(
            reinterpret_cast<const char*>(data.data() + data.pos() + sizeof(int)),
            data.size() - sizeof(int));
        GlobalLogger->debug(
            "Append app logs {}, content: {}, value type {}", idx, content,
            entry->get_val_type()); // Добавление печати журнала.
        vector_database->writeWALLogWithID(idx, content);
    } else {
        buffer& data = clone->get_buf();
        std::string content(
            reinterpret_cast<const char*>(data.data() + data.pos()), data.size());
        GlobalLogger->debug(
            "Append other logs {}, content: {}, value type {}", idx, content,
            entry->get_val_type()); // Добавление печати журнала.
    }
    if (disk_emul_delay) { // Эмуляция задержки записи на диск.
        uint64_t cur_time = timer_helper::get_timeofday_us();
        disk_emul_logs_being_written[cur_time + disk_emul_delay * 1000] = idx;
        disk_emul_ea_.invoke(); // Триггер события эмуляции диска.
    }
    return idx; // Возвращение индекса текущей записи журнала.
}

```


Можно увидеть, что функция `append` отвечает за обработку пользовательских запросов на запись журнала данных из подсистемы NuRaft. Эта функция, вызывая функцию `writeWALLogWithID` объекта `vector_database`, персистентно записывает эти данные журнала в систему. Этот механизм позволяет системе восстанавливать данные через журнал в случае сбоя на одном узле.

Функция `writeWALLogWithID` была добавлена в нашей системе предзаписи журнала в главе 4 и позволяет записывать предзапись на основе указанного ID. Код этой функции представлен ниже.

```
void VectorDatabase::writeWALLogWithID(uint64_t log_id, const std::string& data) {
    std::string operation_type = "upsert"; // По умолчанию operation_type
                                           // равно upsert.
    std::string version = "1.0"; // Устанавливаем фактический номер версии.
    // Вызов функции writeWALRawLog объекта persistence_.
    persistence_.writeWALRawLog(log_id, operation_type, data, version);
}
```

3. Функция `commit`

Очевидно, что простого копирования журнала на узлы недостаточно, необходимо также фактически зафиксировать эти данные на главном и подчиненном узлах. Только так последующие запросы на чтение смогут действительно получить доступ к данным. Подсистема NuRaft предоставляет соответствующий пользовательский интерфейс, чтобы помочь разработчикам реализовать эту функцию. Нам нужно реализовать функции `commit` и `pre_commit` в классе `log_state_machine`, определенном в разделе 5.1.2. Ниже представлен базовый код реализации функции `commit`, который приводится в качестве примера.

```
ptr<buffer> log_state_machine::commit(const ulong log_idx, buffer& data) {
    std::string content(reinterpret_cast<const char*>(
        data.data() + data.pos() + sizeof(int)),
        data.size() - sizeof(int)); // Разбор содержимого журнала и вывод.
    GlobalLogger->debug("Commit log_idx: {}", content: {}, log_idx, content);
    // Разбор JSON-запроса.
    rapidjson::Document json_request;
    json_request.Parse(content.c_str());
    uint64_t label = json_request[REQUEST_ID].GetUint64();
    // Обновление последнего зафиксированного индекса журнала.
    last_committed_idx = log_idx;
    // Получение типа индекса из параметров запроса.
    IndexFactory::IndexType indexType =
        vector_database->getIndexTypeFromRequest(json_request);
    // Запись или обновление данных в векторной базе данных.
    vector_database->upsert(label, json_request, indexType);
    // Возврат индекса журнала Raft в качестве результата.
    ptr<buffer> ret = buffer::alloc(sizeof(log_idx));
    buffer_serializer bs(ret);
    bs.put_u64(log_idx);
    return ret;
}
```

С точки зрения реализации этот процесс относительно прост. Нам нужно лишь получить из подсистемы NuRaft данные, которые необходимо зафиксировать, затем вызвать функцию обновления `vector_database` для записи результата, тем самым завершив фиксацию данных. Как только вызывается интерфейс `commit`, данные становятся доступными для запросов пользователей через интерфейс. На этом этапе мы можем подтвердить клиенту успешную запись данных.

4. Функция `upsertHandler`

Далее наша задача – обновить функцию `upsertHandler` в классе `HttpServer`. Изначально эта функция напрямую использовала функцию `upsert` класса `vector_database`. Теперь ее нужно заменить на логику записи, основанную на подсистеме NuRaft. Новая функция `upsertHandler` принимает запросы на запись от клиента, добавляет содержимое запроса в журнал кластера Raft и возвращает клиенту JSON-ответ, указывающий на результат обработки запроса. Базовый код реализации представлен ниже.

```
void HttpServer::upsertHandler(const httplib::Request& req, httplib::Response&
res) {
    GlobalLogger->debug("Received upsert request");
    // Разбор JSON-запроса.
    rapidjson::Document json_request;
    json_request.Parse(req.body.c_str());
    uint64_t label = json_request[REQUEST_ID].GetUint64();
    // Получение типа индекса из параметров запроса.
    IndexFactory::IndexType indexType = getIndexTypeFromRequest(json_request);
    raft_stuff_->appendEntries(req.body); // Добавление новой записи журнала
                                         // в кластер.

    // Построение JSON-ответа.
    rapidjson::Document json_response;
    json_response.SetObject();
    rapidjson::Document::AllocatorType& response_allocator =
        json_response.GetAllocator();
    json_response.AddMember(RESPONSE_RETCODE,
                           RESPONSE_RETCODE_SUCCESS,
                           response_allocator);

    // Добавление retCode в ответ.
    setJsonResponse(json_response, res); // Установка содержимого ответа.
}
```

► Обновление версии v0.3

На этом этапе мы реализовали репликацию данных между главным и подчиненными узлами. Используя пользовательские интерфейсы, предоставляемые подсистемой NuRaft, мы смогли реализовать основные процессы без необходимости писать избыточный код самого протокола Raft. Это пример того, как открытое программное обеспечение и лучшие практики разработки помогают быстро достичь целей.

Обозначим эту версию как v0.3, она поддерживает установление отношения главный–подчиненный и функцию репликации данных между узлами.

В табл. 5.2 приведены новые и обновленные модули, добавленные функции версии v0.3 для реализации отношений главный–подчиненный и репликации данных после внедрения NuRaft.

Таблица 5.2. Новые и обновленные модули, добавленные функции версии v0.3

Модуль	Затронутые файлы	Описание
<code>inmem_state_mgr</code>	<code>in_memory_state_mgr.h</code>	Реализует пользовательские функции обновления состояния и информации узлов в подсистеме NuRaft
<code>log_state_machine</code>	<code>log_state_machine.h</code> <code>log_state_machine.cpp</code>	Реализует функцию фиксации данных на нескольких узлах в подсистеме NuRaft
<code>inmem_log_store</code>	<code>in_memory_log_store.h</code> <code>in_memory_log_store.cpp</code>	Реализует функцию репликации журнала между несколькими узлами в подсистеме NuRaft
<code>RaftStuff</code>	<code>raft_stuff.h</code> <code>raft_stuff.cpp</code>	Является оберткой для подсистемы NuRaft, предоставляя единый вход для информации Raft

5.2. УПРАВЛЕНИЕ ТРАФИКОМ КЛАСТЕРА

В разделе 5.1 мы создали распределенную систему с главным и подчиненным узлами, используя протокол Raft. Если необходимо получить доступ к главному или подчиненному узлу этой распределенной системы из клиентской программы, существует простой способ реализации – фиксированная конфигурация адресов доступа к главному и подчиненному узлам на клиенте. Однако в случае переизбрания главного или подчиненного узла такой метод может потребовать обновления конфигурации на клиенте, что неудобно для разработчиков.

Для решения этой проблемы мы предложили альтернативное решение: установить прокси-сервер `ProxyServer` перед `VdbServer`. Этот прокси-сервер скрывает детали главного и подчиненного узлов, и клиенту необходимо лишь установить соединение с прокси-сервером. Когда идентификация главного или подчиненного узла изменяется, `ProxyServer` способен обнаружить это изменение и выполнить фактическое переключение запросов вместо клиента.

При разработке такого прокси-сервера мы столкнулись с вопросом: как он может обнаруживать изменения системы в Raft-кластере позади себя? Для этого мы введем модуль управления системными метаданными `MasterServer`, который отвечает за отслеживание состояния Raft-кластера. `MasterServer` регулярно общается с каждым узлом в Raft-кластере, чтобы выявить любые изменения ролей главного и подчиненного узлов или отказ узла, и на основе этого обновляет информацию о состоянии кластера. На основе данных в реальном времени, предоставляемых `MasterServer`, `ProxyServer` динамически корректирует свою стратегию перенаправления трафика, чтобы все запросы могли эффективно перенаправляться на правильный узел, будь то главный или подчиненный.

5.2.1. Управление метаданными кластера

```

Управление метаданными кластера
├─ Об управлении метаданными
│  └─ MasterServer: управление метаданными распределенного кластера
│     └─ Структура метаданных: экземпляры, информация об узлах
│        └─ Схема персистентности: etcd
├─ Формат хранения метаданных
│  └─ Использование nodeId в качестве ключа, информации об узле в качестве значения
├─ Реализация управления метаданными
│  └─ Класс MasterServer
│     └─ Реализация HTTP-интерфейса инициализации клиента etcd,
│         предоставление HTTP-порта
├─ Реализация HTTP-интерфейса
│  └─ addNode: добавление узла
│     └─ getNodeInfo: получение информации об узле
│        └─ removeNode: удаление узла
│           └─ getInstance: получение информации об экземпляре

```

1. Об управлении метаданными

MasterServer отвечает за управление метаданными нескольких распределенных кластеров. Нам необходимо сначала определить структуру метаданных и выбрать подходящую схему персистентности, чтобы обеспечить определенную степень отказоустойчивости системы управления метаданными.

Метаданные содержат следующую важную информацию.

- Информация об экземпляре, указывающая, к какому экземпляру относятся эти метаданные.
- Информация об узле, соержащая метаданные одного узла в рамках экземпляра, включая `nodeId`, `url`, `role` и `status`.

Несколько узлов составляют один экземпляр, а система управляет несколькими экземплярами.

Учитывая, что наши требования к управлению метаданными относительно просты и мы стремимся к низкой задержке доступа в будущем, мы выбрали зрелую, открытую распределенную систему хранения «ключ–значение» etcd. Хранилище etcd предоставляет возможности отказоустойчивости для нескольких узлов и обладает низкой задержкой доступа, что делает его очень подходящим для хранения метаданных, которыми управляет MasterServer.

2. Формат хранения метаданных

Основываясь на необходимости управления метаданными и особенностях системы хранения etcd, мы разработали формат хранения «ключ–значение».

```

key: /instances/instance1/nodes/node123
value: {"instanceId":"instance1","nodeId":"node123","url":"http://127.0.0.1:8080",
       "role":0,"status":1}

```

```
key: /instances/instance1/nodes/node125
value: {"instanceId":"instance1","nodeId":"node125","url":"http://127.0.0.1:9090",
       "role":0,"status":1}
```

Ниже приведено краткое описание.

- Формат ключа (key): `/instances/ID` экземпляра/`nodes/ID` узла.
- Формат значения (value): строка JSON, содержащая важную метаинформацию об этом узле:
 - ◆ `url` указывает адрес доступа к узлу;
 - ◆ `role` указывает роль узла: 0, если узел является главным, и 1, если узел является подчиненным;
 - ◆ `status` указывает состояние узла: 1, если узел находится в нормальном состоянии, и 0, если узел находится в аномальном состоянии.

Мы закодировали метаданные об узле в двух строках – ключе и значении. Преимущество такого формата заключается в том, что, когда нам нужно получить метаданные узла с ID `nodeId`, мы можем на основании ID экземпляра и ID узла получить его полный ключ и напрямую получить соответствующее значение метаданных. Кроме того, если мы хотим получить информацию обо всех узлах в экземпляре с ID `instanceId`, мы можем использовать в запросе функцию префиксного соответствия хранилища etcd.

3. Реализация управления метаданными

Ниже приведена базовая реализация класса `MasterServer`.

```
class MasterServer {
public:
    explicit MasterServer(const std::string& etcdEndpoints, int httpPort);
    void run();
private:
    etcd::Client etcdClient_;
    void getNodeInfo(const httpLib::Request& req, httpLib::Response& res);
    void addNode(const httpLib::Request& req, httpLib::Response& res);
    void removeNode(const httpLib::Request& req, httpLib::Response& res);
    void getInstance(const httpLib::Request& req, httpLib::Response& res);
};
```

Этот фрагмент кода определяет класс `MasterServer` с архитектурой, ориентированной на клиент-серверную модель программирования. В этой архитектуре `MasterServer` выступает в роли сервера, а функции класса используются для обработки определенных HTTP-запросов. Переменная-член `etcdClient_` предоставляет возможность взаимодействия с кластером etcd. Ниже приведено краткое описание класса `MasterServer` и его функций.

Конструктор

Принимает два параметра: строку `etcdEndpoints`, представляющую точки доступа к кластеру etcd, и целое число `httpPort`, обозначающее порт, на котором `MasterServer` будет прослушивать HTTP-запросы.

Функция-член `void run()`

Используется для запуска сервера `MasterServer` и начала прослушивания HTTP-запросов.

Приватная переменная-член `etcdClient_`

Объект клиента для взаимодействия с кластером etcd.

Приватные функции-члены

`addNode`: обрабатывает HTTP-запросы на добавление нового узла.

`getNodeInfo`: обрабатывает HTTP-запросы на получение информации об узле.

`removeNode`: обрабатывает HTTP-запросы на удаление существующего узла.

`getInstance`: обрабатывает HTTP-запросы на получение информации об экземпляре.

Реализация этих интерфейсов зависит от клиента etcd, инициализированного в конструкторе `MasterServer`, который управляет метаданными. Далее подробно рассмотрим реализацию функции `addNode`.

```
void MasterServer::addNode(const httplib::Request& req, httplib::Response& res) {
    rapidjson::Document doc;
    doc.Parse(req.body.c_str()); // Разбор JSON-данных запроса.
    if (!doc.IsObject()) { // Если разбор не удался или формат данных неверный.
        setResponse(res, 1, "Invalid JSON format"); // Установить код состояния
                                                    // и сообщение ответа.
        return;
    }
    try {
        std::string instanceId = doc["instanceId"].GetString(); // Получение ID
                                                                // экземпляра.
        std::string nodeId = doc["nodeId"].GetString(); // Получение ID узла.
        std::string etcdKey = "/instances/" +
            instanceId + "/nodes/" + nodeId; // Формирование ключа etcd.
        rapidjson::StringBuffer buffer;
        rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
        doc.Accept(writer); // Сериализация JSON-данных.
        etcdClient_.set(etcdKey, buffer.GetString()).get(); // Сохранение
                                                            // данных в etcd.
        setResponse(res, 0, "Node added successfully"); // Ответ успешного
                                                        // выполнения.
    } catch (const std::exception& e) {
        setResponse(res, 1, std::string("Error accessing etcd: ") + e.what());
    }
    // Ответ при исключении.
}
}
```

Функция `addNode` сначала извлекает из клиентского запроса информацию об `instanceId` и узле. Затем она объединяет эту информацию в строку `etcdKey`. После этого функция использует строку JSON из запроса в качестве значения и вызывает функцию `set` клиента `etcdClient_` для сохранения этих данных.

Функции `getNodeInfo`, `removeNode` и `getInstance` реализуются аналогично.

Кроме того, класс `MasterServer` предоставляет HTTP-интерфейс, который позволяет внешним системам получать информацию о метаданных текущей системы через HTTP-запросы. Соответствующие HTTP-запросы реализованы на основе библиотеки `cpp-httplib`.

Код регистрации HTTP-сервисов в `MasterServer` выглядит следующим образом:

```
// Инициализация клиента etcd с передачей информации о точках доступа etcd,
// инициализация номера HTTP-порта.
MasterServer::MasterServer(const std::string& etcdEndpoints, int httpPort)
    : etcdClient_(etcdEndpoints)
    , httpPort_(httpPort) {
    // Настройка правил перенаправления HTTP-сервера.
    httpServer_.Get("/getNodeInfo", [this](const httplib::Request& req,
                                           httplib::Response& res) {
        // Обработка запроса getNodeInfo, предоставление функции запроса
        // информации об узле.
        getNodeInfo(req, res);
    });
    httpServer_.Post("/addNode", [this](const httplib::Request& req,
                                       httplib::Response& res) {
        // Обработка запроса addNode, используется для добавления нового узла
        // в кластер.
        addNode(req, res);
    });
    httpServer_.Delete("/removeNode", [this](const httplib::Request& req,
                                              httplib::Response& res) {
        // Обработка запроса removeNode, используется для удаления узла из кластера.
        removeNode(req, res);
    });
    httpServer_.Get("/getInstance", [this](const httplib::Request& req,
                                           httplib::Response& res) {
        // Обработка запроса getInstance, предоставление функции получения
        // информации об экземпляре.
        getInstance(req, res);
    });
}
```

После завершения написания HTTP-интерфейсов протестируем соответствующие интерфейсы с помощью следующей команды:

```
Запрос:
curl "http://localhost:6060/getInstance?instanceId=instance1"
Ответ:
{"retCode":0,"msg":"Instance info retrieved successfully", \
"data":{"instanceId":"instance1","nodes":[ \
{"instanceId":"instance1","nodeId":"node123", \
"url":"http://127.0.0.1:8080","role":0,"status":1}, \
{"instanceId":"instance1","nodeId":"node125", \
"url":"http://127.0.0.1:9090","role":0,"status":1}]}}
```

Судя по запросу и ответу, [MasterServer](#) уже обладает способностью управлять и запрашивать метаданные текущей распределенной системы. Внешние системы могут полагаться на предоставляемый классом [MasterServer](#) HTTP-интерфейс для получения состояния и информации обо всех узлах текущей системы.

5.2.2. Единый входной поток

Единый входной поток

- └─ Класс [ProxyServer](#)
 - └─ Назначение: предоставление единого входа, скрытие информации о [VdbServer](#) позади
 - └─ Структура [NodeInfo](#): хранение информации о задних узлах
- └─ Функция [fetchAndUpdateNodes](#)
 - └─ Периодическое обновление информации об узлах из [MasterServer](#)
 - └─ Использование двойного массива и атомарного индекса для избежания конкуренции потоков
- └─ Асинхронный таймер-поток для обновления информации об узлах
- └─ Функция [forwardRequest](#)
 - └─ Использование стратегии опроса для выбора узла
 - └─ Построение целевого URL и перенаправление запроса
 - └─ Получение ответа и передача клиенту

Имея интерфейс управления метаданными класса [MasterServer](#), мы можем приступить к реализации основной части класса [ProxyServer](#). Его основное назначение заключается в предоставлении единого входного потока, чтобы клиентам не нужно было знать конкретную информацию о работе сервера [VdbServer](#).

1. Класс [ProxyServer](#)

Прежде всего нам нужно в реальном времени хранить в [ProxyServer](#) информацию об адресах сервисов позади него, необходимых для перенаправления. Эта информация должна периодически обновляться из [MasterServer](#), чтобы гарантировать ее актуальность. Ниже приведено определение класса [ProxyServer](#).

```
// Структура информации об узле
struct NodeInfo {
    std::string nodeId;
    std::string url;
    int role; // Например, 0 обозначает главный узел, 1 - подчиненный.
};
class ProxyServer {
public:
    ProxyServer(const std::string& masterServerHost,
                int masterServerPort,
                const std::string& instanceId);
    ~ProxyServer();
    void start(int port);
private:
    CURL* curlHandle_;
    std::vector<NodeInfo> nodes_[2]; // Использование двух массивов.
```



```
std::atomic<int> activeNodesIndex_; // Указатель на текущий активный
                                   // индекс массива.
std::atomic<size_t> nextNodeIndex_; // Индекс для опроса.
void fetchAndUpdateNodes(); // Получение и обновление информации об узлах.
void startNodeUpdateTimer(); // Запуск таймера обновления узлов.
};
```

Из приведенного выше кода видно, что мы определили структуру с именем `NodeInfo` для представления ключевой информации о задействованных узлах, управляемых текущим экземпляром `ProxyServer`. Основные данные включают ID узла, URL-адрес доступа и информацию о роли узла (указывает, является ли узел главным или подчиненным).

В последующих операциях данные `NodeInfo` будут использоваться в двух разных потоках. Поток 1 будет взаимодействовать с `MasterServer`, периодически обновляя информацию об узлах. Поток 2 будет выполнять задачи по перенаправлению на основе текущей информации об узлах (включая URL и роль). Таким образом, оба потока будут одновременно обращаться к этим данным в памяти, что может привести к проблеме гонки данных между потоками, на которую необходимо обратить особое внимание при реализации. Если два потока одновременно обращаются к одной и той же области памяти, это может привести к аномальному завершению программы.

Обычно для решения этой проблемы используют мьютексы, то есть блокировки чтения-записи. Когда один поток работает с определенной областью памяти, другие потоки должны ждать освобождения блокировки, прежде чем получить к ней доступ. Однако ожидание блокировки может привести к значительным задержкам, поэтому это не является высокопроизводительным решением. Лучшим решением является использование метода обмена массивами, при котором можно предоставить два независимых массива `NodeInfo` и два атомарных поля индекса для реализации этого подхода. Ниже приведено определение соответствующих ключевых структур данных.

```
std::vector<NodeInfo> nodes_[2]; // Использование двух массивов.
std::atomic<int> activeNodesIndex_; // Указывает текущий активный индекс массива.
std::atomic<size_t> nextNodeIndex_; // Индекс для опроса.
```

2. Функция `fetchAndUpdateNodes`

На основе приведенных выше структур данных приведем базовую реализацию функции `fetchAndUpdateNodes` для обновления информации об узлах в классе `ProxyServer`.

```
void ProxyServer::fetchAndUpdateNodes() {
    // Формирование URL запроса, включая адрес MasterServer, порт и ID экземпляра.
    std::string url = "http://" + masterServerHost_ + ":" +
                     std::to_string(masterServerPort_) +
                     "/getInstance?instanceId=" + instanceId_;
    // Установка параметров cURL, конкретный код настройки здесь опущен.
    ...
}
```

```

CURLcode curl_res = curl_easy_perform(curlHandle_); // Выполнение cURL-запроса
                                                    // и получение результатов.
if (curl_res != CURLE_OK) { // Если cURL-запрос не удался, запись ошибки
                            // и возврат.
    GlobalLogger->error("cURL request failed: {}",
                       curl_easy_strerror(curl_res));
    return;
}
// Разбор данных ответа в JSON-объект.
rapidjson::Document doc;
if (doc.Parse(response_data.c_str()).HasParseError()) {
    GlobalLogger->error("Failed to parse JSON response");
    return;
}
// Получение индекса неактивного массива узлов для обновления информации.
int inactiveIndex = activeNodesIndex_.load() ^ 1;
// Очистка неактивного массива узлов для добавления новой информации.
nodes_[inactiveIndex].clear();
// Перебор массива узлов в JSON-ответе, извлечение информации о каждом узле.
const auto& nodesArray = doc["data"]["nodes"].GetArray();
for (const auto& nodeVal : nodesArray) {
    NodeInfo node;
    node.nodeId = nodeVal["nodeId"].GetString();
    node.url = nodeVal["url"].GetString();
    node.role = nodeVal["role"].GetInt();
    // Добавление разобранной информации об узле в неактивный массив узлов.
    nodes_[inactiveIndex].push_back(node);
}
activeNodesIndex_.store(inactiveIndex); // Атомарное переключение индекса
                                        // активного массива узлов.
GlobalLogger->info("Nodes updated successfully"); // Запись в журнал
                                                // успешного обновления узлов.
}

```

Из приведенного кода видно, что мы сначала используем выражение `activeNodesIndex_.load() ^ 1`, чтобы получить позицию текущего неиспользуемого элемента массива `NodeInfo`. Затем обновляем данные `NodeInfo` в этой позиции на основе информации об узлах, возвращенной `MasterServer`. Хотя количество информации об узлах, которую нужно обновить за один раз, может быть значительным, что может привести к длительному процессу обновления, использование неактивного элемента массива `NodeInfo` минимизирует влияние этого обновления на фактический поток перенаправления `ProxyServer`.

После завершения обновления информации об узлах элемент массива, соответствующий `inactiveIndex`, хранит самую последнюю информацию об узлах. Мы используем операцию `activeNodesIndex_.store(inactiveIndex)`, чтобы атомарно обновить значение `inactiveIndex` в `activeNodesIndex_`. Таким образом, при следующем получении информации об узлах `ProxyServer` получит самую актуальную информацию. Благодаря динамическому переключению двух массивов мы можем реализовать высокопроизводительный механизм обновления и доступа к метаданным.

После реализации этой функции мы можем запустить поток асинхронного таймера в `ProxyServer` для обновления `NodeInfo`, тем самым реализуя периодическое обновление информации об узлах.

Ниже приведен код реализации регистрации асинхронного таймера.

```
void ProxyServer::startNodeUpdateTimer() {
    // Создание нового потока, захватывающего указатель this текущего объекта.
    std::thread([this]() {
        // Вход в цикл, продолжается, пока флаг running_ равен true.
        while (running_) {
            // Поток засыпает на 30 секунд, это интервал таймера.
            std::this_thread::sleep_for(std::chrono::seconds(30));
            // После пробуждения вызывается функция для получения
            // и обновления информации об узлах.
            fetchAndUpdateNodes();
        }
    }).detach(); // Отсоединение потока для работы, независимо от основной программы.
}
```

Конструктор `ProxyServer` вызывает метод `startNodeUpdateTimer()`, чтобы запустить функцию обновления, обеспечивая асинхронное обновление соответствующей информации об узлах.

3. Функция `forwardRequest`

На основе информации о узлах, обновляемой в реальном времени, далее мы реализуем логику переадресации в `ProxyServer`. Метод `forwardRequest` класса `ProxyServer` предназначен для переадресации полученных HTTP-запросов на узлы позади себя. Его код представлен ниже.

```
void ProxyServer::forwardRequest(const httplib::Request& req,
    httplib::Response& res, const std::string& path) {
    int activeIndex = activeNodesIndex_.load(); // Получение индекса текущего
                                                // активного массива узлов.
    if (nodes_[activeIndex].empty()) { // Если массив пуст, значит нет
                                        // доступных узлов.
        ... // Логика обработки ошибок и восстановления узлов опущена.
        return;
    }
    // Использование стратегии опроса для выбора узла из массива активных узлов.
    size_t nodeIndex = nextNodeIndex_.fetch_add(1)
        % nodes_[activeIndex].size();
    const auto& targetNode = nodes_[activeIndex][nodeIndex]; // Получение
    // информации о выбранном узле.
    std::string targetUrl = targetNode.url + path; // Построение полного URL
                                                // целевого узла.
    GlobalLogger->info("Forwarding request to: {}", targetUrl); // Запись
    // информации о переадресации запроса в журнал.
    ... // Настройка параметров cURL-запроса, конкретный код настройки опущен.
    std::string response_data; // Определение контейнера для хранения данных ответа.
    curl_easy_setopt(curlHandle_, CURLOPT_WRITEDATA, &response_data);
```

```

CURLcode curl_res = curl_easy_perform(curlHandle_);
if (curl_res != CURLE_OK) { // При сбое запроса записываем ошибку и обрабатываем.
    ... // Обработка неудачного cURL-запроса опущена.
} else {
    // Если запрос выполнен, записываем информацию о полученном ответе от сервера.
    GlobalLogger->info("Received response from server");
    if (response_data.empty()) { // Если данные ответа пусты,
        // необходима обработка.
        ... // Детали обработки опущены.
    } else {
        // Установка данных ответа в HTTP-ответ и установка типа
        // содержимого как application/json.
        res.set_content(response_data, "application/json");
    }
}
}

```

Из логики переадресации видно, что функция сначала использует `activeNodesIndex_` для получения текущей позиции массива `NodeInfo`. Поскольку здесь не используется механизм блокировки, потери производительности минимальны. Текущий массив не обновляется другими потоками, поэтому мы также можем безопасно его использовать. Далее, на основе URL-адреса внутреннего сервера `VdbServer` из `NodeInfo` мы строим адрес для переадресации и отправляем запрос на фактический узел с помощью вызова API, связанных с cURL. После получения ответа от узла мы передаем возвращаемое значение клиенту.

Таким образом, с помощью классов `ProxyServer` и `MasterServer` мы повысили удобство использования системы. Разработчикам достаточно обращаться к адресу `ProxyServer`, который будет переадресовывать реальные запросы на внутренний `VdbServer`.

Так как `ProxyServer` и `MasterServer` являются серверами, не сохраняющими состояние, мы можем развернуть несколько узлов для повышения их доступности. Переадресацию на несколько `ProxyServer` и `MasterServer` можно выполнять через отдельный сервис балансировки нагрузки. Для вызывающей стороны достаточно использовать два отдельных адреса сервиса балансировки нагрузки для доступа к этим двум сервисам. Если какой-либо узел `ProxyServer` или `MasterServer` выйдет из строя, сервис балансировки нагрузки исключит проблемный узел на основе собственной механики обнаружения трафика.

5.2.3. Разделение чтения и записи

Разделение чтения и записи

- ├─ Распознавание запросов
 - ├─ Определение коллекции путей запросов на чтение/запись, идентификация типа запроса
 - └─ Переадресация по стратегии
 - └─ `forwardRequest`: реализация логики переадресации запросов

В разделе 5.2.2 мы реализовали взаимодействие `ProxyServer` и `MasterServer`, наделив `ProxyServer` базовой способностью к переадресации трафика. Однако из-за различий в ролях узлов они фактически выполняют разные функции. Напри-

мер, главный узел может обрабатывать запросы как на чтение, так и на запись, тогда как подчиненный узел может обрабатывать только запросы на чтение.

1. Распознавание запросов

Далее мы планируем внедрить в класс `ProxyServer` механизм распознавания для разделения чтения и записи. Когда клиент инициирует запрос на запись, трафик будет переадресовываться на главный узел. Когда клиент инициирует запрос на чтение, будет происходить балансировка нагрузки между несколькими узлами.

Сначала в переменных-членах класса `ProxyServer` определим коллекции путей запросов на чтение и запись.

```
std::set<std::string> readPaths_; // Коллекция путей запросов на чтение.
std::set<std::string> writePaths_; // Коллекция путей запросов на запись.
```

В конструкторе `ProxyServer` мы инициализируем и настраиваем эти пути запросов на чтение и запись.

```
readPaths_ = {"/search"}; // Определение пути запросов на чтение.
writePaths_ = {"/upsert"}; // Определение пути запросов на запись.
```

2. Переадресация по стратегии

На основе этих двух инициализированных коллекций путей мы добавим стратегию переадресации на основе пути доступа в фактическую функцию переадресации, расширяя логику разделения чтения и записи, описанную в разделе 5.2.2.

```
void ProxyServer::forwardRequest(const httplib::Request& req,
    httplib::Response& res,
    const std::string& path) {
    ... // Код анализа параметров опущен.
    size_t nodeIndex = 0; // Инициализация индекса узла, по умолчанию 0.
    // Если путь запроса находится в списке запросов на запись, выполняем
    // следующие действия.
    if (writePaths_.find(path) != writePaths_.end()) {
        for (size_t i = 0; i < nodes_[activeIndex].size(); ++i) { // Поиск
            // главного узла.
            if (nodes_[activeIndex][i].role == 0) {
                nodeIndex = i; // Найден главный узел, обновляем индекс узла.
                break;
            }
        }
    } else { // Если путь запроса не в списке запросов на запись, выполняем
        // следующие действия.
        // Опрос для выбора узла с любой ролью, использование fetch_add для
        // потокобезопасной операции инкремента.
        nodeIndex = nextNodeIndex_.fetch_add(1) % nodes_[activeIndex].size();
    }
    ... // Код переадресации с помощью cURL опущен.
}
```

В реализации функции `forwardRequest` мы настроили пути для чтения и записи, реализовав разделение чтения и записи на уровне запросов. Конечно, стратегия переадресации может быть более разнообразной и сложной. Например, можно использовать переадресацию на основе наименьшего числа подключений или в зависимости от фактической нагрузки на целевой узел и т. д. Если вас интересуют эти стратегии, вы можете расширить и исследовать их на основе текущего кода.

5.2.4. Обеспечение согласованности чтения и записи

В нашей системе репликация данных между главным и подчиненными узлами может осуществляться в асинхронном режиме. Это означает, что данные, записанные на главный узел, становятся доступными сразу, а подчиненные узлы из-за задержки репликации могут прочитать данные позже. Для клиентов, которые хотят сразу читать данные после записи, наш `ProxyServer` должен предоставить способ чтения с более сильной гарантией согласованности.

Мы можем реализовать эту функцию, введя параметр клиента `forceMaster`. Ниже приведена реализация оптимизированной функции переадресации.

```
void ProxyServer::forwardRequest(const httplib::Request& req,
    httplib::Response& res,
    const std::string& path) {
    ... // Код анализа параметров опущен.

    // Проверяем, содержит ли запрос параметр принудительной переадресации
    // на главный узел.
    bool forceMaster = (req.has_param("forceMaster") &&
        req.get_param_value("forceMaster") == "true");
    // Если требуется принудительное использование главного узла или запрашивается
    // операция записи (путь записи существует в коллекции путей записи).
    if (forceMaster || writePaths_.find(path) != writePaths_.end()) {
        for (size_t i = 0; i < nodes_[activeIndex].size(); ++i) {
            if (nodes_[activeIndex][i].role == 0) {
                nodeIndex = i;
                break;
            }
        }
    }
    ... // Код переадресации запросов на чтение и выполнения переадресации
    // с помощью cURL опущен.
}
```

Как видно из кода, реализация этой функции относительно проста: нам нужно лишь проверить, содержит ли запрос параметр `forceMaster`. Если параметр `forceMaster` равен `true`, мы принудительно выбираем главный узел в качестве цели переадресации.

► Обновление версии v0.4

На этом этапе мы переходим к версии v0.4, включающей классы `MasterServer` и `ProxyServer`. Эта версия вводит унифицированный модуль управления мета-

данными и модуль проксирования. Разработчики могут легко получить доступ к главным и подчиненным узлам системы через унифицированный вход [ProxyServer](#), что значительно повышает удобство использования системы. В табл. 5.3 приведены новые и обновленные модули, добавленные функции версии v0.4.

Таблица 5.3. Новые модули и функции, введенные в v0.4

Модуль	Связанные файлы	Описание
MasterServer	master_server.h master_server. cpp vdb_server_master.cpp	Реализован модуль управления метаданными, предоставляющий внешние адреса доступа узлов, информацию о роли и состоянии. MasterServer работает как независимый процесс
ProxyServer	proxy_server.h proxy_server. cpp vdb_server_proxy.cpp	Реализован модуль проксирования, который может получать адреса для переадресации из системы метаданных, поддерживает разделение чтения и записи и согласованное чтение. ProxyServer работает как независимый процесс

5.3 УПРАВЛЕНИЕ ИСКЛЮЧЕНИЯМИ В КЛАСТЕРЕ

В предыдущих главах мы построили сложную распределенную систему, объединяющую [VdbServer](#), [MasterServer](#) и [ProxyServer](#). Эта система не только может выполнять репликацию данных между узлами, но и предоставляет единый вход через [ProxyServer](#), что облегчает использование разработчиками. Однако, учитывая неизбежность отказов отдельных узлов в распределенной системе, нам необходимо уделить больше внимания обеспечению определенной устойчивости системы к отказам узлов.

5.3.1. Обнаружение нового главного узла

- Обнаружение нового главного узла
 - Функция `startNodeUpdateTimer`
 - Создание и запуск потока для периодического обновления
 - Функция `updateNodeStates`
 - Получение списка узлов из `etcd`
 - Отправка HTTP-запроса на `VdbServer` для получения состояния узлов
 - Обновление информации о роли узлов в `etcd` на основе возвращенного состояния

В текущей системе в [VdbServer](#) уже реализовано автоматическое избрание главного узла с помощью `NuRaft`. То есть, если главный узел выходит из строя, другие узлы автоматически выбирают новый главный узел с наивысшим приоритетом через механизм голосования.

1. Функция `startNodeUpdateTimer`

Способность `MasterServer` узнавать об изменении главного узла является ключевой для распознавания отказов. Поэтому необходимо добавить в [MasterServer](#) поток

таймера, который будет периодически проверять у каждого экземпляра `VdbServer`, изменилось ли состояние главного узла, и обновлять информацию о состоянии узлов. Для реализации этого таймера мы используем асинхронный поток, ниже приведен код реализации таймера `startNodeUpdateTimer` (он практически идентичен коду реализации асинхронного таймера, который мы запускаем в `ProxyServer`).

```
void MasterServer::startNodeUpdateTimer() {
    std::thread([this]() {
        while (true) {
            std::this_thread::sleep_for(std::chrono::seconds(30));
            updateNodeStates();
        }
    }).detach();
}
```

Реализация этого таймера относительно проста, он запускается в конструкторе `MasterServer`. После запуска таймер будет периодически засыпать на определенное время (например, здесь на 30 с), после чего вызывается функция обновления информации об узлах `updateNodeStates` для фактического обнаружения нового главного узла.

2. Функция `updateNodeStates`

Код реализации функции `updateNodeStates`, которая выполняет обновление информации о состоянии, представлен ниже.

```
void MasterServer::updateNodeStates() {
    ... // Предыдущий код опущен.
    try {
        std::string nodesKeyPrefix = "/instances/";
        GlobalLogger->info("Fetching nodes list from etcd");
        // Получение списка узлов из etcd.
        etcd::Response etcdResponse = etcdClient_.ls(nodesKeyPrefix).get();
        for (size_t i = 0; i < etcdResponse.keys().size(); ++i) {
            const std::string& nodeKey = etcdResponse.keys()[i];
            const std::string& nodeValue = etcdResponse.values()[i].as_string();
            // Разбор информации об узле.
            rapidjson::Document nodeDoc;
            nodeDoc.Parse(nodeValue.c_str());
            // Построение URL для запроса состояния узла.
            std::string getNodeUrl =
                std::string(nodeDoc["url"].GetString()) +
                "/admin/getNode";
            // Выполнение запроса HTTP GET для получения информации о состоянии узла/
            CURLcode res = curl_easy_perform(curl);
            // Разбор ответа о состоянии узла.
            rapidjson::Document getNodeResponse;
            getNodeResponse.Parse(responseStr.c_str());
            // Обновление информации о роли узла.
            const rapidjson::Value& node = getNodeResponse["node"];
```



```

        if (node.HasMember("state") && node["state"].IsString()) {
            std::string state = node["state"].GetString();
            // Определение новой роли узла: leader - 0, follower - 1.
            int newRole = (state == "leader") ? 0 : 1;
            // Если роль узла не изменилась, пропустить обновление.
            if (nodeDoc.HasMember("role") &&
                nodeDoc["role"].GetInt() == newRole) {
                GlobalLogger->debug(
                    "No role update needed for node {}", nodeKey);
                continue;
            }
            // Обновление информации об узле в etcd.
            nodeDoc["role"].SetInt(newRole);
            rapidjson::StringBuffer buffer;
            rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
            nodeDoc.Accept(writer);
            etcdClient_.set(nodeKey, buffer.GetString()).get();
        }
    } catch (const std::exception& e) {
        // Захват исключения и запись ошибки в журнал.
        GlobalLogger->error(
            "Exception while updating node states: {}",
            e.what());
    }
    ... // Последующий код опущен.
}

```

В функции `updateNodeStates` класс `MasterServer` сначала получает из хранилища `etcd` информацию о всех узлах, соответствующих текущим экземплярам, и с использованием URL-адреса узла строит запрос `/admin/getNode`, который отправляется на сервер `VdbServer`. Когда `VdbServer` получает этот запрос, он возвращает текущую информацию об узле, включая поле `state`. Для главного узла значение поля `state` будет `"leader"`, для подчиненного – `"follower"`.

`MasterServer` проверяет, изменилась ли идентификационная информация узла, возвращенная `VdbServer`, по сравнению с информацией в хранилище `etcd`. Если идентификационная информация отличается, `MasterServer` обновляет информацию об узле в `etcd` в соответствии с последними данными.

Таким образом, если на `VdbServer` происходит переключение главный–подчиненный, `MasterServer` своевременно обнаруживает это изменение посредством периодического обновления интерфейса.

5.3.2. Обнаружение отказа подчиненного узла

Обнаружение отказа подчиненного узла

- ├─ Счетчик ошибок `nodeErrorCounts`
- ├─ Запись количества обнаружений аномалий в узле
- ├─ Функция `updateNodeStates`
 - ├─ Получение списка узлов из `etcd`
 - ├─ Перебор узлов, выполнение проверки работоспособности и обновление состояния
 - ├─ Захват исключений и запись в журнал

1. Счетчик ошибок `nodeErrorCounts`

Помимо случаев, когда главный узел выходит из строя и подчиненный узел выбирается новым главным, `VdbServer` также может столкнуться с ситуацией, когда подчиненный узел становится недоступным из-за отказа локальной системы. `MasterServer` также должен распознавать и обрабатывать такие ситуации. Поэтому мы продолжаем оптимизировать реализацию функции `updateNodeStates`.

Для обнаружения текущего состояния узла добавим в класс `MasterServer` счетчик ошибок `nodeErrorCounts`, который используется для записи количества обнаружений аномалий в узле.

```
std::map<std::string, int> nodeErrorCounts;
```

2. Функция `updateNodeStates`

В функции `updateNodeStates` мы будем увеличивать значение этого счетчика ошибок. Как только счетчик достигнет определенного порога, состояние узла будет изменено до состояния отказа. Таким образом, внешняя система сможет обнаружить аномальное состояние узла через соответствующее поле.

```
void MasterServer::updateNodeStates() {
    // Определение префикса пути для хранения информации об узлах в etcd.
    std::string nodesKeyPrefix = "/instances/";
    // Запись операции получения списка узлов из etcd.
    GlobalLogger->info("Fetching nodes list from etcd");
    // Получение информации обо всех узлах с соответствующим префиксом пути
    // через клиент etcd.
    etcd::Response etcdResponse = etcdClient_.ls(nodesKeyPrefix).get();
    // Перебор всех пар «ключ-значение» узлов, возвращенных etcd.
    for (size_t i = 0; i < etcdResponse.keys().size(); ++i) {
        ... // Получение ключа узла (nodeKey) и связанного документа (nodeDoc),
            // код получения и разбора опущен.
        CURLcode res = curl_easy_perform(curl);
        bool needsUpdate = false;
        if (res != CURLE_OK) {
            GlobalLogger->error("curl_easy_perform() failed: {}",
                               curl_easy_strerror(res));
            nodeErrorCounts[nodeKey]++; // Увеличение счетчика ошибок узла.
            // Если счетчик ошибок достиг определенного порога и состояние
            // узла не равно 0 (аномалия), установить состояние узла в 0.
            if (nodeErrorCounts[nodeKey] >= 5 &&
                nodeDoc["status"].GetInt() != 0) {
                nodeDoc["status"].SetInt(0); // Установить состояние в 0.
                needsUpdate = true;
            }
        } else {
            // Если запрос cURL выполнен, сбросить счетчик ошибок узла.
            nodeErrorCounts[nodeKey] = 0;
            // Если состояние узла не равно 1 (нормально), установить
            // состояние узла в 1.
            if (nodeDoc["status"].GetInt() != 1) {
```

```

        nodeDoc["status"].SetInt(1); // Установить состояние в 1.
        needsUpdate = true;
    }
    ... // Дополнительная логика обновления состояния опущена.
}
// Если состояние узла требует обновления, сериализовать документ узла
// и сохранить в etcd.
if (needsUpdate) {
    // Создание буфера текста rapidjson и записывающего устройства.
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
    nodeDoc.Accept(writer); // Запись документа узла в буфер.
    etcdClient_.set(nodeKey, buffer.GetString()).get(); // Обновление
    // информации об узле в etcd.
    // Запись информации об обновлении состояния узла.
    GlobalLogger->info("Updated node {} with new status and role",
                     nodeKey);
}
}
// Перехват и запись возможных исключений.
catch (const std::exception& e) {
    GlobalLogger->error("Exception while updating node states: {}",
                     e.what());
}
// Другие возможные операции, здесь опущены.
...
}

```

В текущей реализации, если при обновлении в [MasterServer](#) информации об узле через запрос [/admin/getNode](#) обнаруживается, что нельзя подключиться к какому-либо узлу, фиксируется количество неудачных запросов к этому узлу в [nodeErrorCounts](#). Как только количество неудач достигает заданного порога, [MasterServer](#) изменяет состояние этого узла в etcd на состояние отказа. Таким образом, когда внешняя система впоследствии получает актуальную информацию об узлах через [MasterServer](#), любой узел в состоянии отказа будет идентифицирован.

5.3.3. Реализация переключения при отказе

На данном этапе сервер [MasterServer](#) уже способен распознавать переключение главного узла и отказы подчиненного узла, а также обновлять соответствующую информацию об узле в метаданных системы после возникновения отказа.

Для реализации окончательного переключения потока при отказе необходимо добавить соответствующую логику распознавания и обработки отказов в [ProxyServer](#). Ниже приведен обновленный код функции [fetchAndUpdateNodes](#).

```

void ProxyServer::fetchAndUpdateNodes() {
    // Запись операции получения информации об узлах от MasterServer.
    GlobalLogger->info("Fetching nodes from master server");
}

```

```

std::string url = "http://" + masterServerHost_ + ":" +
    std::to_string(masterServerPort_) +
// Построение URL-адреса запроса, включая адрес MasterServer,
// порт и ID экземпляра.
"/getInstance?instanceId=" + instanceId_;
// Запись информации о запрашиваемом URL-адресе для отладки.
GlobalLogger->debug("Requesting URL: {}", url);
// Получение индекса массива неактивных узлов для последующего обновления
// информации об узлах; операция XOR используется для получения индекса
// другого массива.
int inactiveIndex = activeNodesIndex_.load() ^ 1;
// Очистка массива неактивных узлов, подготовка к добавлению новой
// информации об узлах.
nodes_[inactiveIndex].clear();
// Получение массива узлов из JSON-ответа.
const auto& nodesArray = doc["data"]["nodes"].GetArray();
// Перебор массива узлов, извлечение информации о каждом узле.
for (const auto& nodeVal : nodesArray) {
    // Обработка только узлов со статусом 1 (нормальный).
    if (nodeVal["status"].GetInt() == 1) {
        NodeInfo node; // Создание нового экземпляра NodeInfo.
        node.nodeId = nodeVal["nodeId"].GetString(); // Установка ID узла.
        node.url = nodeVal["url"].GetString(); // Установка URL узла.
        node.role = nodeVal["role"].GetInt(); // Установка роли узла.
        // Добавление новой информации об узле в массив неактивных узлов.
        nodes_[inactiveIndex].push_back(node);
    } else {
        // Если статус узла не равен 1, запись информации о пропуске этого узла.
        GlobalLogger->info("Skipping inactive node: {}",
            nodeVal["nodeId"].GetString());
    }
}
}
}

```

Из этого кода видно, что, когда **ProxyServer** обновляет информацию об узлах от **MasterServer**, он обрабатывает только узлы со статусом 1 (нормальный). Иными словами, как только **MasterServer** распознает аномалию и помечает состояние узла как 0 (аномалия), **ProxyServer** при следующем обновлении информации об узлах исключит этот отказавший узел из списка пересылки. Таким образом, последующий новый поток больше не будет перенаправляться на этот отказавший узел.

В то же время в логике обновления, если узел из подчиненного узла превращается в главный узел, **ProxyServer** также способен распознать это изменение и перенаправить поток записи на новый главный узел.

Несмотря на то что отказавший узел быстро распознается и исключается, система тем не менее в этот момент находится в состоянии с недостаточным количеством узлов. Поэтому необходимо наличие вспомогательной системы или системы мониторинга для проверки таких ситуаций, чтобы в случае необходимости автоматически или вручную создать новые узлы и добавить их в кластер Raft. Новые узлы станут новыми подчиненными узлами. Став новыми подчиненными узлами, они могут быть добавлены в единое управление

MasterServer с помощью команды конфигурации. Как только узел войдет в единое управление **MasterServer**, прокси-сервер **ProxyServer** также через механизм периодического обновления получит информацию об этом узле и перенаправит на него соответствующий поток, тем самым восстанавливая систему до состояния высокой доступности.

► **Обновление версии v0.5**

На этом этапе мы переходим к версии v0.5, поддерживающую обработку кластерных аномалий. Эта версия, оптимизируя серверы **MasterServer** и **ProxyServer**, поддерживает динамическое распознавание изменений главного узла и отказов подчиненных узлов, тем самым действительно реализуя автоматическое восстановление системы после отказа автономного узла. В табл. 5.4 приведены новые и обновленные модули, добавленные функции версии v0.5.

Таблица 5.4. Новые и обновленные модули, добавленные функции версии v0.5

Модуль	Связанные файлы	Описание
MasterServer	master_server.h master_server.cpp	В MasterServer добавлен асинхронный поток обнаружения, периодически проверяющий состояние узлов и обновляющий метаданные системы
ProxyServer	proxy_server.h proxy_server.cpp	При обновлении метаданных из ProxyServer исключаются аномальные узлы

На рис. 5.4 изображена текущая архитектура векторной базы данных, обладающей определенной распределенной способностью к отказоустойчивости, после объединения трех модулей: **MasterServer**, **ProxyServer** и **VectorDatabase**.

5.4. СЕГМЕНТИРОВАНИЕ КЛАСТЕРА

В предыдущих главах наша система постепенно совершенствовалась, однако в плане масштабируемости она все еще сталкивается с вызовами. Текущая система позволяет выполнять операции записи только главному узлу, а предел вертикального масштабирования автономной системы ограничивает возможности расширения. Как только достигается предел автономной системы, становится невозможно поддерживать более масштабные службы системы. Обычным решением этой проблемы является разделение кластера, но это требует определенных изменений на уровне бизнес-логики.

В распределенных системах способность к горизонтальному масштабированию является важным показателем их передовых характеристик. В этом разделе на основе существующей системы будет предложена более мощная возможность сегментирования. Мы разрешим сегментировать запросы пользователей с использованием ключа сегментирования, что позволит более эффективно распределять запросы на запись по различным узлам. Также для запросов мы учтем особенности векторных данных, чтобы в условиях много-сегментности возвращать корректные результаты в соответствии с фактическими требованиями пользователей.

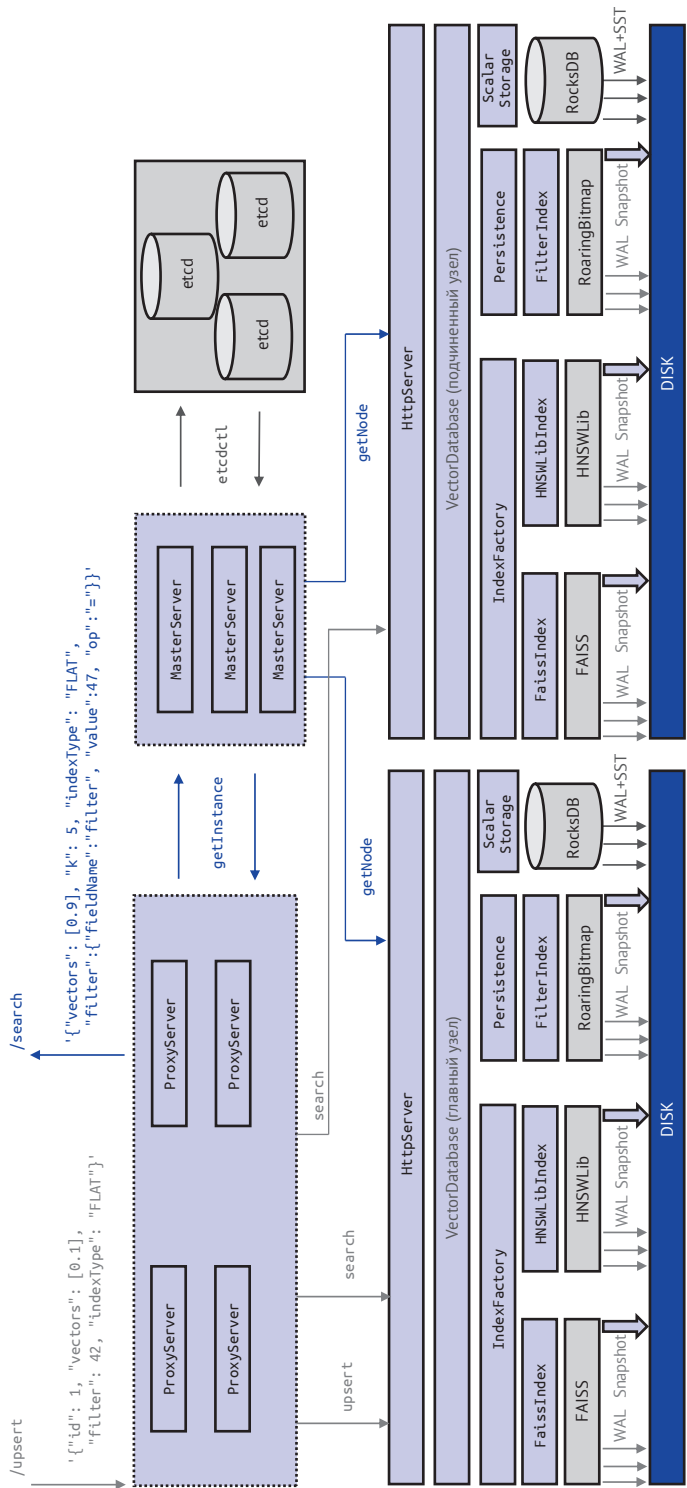


Рис. 5.4. Архитектура векторной базы данных версии v0.5

5.4.1. Настройка стратегии сегментирования кластера

Настройка стратегии сегментирования кластера

- └ Структура метаданных сегментирования
 - └ key: /instancesConfig/{instanceId}/partitionConfig
 - └ value: строка JSON, содержащая информацию о конфигурации сегментирования
- └ Интерфейс управления сегментированием
 - └ getPartitionConfig: получение информации о конфигурации сегментирования
 - └ updatePartitionConfig: обновление информации о конфигурации сегментирования
 - └ doUpdatePartitionConfig: создание информации о сегментировании и ее хранение в etcd
 - └ вызов через HTTP-запрос для обновления информации о конфигурации сегментирования

При реализации функции сегментирования кластера первым шагом является хранение стратегии сегментирования в системе метаданных, чтобы **MasterServer** мог управлять стратегией сегментирования, связанной с определенным экземпляром **InstanceId**. Затем через **MasterServer** будет выполняться обновление метаданных сегментирования и предоставляться актуальная информация о метаданных через интерфейс.

1. Структура метаданных сегментирования

Для эффективной идентификации текущей ситуации с сегментированием в системе мы разработали следующую структуру метаданных сегментирования:

```
key: /instancesConfig/instance1/partitionConfig
value: {"partitionKey":"id", "numberOfPartitions":2, "partitions":[{"partitionId":0,"nodeId":"node123"}, {"partitionId":0,"nodeId":"node124"}, {"partitionId":1,"nodeId":"node125"}, {"partitionId":1,"nodeId":"node126"}]}
```

Эта структура содержит следующую ключевую информацию:

- **key**: этот ключ уникально идентифицирует элемент конфигурации, указывая, что это информация о конфигурации сегментирования для экземпляра **instance1**;
- **value**: значение представляет собой объект JSON, содержащий детальную информацию о конфигурации сегментирования. Поля в объекте JSON определяют правила сегментирования:
 - ◆ ключ данных сегментирования: **partitionKey** указывает ключ данных, используемый для сегментирования, здесь это **id**. В алгоритме сегментирования этот ключ используется для определения, в каком сегменте должны храниться данные;
 - ◆ количество сегментов: значение **numberOfPartitions** равно 2, то есть весь набор данных разделен на два сегмента;
 - ◆ список сегментов: **partitions** – это массив, содержащий информацию обо всех сегментах. Каждый сегмент представлен объектом, содержащим два поля. **partitionId** – это уникальный идентификатор сегмента, в данном примере это 0 или 1, что соответствует ID двух сегментов. **nodeId** – это идентификатор узла, хранящего данные сегмента, здесь это **node123**, **node124**, **node125** или **node126**.

В этой конфигурации сегментирования мы видим, что имеется два сегмента, каждый из которых распределен между двумя различными узлами. Такая конфигурация позволяет системе продолжать иметь доступ к данным в случае отказа узла, а также увеличивать обработку и емкость системы путем увеличения количества сегментов.

2. Интерфейс управления сегментированием

Добавим в `MasterServer` интерфейсы для получения и обновления информации о сегментировании.

```
void getPartitionConfig(const httplib::Request& req, httplib::Response& res);
void updatePartitionConfig(const httplib::Request& req, httplib::Response& res);
```

Интерфейс `updatePartitionConfig` реализует прием запросов разработчиков на конфигурацию сегментирования, извлечение соответствующих параметров сегментирования из запроса и вызов внутренней функции `doUpdatePartitionConfig`. Базовый код реализации приведен ниже.

```
void MasterServer::doUpdatePartitionConfig(
    const std::string& instanceId,
    const std::string& partitionKey,
    int numberOfPartitions,
    const std::list<Partition>& partitions) {
    rapidjson::Document doc;
    doc.SetObject();
    rapidjson::Document::AllocatorType& allocator = doc.GetAllocator();
    // Установка ключа сегментирования и количества сегментов.
    doc.AddMember("partitionKey",
        rapidjson::Value(partitionKey.c_str(), allocator), allocator);
    doc.AddMember("numberOfPartitions", numberOfPartitions, allocator);
    // Установка информации о конфигурации сегментирования.
    rapidjson::Value partitionArray(rapidjson::kArrayType);
    for (const auto& partition : partitions) {
        rapidjson::Value partitionObj(rapidjson::kObjectType);
        partitionObj.AddMember("partitionId", partition.partitionId, allocator);
        partitionObj.AddMember("nodeId",
            rapidjson::Value(partition.nodeId.c_str(), allocator), allocator);
        partitionArray.PushBack(partitionObj, allocator);
    }
    doc.AddMember("partitions", partitionArray, allocator);
    // Запись конфигурации в etcd.
    rapidjson::StringBuffer buffer;
    rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
    doc.Accept(writer);
    std::string etcdKey = "/instancesConfig/" + instanceId + "/partitionConfig";
    etcdClient_.set(etcdKey, buffer.GetString()).get();
    GlobalLogger->info("Updated partition config for instance {}", instanceId);
}
```


С помощью этого интерфейса обновления экземпляр **MasterServer** сохраняет информацию о сегментировании кластера в системе хранения etcd.

Затем через функцию **getPartitionConfig** предоставляется HTTP-интерфейс. Через этот интерфейс **getPartitionConfig** обращается к системе etcd и возвращает соответствующую информацию о сегментировании. После реализации этого интерфейса мы можем использовать следующую команду для получения конфигурации сегментирования от **MasterServer**:

```
Запрос:
curl "http://localhost:6060/getPartitionConfig?instanceId=instance1"
Ответ:
{"retCode":0,"msg":"Partition config retrieved successfully","data":
{"partitionKey":"id","numberOfPartitions":2,"partitions":[
{"partitionId":0,"nodeId":"node123"}, {"partitionId":0,"nodeId":
"node124"}, {"partitionId":1,"nodeId":"node125"}, {"partitionId":1,
"nodeId":"node126"}]}}
```

5.4.2. Перенаправление запросов в соответствии со стратегией сегментирования

```
Перенаправление запросов в соответствии со стратегией сегментирования
├─ Обновление информации о сегментировании
│ └─ startPartitionUpdateTimer: таймер запускает обновление
│   └─ fetchAndUpdatePartitionConfig: выполняет обновление информации о сегментах
├─ Оптимизация переадресации запросов
│ └─ forwardRequest: переадресация запроса в соответствии со значением ключа
сегментирования
├─ Извлечение ключа сегментирования
│ └─ extractPartitionKeyValue: извлечение значения, соответствующего ключу
сегментирования
├─ Широковещательная рассылка запросов без ключа сегментирования
│ └─ broadcastRequestToAllPartitions: широковещательная рассылка запросов без
ключа сегментирования на все сегменты
│   └─ processAndRespondToBroadcast: обработка ответа на широковещательную рас-
сылку и возврат результата
├─ Переадресация запросов с ключом сегментирования
│ └─ calculatePartitionId: вычисление ID сегмента на основе значения ключа
сегментирования
│   └─ selectTargetNode: выбор соответствующего целевого узла
│     └─ forwardToTargetNode: переадресация запроса на целевой узел
```

После реализации управления соответствующей информацией о сегментах в **MasterServer** перейдем к сегментированию клиентских запросов в **ProxyServer** на основе этой информации.

1. Обновление информации о сегментировании

Прежде всего необходимо периодически обновлять в **ProxyServer** соответствующую информацию о сегментировании. Поскольку частота обновления этой о сегментировании отличается от частоты обновления информации об узлах

(обновление информации о сегментировании происходит не так часто), мы решили запустить отдельный асинхронный поток для реализации нового таймера. Код реализации таймера `startPartitionUpdateTimer` приведен ниже.

```
void ProxyServer::startPartitionUpdateTimer() {
    std::thread([this]() {
        while (running_) {
            std::this_thread::sleep_for(std::chrono::minutes(5));
            // Вызов функции для обновления конфигурации сегментирования.
            fetchAndUpdatePartitionConfig();
        }
    }).detach();
}
```

В таймере мы вызываем функцию реализации получения конфигурации сегментирования `fetchAndUpdatePartitionConfig`, код которой приведен ниже.

```
void ProxyServer::fetchAndUpdatePartitionConfig() {
    std::string url = "http://" + masterServerHost_ + ":" +
        std::to_string(masterServerPort_) +
        "/getPartitionConfig?instanceId=" +
        instanceId_;
    // Использование libcurl для отправки HTTP GET-запроса.
    CURLcode curl_res = curl_easy_perform(curlHandle_);
    // Проверка успешности запроса.
    if (curl_res != CURLE_OK) {
        GlobalLogger->error("Failed to fetch partition config: {} ",
            curl_easy_strerror(curl_res));
        return;
    }
    // Разбор данных ответа и обновление массива nodePartitions_.
    rapidjson::Document doc;
    if (doc.Parse(response_data.c_str()).HasParseError()) {
        GlobalLogger->error("Failed to parse JSON response");
        return;
    }
    ... // Код обработки исключений при разборе данных ответа опущен.
    // Получение индекса неактивного массива узлов.
    int inactiveIndex = activePartitionIndex_.load() ^ 1;
    // Очистка данных неактивного массива узлов.
    nodePartitions_[inactiveIndex].nodesInfo.clear();
    nodePartitions_[inactiveIndex].partitionKey_ =
        doc["data"]["partitionKey"].GetString();
    // Получение partitionKey и numberOfPartitions.
    nodePartitions_[inactiveIndex].numberOfPartitions_ =
        doc["data"]["numberOfPartitions"].GetInt();
    // Заполнение данных nodePartitions_[inactiveIndex].
    const auto& partitionsArray = doc["data"]["partitions"].GetArray();
    for (const auto& partitionVal : partitionsArray) {
        int partitionId = partitionVal["partitionId"].GetInt();
        std::string nodeId = partitionVal["nodeId"].GetString();
```

```

// Поиск или создание нового объекта NodePartitionInfo.
auto it = nodePartitions_[inactiveIndex].nodesInfo.find(partitionId);
if (it == nodePartitions_[inactiveIndex].nodesInfo.end()) {
    // Создание нового объекта NodePartitionInfo.
    NodePartitionInfo newPartition;
    newPartition.partitionId = partitionId;
    nodePartitions_[inactiveIndex].nodesInfo[partitionId] =
        newPartition;
    it = std::prev(nodePartitions_[inactiveIndex].nodesInfo.end());
}
// Добавление информации об узле.
NodeInfo nodeInfo;
nodeInfo.nodeId = nodeId;
// nodeInfo.url и nodeInfo.role необходимо получить или задать.
it->second.nodes.push_back(nodeInfo);
}
// Атомарное переключение индекса активного массива узлов.
activePartitionIndex_.store(inactiveIndex);
}

```

В функции `fetchAndUpdatePartitionConfig` мы отправляем запрос `getPartitionConfig` на сервер `MasterServer` для получения информации о сегментах, соответствующей текущему экземпляру `InstanceId`. Получив информацию о сегментировании, мы применяем метод замены двойного массива для оптимизации производительности. Выполнение операции заключается в получении текущего неиспользуемого элемента массива конфигурации сегментирования с помощью `activePartitionIndex_.load() ^ 1` и последующем обновлении актуальной информации о конфигурации сегментов в этом свободном элементе. После завершения обновления конфигурации сегментирования значение `inactiveIndex` атомарно заменяется на `activePartitionIndex`. Этот метод аналогичен методу обновления информации об узлах, описанному в разделе 5.2.2, и обеспечивает более высокую скорость обновления и лучшую производительность чтения информации о сегментировании.

2. Оптимизация переадресации запросов

После получения этой информации нам нужно использовать конфигурацию сегментирования для оптимизации функции `forwardRequest`, код которой приведен ниже.

```

void ProxyServer::forwardRequest(const httpLib::Request& req,
httpLib::Response& res,
    const std::string& path) {
    // Извлечение значения, соответствующего ключу сегментирования.
    std::string partitionKeyValue;
    // Если не найдено, запрос будет отправлен на все сегменты.
    if (!extractPartitionKeyValue(req, partitionKeyValue)) {
        GlobalLogger->debug("Partition key value not found, broadcasting
request to all partitions");
        broadcastRequestToAllPartitions(req, res, path);
        return;
    }
}

```

```

    int partitionId = calculatePartitionId(partitionKeyValue); // Вычисление ID
                                                                // сегмента.
    NodeInfo targetNode; // Выбор целевого узла.
    if (!selectTargetNode(req, partitionId, path, targetNode)) {
        res.status = 503; // Если подходящий узел не найден, возвращается
                        // статус 503.
        res.set_content("No suitable node found for forwarding", "text/plain");
        return;
    }
    // Переадресация запроса на целевой узел.
    forwardToTargetNode(req, res, path, targetNode);
}

```

В этой функции мы провели рефакторинг процесса переадресации, разделив его на четыре основных этапа.

- Этап 1: извлечение значения, соответствующего ключу сегментирования, из запроса.
- Этап 2.1: если ключ сегментирования отсутствует, запрос переадресуется на все сегменты через интерфейс широковещательной рассылки.
- Этап 2.2: если ключ сегментирования присутствует, вычисляется ID сегмента, соответствующий значению ключа сегментирования.
- Этап 3: на основе ID сегмента выбирается целевой узел из потенциальных узлов для переадресации.
- Этап 4: запрос переадресуется на фактический узел, и возвращается результат выполнения.

3. Извлечение ключа сегментирования

Теперь давайте подробно рассмотрим детали реализации кода этих этапов. В качестве примера возьмем функцию `extractPartitionKeyValue` из этапа 1, код которой приведен ниже.

```

// Извлечение из запроса значения, соответствующего ключу сегментирования.
bool ProxyServer::extractPartitionKeyValue(const httplib::Request& req,
std::string& partitionKeyValue) {
    GlobalLogger->debug("Extracting partition key value from request");
    rapidjson::Document doc; // Разбор тела запроса в формате JSON.
    // Если разбор не удался, записывается ошибка и возвращается false.
    if (doc.Parse(req.body.c_str()).HasParseError()) {
        GlobalLogger->debug("Failed to parse request body as JSON");
        return false;
    }
    // Получение текущего активного индекса сегмента.
    int activePartitionIndex = activePartitionIndex_.load();
    // Получение информации о конфигурации сегмента.
    const auto& partitionConfig = nodePartitions_[activePartitionIndex];
    // Проверка наличия ключа сегментирования в запросе.
    if (!doc.HasMember(partitionConfig.partitionKey_.c_str())) {
        // Если ключ сегментирования не найден, записывается ошибка
        // и возвращается false.
    }
}

```

```

        GlobalLogger->debug("Partition key not found in request");
        return false;
    }
    // Получение значения, соответствующего ключу сегментирования.
    const rapidjson::Value& keyVal = doc[partitionConfig.partitionKey_.c_str()];
    if (keyVal.IsString()) {
        // Если значение, соответствующее ключу сегментирования, является
        // строкой, оно присваивается переменной partitionKeyValue.
        partitionKeyValue = keyVal.GetString();
    } else if (keyVal.IsInt()) {
        // Если значение ключа сегментирования является целым числом,
        // преобразовать его в строку и присвоить переменной partitionKeyValue.
        partitionKeyValue = std::to_string(keyVal.GetInt());
    } else {
        // Если значение ключа сегментирования другого типа, зафиксировать
        // ошибку и вернуть false.
        GlobalLogger->debug("Unsupported type for partition key");
        return false;
    }
    // Зафиксировать успешно извлеченное значение ключа сегментирования
    // и вернуть true.
    GlobalLogger->debug("Partition key value extracted: {}",
partitionKeyValue);
    return true;
}

```

4. Широковещательный запрос без ключа сегментирования

На этапе 1 мы определяем, содержит ли текущий запрос ключ сегментирования. Если в запросе его нет, необходимо выполнить широковещательную рассылку, то есть переадресовать запрос всем сегментам. Затем мы агрегируем результаты, возвращенные всеми сегментами, и сортируем их, чтобы предоставить пользователю необходимые результаты. Реализация функции `broadcastRequestToAllPartitions` представлена ниже.

```

void ProxyServer::broadcastRequestToAllPartitions(const httpLib::Request& req,
                                                httpLib::Response& res,
                                                const std::string& path) {
    rapidjson::Document doc; // Разбор запроса для получения значения k.
    doc.Parse(req.body.c_str());
    if (doc.HasParseError() || !doc.HasMember("k") || !doc["k"].IsInt()) {
        res.status = 400;
        res.set_content("Invalid request: missing or invalid 'k'", "text/plain");
        return;
    }
    int k = doc["k"].GetInt();
    int activePartitionIndex = activePartitionIndex_.load();
    const auto& partitionConfig = nodePartitions_[activePartitionIndex];
    std::vector<std::future<httpLib::Response>> futures;
    std::unordered_set<int> sentPartitionIds;
    for (const auto& partition : partitionConfig.nodesInfo) {

```

```

    int partitionId = partition.first;
    if (sentPartitionIds.find(partitionId) != sentPartitionIds.end()) {
        // Если запрос уже отправлен, пропустить.
        continue;
    }
    futures.push_back(std::async(std::launch::async,
                                &ProxyServer::sendRequestToPartition,
                                this, req, path, partition.first));
    // После отправки запроса добавить ID сегмента в отправленный набор.
    sentPartitionIds.insert(partitionId);
}
// Сбор и обработка ответов.
std::vector<httplib::Response> allResponses;
for (auto& future : futures) {
    allResponses.push_back(future.get());
}
// Обработка ответов, включая сортировку и извлечение не более k результатов.
processAndRespondToBroadcast(res, allResponses, k);
}

```

Функция фактической переадресации формирует соответствующий пакет запроса, реализация функции `sendRequestToPartition` представлена ниже.

```

httplib::Response ProxyServer::sendRequestToPartition(const httplib::Request&
originalReq,
const std::string& path, int partitionId) {
    NodeInfo targetNode; // Выбор целевого узла.
    if (!selectTargetNode(originalReq, partitionId, path, targetNode)) {
        // Если не удалось выбрать целевой узел, вернуть объект ответа
        // с кодом состояния 503.
        httplib::Response httpResponse;
        httpResponse.status = 503;
        return httpResponse;
    }
    std::string targetUrl = targetNode.url + path; // Формирование целевого URL.
    CURL* curl = curl_easy_init();
    ... // Код переадресации с использованием cURL опущен.
    return httpResponse;
}

```

Эта функция переадресации использует функцию выбора целевого узла по ID сегмента, часть `selectTargetNode` будет рассмотрена далее в этом разделе.

Возвращаясь к реализации функции широковещательной рассылки, эта функция отправляет запрос всем сегментным узлам в системе. Запросы отправляются асинхронно нескольким узлам. При получении ответов от нескольких узлов мы обрабатываем эти результаты с помощью функции `processAndRespondToBroadcast`, сортируем все результаты и возвращаем первые k результатов, соответствующих условиям. Реализация функции `processAndRespondToBroadcast` представлена ниже.

```

void ProxyServer::processAndRespondToBroadcast(
    http::Response& res,
    const std::vector<http::Response>&
    allResponses, uint k) {
    struct CombinedResult {
        double distance;
        double vector;
    };
    std::vector<CombinedResult> allResults;
    // Разбор и объединение ответов.
    for (const auto& response : allResponses) {
        if (response.status == 200) {
            rapidjson::Document doc;
            doc.Parse(response.body.c_str());
            if (!doc.HasParseError() && doc.IsObject() &&
                doc.HasMember("vectors") && doc.HasMember("distances"))
            {
                const auto& vectors = doc["vectors"].GetArray();
                const auto& distances = doc["distances"].GetArray();
                for (rapidjson::SizeType i = 0; i < vectors.Size(); ++i) {
                    CombinedResult result = {
                        distances[i].GetDouble(),
                        vectors[i].GetDouble()
                    };
                    allResults.push_back(result);
                }
            }
        }
    }
    // Сортировка allResults по distances.
    std::sort(
        allResults.begin(),
        allResults.end(),
        [](const CombinedResult& a, const CombinedResult& b) {
            // Использовать distance в качестве ключа сортировки.
            return a.distance < b.distance;
        }
    );
    // Извлечение не более k результатов.
    if (allResults.size() > k) {
        allResults.resize(k);
    }
    // Формирование окончательного ответа.
    rapidjson::Document finalDoc;
    finalDoc.SetObject();
    rapidjson::Document::AllocatorType& allocator = finalDoc.GetAllocator();
    rapidjson::Value finalVectors(rapidjson::kArrayType);
    rapidjson::Value finalDistances(rapidjson::kArrayType);
    for (const auto& result : allResults) {
        finalVectors.PushBack(result.vector, allocator);
        finalDistances.PushBack(result.distance, allocator);
    }
}

```

```

finalDoc.AddMember("vectors", finalVectors, allocator);
finalDoc.AddMember("distances", finalDistances, allocator);
finalDoc.AddMember("retCode", 0, allocator);
rapidjson::StringBuffer buffer;
rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
finalDoc.Accept(writer);
res.set_content(buffer.GetString(), "application/json");
}

```

В функции `processAndRespondToBroadcast` мы проходим по результатам запросов, возвращенных с нескольких узлов, и сортируем их по элементу `distances`, чтобы получить первые k элементов, соответствующих условиям, и затем возвращаем эти результаты клиенту.

5. Переадресация запроса с ключом сегментирования

Теперь проанализируем случай, когда в запросе присутствует ключ сегментирования. Сначала мы получаем значение, соответствующее этому ключу, и вычисляем соответствующий ID сегмента с помощью функции `calculatePartitionId`, реализация которой представлена ниже.

```

int ProxyServer::calculatePartitionId(
    const std::string& partitionKeyValue) {
    GlobalLogger->debug(
        "Calculating partition ID for key value: {}", partitionKeyValue);
    int activePartitionIndex = activePartitionIndex_.load();
    const auto& partitionConfig = nodePartitions_[activePartitionIndex];
    // Обработка partitionKeyValue с помощью хеш-функции.
    std::hash<std::string> hasher;
    size_t hashValue = hasher(partitionKeyValue);
    // Вычисление ID сегмента с использованием хеш-значения.
    int partitionId =
        static_cast<int>(hashValue % partitionConfig.numberOfPartitions_);
    GlobalLogger->debug("Calculated partition ID: {}", partitionId);
    return partitionId;
}

```

В этой функции мы сначала преобразуем значение, соответствующее ключу сегментирования, в хеш-значение (`hashValue`) с помощью хеш-алгоритма. Затем мы используем это хеш-значение для выполнения операции по модулю с общим количеством сегментов, и полученный целочисленный результат является ID сегмента, к которому следует переадресовать запрос.

После получения ID сегмента мы используем функцию `selectTargetNode` для выбора соответствующего целевого узла для переадресации. Реализация функции `selectTargetNode` представлена ниже.


```

bool ProxyServer::selectTargetNode(const httplib::Request& req,
                                   int partitionId,
                                   const std::string& path,
                                   NodeInfo& targetNode) {
    // Запись в журнал выбора целевого узла.
    GlobalLogger->debug("Selecting target node for partition ID: {}",
                      partitionId);
    // Проверка необходимости принудительной переадресации на главный узел.
    bool forceMaster = req.has_param("forceMaster") &&
        req.get_param_value("forceMaster") == "true";
    // Получение индекса массива активных узлов.
    int activeNodeIndex = activeNodesIndex_.load();
    // Получение конфигурации активного сегмента.
    const auto& partitionConfig =
        nodePartitions_[activePartitionIndex_.load()];
    // Получение информации о сегментных узлах.
    const auto& partitionNodes = partitionConfig.nodesInfo.find(partitionId);
    // Если информация о сегментных узлах не найдена, зафиксировать ошибку
    // и вернуть false.
    if (partitionNodes == partitionConfig.nodesInfo.end()) {
        GlobalLogger->error("No nodes found for partition ID: {}",
                          partitionId);
        return false;
    }
    // Получение всех доступных узлов.
    std::vector<NodeInfo> availableNodes;
    for (const auto& partitionNode : partitionNodes->second.nodes) {
        // Поиск информации об узле.
        auto it = std::find_if(
            nodes_[activeNodeIndex].begin(),
            nodes_[activeNodeIndex].end(),
            [&partitionNode](const NodeInfo& n) {
                return n.nodeId == partitionNode.nodeId;
            }
        );
        // Если информация об узле найдена, добавить его в список
        // доступных узлов.
        if (it != nodes_[activeNodeIndex].end()) {
            availableNodes.push_back(*it);
        }
    }
    // Если доступных узлов нет, зафиксировать ошибку и вернуть false.
    if (availableNodes.empty()) {
        GlobalLogger->error("No available nodes for partition ID: {}",
                          partitionId);
        return false;
    }
    if (forceMaster || writePaths_.find(path) != writePaths_.end()) {
        for (const auto& node : availableNodes) {
            if (node.role == 0) {

```

```

        targetNode = node;
        return true;
    }
}
GlobalLogger->error("No master node available for partition ID: {}",
    partitionId);
return false;
} else {
    size_t nodeIndex = nextNodeIndex_.fetch_add(1) % availableNodes.size();
    targetNode = availableNodes[nodeIndex];
    return true;
}
}

```

В логике определения целевого узла для переадресации мы используем метод, аналогичный предыдущему. Сначала из всех кандидатов выбираются узлы, соответствующие ID целевого узла текущей задачи переадресации, и они сохраняются в списке `availableNodes`. Затем мы анализируем путь запроса, чтобы определить, является ли он запросом на чтение или запись, и выбираем подходящий узел в соответствии с логикой разделения чтения и записи. Также учитывается параметр `forceMaster`.

В завершение мы получаем информацию о целевом узле, удовлетворяющем условиям, и передаем ее в функцию `forwardToTargetNode` для выполнения задачи переадресации. Реализация этой функции приведена ниже.

```

void ProxyServer::forwardToTargetNode(const httplib::Request& req,
                                     httplib::Response& res,
                                     const std::string& path,
                                     const NodeInfo& targetNode) {
    GlobalLogger->debug("Forwarding request to target node: {}",
        targetNode.nodeId);
    // Формирование целевого URL-адреса.
    std::string targetUrl = targetNode.url + path;
    GlobalLogger->info("Forwarding request to: {}", targetUrl);
    // Выполнение запроса cURL.
    ... // Код выполнения переадресации с помощью cURL опущен.
}

```

Эта функция использует cURL для выполнения запроса к целевому интерфейсу. После получения ответа от внутреннего узла она заполняет и возвращает результат клиенту. Таким образом, наш `ProxyServer` завершает выполнение всех функций переадресации на основе информации о сегментировании из экземпляра `MasterServer`. Для запросов без ключа сегментирования мы используем широковещательную рассылку к каждому сегменту системы. Для запросов с ключом сегментирования мы непосредственно переадресуем их в соответствующий сегмент согласно правилам сегментирования и возвращаем полученный результат клиенту.

► Обновление версии v0.6

На этом этапе мы переходим к версии v0.6, которая поддерживает правила сегментирования. В этой версии был оптимизирован модуль **MasterServer** для поддержки настройки информации о сегментировании, а также оптимизирован **ProxyServer** для поддержки переадресации запросов на основе сегментирования. В табл. 5.5 приведены новые и обновленные модули, добавленные функции версии v0.6.

Таблица 5.5. Новые и обновленные модули, добавленные функции версии v0.6

Модуль	Затронутые файлы	Описание
MasterServer	master_server.h, master_server.cpp	В MasterServer добавлена функция управления информацией о сегментировании кластера
ProxyServer	proxy_server.h, proxy_server.cpp	В ProxyServer добавлена поддержка переадресации запросов на основе информации о сегментировании кластера

5.5. РЕЗЮМЕ

В этой главе мы продолжили расширение ключевых функций на основе ранее реализованной автономной векторной базе данных.

Репликация данных в архитектуре

«главный – подчиненный»

Мы исследовали широко используемый в индустрии распределенный протокол Raft и узнали о легковесной реализации с открытым исходным кодом – NuRaft. Библиотека NuRaft, разработанная командой eBau, предоставляет функции репликации данных и выбора узлов на основе протокола Raft. Благодаря этим функциям мы расширили **VdbServer** от автономной версии до версии, поддерживающей архитектуру «главный – подчиненный». На этом этапе мы реализовали репликацию данных между главным и подчиненными узлами и зафиксировали эти журнальные данные в системе. На этом этапе наша система получила распределенную архитектуру, способную выполнять переизбрание главного узла при отказе одного из узлов, после которого подчиненный узел может продолжать работу в качестве главного.

Модуль управления метаданными **MasterServer**

На этом этапе главный и подчиненный узлы предоставляют услуги независимо, и внешним системам сложно определить, кто из них является главным, а кто подчиненным. Поэтому наша система требует добавления модуля управления метаданными для унифицированного управления этой информацией. Мы создали новый модуль **MasterServer**, который, основываясь на распределен-

ных возможностях хранения хранилища etcd, сохраняет конфигурацию метаданных системы и предоставляет HTTP-интерфейс, упрощающий другим системам получение текущей информации о метаданных.

Модуль прокси-сервера ProxyServer

Для облегчения доступа разработчиков к VdbServer мы ввели в систему унифицированный модуль прокси-сервера ProxyServer. Он регулярно получает метаданные каждого узла из MasterServer и может определить, кто из узлов является главным, а кто подчиненным. На основе этой информации ProxyServer реализует разделение чтения и записи и принудительное чтение с главного узла. Благодаря этим функциям удобство использования нашей системы значительно возросло.

Стратегия сегментирования

Наконец, для решения проблемы масштабируемости при записи на один узел мы ввели стратегию сегментирования. MasterServer контролирует конфигурацию сегментирования текущего экземпляра. На основе этой конфигурации ProxyServer может переадресовывать запросы в зависимости от наличия ключа сегментирования. Для запросов без ключа сегментирования ProxyServer использует широковещательную рассылку ко всем сегментам и агрегирует запросы для возврата окончательного результата. Для запросов с ключом сегментирования он переадресует запрос на фактическое выполнение в соответствующий сегмент на основе системной конфигурации.

Благодаря оптимизации в этой главе класс VdbServer прошел эволюцию от автономной до распределенной векторной базы данных, приобретая базовые возможности распределенной системы к отказоустойчивости и масштабируемости. На рис. 5.5 изображена распределенная архитектура этой версии.

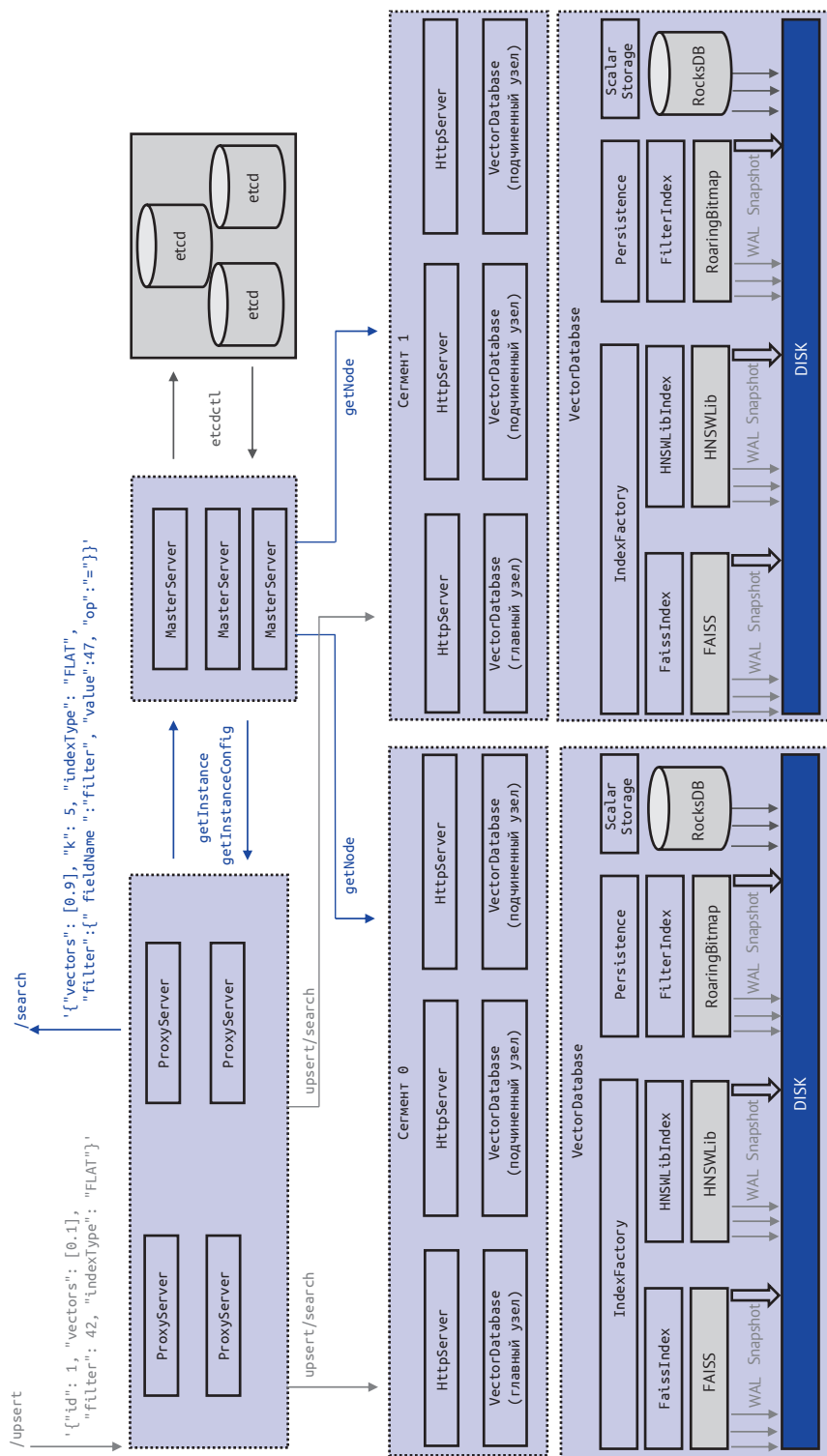


Рис. 5.5. Архитектура распределенной векторной базы данных версии v0.6

Глава 6

Оптимизация векторной базы данных

Мы склонны переоценивать эффект технологий в краткосрочной перспективе
и недооценивать его – в долгосрочной.

Рой Амара. Закон Амары

После работы, проведенной в предыдущих главах, мы успешно создали векторную базу данных со свойствами распределенности и отказоустойчивости, которая удовлетворяет базовые потребности разработчиков. Однако развитие ИИ идет семимильными шагами, и с увеличением числа ИИ-приложений, построенных на основе векторных баз данных, неизбежно растет и объем векторных данных. В будущем наша векторная база данных должна будет справляться с более масштабными сценариями эксплуатации. Если не спланировать заранее, накопление данных до определенного уровня может привести к непредсказуемым потерям. Поэтому на этапе проектирования векторной базы данных необходимо тщательно продумать вопросы оптимизации. Для упрощения мы сосредоточимся на трех направлениях оптимизации: производительности, затратах и удобстве использования.

В плане производительности мы обычно оптимизируем как аппаратные, так и программные уровни, чтобы снизить общую задержку на всем пути запроса в системе. Векторная база данных с более низкой задержкой предоставляет предприятиям лучшую основу для создания более качественных ИИ-приложений. В этой главе основное внимание уделяется оптимизации векторных вычислений, алгоритмов запросов и протоколов связи.

Что касается затрат, наша цель – снизить операционные затраты на уровне всего сервиса при сохранении той же мощности обслуживания. Для векторной базы данных, обслуживающей более крупные конечные приложения, контроль операционных затрат является ключевым элементом для достижения долгосрочной устойчивой бизнес-модели. В этой главе мы рассмотрим текущие технические решения и архитектуру развертывания, а также изучим подходы к оптимизации затрат при гибридном развертывании и развертывании на одном узле.

В отношении удобства использования мы стремимся к всестороннему улучшению системы, чтобы обеспечить бесшовный и эффективный опыт исполь-

зования для разработчиков. Это позволит предприятиям сосредоточиться на своей бизнес-логике и анализе данных, затрачивая меньше времени и средств на создание ИИ-приложений и не отвлекаясь на управление и эксплуатацию базы данных. В этой главе мы рассмотрим, как улучшить удобство использования базы данных с помощью пакета SDK, аутентификации доступа и резервного копирования данных.

Эти оптимизации охватывают лишь 20 % исключительных сценариев, но они крайне важны для создания более мощной векторной базы данных и требуют терпения для постепенной реализации.

Глава будет развиваться в этих трех направлениях, мы будем стремиться создать векторную базу данных, ориентированную на будущее, и предлагать идеи для оптимизации обслуживания более масштабных бизнес-сценариев. Следует отметить, что в этой главе предлагается оптимизация в каждом направлении на достаточно базовом простом уровне. Для создания мощной векторной базы данных необходимо продолжать оптимизацию в этих трех направлениях и учитывать больше факторов. Если вас интересует этот вопрос, вы можете продолжить исследование на основе идей, изложенных в этой главе.

6.1. ОПТИМИЗАЦИЯ ПРОИЗВОДИТЕЛЬНОСТИ

Производительность программной системы обычно выражается в общей задержке обработки запросов. Снижение задержки может значительно улучшить опыт конечных пользователей и открыть больше возможностей для расширения бизнеса. Особенно для онлайн-систем, где одно действие конечного пользователя часто требует нескольких запросов к серверу базы данных. Если хранилище данных этого ИИ-приложения основано на векторной базе данных, то скорость ее отклика будет напрямую связана с удобством работы конечного пользователя.

В индустрии разработки программного обеспечения существует выражение: «Аппаратное обеспечение прогрессирует за год, а программное – за три года». Это можно понимать так, что итеративная оптимизация на уровне аппаратного обеспечения за один год может дать более значительный эффект, чем программная оптимизация, на которую уйдет три года. Поэтому в этом разделе мы сначала рассмотрим, как для повышения производительности системы эффективно использовать аппаратные ресурсы. На основе аппаратной оптимизации мы далее проанализируем существующую программную архитектуру и внедрим меры по оптимизации на программном уровне. Наша цель – добиться повышения производительности как на аппаратном, так и на программном уровне, чтобы оптимизировать производительность всей системы. В этом разделе основное внимание уделяется оптимизации вычислений векторов, алгоритмов запросов и протоколов связи.

6.1.1. Оптимизация векторных вычислений с использованием набора инструкций

В разделе 1.1.3 мы узнали, что для вычисления сходства двух векторов система выполняет множество векторных вычислений, включая косинус, скалярное

произведение и евклидово расстояние. Эти вычисления потребляют значительное количество ресурсов CPU и являются одной из наиболее затратных частей системы. Если мы сможем оптимизировать их с использованием специфического набора инструкций процессора, это может привести к значительному повышению производительности системы.

Основные производители CPU обычно предоставляют набор инструкций с одной командой и множеством данных (single-instruction multiple-data, SIMD). Например, набор инструкций AVX-512 обладает шириной регистра в 512 бит и двумя 512-битными блоками совмещенного умножения-сложения (fused multiply-add, FMA). Эти характеристики позволяют AVX-512 эффективно выполнять параллельные вычисления, например загружать несколько чисел с плавающей запятой за один раз для параллельных вычислений.

Предположим, что тип данных векторов – это 32-битные числа с плавающей запятой одинарной точности. Используя набор инструкций AVX-512, мы можем при вычислении сходства векторных данных одновременно загрузить 16 размерностей вектора запроса A и 16 размерностей вектора B, хранящегося в векторной базе данных, в соответствующие регистры для параллельных вычислений. Теоретически такой подход значительно повышает эффективность вычислений и степень параллелизма, что позволяет существенно улучшить производительность всей базы данных.

Ниже приведена реализация функции вычисления сходства векторов (евклидово расстояние L2) с использованием набора инструкций AVX-512.

```
static float L2SqrSIMD16ExtAVX512(const void *pVect1v, const void *pVect2v,
const void *qty_ptr) {
    float *pVect1 = (float *) pVect1v; // Преобразование входного указателя
                                     // в тип float.
    float *pVect2 = (float *) pVect2v;
    size_t qty = *((size_t *) qty_ptr); // Получение количества векторов.
    float PORTABLE_ALIGN64 TmpRes[16]; // Определение массива для временного
                                     // хранения результатов.
    size_t qty16 = qty >> 4; // Вычисление шестнадцатой части количества векторов.
    const float *pEnd1 = pVect1 + (qty16 << 4); // Вычисление адреса окончания цикла.
    __m512 diff, v1, v2; // Определение переменных регистров AVX-512.
    __m512 sum = __m512_set1_ps(0); // Инициализация суммы квадратов
                                     // разностей нулем.
    while (pVect1 < pEnd1) { // Цикл по данным векторов, обработка 16 элементов
                             // за раз.
        v1 = _mm512_loadu_ps(pVect1); // Загрузка данных первого вектора.
        pVect1 += 16; // Перемещение указателя на следующую группу из 16 элементов.
        v2 = _mm512_loadu_ps(pVect2); // Загрузка данных второго вектора.
        pVect2 += 16;
        diff = _mm512_sub_ps(v1, v2); // Вычисление разности векторов.
        // Вычисление квадратов разностей и их суммирование.
        sum = _mm512_add_ps(sum, _mm512_mul_ps(diff, diff));
    }
    // Сохранение суммы квадратов разностей во временный массив.
    __m512_store_ps(TmpRes, sum);
    // Суммирование 16 элементов временного массива для получения
    // окончательного сходства.
```



```

float res = TmpRes[0] + TmpRes[1] + TmpRes[2] + TmpRes[3] + TmpRes[4] +
TmpRes[5] + TmpRes[6] + TmpRes[7] + TmpRes[8] + TmpRes[9] + TmpRes[10] +
TmpRes[11] + TmpRes[12] + TmpRes[13] + TmpRes[14] + TmpRes[15];
return (res); // Возврат окончательного результата.
}

```

Этот код использует мощные возможности набора инструкций AVX-512, реализуя эффективное вычисление сходства векторов (евклидово расстояние L2) с помощью параллельных вычислений и оптимизированного алгоритма. Мы знаем, что для вычисления евклидова расстояния L2 необходимо вычислить квадрат разности между каждой размерностью векторов и затем суммировать их. Используя функции параллельной загрузки (`_mm512_loadu_ps`), параллельного вычитания (`_mm512_sub_ps`), параллельного умножения (`_mm512_mul_ps`) и параллельного сложения (`_mm512_add_ps`), мы можем одновременно обработать 16 чисел с плавающей запятой и вычислить сумму квадратов разностей этой группы из 16 чисел. Далее, с помощью функции `_mm512_store_ps` эти 16 чисел с плавающей запятой переносятся в массив результата. В завершение суммируются 16 элементов этого массива, чтобы получить окончательный результат евклидова расстояния L2.

Стоит отметить, что здесь AVX-512 используется лишь в качестве примера. Вы можете писать код и проверять его эффективность в зависимости от реальных аппаратных ресурсов. Практическое применение и эффект этого метода могут варьироваться в зависимости от различных аппаратных конфигураций.

6.1.2. Оптимизация алгоритмов запросов

Хотя оптимизация с использованием аппаратного обеспечения может быстро привести к значительному улучшению производительности, обновление и замена аппаратного обеспечения обычно происходят медленнее, чем программного обеспечения. Кроме того, поскольку программное обеспечение легко распространяется, мы можем постоянно проводить быстрые итерации на уровне программного обеспечения в соответствии с реальными сценариями применения. Такие оптимизации можно быстро доставлять клиентам, тем самым улучшая их опыт.

В ранее созданной подсистеме запросов мы реализовали функцию гибридного запроса, сочетающую битовые карты и сходство векторов. Эта функция позволяет при запросе определенного вектора сравнивать только те векторы, которые соответствуют определенным фильтрующим меткам, что позволяет осуществлять более детализированный запрос векторов.

Рассмотрим следующий сценарий: у нас есть коллекция векторных данных населения, содержащая 10 млн образцов, в которой каждый вектор представляет отпечаток пальца человека и хранит информацию о районе, в котором он находится. Сейчас нам нужно запросить вектор отпечатка пальца и определить, является ли он сильно похожим на отпечатки пальцев жителей района А (предположим, в районе А проживает 1000 человек). Используя предыдущую реализацию, мы сначала получаем битовую карту всех векторов, принадлежащих этому району, а затем передаем эти битовые карты в подсистему запросов векторов. В процессе запроса мы отфильтровываем векторы, не соответствующие условиям, но фактический запрос все равно осуществляется среди 10 млн векторов.

На программном уровне здесь существует определенное пространство для оптимизации. Поскольку мы создали битовую карту, детализированную до уровня района, количество векторов, удовлетворяющих условиям, уже сокращено до 1000. Поэтому мы можем оптимизировать стратегию запроса: в памяти поочередно вычислять сходство между вектором в запросе и этими 1000 векторами, в конечном итоге оставляя только целевой результат. Такой подход не требует выполнения запроса среди 10 млн векторов.

Этот метод поиска в подсистеме запросов векторной базы данных называется «эволюция запроса», то есть использование различных алгоритмов запроса в зависимости от масштаба данных. Ниже приведен код для реализации запроса ближайших соседей в памяти.

```
// Класс VectorSimilarity предоставляет функции для вычисления сходства векторов.
class VectorSimilarity {
public:
    // Перечисляемый тип MetricType для указания различных функций
    // вычисления сходства.
    enum class MetricType { COSINE, INNER_PRODUCT, EUCLIDEAN };
    // Статическая функция для вычисления сходства между двумя векторами.
    static float compute_similarity(const std::vector<float>& v1,
                                   const std::vector<float>& v2,
                                   MetricType metric) {
        // Выбор функции вычисления сходства в зависимости от значения metric.
        switch (metric) {
        case MetricType::COSINE:
            return cosine_similarity(v1, v2); // Косинусное сходство.
        case MetricType::INNER_PRODUCT:
            return inner_product(v1, v2); // Скалярное произведение.
        case MetricType::EUCLIDEAN:
            return -euclidean_distance(v1, v2); // Отрицательное евклидово
            // расстояние.
        default:
            // Если metric не является допустимым типом метрики, вызывается
            // исключение.
            throw std::invalid_argument("Unknown metric type");
        }
    }
    // Статическая функция для поиска k векторов, наиболее похожих на целевой вектор
    static std::vector<std::pair<float, int>> find_top_k_similar_vectors(
        const std::vector<float>& target_vector,
        const std::vector<std::vector<float>>& vectors,
        int k,
        MetricType metric
    ) {
        // Использование очереди с приоритетом (максимальная куча) для хранения
        // k наиболее похожих векторов.
        std::priority_queue<std::pair<float, int>> pq;
        // Обход всех векторов, вычисление их сходства с целевым вектором
        // и добавление в очередь с приоритетом.
```

```

    for (int i = 0; i < vectors.size(); ++i) {
        float similarity = compute_similarity(target_vector, vectors[i],
                                             metric);
        // Добавление пары значений сходства и индекса в очередь.
        pq.push(std::make_pair(similarity, i));
        if (pq.size() > k) {
            pq.pop(); // Если размер очереди превышает k, удаляется
                     // верхний элемент.
        }
    }
    // Динамический массив для хранения k ближайших соседей.
    std::vector<std::pair<float, int>> top_k;
    // Извлечение всех элементов из очереди с приоритетом и сохранение
    // их в динамическом массиве в порядке убывания сходства.
    while (!pq.empty()) {
        top_k.push_back(pq.top());
        pq.pop();
    }
    // Обращение порядка динамического массива, чтобы вектор с наибольшим
    // сходством был первым.
    std::reverse(top_k.begin(), top_k.end());
    return top_k;
}

private:
    // Вычисление и возврат косинусного сходства между двумя векторами.
    static float cosine_similarity(const std::vector<float>& v1, const
std::vector<float>& v2) {
        float inner_product = 0.0f, norm_v1 = 0.0f, norm_v2 = 0.0f;
        // Вычисление скалярного произведения и норм векторов.
        for (size_t i = 0; i < v1.size(); ++i) {
            inner_product += v1[i] * v2[i];
            norm_v1 += v1[i] * v1[i];
            norm_v2 += v2[i] * v2[i];
        }
        // Вычисление результата по формуле косинусного сходства.
        return inner_product / (sqrt(norm_v1) * sqrt(norm_v2));
    }

    // Вычисление и возврат скалярного произведения двух векторов.
    static float inner_product(const std::vector<float>& v1, const
std::vector<float>& v2) {
        float result = 0.0f;
        for (size_t i = 0; i < v1.size(); ++i) {
            result += v1[i] * v2[i];
        }
        return result;
    }

    // Вычисление и возврат евклидова расстояния между двумя векторами.
    static float euclidean_distance(const std::vector<float>& v1, const
std::vector<float>& v2) {
        float result = 0.0f;
        for (size_t i = 0; i < v1.size(); ++i) {
            result += (v1[i] - v2[i]) * (v1[i] - v2[i]);
        }
    }

```

```

    }
    return sqrt(result);
}
};
#endif // IN_MEM_VECTOR_SIMILARITY_H

```

Из кода видно, что класс `VectorSimilarity` реализует функцию `find_top_k_similar_vectors`, которая позволяет находить k ближайших соседей к вектору из списка векторов. Основные элементы класса `VectorSimilarity` представлены ниже.

- Перечисление `MetricType`: используется для определения типа метрики сходства, включая косинусное сходство, скалярное произведение и евклидово расстояние.
- Функция-член `compute_similarity`: предназначена для вычисления сходства между двумя векторами. Она выбирает соответствующую функцию вычисления сходства в зависимости от переданного типа метрики и возвращает значение сходства.
- Функция `find_top_k_similar_vectors` предназначена для нахождения k наиболее похожих векторов относительно целевого вектора. Она использует очередь с приоритетом (максимальная куча) для хранения текущих k наиболее похожих векторов. Если при вычислении сходства нового вектора с целевым очередь заполнена, удаляется наименее похожий вектор. В итоге возвращается динамический массив, содержащий k наиболее похожих векторов.
- Внутренние статические приватные функции `cosine_similarity`, `inner_product` и `euclidean_distance` предназначены для вычисления сходства по различным метрикам. Они реализуют функции вычисления косинусного сходства, скалярного произведения и евклидова расстояния соответственно.

Если вас заинтересовала эта тема, вы можете попробовать объединить данную функцию с гибридной стратегией запроса, реализованной в разделе 4.2.3. Например, можно установить условие оптимизации запроса: при количестве записей в битовой карте менее 1000 переключаться на режим соответствия в памяти. Применение такой «эволюции запроса» к нашему крупномасштабному индексу, построенному на основе библиотек FAISS или HNSWLib, может значительно повысить производительность.

6.1.3. Оптимизация протокола связи

Помимо эффективности запроса векторов, после введения в нашу систему сервера `ProxyServer` эффективность передачи данных между `ProxyServer` и внутренним сервером `VdbServer` также является одним из ключевых факторов, влияющих на общую производительность системы.

В текущей версии для связи между двумя модулями используется протокол HTTP. Он ценится за высокую читаемость и удобство отладки в процессе разработки, но, поскольку этот протокол не является оптимальным с точки зрения производительности, мы рассматриваем возможность использования более производительного протокола связи для повышения эффективности передачи.

В условиях высоких требований к производительности внутренней связи в распределенной системе часто выбирают протоколы удаленного вызова процедур (RPC). Распространенные в отрасли протоколы такого типа включают gRPC от Google, tRPC от Tencent, bRPC от Baidu и др. Разработчики могут выбрать подходящую реализацию RPC исходя из реальных условий своего проекта, учитывая производительность, удобство интерфейса и поддержку экосистемы. Пример кода ниже основан на подсистеме gRPC и демонстрирует процесс модификации серверов [ProxyServer](#) и [VdbServer](#). Модификация должна проводиться одновременно как в [VdbServer](#), так и в [ProxyServer](#) и включает четыре основных шага.

Первый шаг: определение службы gRPC (файл .proto).

```
syntax = "proto3";
package vdb;
service VectorDatabaseService {
    // Определение клиентского потока RPC для записи и обновления векторных данных.
    rpc Upsert(stream UpsertRequest) returns (UpsertResponse) {}
}
message UpsertRequest {
    uint64 id = 1; // Уникальный идентификатор вектора.
    repeated float vectors = 2; // Векторные данные для записи или обновления.
    // Тип индекса, необязательное поле, указывает способ индексации вектора.
    string indexType = 3;
}
message UpsertResponse {
    int32 retCode = 1; // Код возврата, 0 – успех, другие значения – сбой.
    string errorMsg = 2; // Сообщение об ошибке, необязательное поле.
}
```

Второй шаг: реализация сервера gRPC.

Этот шаг требует использования компилятора Protocol Buffers (protoc) для обработки ранее созданного файла .proto и преобразования его в исходный код на C++, а именно генерации файла определения [VectorDatabaseService](#). Затем мы реализуем сервер gRPC на основе этого файла.

```
class VectorDatabaseServiceImpl final
    : public VectorDatabaseService::Service {
public:
    // Реализация функции Upsert RPC.
    grpc::Status Upsert(
        grpc::ServerContext* context,
        grpc::ServerReader<UpsertRequest>* reader,
        UpsertResponse* response) override {
        UpsertRequest request;
        // Циклическое чтение каждого запроса из клиентского потока.
        while (reader->Read(&request)) {
            // Обработка каждого запроса, например запись или обновление
            // векторных данных, параметры вызова upsert здесь опущены.
            vector_database_>upsert(...);
        }
    }
}
```

```
response->set_retcode(0); // Установка кода ответа (0 означает успех).
return grpc::Status::OK; // Возвращение статуса, OK означает нормальную
                          // работу.
}
};
```

Третий шаг: запуск сервера gRPC.

```
void RunServer() {
    std::string server_address("0.0.0.0:50051");
    VectorDatabaseServiceImpl service;
    grpc::ServerBuilder builder;
    builder.AddListeningPort(server_address, grpc::InsecureServerCredentials());
    builder.RegisterService(&service);
    std::unique_ptr<grpc::Server> server(builder.BuildAndStart());
    std::cout << "Server listening on " << server_address << std::endl;
    server->Wait();
}
```

Логика работы выглядит следующим образом.

1. Указание адреса сервера 0.0.0.0:50051, что означает прослушивание порта 50051 на всех сетевых интерфейсах локальной машины.
2. Создание экземпляра `VectorDatabaseServiceImpl` в качестве реализации сервиса.
3. Использование класса `grpc::ServerBuilder` для создания конструктора сервера `builder` и передача адреса сервера и учетных данных через `AddListeningPort`. В нашем примере используются небезопасные параметры серверных учетных данных для добавления порта прослушивания, в производственной среде так делать не следует.
4. Регистрация объекта сервиса в конструкторе с помощью метода `RegisterService`.
5. Вызов метода `BuildAndStart` для сборки и запуска сервера, вывод адреса сервера в консоль, что означает, что сервер выполняет прослушивание адреса и порта.
6. Вызов `server->Wait()`, чтобы перевести сервер в состояние ожидания подключения клиентов.

Четвертый шаг: модификация **ProxyServer** – добавление клиента gRPC.

```
void ForwardUpsertRequests() {
    // Создание клиентского канала gRPC.
    auto channel = grpc::CreateChannel("vdbServer_address:50051",
        grpc::InsecureChannelCredentials());
    // Создание экземпляра.
    std::unique_ptr<VectorDatabaseService::Stub> stub_ =
        VectorDatabaseService::NewStub(channel);
    grpc::ClientContext context; // Создание клиентского контекста
                                // и объекта ответа.
}
```

```

UpsertResponse response;
auto stream = stub_->Upsert(&context, &response); // Создание потокового RPC.
for (...) { // Отправка нескольких запросов, код обхода запросов опущен.
    UpsertRequest request;
    ... // Здесь опущена часть кода заполнения данных запроса.
    if (!stream->Write(request)) {
        break; // Обработка ошибки записи.
    }
}
stream->WritesDone(); // Закрытие потока записи.
// Завершение потокового RPC, получение состояния.
grpc::Status status = stream->Finish();
if (!status.ok()) {
    ... // Обработка ошибки.
}
}

```

С помощью вышеуказанных четырех шагов мы успешно реализовали сервер gRPC для [VdbServer](#). Это позволяет [ProxyServer](#) посредством протокола gRPC в режиме постоянного соединения непрерывно отправлять несколько запросов к [VdbServer](#). По сравнению с решениями на основе HTTP реализация на основе gRPC имеет явные преимущества в производительности. Если вас интересует этот вопрос, вы можете на основе этой структуры провести дальнейшую модификацию и оптимизацию [ProxyServer](#) и [VdbServer](#).

6.1.4. Настройка инструмента для эталонного тестирования

```

Настройка инструмента для эталонного тестирования
├─ Ключевые показатели системы
│  └─ Задержка доступа: задержка р99 и средняя задержка
│  └─ Пропускная способность: производительность системы
│  └─ Полнота: точность запроса
├─ Подготовка тестовых данных
│  └─ Функция generateTestDataBoth
├─ Выполнение тестовых действий
│  └─ Функция performOperation
└─ Вычисление тестовых показателей
    └─ Функция executeBenchmark

```

1. Ключевые показатели системы

Для постоянной оптимизации производительности векторной базы данных нам необходимо сначала точно оценить текущее состояние производительности. Этот процесс зависит от мониторинга и анализа данных о производительности и требует учета реальных бизнес-требований. В области векторных баз данных оптимизация производительности обычно сосредоточена на трех ключевых системных показателях: задержке доступа, пропускной способности и полноте. Мы уже рассматривали эти три показателя в разделе 3.1.3, здесь кратко напомним ключевую информацию.

- **Задержка доступа:** отражает время, необходимое векторной базе данных для обработки запроса. Для более точной оценки производительности системы обычно также учитывают задержку р99 и среднюю задержку. Задержка р99 означает, что в серии запросов 99 % запросов имеют время отклика ниже этой задержки. Средняя задержка – это среднее время отклика серии запросов, то есть сумма времени отклика всех запросов, деленная на количество запросов.
- **Пропускная способность:** измеряет общее количество запросов, которые векторная база данных может обработать за единицу времени, обычно измеряется в запросах в секунду (QPS). В условиях полной загрузки ресурсов пропускная способность может отражать предельную производительность системы.
- **Полнота:** ключевой показатель точности запроса в векторной базе данных. Представляет собой отношение числа фактически возвращенных векторной базой данных наиболее похожих векторов к общему числу фактически похожих векторов, хранящихся в базе данных. Этот показатель является одним из основных отличий векторной базы данных от других традиционных баз данных. Различные уровни полноты тесно связаны с задержкой и пропускной способностью экземпляра. Более высокая полнота обычно означает большее потребление ресурсов.

В области баз данных эталонное тестирование (бенчмарк) – это метод оценки и сравнения производительности различных систем управления базами данных. Оно позволяет путем имитации определенной рабочей нагрузки количественно оценить показатели производительности системы, такие как скорость обработки транзакций, время отклика на запросы, пропускная способность системы. Эталонное тестирование обычно следует определенным стандартам или спецификациям, чтобы обеспечить сопоставимость и объективность результатов тестирования. Эталонное тестирование может быть основано на отраслевых стандартах или иметь произвольные настройки. Например, в традиционной области баз данных требования обычно определяются Советом по производительности обработки транзакций (Transaction Processing Performance Council, TPC) или другими организациями. В области векторных баз данных текущие стандарты тестирования еще не унифицированы, предприятиям необходимо разрабатывать инструменты тестирования и наборы тестовых случаев, наиболее соответствующие их бизнес-потребностям.

Настройка инструмента для эталонного тестирования включает в себя три основные части: подготовку тестовых данных, выполнение тестовых действий и вычисление тестовых показателей. Далее мы последовательно реализуем эти три компонента.

2. Подготовка тестовых данных

Ниже приведен основной код реализации функции `generateTestDataBoth`, используемой для генерации тестовых данных.


```

std::pair<std::vector<Document>, std::vector<Document>>
generateTestDataBoth(int numberOfRequests,
    int dim) {
    std::vector<Document> writeData; // Вектор объектов Document для записи.
    std::vector<Document> readData; // Вектор объектов Document для чтения.
    std::random_device rd; // Инициализация генератора случайных чисел.
    std::mt19937 gen(rd());
    // Распределение для равномерной генерации случайных чисел
    // в интервале [0.0, 1.0).
    std::uniform_real_distribution<> dis(0.0, 1.0);
    for (int i = 0; i < numberOfRequests; ++i) { // Цикл для генерации
запросов.
        std::vector<double> readVectorValues; // Пустой вектор для вектора чтения.
        for (int j = 0; j < dim; ++j) { // Создание dim случайных значений
// для вектора чтения.
            readVectorValues.push_back(dis(gen));
        }
        // Создание объекта Document, содержащий вектор чтения.
        Document readDoc = createDocumentWithVector(readVectorValues);
        // Добавление членов id, k и indexType в объект Document для чтения.
        readDoc.AddMember("id", i, readDoc.GetAllocator());
        readDoc.AddMember("k", 5, readDoc.GetAllocator());
        readDoc.AddMember("indexType", "FLAT", readDoc.GetAllocator());
        // Добавление объекта Document для чтения в вектор readData.
        readData.push_back(std::move(readDoc));
        // Цикл для генерации данных для 5 операций записи.
        for (int writeCount = 0; writeCount < 5; ++writeCount) {
            // Создание пустого вектора для хранения значений вектора записи.
            std::vector<double> writeVectorValues;
            // Генерация dim случайных смещений для вектора записи.
            for (double val : readVectorValues) {
                double offset = dis(gen) * 0.002 - 0.001;
                writeVectorValues.push_back(val + offset);
            }
            // Создание объекта Document, содержащего вектор записи.
            Document writeDoc = createDocumentWithVector(writeVectorValues);
            // Добавление членов id и indexType в объект Document для записи.
            writeDoc.AddMember(
                "id", i * 10000 + writeCount, writeDoc.GetAllocator()
            );
            writeDoc.AddMember("indexType", "FLAT", writeDoc.GetAllocator());
            // Добавление объекта Document для записи в вектор writeData.
            writeData.push_back(std::move(writeDoc));
        }
    }
    // Возврат объекта pair, содержащего сгенерированные данные для записи и чтения.
    return std::make_pair(std::move(writeData), std::move(readData));
}

```

В функции `generateTestDataBoth` мы генерируем необходимые для последующих тестов данные записи и запроса, представленные в формате JSON. Чтобы

сократить время обработки данных во время тестирования, мы заранее подготавливаем тестовые данные, чтобы их можно было использовать непосредственно при выполнении тестов. В этом процессе есть один ключевой момент, требующий особого внимания: необходимо четко идентифицировать данные, которые должны быть возвращены для каждого запроса. Это означает, что мы должны заранее определить целевые результаты запроса. Эти данные будут использоваться для последующей оценки полноты, что является уникальным аспектом эталонного тестирования векторных баз данных.

Для достижения этой цели мы применили специфический алгоритм. Для каждого случайно сгенерированного вектора запроса мы генерируем 5 векторов записи в его окрестности. ID этих векторов записи устанавливаются как $i * 10000 + writeCount$. Такая схема отображения позволяет нам при использовании i -го вектора для запроса определить результаты запроса по возвращаемому ID. Если возвращаемый ID не может быть найден по той же схеме отображения, то этот результат будет отмечен как ошибка полноты.

3. Выполнение тестовых действий

После реализации функции `generateTestDataBoth` далее реализуем функцию `performOperation` для выполнения теста и измерения соответствующих показателей.

```
std::chrono::duration<double> performOperation(const Document& doc,
    const char* url, rapidjson::Document* responseDoc = nullptr) {
    CURL* curl = curl_easy_init(); // Инициализация сессии cURL.
    // Объект duration для записи времени выполнения операции.
    std::chrono::duration<double> duration(0);
    // Если инициализация сессии cURL прошла успешно, создается два объекта
    // для сериализации JSON-документа.
    if (curl) {
        StringBuffer buffer;
        Writer<StringBuffer> writer(buffer);
        doc.Accept(writer); // Сериализация входного JSON-документа в строку.
        std::string jsonStr = buffer.GetString(); // Получение сериализованной
        // строки JSON.

        // Печать URL и содержимое JSON-запроса, который будет отправлен.
        spdlog::info("Sending request to {}: {}", url, jsonStr);
        // Установка URL и данных POST-поля для сессии cURL.
        curl_easy_setopt(curl, CURLOPT_URL, url);
        curl_easy_setopt(curl, CURLOPT_POSTFIELDS, buffer.GetString());
        std::string response; // Определение строки для хранения данных ответа.
        // Функция обратного вызова записи и указатель данных для сессии cURL.
        curl_easy_setopt(curl, CURLOPT_WRITEFUNCTION, callbackFunction);
        curl_easy_setopt(curl, CURLOPT_WRITEDATA, &response);
        // Запись времени начала операции.
        auto start = std::chrono::high_resolution_clock::now();
        CURLcode res = curl_easy_perform(curl); // Выполнение запроса cURL.
        // Запись времени окончания операции.
        auto end = std::chrono::high_resolution_clock::now();
```

```

// Если запрос cURL завершился неудачно, записываем сообщение об ошибке.
if (res != CURLE_OK) {
    spdlog::error("curl_easy_perform() failed: {}",
        curl_easy_strerror(res));
} else if (responseDoc) {
    // Если предоставлен объект ответа, пытаемся разобрать его содержимое.
    responseDoc->Parse(response.c_str());
    // Если разбор завершился неудачно, записываем сообщение об ошибке.
    if (responseDoc->HasParseError()) {
        spdlog::error("Failed to parse response: {}", response);
    }
}
duration = end - start; // Вычисляем и записываем общее время выполнения.
curl_easy_cleanup(curl); // Освобождаем ресурсы сессии cURL.
}
return duration; // Возвращаем время выполнения операции.
}

```

Эта функция использует библиотеку `cpp-libcurl` для построения запросов. Для теста обновления функция отправляет запрос на интерфейс `/upsert`, а для теста запроса – на интерфейс `/search`. Чтобы как можно точнее измерить фактическое время от отправки запроса до получения ответа, мы стараемся ограничить код для измерения времени выполнения внутри этой функции. Кроме того, функция предоставляет параметр `responseDoc`, позволяющий вернуть результат запроса вызывающей стороне.

4. Вычисление тестовых показателей

Чтобы ускорить оценку тестирования всей системы, мы применили многопоточное выполнение тестовых действий в функции `performOperation`. Код реализации функции `executeBenchmark` для выполнения эталонного тестирования представлен ниже.

```

// Функция executeBenchmark выполняет эталонное тестирование, отправляя запросы
// на запись и чтение на указанный URL и вычисляя тестовые показатели.
void executeBenchmark(const std::vector<Document>& testData, int numThreads,
    TestType testType, const std::string& writeUrl,
    const std::string& readUrl, bool checkReadResponse = false) {
    // Создание динамического массива потоков и мьютекса для управления
    // параллельными операциями.
    std::vector<std::thread> threads;
    std::mutex mutex;
    std::vector<std::chrono::duration<double>> durations; // Время выполнения
                                                         // потоков.
    std::vector<double> recallRates(numThreads, 0.0); // Полнота каждого потока.
    // Вычисление размера блока тестовых данных для каждого потока и остаток.
    int blockSize = testData.size() / numThreads;
    int remainder = testData.size() % numThreads;
    // Создание нескольких потоков и распределение задач.

```

```

for (int i = 0; i < numThreads; ++i) {
    threads.emplace_back([&, i]() {
        // Вычисление рабочего диапазона текущего потока.
        int start = i * blockSize + std::min(i, remainder);
        int end = start + blockSize + (i < remainder ? 1 : 0);
        int mismatchCount = 0;
        int totalCount = 0;
        // Перебор тестовых данных, за которые отвечает текущий поток.
        for (int j = start; j < end; ++j) {
            const Document& doc = testData[j];
            std::chrono::duration<double> duration;
            // Выполнение операции записи или чтения в зависимости
            // от типа теста.
            if (testType == TestType::Write) {
                duration = performOperation(doc, writeUrl.c_str());
            }
            if (testType == TestType::Read) {
                // При чтении, возможно, потребуется проверить содержимое
                // ответа.
                if (checkReadResponse) {
                    rapidjson::Document responseDoc;
                    duration = performOperation(doc, readUrl.c_str(),
&responseDoc);

                    // При успешном разборе проверить правильность.
                    if (!responseDoc.HasParseError()
                        && responseDoc.IsObject()) {
                        const auto& vectors =
                            responseDoc["vectors"].GetArray();
                        int requestId = doc["id"].GetInt();
                        for (const auto& v : vectors) {
                            int returnedId = v.GetInt() / 10000;
                            if (returnedId != requestId) {
                                mismatchCount++;
                            }
                        }
                        totalCount++;
                    }
                }
                // Вычисление полноты текущего потока.
                double recallRate = totalCount > 0 ?
                    1.0 - static_cast<double>(mismatchCount)
                    / totalCount : 0.0;
                recallRates[i] = recallRate;
            } else {
                duration = performOperation(doc, readUrl.c_str());
            }
        }
        // Добавление времени выполнения в динамический массив.
        {
            std::lock_guard<std::mutex> lock(mutex);
            durations.push_back(duration);
        }
    });
}

```

```

// Ожидание завершения всех потоков.
for (auto& thread : threads) {
    thread.join();
}

// Вычисление общей и средней длительности и 99-го перцентиля (p99)
// длительности.
double totalDuration = std::accumulate(durations.begin(), durations.end(),
    0.0, [](double sum,
// Перевод в миллисекунды.
const std::chrono::duration<double>& dur) { return sum + dur.count(); })
    * 1000.0;
double averageDuration = totalDuration / durations.size();
std::sort(durations.begin(), durations.end());
size_t p99Index = std::max(0, std::min(
    static_cast<int>(durations.size() * 99 / 100) - 1,
    static_cast<int>(durations.size()) - 1));

// Перевод в миллисекунды.
double p99Duration = durations[p99Index].count() * 1000.0;

// Общее время обработки по-прежнему измеряется в секундах.
double throughput = durations.size() / (totalDuration / 1000.0);

// Вывод тестовых показателей.
spdlog::info("Average duration: {} milliseconds", averageDuration);
spdlog::info("P99 duration: {} milliseconds", p99Duration);
spdlog::info("Throughput: {} operations per second", throughput);

// Если была проверка ответа, вычислить и вывести общую полноту.
if (checkReadResponse) {
    double overallRecallRate = std::accumulate(recallRates.begin(),
        recallRates.end(), 0.0) / numThreads;
    spdlog::info("Overall recall rate: {}", overallRecallRate);
}
}

```

Функция `executeBenchmark` использует ранее созданный набор данных для оценки теста. Она позволяет разработчикам выбирать количество потоков в зависимости от потребностей и поддерживает различные режимы тестирования, включая тесты записи и чтения. В начале теста функция равномерно распределяет запросы между потоками в соответствии с параметром `numThreads`. Для теста чтения мы осуществляем обратное сопоставление по возвращенному ID. Если сопоставленный ID не совпадает с отправленным, запрос помечается как ошибка полноты. По завершении теста функция преобразует измеренные задержки и результаты полноты в фактические результаты и выводит заключение по данному эталонному тестированию.

На этом мы завершаем разработку инструмента эталонного тестирования. Выполнение тестирования с приведенной ниже конфигурацией в файле `conf.ini` приведет к следующим результатам:

```

conf.ini:
[test]
test_type=2
num_threads=4
num_vectors=10
dim=1
write_url=http://172.19.0.9:8080/upsert
read_url=http://172.19.0.9:8080/search

```

Результаты:

```

[2024-01-20 23:09:14.268] [info] Average duration: 0.9895633 milliseconds
[2024-01-20 23:09:14.268] [info] P99 duration: 1.382547 milliseconds
[2024-01-20 23:09:14.268] [info] Throughput: 1010.5467735111032 operations per second
[2024-01-20 23:09:14.268] [info] Overall recall rate: 1

```

Файл `conf.ini` содержит параметры настройки теста, а вывод журнала отображает некоторые ключевые показатели эталонного тестирования, включая задержку доступа, пропускную способность и полноту. Средняя задержка и задержка p99 показывают скорость отклика операций, пропускная способность демонстрирует способность системы обрабатывать запросы, а значение полноты (1) указывает на то, что все операции чтения корректно возвращали ближайшие векторы, – точность запроса составляет 100 %.

С помощью инструмента эталонного тестирования мы можем постоянно отслеживать производительность системы. Можно использовать этот инструмент после каждого обновления системы для оценки того, не привело ли обновление к значительным потерям производительности по сравнению с предыдущими тестами. Для сценариев, требующих постоянной оптимизации векторной базы данных, использование тестирования на основе эталонных порогов производительности в процессе выпуска версий является рекомендуемой лучшей практикой.

6.2. ОПТИМИЗАЦИЯ ЗАТРАТ

Помимо производительности, разработчики при выборе технического решения обычно уделяют внимание оценке затрат. Особенно на начальном этапе бизнеса, когда интенсивность использования базы данных может быть незначительной, затраты на нее будут составлять небольшую долю от общих расходов на информационную инфраструктуру. Однако нужно учитывать, что с быстрым развитием бизнеса и быстрым ростом объема данных могут измениться и затраты. При оценке стоимости системы обычно используется показатель ТСО (total cost of ownership, общая стоимость владения). ТСО представляет собой общие затраты, необходимые для владения этими ресурсами, и в основном состоит из затрат на оборудование и затрат на обслуживание. В области векторных баз данных мы обычно более детально рассматриваем стоимость хранения данных векторов на каждый гигабайт и стоимость ресурсов на один запрос вектора.

В этом разделе мы попытаемся объединить текущие технические решения и архитектуру развертывания системы, чтобы исследовать некоторые возможные подходы к оптимизации затрат. Следует отметить, что оптимизация затрат

сама по себе является динамическим и непрерывным процессом. Вы можете развивать эти идеи и проводить дальнейшие эксперименты в соответствии с потребностями вашего бизнеса, основные принципы остаются неизменными.

6.2.1. Гибридное развертывание нескольких модулей

Из рис. 5.5 видно, что после добавления **MasterServer** и **ProxyServer** количество задействованных компонентов в нашей системе увеличилось. При развертывании завершенной программы в производственной среде необходимо выбирать соответствующие аппаратные ресурсы в зависимости от фактической нагрузки каждого модуля. На начальном этапе мы уделяем первоочередное внимание доступности системы, поэтому выбираем отдельное развертывание соответствующих модулей, что приводит к получению первой версии архитектурной схемы развертывания, изображенной на рис. 6.1.

Из рисунка видно, что для построения относительно полнофункциональной распределенной векторной базы данных при отдельном развертывании каждого модуля требуются ресурсы в объеме 52 ядра (C) и 76 Гб памяти (G).

- **VdbServer** имеет 6 узлов, каждый узел имеет 4C8G, всего 24C48G.
- **ProxyServer** имеет 4 узла, каждый узел имеет 4C4G, всего 16C16G.
- **MasterServer** имеет 3 узла, каждый узел имеет 2C2G, всего 6C6G.
- Хранилище etcd имеет 3 узла, каждый узел имеет 2C2G, всего 6C6G.

Проведем анализ на основе фактической нагрузки каждого модуля векторной базы данных.

Сервер **VdbServer** отвечает за запись и запросы векторов, так как индексы векторных данных необходимо загружать в память, поэтому он требует относительно большого объема памяти. **ProxyServer** отвечает за пересылку потока данных, а **MasterServer** управляет системой метаданных, их потребность в памяти относительно мала. В условиях такой нагрузки мы можем гибридно развернуть **ProxyServer** и **MasterServer** на серверах **VdbServer**, повторно используя ресурсы CPU сервера **VdbServer**.

Применение такой модели гибридного развертывания позволяет сэкономить 22C22G ресурсов **ProxyServer** и **MasterServer**, снижая потребность всей системы в ресурсах с 52C76G до 30C54G, что уменьшает затраты на CPU примерно на 42 %, а на память – примерно на 29 %. Эффект оптимизации затрат здесь очевиден. Кроме того, на основе этой модели развертывания задержка при пересылке **ProxyServer** на локальные сервисы также становится меньше. Гибридное развертывание, помимо экономии затрат, позволяет в определенной степени оптимизировать задержку системы. Скорректированная архитектура гибридного развертывания изображена на рис. 6.2.

Однако такая модель развертывания имеет и побочные эффекты. Если один сервер выйдет из строя, количество затронутых модулей увеличится, что может привести к снижению доступности.

При эксплуатации крупномасштабных систем два ключевых фактора, которые обычно необходимо учитывать в комплексе, – это стоимость инфраструктуры и стабильность, которые взаимно влияют друг на друга. Конкретный способ развертывания в конечном итоге должен определяться на основе фактических бизнес-сценариев.

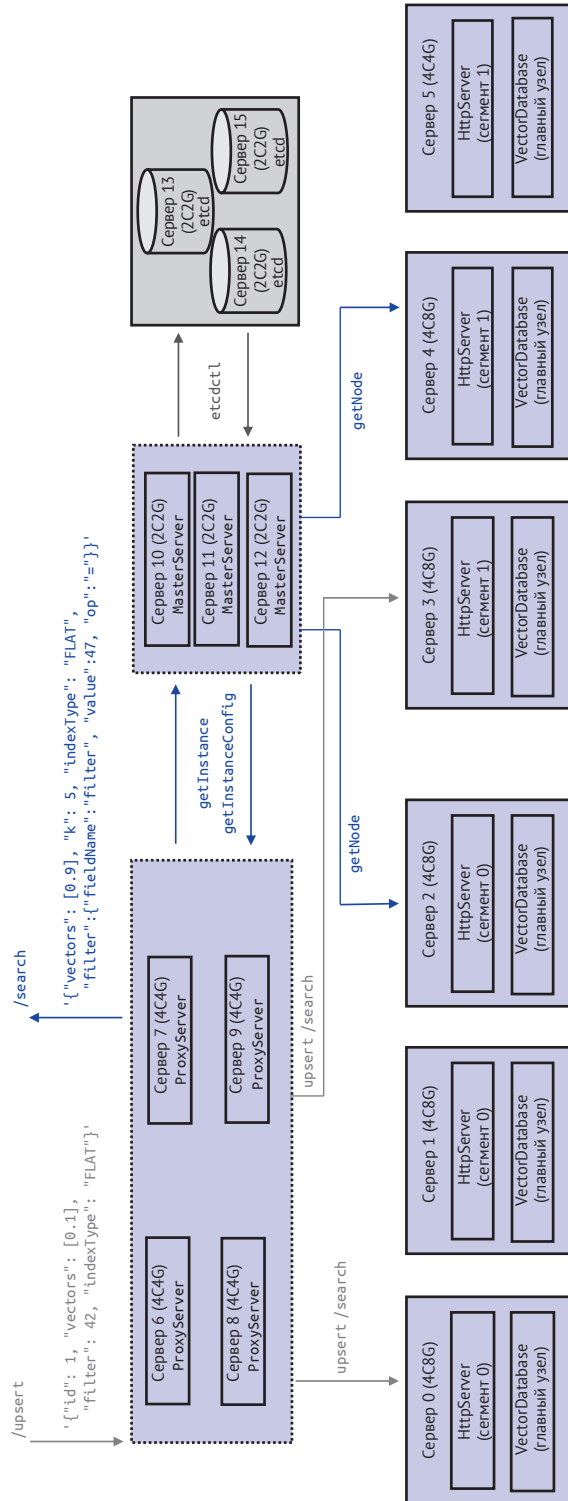


Рис. 6.1. Схема развертывания компонентов по отдельности

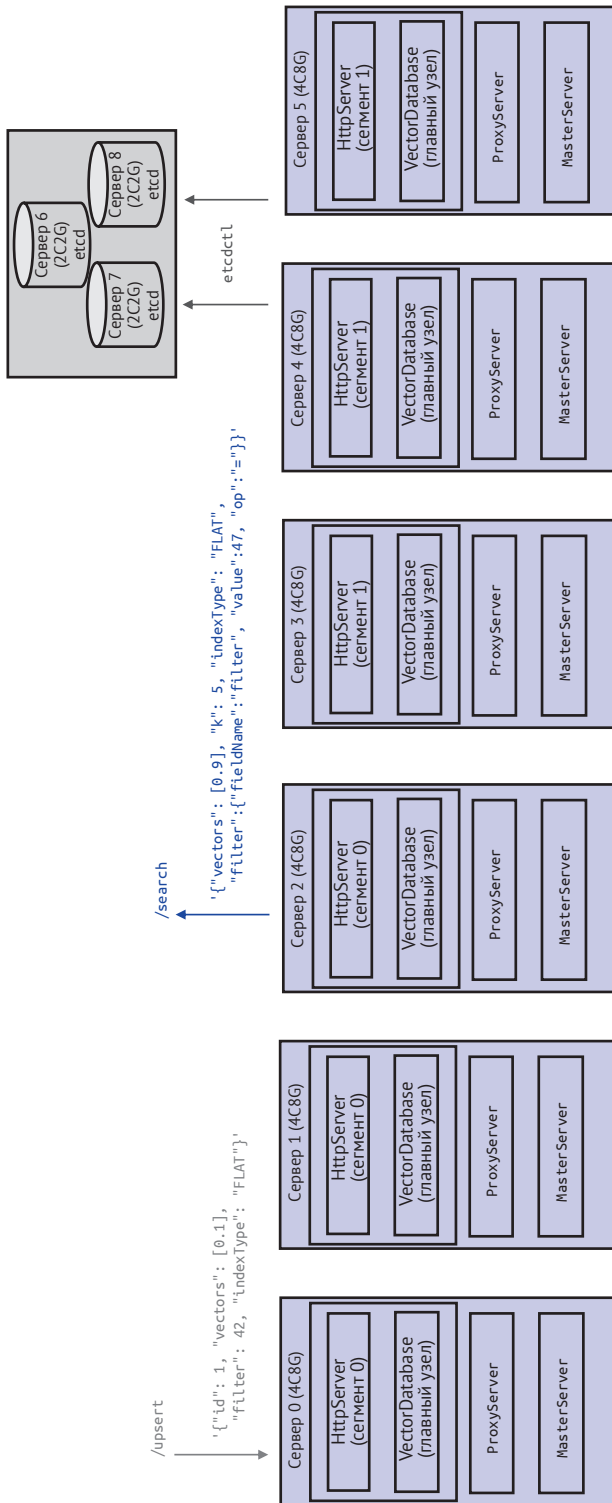


Рис. 6.2. Схема гибридного развертывания компонентов

6.2.2. Развертывание на одном узле

Мы продолжаем анализировать текущую структуру затрат системы, основываясь на модели развертывания из раздела 6.2.1. На данный момент каждый сегмент развертывает 3 сервера **VdbServer**, что обусловлено необходимостью обеспечения высокой надежности и доступности системы. Высокая надежность означает, что данные, уже записанные в векторную базу данных, не будут потеряны в случае отказа любого узла, что требует использования нескольких реплик для обеспечения надежности данных. Высокая доступность означает, что в случае аномалии любого узла **MasterServer** и **ProxyServer** могут быстро обнаружить отказ узла и перенаправить пользовательский трафик на работоспособный узел, что также требует использования нескольких реплик для обеспечения доступности системы.

Конечно, надежность и доступность здесь достигаются за счет «дополнительных затрат». Мы можем продолжить анализировать потенциальные сценарии применения. Например, в терминологии баз данных мы называем определенные сценарии отображения отчетов и анализа небольших объемов данных для администраторов «офлайн-сценариями» или «сценариями Ad Hoc». В таких офлайн-сценариях мы часто храним большие объемы данных, но имеем большую терпимость ко времени недоступности системы, при условии, что система может гарантировать отсутствие потери данных. В таких сценариях можно оптимизировать архитектуру, комбинируя показатели репликации и технологии облачных вычислений.

Простой вариант решения заключается в следующем: развернуть один узел для **VdbServer**, разместив на этом же узле **MasterServer** и **ProxyServer**. Таким образом, мы можем сократить ресурсы, используемые четырьмя серверами **VdbServer**, с 16C32G до 14C22G, а общие затраты ресурсов снизятся с 30C54G до 14C22G. Затраты на CPU уменьшатся примерно на 53 %, а затраты на память – на 59 %. Однако этот подход приводит к неприемлемой проблеме: наши данные будут храниться только на одном узле, и в случае неустраняемого сбоя жесткого диска данные будут утрачены навсегда. Поэтому в это решение нужно внести некоторые изменения. **VdbServer** может хранить данные в удаленной распределенной системе, например в облачной распределенной блочной системе хранения или в собственной распределенной блочной системе хранения. Удаленная распределенная блочная система хранения предоставляет механизм множественных реплик, а для **VdbServer** это выглядит как использование локальной файловой системы для записи файлов без необходимости вносить дополнительные изменения в код.

Возможно, вы знаете, что удаленная распределенная блочная система хранения также требует дополнительных ресурсов CPU и памяти для предоставления услуг, что, безусловно, увеличивает затраты. Однако благодаря многолетнему накоплению знаний в области компьютерных наук распределенные блочные системы хранения обладают растущими преимуществами в отношении ресурсных требований. Несмотря на то что система нуждается в большем количестве ресурсов CPU и памяти для управления и координации хранения и доступа к данным на нескольких узлах, ее кластерная модель обслуживания позволяет нескольким приложениям совместно использовать ресурсы хранения. Этот механизм совместного использования снижает затраты на хранение

для каждого отдельного приложения, а сэкономленные ресурсы CPU и памяти зачастую компенсируют дополнительные расходы, связанные с использованием распределенной системы хранения. Кроме того, хотя распределенное хранение может увеличить задержку записи данных, в большинстве нерелевантных сценариев это обычно не является ключевым показателем производительности для разработчиков.

Скорректированная архитектура развертывания на одном узле представлена на рис. 6.3.

Таким образом, мы построили модель развертывания на одном узле на основе распределенной блочной системы хранения. В случае сбоя на одном узле мы можем автоматически добавить другой узел, затем развернуть приложение на новом узле и запустить его. Удаленная распределенная блочная система хранения обеспечит персистентность векторных данных в системе. На основе этих персистентных данных мы можем завершить восстановление после сбоя на одном узле.

6.3. ОПТИМИЗАЦИЯ УДОБСТВА ИСПОЛЬЗОВАНИЯ

Продукт базы данных представляет собой сложную техническую систему, для эффективной работы которой требуется тесное сотрудничество разработчиков. В качестве примера можно привести реляционную систему баз данных: если разработчики неправильно настроят индекс, скорость выполнения запросов может значительно снизиться, и даже одна строка запроса может исчерпать системные ресурсы. Аналогичные вызовы стоят и перед разработчиками векторной базы данных. При различных объемах векторных данных необходимо выбирать соответствующие параметры создания индекса и правильно настраивать фильтры при запросах для получения точных результатов. Поэтому использование векторной базы данных должно быть максимально простым и понятным, чтобы избежать недоразумений. Здесь важную роль играет хорошо продуманный пакет SDK.

Кроме того, векторная база данных должна учитывать и вопросы безопасности. Данные, хранящиеся в базе данных, очевидно, должны быть защищены от несанкционированного доступа. Учитывая возможность случайного удаления данных разработчиками, векторная база данных должна предоставлять функцию резервного копирования данных. Таким образом, в случае непреднамеренной утраты данных система может восстановить данные до определенного временного момента. Интеграция этих функций, безусловно, повысит удобство использования векторной базы данных.

6.3.1. Пакет SDK

В предыдущих системах мы предоставляли доступ к системе через HTTP URL, что было относительно удобно для отладки. Однако в крупных командах разработчиков обычно используется собственная подсистема разработки, и необходимо интегрировать в нее функции доступа к векторной базе данных. Учитывая, что потребности разработчиков могут различаться, наиболее разумным решением для поставщика базы данных является предоставление единого комплекта разработки (SDK).

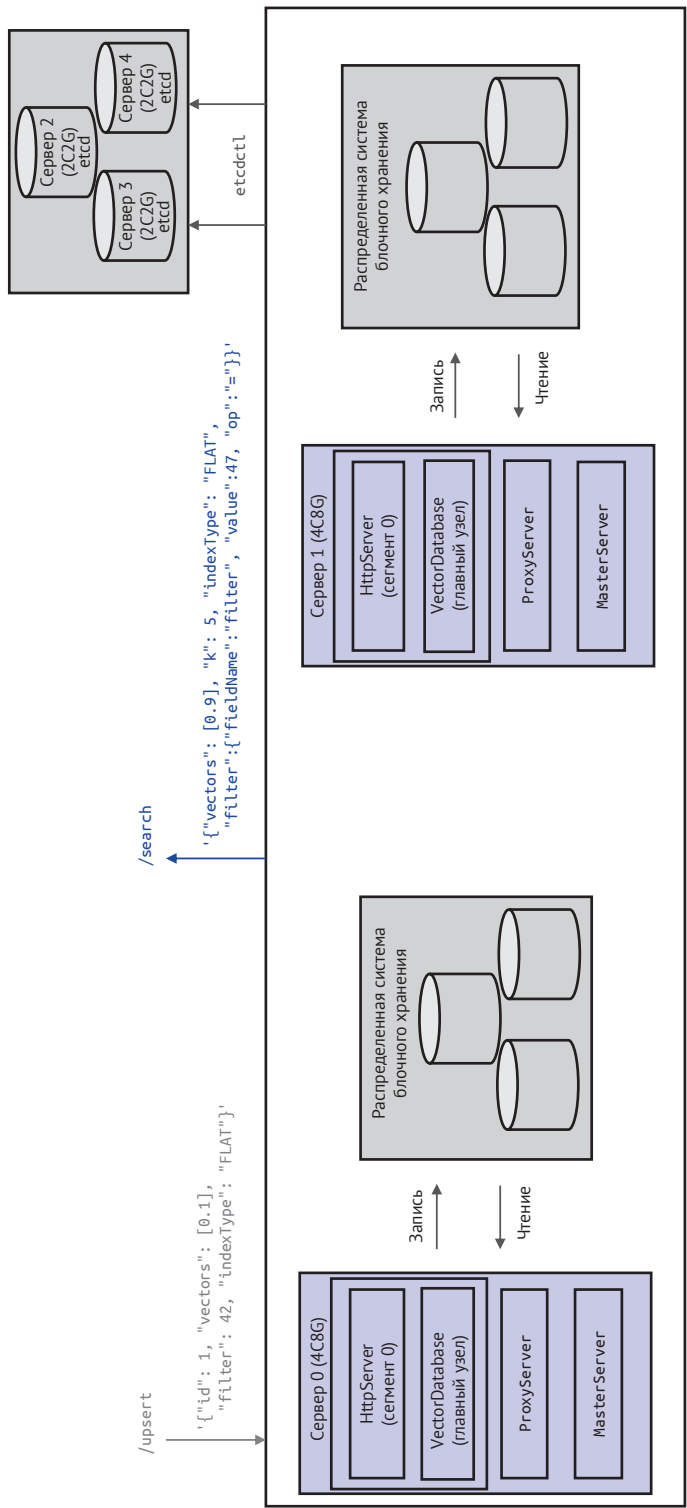


Рис. 6.3. Архитектура развертывания на одном узле

Учитывая, что в ИИ-сценариях наиболее часто используемым языком программирования является Python, мы решили реализовать версию SDK на основе этого языка. Для удобства распространения и использования в сети мы инкапсулируем SDK в пространство имен под названием `vector_db_sdk` (в Python это обычно называется модулем). Такой подход не только помогает избежать конфликтов имен, но и значительно повышает читаемость и поддерживаемость кода. Таким образом, мы стремимся упростить процесс интеграции векторной базы данных, чтобы она лучше соответствовала реальным потребностям и рабочим процессам разработчиков.

Формат организации кода следующий:

- `vector_db_sdk/_init_.py` – файл инициализации модуля;
- `vector_db_sdk/client.py` – файл с реализацией класса `VectorDatabaseSDK`.

Ниже приведена реализация класса `VectorDatabaseSDK` в `client.py`.

```
import requests # Импорт библиотеки requests для отправки HTTP-запросов.
import json # Импорт библиотеки json для обработки JSON-данных.
import logging # Импорт библиотеки logging для записи журналов.

# Класс VectorDatabaseSDK предоставляет интерфейс для взаимодействия
# с векторной базой данных.
class VectorDatabaseSDK:
    def __init__(self, host, port):
        # Установка базового URL и заголовков запроса при инициализации.
        self.base_url = f"http://{host}:{port}"
        self.headers = {"Content-Type": "application/json"}
        # Создание объекта logger для записи журнала.
        self.logger = logging.getLogger(__name__)

    # Приватная функция для отправки HTTP-запросов и обработки ответов.
    def _send_request(self, endpoint, payload):
        try:
            # Отправляем POST-запрос на указанный доступный пункт и передаем
            # payload в формате JSON.
            response = requests.post(
                f"{self.base_url}/{endpoint}", json=payload, headers=self.headers
            )
            # Если код состояния ответа указывает на успешный запрос,
            # возвращаем содержимое ответа в формате JSON.
            response.raise_for_status()
            return response.json()
        except requests.RequestException as e:
            # Если запрос не выполнен (например, возникли проблемы с сетью
            # или ошибка сервера), записываем информацию об ошибке.
            self.logger.error(f"Request failed: {e}")
            return None

    # Публичная функция для записи и обновления векторных данных в базе данных.
    def upsert(self, id, vectors, int_field, index_type):
        # Подготовка данных для записи или обновления.
```

```

payload = {
    "id": id, # Уникальный идентификатор векторных данных.
    "vectors": vectors, # Векторные данные.
    "int_field": int_field, # Значение целочисленного поля.
    "indexType": index_type # Тип индекса.
}
# Вызов функции _send_request для отправки данных и обработки ответа.
return self._send_request("upsert", payload)

# Публичная функция для выполнения запроса на поиск k наиболее похожих
# векторов и фильтрации результатов по условиям.
def search(self, vectors, k, index_type, field_name, value, op):
    # Подготовка данных запроса.
    payload = {
        "vectors": vectors, # Векторные данные для запроса.
        "k": k, # Количество возвращаемых похожих векторов.
        "indexType": index_type, # Тип индекса, используемый для запроса.
        "filter": { # Условия фильтрации.
            "fieldName": field_name, # Имя поля для фильтрации.
            "value": value, # Значение условия фильтрации.
            "op": op # Оператор, например равно, не равно.
        }
    }
    # Вызов функции _send_request для отправки запроса и обработки ответа.
    return self._send_request("search", payload)

```

В классе `VectorDatabaseSDK` мы реализовали несколько ключевых функций: сначала функцию инициализации `__init__`, затем функцию `upsert` для записи и обновления данных, а также функцию `search` для выполнения операций запроса. Кроме того, мы интегрировали внутреннюю функцию инструмента HTTP-запросов для отправки запросов и получения результатов. Для наглядной демонстрации использования этого SDK мы предоставили простой пример тестовой программы, показывающий, как инициализировать SDK, выполнять запись данных и осуществлять запросы. Эти фрагменты кода имеют четкую структуру, легки для понимания и предназначены для того, чтобы помочь разработчикам быстро освоить SDK и интегрировать его в свои приложения.

```

from vector_db_sdk import VectorDatabaseSDK
def test_vector_db_sdk():
    db_sdk = VectorDatabaseSDK(host="localhost", port=8080)
    # Тестируем интерфейс upsert.
    upsert_result = db_sdk.upsert(
        id=6, vectors=[0.9], int_field=47, index_type="FLAT"
    )
    print("Upsert Result:", upsert_result)
    # Тестируем интерфейс search.
    search_result = db_sdk.search(
        vectors=[0.9], k=5, index_type="FLAT", field_name="int_field",
        value=47, op="="
    )
    print("Search Result:", search_result)

```

```
if __name__ == "__main__":
    test_vector_db_sdk()
```

С помощью класса `VectorDatabaseSDK` работа с векторной базой данных становится более стандартизированной и упорядоченной. Этот SDK позволяет явно передавать соответствующие параметры через интерфейс, что упрощает взаимодействие с векторной базой данных. Кроме того, SDK обладает способностью демонстрировать текущие функции системы, что более наглядно, по сравнению с традиционным способом составления URL запросов, и более удобно для разработчиков. Такой подход не только повышает удобство использования системы, но и усиливает понимание функциональности и производительности базы данных, позволяя более эффективно использовать векторную базу данных для удовлетворения своих потребностей в приложениях.

6.3.2. Аутентификация доступа

Аутентификация доступа

- ├ Хранение метаданных аутентификации
 - | └ Хранение имени пользователя и пароля в системе хранения etcd
 - | └ Введение управления паролями или статического шифрования паролей
- ├ Обновление метаданных аутентификации
 - | └ Добавление переменной-члена для хранения учетных данных пользователя
 - | └ Обеспечение производительности с помощью динамического двойного массива
- ├ Выдача токенов
 - | └ Генерация токена JWT на основе имени пользователя и пароля
 - | └ Распределение токена JWT через интерфейс `handleJwtTokenRequest`
- ├ Проверка токенов
 - | └ Функция `validateJwt` для проверки легитимности токена JWT
 - | └ Интеграция в функцию `forwardRequest` `ProxyServer`
- └ Интеграция аутентификации в SDK
 - └ Добавление функции `authenticate` в `VectorDatabaseSDK` для получения JWT
 - └ Добавление информации об аутентификации в функцию `_send_request`

В разделе 6.3.1 мы разработали простой пакет SDK, который облегчает разработчикам сборку URL запросов. Однако в нашей системе отсутствует функция аутентификации доступа, что означает, что любой, кто знает адрес экземпляра и может подключиться к нему и получить доступ к нашей векторной базе данных, что создает значительные риски для безопасности. Для повышения безопасности мы планируем разработать механизм аутентификации доступа в систему.

Суть механизма аутентификации заключается в разумном использовании серверного пароля. Клиент будет использовать этот пароль для шифрования поля аутентификации в запросе, а сервер, получив запрос, расшифрует эти поля для проверки подлинности. В индустрии существует множество сложных схем аутентификации с использованием токенов, мы выбрали относительно легковесную схему с использованием токенов JWT (JSON web token).

1. Хранение метаданных аутентификации

Для реализации аутентификации JWT нам необходимо сначала внести изменения в класс `MasterServer`, чтобы добавить поддержку хранения имени пользователя и пароля в системе метаданных. Мы выбрали хранение этой информации в системе хранения `etcd` в определенном формате, который выглядит следующим образом:

```
key: /instancesConfig/instance1/credentials
value: {"user2":{"username":"user2","password":"newpassword456"}, "user1":{"username":"user1","password":"newpassword123"}}
```

В системе хранения `etcd` мы храним соответствующие имена пользователей и пароли для каждого экземпляра. Следует особо отметить, что в этом примере пароли хранятся в открытом виде, что не является идеальным с точки зрения безопасности. Вы можете на основании нашего решения ввести дополнительные меры для повышения безопасности системы. Одним из возможных методов является использование общей системы управления ключами (KMS) или статическое шифрование паролей. Таким образом, пароли могут безопасно храниться и расшифровываться в памяти после чтения, что снижает потенциальные риски безопасности.

Кроме того, мы ввели в `MasterServer` две новые функции для управления данными об именах пользователей и паролях, связанных с экземплярами. Эти две функции не только повышают безопасность системы, но и предоставляют более удобные функции управления пользователями. Таким образом, системные администраторы могут легко поддерживать учетные данные пользователей, обеспечивая надлежащий контроль доступа.

```
void updateUserCredentials(const httpLib::Request& req, httpLib::Response& res);
void getUserCredentials(const httpLib::Request& req, httpLib::Response& res);
```

Эти две новые функции управляют данными, хранящимися в хранилище `etcd`, через клиент `etcd`. `updateUserCredentials` обрабатывает HTTP-запросы на обновление информации об учетных данных пользователя, а `getUserCredentials` обрабатывает HTTP-запросы на получение информации об учетных данных пользователя. Чтобы более четко продемонстрировать этот процесс, мы объясним его на примере получения соответствующих метаданных через функцию-член `getUserCredentials` класса `MasterServer`. Код реализации выглядит следующим образом:

```
void MasterServer::getUserCredentials(
    const httpLib::Request& req, httpLib::Response& res) {
    // Получение параметра ID экземпляра из HTTP-запроса.
    auto instanceId = req.get_param_value("instanceId");
    // Построение ключа для хранения информации об учетных данных
    // пользователя в etcd.
```



```

std::string etcdKey = "/instancesConfig/" + instanceId +
    "/credentials";
try {
    // Получение значения соответствующего ключа из клиента etcd.
    etcd::Response etcdResponse = etcdClient_.get(etcdKey).get();
    // Если ответ etcd не успешен, установка кода состояния ответа в 1
    // и возврат сообщения об ошибке.
    if (!etcdResponse.is_ok()) {
        setResponse(res, 1,
            "Error accessing etcd: " +
            etcdResponse.error_message());
        return;
    }
    // Разбор JSON-строки, возвращенной etcd, в объект документа RapidJSON.
    rapidjson::Document doc;
    doc.Parse(etcdResponse.value().as_string().c_str());
    // Если разбор JSON не удался, установка кода состояния ответа в 1
    // и возврат сообщения об ошибке.
    if (!doc.IsObject()) {
        setResponse(res, 1, "Invalid JSON format in etcd");
        return;
    }
    // Если учетные данные успешно получены, установка кода состояния
    // ответа в 0 и возврат успешного сообщения и учетных данных.
    setResponse(res, 0, "User credentials retrieved successfully", &doc);
} catch (const std::exception& e) {
    // Если возникает исключение, установка кода состояния ответа в 1
    // и возврат сообщения об исключении.
    setResponse(res, 1, "Exception occurred: " + std::string(e.what()));
}
}

```

Как видно из кода, функция `getUserCredentials` считывает соответствующую информацию об аутентификации из хранилища etcd и возвращает ее запрашивающей стороне. На основе этого интерфейса мы можем расширить сервер `ProxyServer`, чтобы он мог предоставлять серверные возможности аутентификации на основе имени пользователя и пароля.

2. Обновление метаданных аутентификации

Мы продолжаем расширять определение `ProxyServer` и добавляем в него переменную-член для хранения имен пользователей и паролей.

```

std::unordered_map<std::string, std::pair<std::string, std::string>>
userCredentials_[2];
// Указывает индекс массива учетных данных активных пользователей.
std::atomic<int> activeUserCredentialsIndex_;

```

Этот код определяет две переменные: `userCredentials_` и `activeUserCredentialsIndex_`. `userCredentials_` – это хеш-таблица, используемая для хранения ин-

формации об учетных данных пользователей, в которой каждый пользователь имеет уникальный ключ и связанную с ним пару значений. Для повышения надежности данных эта хеш-таблица имеет два экземпляра. `activeUserCredentialsIndex_` – это атомарное целое число, указывающее индекс экземпляра `userCredentials_`, который в данный момент используется.

Аналогично предыдущим сценариям, где требовалось обновление метаданных с `MasterServer`, здесь для обновления имен пользователей и паролей в памяти `ProxyServer` также используется метод динамического двойного массива для обеспечения высокой производительности. Поскольку код обновления здесь схож с реализацией получения других метаданных с `MasterServer` для `ProxyServer`, мы сосредоточимся только на части, связанной с обновлением учетных данных пользователей. Упрощенная реализация функции-члена `fetchAndUpdateUserCredentials` класса `ProxyServer` представлена ниже.

```
void ProxyServer::fetchAndUpdateUserCredentials() {
    // Запись операции получения информации об учетных данных пользователей
    // с MasterServer.
    GlobalLogger->info("Fetching User Credentials from Master Server");
    // Формирование URL-запроса, включая адрес MasterServer,
    // порт и ID экземпляра.
    std::string url = "http://" + masterServerHost_ + ":"
        + std::to_string(masterServerPort_) +
        "/getUserCredentials?instanceId=" + instanceId_;
    ... // Код отправки запроса и получения ответа через cURL опущен.
    // Получение индекса неактивного массива учетных данных для обновления.
    int inactiveIndex = activeUserCredentialsIndex_.load() ^ 1;
    // Очистка неактивного массива учетных данных, подготовка к добавлению
    // новых данных.
    userCredentials_[inactiveIndex].clear();
    // Перебор объектов пользователей в JSON-ответе.
    const auto& users = doc["data"].GetObject();
    for (const auto& user : users) {
        // Извлечение имени пользователя и пароля из объекта пользователя.
        std::string username = user.name.GetString();
        std::string password = user.value["password"].GetString();
        // Добавление имени пользователя и пароля в неактивный массив
        // учетных данных.
        userCredentials_[inactiveIndex][username] =
            std::make_pair(username, password);
    }
    // Атомарное переключение индекса активного массива учетных данных,
    // чтобы обеспечить непрерывность обслуживания во время обновления.
    activeUserCredentialsIndex_.store(inactiveIndex);
    // Запись информации об успешном обновлении учетных данных пользователей.
    GlobalLogger->info("User Credentials updated successfully");
}
```

Этот интерфейс получает актуальные имена пользователей и пароли, вызывая интерфейс `/getUserCredentials` класса `MasterServer`, затем обновляет их в неактивном массиве и атомарно заменяет индекс текущего активного массива.

3. Выдача токенов

На этой основе далее необходимо реализовать логику выдачи токенов JWT и логику проверки подлинности перед выдачей, основываясь на информации об именах пользователей и паролях. Код реализации приведен ниже.

```
bool ProxyServer::generateJwt(
    const std::string& username, std::string& jwtToken
) {
    // Получение индекса массива учетных данных активных пользователей.
    int gthactiveIndex = activeUserCredentialsIndex_.load();
    auto& credentials = userCredentials_[activeIndex];
    // Проверка наличия имени пользователя в учетных данных.
    auto it = credentials.find(username);
    if (it == credentials.end()) {
        GlobalLogger->error("Username not found: {}", username);
        return false; // Имя пользователя не найдено.
    }
    // Использование имени пользователя и пароля для генерации уникального
    // пароля JWT.
    // Пример: соединение базового пароля и информации о пользователе.
    std::string jwtSecret = "SecretBase" + it->second.first + it->second.second;
    try {
        // Установка срока действия JWT.
        auto token = jwt::create()
            .set_issuer("ProxyServer")
            .set_type("JWT")
            .set_issued_at(std::chrono::system_clock::now())
            // Установка срока действия на 30 минут.
            .set_expires_at(
                std::chrono::system_clock::now() + std::chrono::minutes{30}
            )
            .set_payload_claim("username", jwt::claim(username));
        jwtToken = token.sign(jwt::algorithm::hs256{jwtSecret});
        return true; // Успешная генерация.
    } catch (const std::exception& e) {
        GlobalLogger->error(
            "Failed to generate JWT for {}: {}", username, e.what());
        return false; // Ошибка при генерации JWT.
    }
}
```

В функции `generateJwt` мы на основе комбинации имени пользователя и пароля текущего экземпляра создаем строку `jwtSecret`. Затем, используя функцию `token.sign`, генерируем `jwtToken` с сроком действия 30 мин. Этот `jwtToken` будет возвращаться клиенту для последующей аутентификации при запросах к серверу. Важно отметить, что перед получением `jwtToken` клиент должен пройти проверку подлинности. Ниже представлена функция-член `handleJwtTokenRequest` класса `ProxyServer` для обработки запросов на получение `jwtToken`.

```
void ProxyServer::handleJwtTokenRequest(
    const httplib::Request& req, httplib::Response& res) {
    rapidjson::Document requestDoc; // Объект документа RapidJSON для
    // разбора запроса.
    rapidjson::Document responseDoc; // Объект документа RapidJSON для
    // формирования ответа.
    responseDoc.SetObject(); // Инициализация объекта документа ответа.
    // Получение распределителя объекта документа ответа.
    rapidjson::Document::AllocatorType& allocator = responseDoc.GetAllocator();
    // Разбор JSON-данных из тела запроса.
    if (requestDoc.Parse(req.body.c_str()).HasParseError()) {
        res.status = 400; // При сбое разбора установка кода 400 (Bad Request).
        // Добавление информации об ошибке формата в документ ответа.
        responseDoc.AddMember("error", "Invalid JSON format", allocator);
    } else if (!requestDoc.HasMember("username") ||
        !requestDoc.HasMember("password")) {
        res.status = 400; // Если в запросе нет имени или пароля,
        // установка кода 400 (Bad Request).
        // Добавление информации об ошибке отсутствия имени или пароля
        // в документ ответа.
        responseDoc.AddMember("error",
            "Missing username or password",
            allocator);
    } else {
        // Извлечение имени пользователя из запроса.
        std::string username = requestDoc["username"].GetString();
        // Извлечение пароля из запроса.
        std::string password = requestDoc["password"].GetString();
        // Проверка действительности имени пользователя и пароля.
        if (!validateCredentials(username, password)) {
            res.status = 401; // Если проверка не удалась, установка
            // кода 401 (Unauthorized).
            // Добавление информации об ошибке неверного пароля
            // в документ ответа.
            responseDoc.AddMember("error",
                "Invalid username or password",
                allocator);
        } else {
            std::string jwtToken;
            // Если проверка успешна, генерация токена JWT.
            if (!generateJwt(username, jwtToken)) {
                // Если генерация JWT не удалась, установка кода 500
                // (Internal Server Error).
                res.status = 500;
                // Добавление информации об ошибке генерации JWT
                // в документ ответа.
                responseDoc.AddMember("error",
                    "Failed to generate JWT",
                    allocator);
            } else {
```

```

        // Создание успешного JSON-ответа, содержащего токен JWT.
        responseDoc.AddMember(
            "jwtToken",
            rapidjson::Value().SetString(jwtToken.c_str(),
                                         allocator),
            allocator);
    }
}
// Сериализация объекта документа ответа в строку JSON.
rapidjson::StringBuffer buffer;
rapidjson::Writer<rapidjson::StringBuffer> writer(buffer);
responseDoc.Accept(writer);
// Установка содержимого и типа ответа HTTP.
res.set_content(buffer.GetString(), "application/json");
}

```

В функции `handleJwtTokenRequest` система сначала извлекает из запроса пользователя поля `username` и `password`, затем вызывает функцию `validateCredentials` для проверки этих учетных данных. После успешной проверки используется функция `generateJwt` для генерации токена `jwtToken`, который возвращается пользователю. Реализация `validateCredentials` относительно проста: она лишь сравнивает данные с актуальными именами пользователей и паролями, хранящимися в памяти. Реализация этой функции представлена ниже.

```

bool ProxyServer::validateCredentials(
    const std::string& username, const std::string& password) {
    int activeIndex = activeUserCredentialsIndex_.load();
    auto& credentials = userCredentials_[activeIndex];
    auto it = credentials.find(username);
    if (it != credentials.end() && it->second.second == password) {
        return true; // Имя пользователя и пароль действительны.
    }
    return false; // Имя пользователя или пароль недействительны.
}

```

Реализовав функцию `handleJwtTokenRequest`, можно использовать следующую команду для тестирования работы этого интерфейса:

```

Запрос:
curl -X POST http://localhost/getJwtToken -H "Content-Type: application/json"
-d '{"username":"user2","password":"newpassword456"}'
Ответ:
{"jwtToken":"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJleHAiOjE3MDM2ODkyMjYsImh-
dCI6MTcwMzY4NzQyNiwiOiJjoIUhJveHlTZXJ2ZXIiLCJ1c2VybmFtZSI6InVzZXIyIn0.507Q
qEoyGfqoIdFsaIyQoPCmb49tMwORCvs0qqxSjlc"}

```

Эта команда демонстрирует процесс отправки на сервер запроса HTTP POST с использованием инструмента cURL для получения токена JWT, а также

пример ответа сервера. Функция сервера заключается в приеме запроса, содержащего имя пользователя и пароль, проверке их действительности и, если проверка пройдена, в генерации токена JWT и его возврате клиенту. Токен JWT представляет собой зашифрованную строку, содержащую информацию о пользователе и некоторые метаданные, такие как время истечения, и может использоваться сервером и клиентом для безопасной передачи информации.

4. Проверка токена

Последующие запросы клиента будут содержать данный токен `jwtToken` в информации для аутентификации. Когда `ProxyServer` получает запрос пользователя, он сначала должен проверить действительность `jwtToken`. Только после успешной проверки сервер будет осуществлять переадресацию содержимого. Ниже представлена логика реализации функции проверки токена `validateJwt`.

```
bool ProxyServer::validateJwt(const std::string& jwtToken, std::string& username) {
    try {
        auto decoded = jwt::decode(jwtToken);
        int activeIndex = activeUserCredentialsIndex_.load();
        auto& credentials = userCredentials_[activeIndex];
        // Извлечение имени пользователя.
        username = decoded.get_payload_claim("username").as_string();
        // Проверка токена JWT.
        auto it = credentials.find(username);
        if (it == credentials.end()) {
            return false; // Имя пользователя не существует.
        }
        std::string jwtSecret = "SecretBase" + it->second.first
                               + it->second.second;
        auto verifier = jwt::verify()
            .allow_algorithm(jwt::algorithm::hs256{jwtSecret})
            .with_issuer("ProxyServer");
        verifier.verify(decoded);
        return true; // JWT действителен.
    } catch (...) {
        return false; // Сбой проверки JWT.
    }
}
```

Функция `validateJwt` отвечает за проверку действительности `jwtToken` и `username` в запросе клиента. Этот процесс является обратной операцией по отношению к процессу генерации, он проверяет действительность токена `jwtToken` путем сравнения с именами пользователей и паролями, хранящимися в памяти. Только запросы, прошедшие эту проверку, считаются действительными. Далее мы интегрируем эту функцию в точку входа функции переадресации `forwardRequest` класса `ProxyServer`.

```

void ProxyServer::forwardRequest(const httplib::Request& req, httplib::Response& res,
    const std::string& path) {
    // Извлечение значения заголовка Authorization из HTTP-запроса.
    auto authHeader = req.get_header_value("Authorization");
    std::string jwtToken; // Определение строковой переменной для хранения JWT.
    // Проверка, начинается ли значение заголовка Authorization с "Bearer ",
    // если да, извлечение JWT.
    if (authHeader.rfind("Bearer ", 0) == 0) {
        jwtToken = authHeader.substr(7); // Извлечение JWT после "Bearer ".
    }
    // Определение строковой переменной для хранения извлеченного имени
    // пользователя.
    std::string username;
    // Проверка, является ли JWT действительным, и извлечение имени
    // пользователя.
    if (jwtToken.empty() || !validateJwt(jwtToken, username)) {
        res.status = 401; // Если токен пуст или недействителен, установка
                        // кода 401.
        res.set_content("Invalid or missing JWT", "text/plain"); // Установка
                                                                // содержимого и типа ответа.
        return;
    }
    ... // Дополнительная логика переадресации, такая как выбор целевого узла,
        // построение запроса и т.д., опущена.
    forwardToTargetNode(req, res, path, targetNode); // Переадресация запроса
                                                    // на целевой узел.
}

```

5. Интеграция аутентификации в SDK

На данном этапе сервер `ProxyServer` обладает способностью выдавать токен `jwtToken` и проверять действительность последующих запросов. Далее, на основе реализованного в разделе 6.3.1 пакета SDK на Python мы разработаем соответствующие клиентские функции. Сначала добавим в класс `VectorData-baseSDK` функцию получения токена `authenticate`.

```

def authenticate(self, username, password):
    url = f"{self.base_url}/getJwtToken"
    payload = {"username": username, "password": password}
    response = requests.post(url, json=payload)
    response.raise_for_status()
    self.jwt_token = response.json().get("jwtToken")

```

Функция `authenticate` принимает в качестве параметров имя пользователя и пароль, вызывает интерфейс сервера `getJwtToken`, чтобы получить токен. Этот токен затем сохраняется в атрибуте `jwt_token` текущего объекта. С `jwt_token` можно в дальнейшем отправлять другие запросы, добавляя необходимую информацию для аутентификации.

```
def _send_request(self, endpoint, payload):
    if self.jwt_token:
        self.headers["Authorization"] = f"Bearer {self.jwt_token}"
    try:
        response = requests.post(
            f"{self.base_url}/{endpoint}", json=payload, headers=self.headers
        )
        response.raise_for_status()
        return response.json()
    except requests.RequestException as e:
        self.logger.error(f"Request failed: {e}")
        return None
```

Этот код определяет приватную функцию `_send_request`, которая используется для отправки на сервер запросов HTTP POST. `_send_request` сначала проверяет наличие токена JWT и добавляет его в заголовок запроса для выполнения аутентификации. Затем она использует библиотеку `requests` для отправки запроса POST, в котором `payload` отправляется как данные JSON. Если код состояния ответа сервера указывает на успешное выполнение запроса, функция анализирует и возвращает содержимое ответа в формате JSON. Если в процессе запроса возникает какое-либо исключение (например, проблемы с сетью, недействительный ответ и т. д.), функция регистрирует сообщение об ошибке и возвращает `None`.

После инкапсуляции новой версии SDK на Python с информацией для аутентификации мы можем обновить соответствующую тестовую программу для проверки этих процессов.

```
from vector_db_sdk import VectorDatabaseSDK

def test_vector_db_sdk():
    db_sdk = VectorDatabaseSDK(host="localhost", port=80)
    db_sdk.authenticate(username="user2", password="newpassword456")
    # Тестирование интерфейса upsert.
    upsert_result = db_sdk.upsert(
        id=6, vectors=[0.9], int_field=47, index_type="FLAT"
    )
    print("Upsert Result:", upsert_result)
    # Тестирование интерфейса search.
    search_result = db_sdk.search(
        vectors=[0.9], k=5, index_type="FLAT", field_name="int_field",
        value=47, op="!="
    )
    print("Search Result:", search_result)

if __name__ == "__main__":
    test_vector_db_sdk()
```


С помощью этой тестовой программы мы можем проверить процесс проверки в [ProxyServer](#). Если информация для аутентификации изменена, последующие операции обновления и запроса вернут ошибку аутентификации. Таким образом, мы полностью реализовали систему взаимодействия клиента и сервера для аутентификации. В текущей версии токен JWT истекает через 30 мин, вы можете добавить логику повторной попытки после истечения срока его действия для реализации функции продления.

6.3.3. Резервное копирование данных

В процессе эксплуатации векторной базы данных в корпоративной среде потеря данных из-за отказа локального жесткого диска предотвращается с помощью механизма многократной репликации, которая обеспечивает безопасность данных в случае отказа автономного узла. Рассмотрим ситуацию: из-за ошибочных операций компания случайно удалила векторные данные из базы данных. В этом случае команда удаления также будет синхронизирована и выполнена на распределенных узлах, что приведет к удалению соответствующих данных по всей системе. Хотя это поведение системы соответствует проектно-му, в некоторых случаях оно может не соответствовать требованиям бизнеса.

Для решения таких проблем в отрасли обычно предоставляется независимая функция резервного копирования данных. Она включает в себя регулярное резервное копирование локальных файлов моментальных снимков и предзаписей векторной базы данных, чтобы при незапланированном удалении данных можно было использовать эти резервные файлы для восстановления данных до определенного момента в прошлом. Для реализации резервного копирования векторной базы данных обычно используется система обходного резервного копирования в сочетании с системой базы данных. На рис. 6.4 изображена типичная архитектура системы резервного копирования данных.

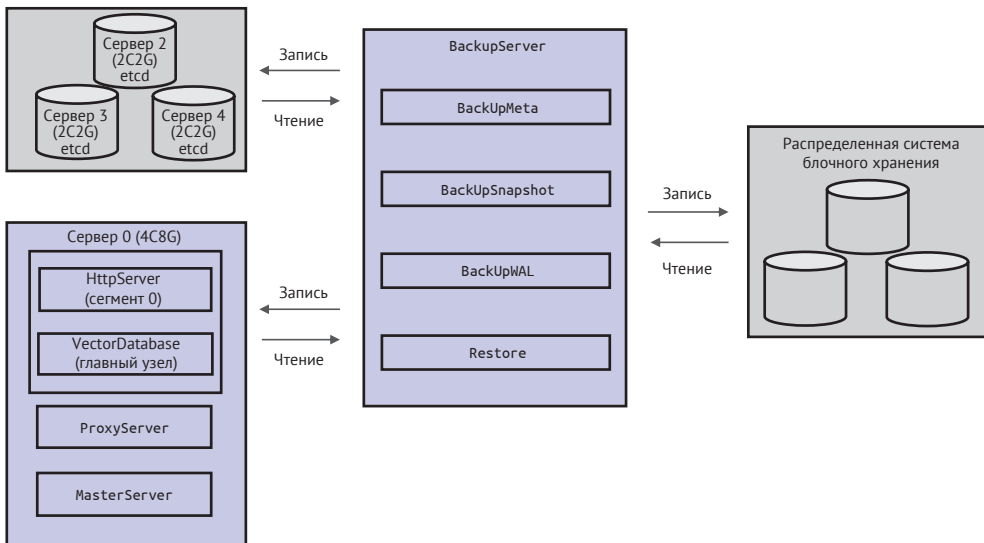


Рис. 6.4. Типичная архитектура системы резервного копирования данных

Для выполнения всего процесса резервного копирования нам необходимо сохранить данные, включая метаданные из хранилища etcd, информацию о моментальных снимках векторных данных на VdbServer и информацию предзаписей. Особое внимание следует уделить тому, как обеспечить их атомарное объединение, поскольку эти три набора данных существуют независимо. Служба резервного копирования обычно реализуется с помощью внешней системы, объединяющей уже существующие инструменты. Ниже представлена реализация процесса резервного копирования в классе BackupServer.

```
class BackupServer:
    # Инициализация сервера резервного копирования.
    def __init__(self, etcd_client, vdb_client, cos_client):
        self.etcd_client = etcd_client
        self.vdb_client = vdb_client
        self.cos_client = cos_client

    # Включение блокировки конфигурации сегментирования.
    def lock_partition_config(self):
        self.etcd_client.lock() # Выполнение операции блокировки через клиент etcd.

    # Резервное копирование конфигурации сегмента в COS
    # (объектное хранилище Tencent Cloud).
    def backup_partition_config_to_cos(self):
        # Получение конфигурации сегмента от MasterServer.
        partition_config = self.etcd_client.get_partition_config()
        # Резервное копирование конфигурации сегмента в COS.
        self.cos_client.save_to_cos(partition_config)

    # Приостановка операций создания новых моментальных снимков на VdbServer.
    def pause_snapshot_operations(self):
        # Приостановка операций моментальных снимков через клиент VdbServer.
        self.vdb_client.pause_snapshots()

    # Резервное копирование файлов моментальных снимков сегмента на VdbServer в COS.
    def backup_snapshots_to_cos(self):
        # Получение всех файлов моментальных снимков сегмента от VdbServer.
        snapshots = self.vdb_client.get_snapshots()
        # Резервное копирование файлов моментальных снимков в COS.
        for snapshot in snapshots:
            self.cos_client.save_to_cos(snapshot)

    # Резервное копирование предзаписи сегмента на VdbServer в COS.
    def backup_wal_logs_to_cos(self):
        # Получение всех WAL сегмента от VdbServer.
        wal_logs = self.vdb_client.get_wal_logs()
        # Резервное копирование предзаписи в COS.
        for wal_log in wal_logs:
            self.cos_client.save_to_cos(wal_log)
```

```

# Возобновление операций создания новых моментальных снимков на VdbServer.
def resume_snapshot_operations(self):
    # Возобновление операций моментальных снимков через клиент VdbServer.
    self.vdb_client.resume_snapshots()

# Освобождение блокировки конфигурации сегмента.
def unlock_partition_config(self):
    self.etcd_client.unlock() # Выполнение операции разблокировки через
                             # клиент etcd.

# Выполнение полного процесса резервного копирования.
def perform_backup(self):
    try:
        self.lock_partition_config()
        self.backup_partition_config_to_cos()
        self.pause_snapshot_operations()
        self.backup_snapshots_to_cos()
        self.backup_wal_logs_to_cos()
        self.resume_snapshot_operations()
    finally:
        self.unlock_partition_config()

```

Ниже приведено подробное описание процесса резервного копирования.

1. Включение блокировки конфигурации сегментирования. В период блокировки информация о сегментировании на [VdbServer](#) не подлежит изменению, чтобы обеспечить совместное использование моментальных снимков и метаданных.
2. Резервное копирование конфигурации сегментирования, управляемой [MasterServer](#), в COS (объектное хранилище Tencent Cloud).
3. Приостановка операций создания новых моментальных снимков в [Vdb-Server](#).
4. Резервное копирование файлов моментальных снимков сегмента [VdbServer](#) в COS.
5. Резервное копирование предзаписи сегмента [VdbServer](#) в виде инкрементальных данных.
6. Возобновление операций создания новых моментальных снимков в [Vdb-Server](#).
7. Освобождение блокировки конфигурации сегментирования, чтобы экземпляр вернулся в нормальное состояние.

Весь процесс резервного копирования показывает, что для обеспечения согласованности метаданных системы и журналов моментальных снимков мы временно прекращаем функцию изменения сегментов в процессе резервного копирования моментальных снимков и избегаем создания новых файлов моментальных снимков во время резервного копирования. На основе пятого шага резервного копирования предзаписи мы можем восстановить данные до состояния последнего момента перед предзаписью. Разработчики могут выбрать частоту резервного копирования в зависимости от реальной бизнес-ситуации, чтобы найти разумный баланс между затратами и безопасностью данных.

6.4. РЕЗЮМЕ

Для удовлетворения потребностей эпохи искусственного интеллекта в хранении и запросах огромных объемов векторных данных в этой главе представлены различные меры оптимизации распределенной векторной базы данных. Эти меры нацелены на оптимизацию с точки зрения производительности, затрат и удобства использования, обеспечивая тем самым способность векторной базы данных непрерывно и эффективно предоставлять услуги.

Оптимизация производительности

Вычисление сходства векторов, являясь основной функцией базы данных, требует значительных ресурсов CPU. Поэтому мы изучили, как более эффективно использовать технологии аппаратной параллелизации. Применяя набор инструкций AVX-512, мы научились выполнять параллельные вычисления для 16 32-битных чисел с плавающей запятой, что значительно увеличило параллелизм вычислительных функций и, следовательно, существенно повысило общую производительность системы. Помимо оптимизации на аппаратном уровне, мы также исследовали методы улучшения на уровне программного обеспечения. В области запросов мы внедрили стратегию «эволюции запроса», то есть выбирали подходящий алгоритм запроса в зависимости от объема данных, особенно для смешанных сценариев, сочетающих фильтрацию по меткам и векторные запросы. Для этого случая мы оптимизировали схему запросов в памяти, когда количество отфильтрованных векторов-кандидатов ниже определенного порога, что дополнительно повысило производительность запросов. В аспекте сетевого подключения в распределенной системе мы выбрали протокол связи на основе gRPC для оптимизации обмена данными между [ProxyServer](#) и [VdbServer](#). В сравнении с традиционным HTTP протокол gRPC обеспечивает более высокую эффективность взаимодействия между модулями.

Оптимизация затрат

С расширением бизнеса объем данных и потребности в доступе будут продолжать расти. Поэтому с точки зрения модульного развертывания мы снизили затраты на развертывание и задержки в коммуникации между модулями, используя гибридное развертывание серверов [ProxyServer](#), [MasterServer](#) и [VdbServer](#). В определенных сценариях, чтобы еще больше снизить затраты, мы предложили схему развертывания на одном узле с уменьшением числа узлов [VdbServer](#), что в сочетании с технологией распределенного блочного хранилища может значительно снизить затраты.

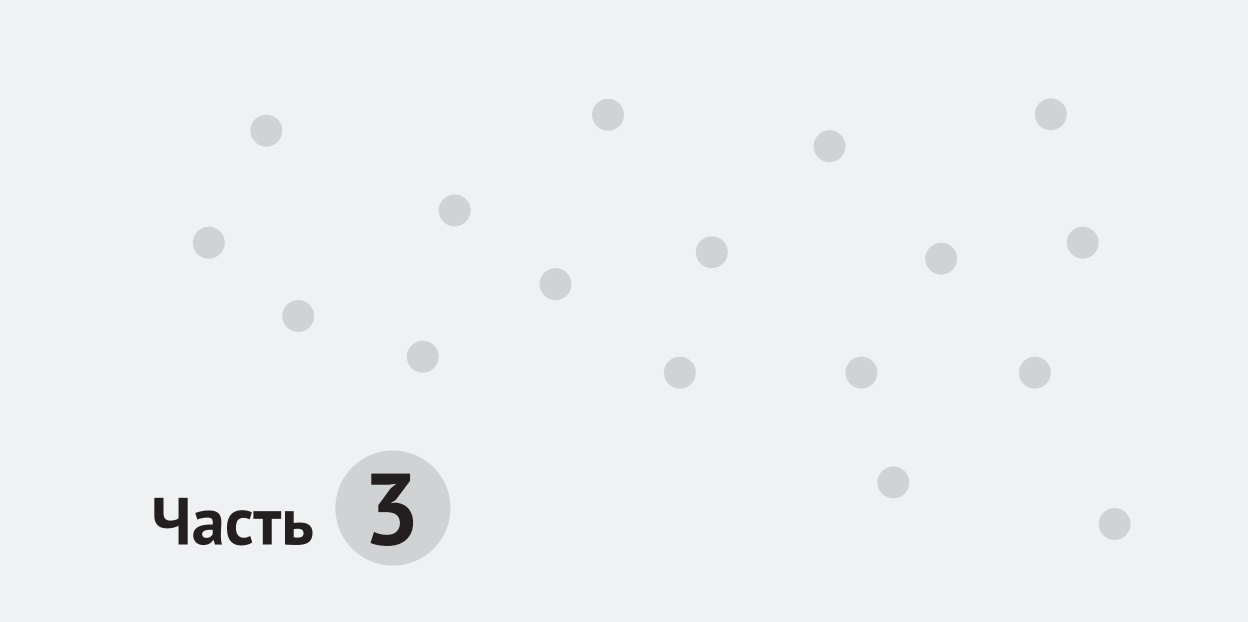
Оптимизация удобства использования

Для повышения эффективности использования разработчиками мы перешли от доступа на основе HTTP к доступу на основе SDK и, взяв в качестве примера широко используемый язык Python, разработали удобный в использовании SDK. Такой подход позволяет разработчикам более стандартизировано и удобно взаимодействовать с векторной базой данных. Также мы уделили внимание безопасности доступа к данным и их хранению, внедрив схему аутентификации на основе токенов JWT, чтобы гарантировать, что только авторизованные системой запросы могут получить доступ к базе данных. Кроме того, для ре-

шения возможных проблем с ошибочным удалением данных мы создали независимую систему резервного копирования данных, включающую резервное копирование метаданных, моментальных снимков данных и данных предзаписи. Таким образом, в случае потери данных разработчики могут восстановить исторические данные.

Конечно, оптимизация производительности и затрат сама по себе является динамическим процессом. Нет абсолютной наилучшей производительности или наилучшей стоимости, более важно в процессе развития бизнеса учитывать собственные условия, постоянно уделять внимание балансу между производительностью и стоимостью и использовать доступные технические средства. В то же время удобство использования векторной базы данных все еще имеет огромный потенциал для улучшения. Например, использование индексов FAISS и HNSW в векторной базе данных довольно сложное, и индустрия пытается внедрить технические решения для автоматической настройки их параметров. Кроме того, для удобства отладки разработчиками система векторной базы данных может также предоставлять консоль управления с графическим интерфейсом.

С помощью разнообразных методов мы можем постоянно укреплять возможности векторной базы данных. Только сохраняя стремление к совершенству, специалисты в области баз данных могут постепенно создать более совершенный продукт.



Часть 3

Практика и перспективы развития векторных баз данных

В предыдущих главах мы изучили основные сведения о векторных базах данных, затем создали автономную векторную базу данных, которая предоставляет основные функции записи и запроса в векторной базе данных. Далее мы добавили возможности главного и подчиненного узлов, а также сегментирование, создав векторную базу данных с распределенными возможностями. В завершение мы провели базовую оптимизацию системы по производительности, затратам и удобству использования, сделав нашу векторную базу данных более совершенной.

Однако в эпоху искусственного интеллекта одного создания векторной базы данных недостаточно для полного раскрытия ее потенциала. Для эффективного изучения и глубокого понимания векторных баз данных необходимо сочетать их с реальными приложениями и внимательно следить за тенденциями развития.

Глава 7

Практические примеры использования векторных баз данных

С бумаги знание поверхностным всегда бывает,
Чтоб глубину постичь, учись делами проверять.

Лу Ю. Зимней ночью читаю, наставляя сына И

В этой главе мы продемонстрируем работу векторных баз данных на двух практических примерах: создании системы поиска по изображениям и построении персональной базы знаний.

7.1. СОЗДАНИЕ СИСТЕМЫ ПОИСКА ПО ИЗОБРАЖЕНИЯМ

В разделе 1.1.4 мы обсуждали типичную ситуацию: вы хотите найти определенные фотографии в личной галерее, но из-за смутных воспоминаний и ограниченной информации не имеете для этого эффективного метода. С векторной базой данных все иначе – функция поиска по изображениям, реализованная с ее помощью, может эффективно решить эту проблему.

Мы можем упростить решение этой задачи и разбить его на следующие три шага.

1. Получение серии изображений и их векторизация с помощью подходящей модели векторизации, целью которой является представление этих изображений в виде векторных данных.
2. Сохранение векторных данных изображений в нашей векторной базе данных через пакет SDK.
3. Получение изображения для запроса, его векторизация с помощью модели векторизации из первого шага, получение вектора, а затем сравнение этого вектора с векторами, сгенерированными и сохраненными на втором шаге. В процессе поиска мы будем искать k ближайших векторов в базе данных, то есть k наиболее похожих изображений, после чего вернем их в порядке убывания сходства.

7.1.1. Векторизация изображений

В приложении для поиска по изображениям часто используется метод извлечения вектора признаков изображения с помощью моделей глубокого обучения. Широко используемые модели включают VGG и ResNet. Для этого примера мы выбрали ResNet-50 для извлечения векторов признаков. ResNet-50 – это универсальная модель, широко применяемая в различных задачах обработки изображений. В среде Python нам нужно подключить библиотеку PyTorch для разработки соответствующего кода. Ниже приведена функция `extract_features`, написанная на Python, которая реализует процесс векторизации изображений.

```
import torch
import torchvision.transforms as transforms
from torchvision.models import resnet50
from PIL import Image

def extract_features(image_path):
    # Загрузка предобученной модели ResNet-50.
    model = resnet50(pretrained=True)
    model.eval()
    # Предварительная обработка изображения.
    preprocess = transforms.Compose([
        transforms.Resize(256),
        transforms.CenterCrop(224),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
    ])
    # Чтение изображения.
    img = Image.open(image_path)
    img_t = preprocess(img)
    batch_t = torch.unsqueeze(img_t, 0)
    # Извлечение признаков.
    with torch.no_grad():
        out = model(batch_t)
    # Преобразование вектора признаков в одномерный массив и возврат.
    return out.flatten().numpy()
```

Ниже приведен детальный анализ логики кода.

1. Импорт необходимых библиотек и модулей:
 - ♦ `torch`: библиотека PyTorch для глубокого обучения и вычислений с тензорами;
 - ♦ `torchvision.transforms`: функции преобразования для предварительной обработки изображений;
 - ♦ `torchvision.models.resnet50`: предобученная модель ResNet-50 для извлечения признаков;
 - ♦ `PIL.Image`: чтение и обработка файлов изображений.
2. Определение функции `extract_features`:

- ♦ функция `extract_features` принимает один параметр `image_path`, значение которого представляет собой путь к файлу изображения, подлежащего обработке. Иными словами, задача этой функции заключается в извлечении вектора признаков из изображения, находящегося по указанному пути.
3. Загрузка предобученной модели ResNet-50:
 - ♦ `model = resnet50(pretrained=True)`: создается экземпляр модели ResNet-50, и параметр `pretrained` устанавливается в значение `True` для загрузки предобученных весов;
 - ♦ `model.eval()`: перевод модели в режим оценки, в этом режиме отключаются некоторые специфические для обучения операции, что позволяет использовать модель для вывода и извлечения признаков.
 4. Определение процесса предварительной обработки изображений:

```
preprocess = transforms.Compose([
    transforms.Resize(256),
    transforms.CenterCrop(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])
```

В этом коде используется модуль `transforms` из PyTorch, который с помощью функции `Compose` объединяет ряд шагов преобразования изображений, приводя входное изображение в формат, подходящий для обработки моделью глубокого обучения. Затем с помощью шага нормализации обеспечивается соответствие распределения данных распределению, использованному при обучении модели:

- `Resize(256)`: изменение размера входного изображения до 256 пикселей, это достигается с помощью интерполяции или других методов передискретизации, чтобы все изображения имели одинаковый размер;
- `CenterCrop(224)`: вырезание области размером 224 пикселя из центра изображения после изменения его размера, это размер входного изображения, используемый многими предобученными нейронными сетями, такими как ResNet;
- `ToTensor()`: преобразование изображения в формате PIL или массива NumPy в тензор PyTorch, что является стандартным форматом входных данных для моделей PyTorch; при этом пиксельные значения автоматически линейно масштабируются из диапазона [0, 255] в диапазон [0, 1];
- `Normalize(...)`: нормализация каждого канала тензорного изображения: сначала из значений вычитается заданное среднее значение (`mean`), затем делится на заданное стандартное отклонение (`std`). Среднее значение и стандартное отклонение предварительно вычисляются на большом наборе данных ImageNet для повышения стабильности процесса обучения и скорости сходимости модели.

5. Чтение и предварительная обработка изображения:

- ♦ `img = Image.open(image_path)`: открытие файла изображения с помощью библиотеки PIL;
- ♦ `img_t = preprocess(img)`: применение к изображению процесса предварительной обработки;
- ♦ `batch_t = torch.unsqueeze(img_t, 0)`: добавление дополнительного измерения пакета к тензору обработанного изображения (это делается потому, что модель обычно ожидает на входе пакет данных, даже если обрабатывается только одно изображение), чтобы сделать его подходящим для ввода в модель.

6. Извлечение признаков:

- ♦ использование контекстного менеджера `torch.no_grad()` для отключения вычисления градиентов, поскольку в процессе извлечения признаков вычисление градиентов не требуется (это полезно для вывода и извлечения признаков, так как позволяет уменьшить потребление памяти и вычислительные затраты);
- ♦ `out = model(batch_t)`: ввод обработанного изображения в модель для получения выходных признаков модели.

7. Преобразование и возврат вектора признаков:

- ♦ `out.flatten().numpy()`: преобразование выходного тензора признаков модели в одномерный массив и его конвертация в массив NumPy;
- ♦ `return out.flatten().numpy()`: возврат массива вектора признаков для дальнейшего использования.

7.1.2. Загрузка и запрос изображений

После реализации функции векторизации изображений далее займемся разработкой удобного для пользователя универсального приложения. Для удобства работы необходимо принимать загружаемые файлы через браузер и выполнять вызовы соответствующих интерфейсов в фоновом режиме. Такие комплексные приложения обычно реализуются на основе многофункциональных фреймворков. Здесь для реализации соответствующих функций мы использовали фреймворк Flask на языке Python. Flask – это довольно легковесный фреймворк приложений, который пользуется популярностью среди разработчиков на Python. Структура каталогов системы поиска по изображениям в нашем примере представлена в табл. 7.1.

Таблица 7.1. Структура каталогов системы поиска по изображениям

Структура каталогов	Содержимое	Описание
static	images script.js	В каталоге images хранятся статические файлы, включая изображения, загруженные пользователями, и изображения, по которым выполняется запрос. В файле script.js хранится код JavaScript для обработки событий на стороне клиента

Структура каталогов	Содержимое	Описание
templates	index.html	Код клиентского интерфейса предоставляет точки входа для загрузки и запроса изображений
vector_db_sdk	client.py __init__.py	Пакет SDK для векторных баз данных, разработанный в главе 6, предоставляет SDK для записи и запроса векторизованных данных
/	app.py image_eb.py	Файл app.py реализует точку входа Flask и обрабатывает запросы клиентов. Файл image_eb.py содержит код из раздела 7.1.1, который реализует функцию векторизации изображений

Этот пример демонстрирует стандартную структуру приложения Flask. Код клиентской части относительно прост, поэтому мы не будем подробно его рассматривать. Сосредоточимся на коде в файле app.py, чтобы понять, как серверная часть принимает изображения, выполняет их векторизацию и поддерживает конечное хранение и запросы. Ниже приведен код запуска и прослушивания в app.py.

```
# Импорт основных компонентов фреймворка для создания веб-приложений.
from flask import Flask, request, jsonify
# Импорт render_template для отрисовки HTML-шаблонов.
from flask import render_template
# Импорт пользовательской функции extract_features для извлечения признаков
из изображений.
from image_eb import extract_features
# Импорт VectorDatabaseSDK для взаимодействия с векторной базой данных.
from vector_db_sdk import VectorDatabaseSDK

import logging # Импорт модуля logging для ведения журнала.
import os # Импорт модуля os для управления функциями операционной системы.

app = Flask(__name__) # Создание экземпляра приложения Flask.
# Инициализация объекта VectorDatabaseSDK с указанием информации о подключении
# к базе данных.
db_sdk = VectorDatabaseSDK(host="172.19.0.9", port=80)
# Аутентификация в базе данных, передача имени пользователя и пароля.
db_sdk.authenticate(username="user2", password="newpassword456")
# Определение маршрута и функции представления для главной страницы.
# Когда пользователь обращается к корневому каталогу, активируется этот маршрут.
@app.route("/")
def index():
    return render_template("index.html")

# При запуске сценария как основной программы выполняется следующий код.
if __name__ == "__main__":
    # Включение режима отладки для удобства разработки и отладки.
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Из кода видно, что при запуске службы мы инициализируем SDK для соединения с векторной базой данных, чтобы иметь возможность работать с векторными данными. Также мы зарегистрировали маршрут для главной страницы, указывающий на страницу `index.html`. Далее нам нужно настроить интерфейс загрузки и запроса изображений. Сначала рассмотрим детали реализации интерфейса загрузки.

```
@app.route("/upload", methods=["POST"])
def upload_image():
    image_file = request.files["image"]
    image_id_str = request.form.get("image_id")
    # Проверка наличия image_id.
    if not image_id_str:
        return jsonify({"message": "Image ID is required"}), 400
    # Попытка преобразовать image_id в целое число.
    try:
        image_id = int(image_id_str)
    except ValueError:
        return jsonify(
            {"message": "Invalid image ID. It must be an integer"}
        ), 400
    filename = image_file.filename # Использование имени загруженного файла.
    image_path = os.path.join("static/images", image_id_str)
    image_file.save(image_path)
    image_features = extract_features(image_path)
    # Обновление записи в базе данных.
    upsert_result = db_sdk.upsert(
        id=image_id, vectors=image_features.tolist(), index_type="FLAT"
    )
    return jsonify({"message": "Image uploaded successfully", "id": image_id})
```

Функция `upload_image` сначала получает из запроса содержимое файла изображения и его ID. Затем изображение сохраняется в каталоге `static/images`, и ему присваивается имя, соответствующее ID изображения. После этого с помощью функции `extract_features` изображение подвергается векторизации на основе пути хранения, в результате чего получается массив векторов `image_features`. Имея эти векторные данные, мы используем функцию `upsert` из `db_sdk`, чтобы сохранить векторные данные и их ID в векторной базе данных, и возвращаем результат клиенту.

После реализации хранения данных рассмотрим реализацию запроса данных.

```
@app.route("/search", methods=["POST"])
def search_image():
    image_file = request.files["image"]
    image_path = os.path.join("static/images", "temp_image.jpg")
    image_file.save(image_path)
    image_features = extract_features(image_path)
    # Использование извлеченного вектора признаков для запроса.
    search_result = db_sdk.search(
        vectors=image_features.tolist(), k=5, index_type="FLAT"
    )
```

```
# Преобразование результатов запроса в пути к изображениям или
# соответствующие данные.
image_urls = [
    f"/static/images/{int(image_id)}" for image_id in search_result["vectors"]
]
return jsonify(
    {
        "data": search_result,
        "distances": search_result["distances"],
        "image_urls": image_urls
    }
)
```

В реализации функции запроса мы сначала получаем содержимое файла из запроса и сохраняем его в каталоге `static/images`. Учитывая, что файл запроса используется временно, мы называем его `temp_image.jpg`. Далее, с помощью функции векторизации `extract_features` файл подвергается векторизации, в результате чего получается вектор запроса, представляющий это изображение. Затем мы используем функцию `db_sdk.search` для поиска в векторной базе данных 5 наиболее похожих изображений и возвращаем клиенту их ID. Клиент может использовать URL этих изображений для отображения списка наиболее похожих изображений.

7.1.3. Обзор системы

На рис. 7.1 изображена упрощенная версия реализации системы поиска по изображениям на стороне клиента.

Загрузка изображения

Image ID Файл не выбран

Поиск изображения

Файл не выбран

Просмотр

Результаты поиска

Рис. 7.1. Базовый интерфейс системы поиска по изображениям

Страница содержит следующие части:

- загрузка изображений: поддерживает выбор локальных файлов и ввод ID изображения, после чего загружает их на сервер;
- запрос изображений: позволяет выбрать локальный файл для поиска на сервере 5 наиболее похожих изображений;
- просмотр: после выбора файла локально отображается выбранное изображение;
- результаты запроса: отображается 5 наиболее похожих изображений, возвращенных сервером.

На рис. 7.2 демонстрируется пример, как в наборе изображений хомяков и попугаев с помощью этой системы можно найти изображение хомяка. Результат выводит наиболее похожие изображения в системе.

Загрузка изображения

Image ID Файл не выбран

Поиск изображения

hamster.jpeg

Просмотр

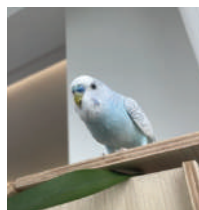
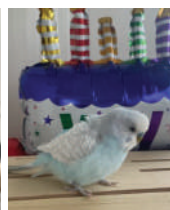
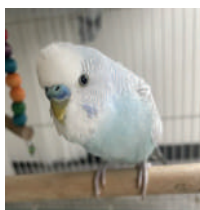


Рис. 7.2. Демонстрация результатов запроса системы поиска по изображениям

Таким образом, мы завершили разработку прототипа системы поиска по изображениям. Эта система позволяет пользователям загружать изображения для создания библиотеки изображений и индекса векторов изображений. На основе этой библиотеки пользователь может загрузить любое изображение для поиска похожих изображений в библиотеке. Вся система – от клиентского интерфейса до векторизации изображений, записи в базу данных и до самой реализации векторной базы данных – полностью реализована. Это 100%-ная независимая замкнутая система. Если вас интересует это направление, вы можете на основе этой версии дополнительно улучшить клиентский интерфейс и добавить больше функций.

7.2. СОЗДАНИЕ ПЕРСОНАЛЬНОЙ БАЗЫ ЗНАНИЙ

В области обработки векторных данных, помимо обработки изображений, мы часто сталкиваемся с приложениями, выполняющими запросы и анализ текстов на основе уже имеющихся знаний. В таких сценариях пользователи могут векторизовать свои личные знания и сохранить их в векторной базе данных, после чего использовать эти знания для выполнения запросов и в сочетании

с большой языковой моделью отвечать на различные вопросы. Мы называем такую систему персональной базой знаний.

Для создания такой персональной базы знаний обычно необходимо пройти несколько ключевых этапов.

1. Предобработка личных знаний: включает сегментацию большого объема знаний для их отдельной векторизации с помощью модели векторизации.
2. Векторизация знаний: включает векторизацию сегментированных данных и их сохранение в векторной базе данных.
3. Управление базой знаний: предоставляет комплексное приложение, поддерживающее загрузку пользователем файлов базы знаний.
4. Система вопросов и ответов: в комплексном приложении предоставляется интерфейс для вопросов, после чего система принимает вопросы пользователей, сопоставляет их с наиболее похожими сегментами данных и передает большой языковой модели, которая отвечает и возвращает ответ пользователю.

7.2.1. Предобработка знаний

В этом примере наша персональная база знаний поддерживает формат Markdown. Markdown – это формат хранения текста с повышенным уровнем читаемости. На основе этого я продемонстрирую процесс предобработки персональной базы знаний. Другие форматы, такие как PDF, Word и т. д., можно обработать аналогичным образом, и вы можете расширить этот подход.

Ключевой аспект предобработки личных знаний заключается в сегментации данных. Каждый сегмент данных затем передается модели векторизации для обработки. Качество данных во многом зависит от эффективности сегментации. Обычно объединение логически связанных абзацев является хорошей начальной практикой. Ниже приведен пример кода для сегментации данных.

```
import markdown
from bs4 import BeautifulSoup # Для анализа и обработки HTML-документов.
# Для автоматической загрузки предобученных моделей и токенизаторов.
from transformers import AutoTokenizer, AutoModel
import torch # Для вычислений в области глубокого обучения.

# Определение функции для преобразования текста в формате Markdown в HTML.
def markdown_to_html(markdown_text):
    # Использование функции markdown для преобразования.
    return markdown.markdown(markdown_text)

# Определение функции для разделения HTML-текста на несколько абзацев.
def split_html_into_segments(html_text):
    soup = BeautifulSoup(html_text, "html.parser") # Анализ HTML-документа.
    segments = [] # Инициализация списка для хранения разделенных абзацев.
    # Поиск в HTML-документе заголовков, абзацев, неупорядоченных
    и упорядоченных списков.
    for tag in soup.find_all(["h1", "h2", "h3", "h4", "h5", "h6", "p", "ul", "ol"]):
```

```
# Извлечение текстового содержимого каждого тега и добавление
# в список абзацев.
segments.append(tag.get_text())
return segments # Возврат списка, содержащего все абзацы.
```

Этот код содержит две функции: `markdown_to_html`, которая преобразует текст в формате Markdown в формат HTML, и `split_html_into_segments`, которая разделяет HTML-документ на несколько абзацев, таких как заголовки, абзацы, списки и т. д.

Основная причина выбора преобразования Markdown в HTML заключается в том, что существуют относительно зрелые инструменты для обработки HTML-формата, что делает последующую обработку данных более удобной. Кроме того, модели обычно лучше работают с данными в фиксированном формате. Конечно, выбор формата данных является необязательной стратегией, и конкретный выбор должен определяться в зависимости от конкретного бизнес-сценария. Основная цель – сохранить определенный формат, чтобы облегчить последующую обработку и понимание модели.

После преобразования формата мы используем библиотеку Beautiful Soup для сегментации абзацев. В этом процессе все отдельные абзацы и табличные данные объединяются в минимальные единицы абзацев. Текущая стратегия сегментации относительно проста и имеет много возможностей для оптимизации. Например, можно рассмотреть добавление последнего предложения предыдущего абзаца и первого предложения следующего абзаца в текущий сегмент, интеграцию заголовков внутрь абзацев, контроль количества символов в каждом абзаце и другие стратегии оптимизации, которые вы можете постепенно адаптировать и внедрять в зависимости от ваших потребностей.

7.2.2. Векторизация знаний

После выполнения предобработки знаний мы получили сегментированные данные, удобные для последующей векторизации. В настоящее время существует множество зрелых моделей, способных преобразовывать строки в векторы. В этом примере мы выбрали модель BGE (BAAI General Embedding, универсальная векторизация Пекинского института искусственного интеллекта), которая демонстрирует относительно хорошие результаты. Ниже приведен пример кода для соответствующей векторизации.

```
from transformers import AutoTokenizer, AutoModel
import torch

# Определение функции для преобразования текстовых сегментов в векторное
# представление.
def vectorize_segments(segments):
    # Использование предобученного токенизатора и модели (китайская модель
    BAAI/bge-large-zh-v1.5).
    tokenizer = AutoTokenizer.from_pretrained("BAAI/bge-large-zh-v1.5")
    model = AutoModel.from_pretrained("BAAI/bge-large-zh-v1.5")
```



```
# Установка модели в режим оценки, отключение используемых при обучении
# параметров, таких как dropout.
model.eval()
# Кодирование текстовых сегментов с использованием токенизатора, добавление
# необходимой подгонки и обрезки, возвращение в формате тензоров PyTorch.
encoded_input = tokenizer(
    segments, padding=True, truncation=True, return_tensors="pt"
)
# Использование контекстного менеджера для освобождения вычислительного
# графа после выполнения блока кода.
with torch.no_grad():
    # Передача закодированного ввода в модель для получения вывода.
    model_output = model(**encoded_input)
# Извлечение результатов векторизации предложений из вывода модели.
sentence_embeddings = model_output[0][:, 0]
# Нормализация результатов векторизации предложений по L2 для последующего
# сравнения сходства или кластерного анализа.
sentence_embeddings = torch.nn.functional.normalize(
    sentence_embeddings, p=2, dim=1
)
return sentence_embeddings # Возврат обработанных результатов
                           # векторизации предложений.
```

Функция `vectorize_segments` использует классы `AutoTokenizer` и `AutoModel` из библиотеки `transformers` для импорта китайской векторной модели и ее токенизатора под названием BAAI/bge-large-zh-v1.5. Эта модель находится в предобученном режиме и может быть использована в коде напрямую. Сначала мы используем токенизатор для сегментации, затем выполняем векторизацию с помощью модели. Результаты векторизации сохраняются в переменной `sentence_embeddings`. В заключение с помощью вызова метода `torch.nn.functional.normalize` производится нормализация, что облегчает последующие вычисления сходства векторов. Нормализованные векторы затем можно сохранить в векторной базе данных.

7.2.3. Управление базой знаний

Для удобства работы с базой знаний необходимо предоставить клиентский интерфейс. Через этот интерфейс пользователи могут легко вводить знания. Как и в разделе 7.1.2, мы выбрали Flask в качестве основы для создания интерфейса ввода и последующего интерфейса запросов для базы знаний. Структура каталогов файлов с кодом представлена в табл. 7.2.

Таблица 7.2. Структура каталогов системы персональной базы знаний

Структура каталогов	Содержимое	Описание
static	script.js	Код JavaScript для обработки событий на стороне клиента
templates	index.html	Код клиентского интерфейса, предоставляет точки входа для загрузки знаний и ответов на вопросы, связанные с этими знаниями

Структура каталогов	Содержимое	Описание
vector_db_sdk	client.py __init__.py	Пакет SDK для векторных баз данных, разработанный в главе 6, предоставляет SDK для записи и запроса векторизованных данных
/	app.py markdown_processor.py	Файл app.py реализует точку входа Flask и обрабатывает запросы клиентов. Файл markdown_processor.py содержит код из разделов 7.2.1 и 7.2.2 для предварительной обработки знаний и векторизации

Далее мы подробно рассмотрим детали кода в файле app.py, чтобы понять, как обрабатывается ввод знаний, векторизация, а также поддержка хранения и системы вопросов и ответов. Поскольку код запуска и прослушивания в app.py схож с кодом в разделе 7.1.2, мы не будем его повторять, а начнем с подробного описания части ввода знаний.

```
# Определение маршрута для обработки HTTP-запросов upload, используемого
# для загрузки файлов.
@app.route("/upload", methods=["POST"])
def upload():
    if "file" not in request.files:
        return jsonify({"error": "No file part in the request"}), 400
    file = request.files["file"]
    if file.filename == "":
        return jsonify({"error": "No file selected for uploading"}), 400
    # Преобразование текста Markdown в векторное представление
    # (предполагается кодировка UTF-8).
    markdown_text = file.read().decode("utf-8")
    html_text = markdown_to_html(markdown_text)
    segments = split_html_into_segments(html_text)
    vectors = vectorize_segments(segments)
    # Использование VectorDatabaseSDK для загрузки текстов сегментов
    # и соответствующих векторов в базу данных.
    for i, (segment, vector) in enumerate(zip(segments, vectors)):
        vector_id = i + 1 # Использование индекса сегмента плюс 1 в качестве ID.
        db_sdk.upsert(id=vector_id, vectors=vector.tolist(), text=segment)
    return jsonify({"message": "File has been processed"})
```

В функции `upload` на серверной стороне мы принимаем загруженный клиентом файл Markdown. Сначала мы преобразуем его в формат HTML, а затем выполняем сегментацию. Исходный текст после сегментации сохраняется в переменной `vectors`. На основе этой переменной мы используем `db_sdk.upsert` для записи векторных данных в векторную базу данных. Особое внимание следует уделить тому, что, помимо векторных данных, мы также сохраняем сегментированные текстовые элементы. Эти текстовые элементы будут использоваться в дальнейшем в системе вопросов и ответов с применением большой языковой модели.

7.2.4. Система вопросов и ответов

После реализации функции `upload`, поддерживающей загрузку файлов Markdown, личные знания пользователя теперь хранятся в виде небольших сегментов. На следующем шаге мы реализуем систему вопросов и ответов. Ниже приведена реализация соответствующего кода.

```
@app.route("/search", methods=["POST"])
def search():
    data = request.get_json()
    search_text = data.get("search")
    # Добавление префикса к строке запроса.
    instruction = (
        "Создайте графическое представление этого предложения "
        "для использования при поиске связанных статей: "
    )
    search_text_with_instruction = instruction + search_text
    # Векторизация модифицированного запроса.
    search_vector = vectorize_segments([search_text_with_instruction])[0].tolist()
    # Использование VectorDatabaseSDK для поиска ближайших векторов.
    search_results = db_sdk.search(vectors=search_vector, k=5)
    # Формирование списка сообщений для взаимодействия с API большой
    # языковой модели.
    messages = [
        {
            "role": "system",
            "content": (
                "You are a helpful assistant. Answer questions based solely "
                "on the provided content without making assumptions or adding "
                "extra information."
            )
        }
    ]
    result_ids = search_results.get("vectors", []) # Разбор результатов запроса.
    # Поиск соответствующего текстового содержания по ID результатов запроса.
    for result_id in result_ids:
        # Убедиться, что ID имеет целочисленный тип.
        if not isinstance(result_id, int):
            result_id = int(result_id)
        query_result = db_sdk.query(id=result_id)
        text = query_result.get("text", "")
        messages.append({"role": "assistant", "content": text})
    messages.append({"role": "user", "content": search_text})
    # Отправка запроса в OpenAI и получение ответа.
    response = client.chat.completions.create(
        model="gpt-3.5-turbo", messages=messages
    )
    answer = response.choices[0].message.content
    return jsonify({"answer": answer})
```

Данная реализация разделена на три этапа.

1. Мы извлекаем исходный текст вопроса пользователя из параметра `search`. Важно отметить, что в соответствии с лучшими практиками модели BGE для оптимизации векторного запроса мы добавляем дополнительный текст инструкции перед текстом вопроса пользователя. Затем мы проводим векторизацию объединенного вопроса с помощью `vectorize_segments` и сохраняем результат в переменной `search_vector`.
2. Мы выполняем запрос к векторной базе данных с использованием `db_sdk.search` для векторизированных данных `search_vector`, чтобы получить 5 наиболее схожих сегментных векторных данных. Возвращенные данные содержат ID этих сегментов, которые служат основой для взаимодействия с большой языковой моделью.
3. На основе полученных ID наиболее схожих данных мы формируем параметры запроса к большой языковой модели. В этом примере мы используем интерфейс OpenAI, но вы можете выбрать модель, соответствующую вашим требованиям. Параметры, которые мы передаем большой модели, включают системную информацию и определяют задачи, которые модель должна выполнить, как показано ниже.

```
messages = [
    {
        "role": "system",
        "content": (
            "You are a helpful assistant. Answer questions based "
            "solely on the provided content without making "
            "assumptions or adding extra information."
        )
    }
]
```

Отправляем ввод пользователя модели GPT-3.5, получаем сгенерированный ответ и возвращаем его клиенту в формате JSON.

Далее мы используем SDK для получения из векторной базы данных исходного текста и добавляем его в массив `messages`, например `messages.append({"role": "assistant", "content": text})`. Эта часть информации предоставляет большой модели необходимый контекст для ответа на вопросы пользователя. Без этого контекста модели будет сложно ответить на вопросы, связанные с личной базой знаний пользователя.

Наконец, мы добавляем фактический вопрос пользователя в `messages` и отправляем его большой модели для ответа. Реализация кода представлена ниже.

```
# Добавление текста запроса пользователя в список сообщений,
# указывая роль как "user".
messages.append({"role": "user", "content": search_text})
# Использование модели GPT-3.5 (gpt-3.5-turbo) для вызова API и генерации
# продолжения разговора.
response = client.chat.completions.create(model="gpt-3.5-turbo",
messages=messages)
```

```
# Извлечение содержания первого ответа из API.
answer = response.choices[0].message.content
# Упаковка ответа в формат JSON и его возврат,
# клиент может разобрать этот JSON, чтобы получить текст ответа.
return jsonify({'answer': answer})
```

7.2.5. Обзор системы

На этом этапе мы завершили создание персональной базы знаний. Поскольку связанный код HTML и JavaScript относительно прост, мы не будем его рассматривать здесь. На рис. 7.3 изображен интерфейс начального состояния системы.

Загрузка файла Markdown

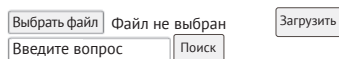


Рис. 7.3. Интерфейс начального состояния персональной базы знаний

Для лучшей демонстрации возможностей персональной базы знаний мы сгенерировали вымышленную историю с помощью большой модели. Этот прием направлен на доказательство того, что знания, используемые моделью для ответа на вопросы, не были заранее использованы в предыдущем обучении. Ниже представлена часть содержимого этого файла в формате Markdown.

```
## Глава первая. Таинственный лес
```

Рядом с тихой деревушкой находится древний и таинственный лес. По легенде, в этом лесу обитают различные фантастические существа и скрыто множество неизвестных тайн. Любопытный мальчик Сяо Мин вместе со своими друзьями решают исследовать этот загадочный лес.

```
### Подготовка
```

Сяо Мин и его лучший друг Сяо Хуа, а также несколько товарищей из деревни, подготовили необходимое снаряжение для приключений: рюкзак, флягу, еду, аптечку и карту окрестностей деревни. Они договорились заботиться друг о друге и не действовать в одиночку, независимо от обстоятельств.

Сяо Мин сказал: «Мы должны быть осторожны, здесь полно опасностей.» Его друг Сяо Хуа с энтузиазмом кивнул, на его лице было предвкушение приключений.

```
### Вход в лес
```

Они пересекли границу деревни и вошли в густой лес. Пейзаж в лесу оказался еще более удивительным, чем они представляли: огромные деревья, казалось, устремлялись в небеса, повсюду росли необычные цветы и травы, а в воздухе порхали разноцветные бабочки, которых они никогда не видели.

Они обнаружили развилку:

- Левая дорога выглядела темной и загадочной, окутанной легким туманом.
- Правая дорога залита солнечным светом и усыпана цветами, выглядит более теплой и приятной.

Сяо Хуа сказал: «Давайте пойдем по правой дороге! Она выглядит более красивой.»

Эта история является вымышленным содержанием, специально созданным для данного теста. Без персональной базы знаний в качестве фона большая языковая модель затрудняется ответить на вопросы, касающиеся содержания истории. На рис. 7.4 показан эффект ответа нашей системы на актуальные вопросы.

Загрузка файла Markdown

Файл обработан, векторы загружены в базу данных

```

{
  "answer": "Это приключенческая история, главными героями которой являются Сяо Мин и Сяо Хуа. В истории они исследуют таинственный лес и встречают различных фантастических существ и персонажей комиксов. Их приключения полны фантазий и трудностей, сохраняя любопытство и смелость, они исследуют чудеса жизни."
}

```

Рис. 7.4. Ответы персональной базы знаний на актуальные вопросы

В этом примере мы используем загруженную вымышленную историю в качестве фоновой информации. Примечательно, что большая языковая модель не отвечает на вопросы произвольно и безосновательно. Напротив, она отвечает, основываясь на предоставленном нами фактическом содержании текста. Если вас интересует это направление, вы можете использовать исходный код, предоставленный в книге, для дальнейшей проверки и тестирования вопросов.

7.3. РЕЗЮМЕ

В этой главе, основываясь на возможностях записи и запроса векторной базы данных, реализованных ранее в книге, мы создали два примера приложений, чтобы помочь читателям глубже понять применение векторных баз данных в реальных бизнес-сценариях.

Первый пример демонстрирует реализацию полноценной системы поиска по изображениям. Ключевые этапы реализации системы включают преобразование изображений в векторы, использование векторной базы данных для хранения и запроса, а также разработку соответствующего веб-приложения. С быстрым развитием технологий векторизации появилось множество моделей векторизации изображений. В этом примере мы выбрали модель ResNet. Простота использования языка Python делает интеграцию этого процесса векторизации в приложение простой задачей. После выполнения векторизации мы легко выполнили задачи записи и запроса данных, используя ранее

разработанный SDK векторной базы данных. Затем с помощью фреймворка Flask мы завершили создание приложения. Благодаря модульному подходу процесс разработки приложения оказался совсем несложным. Это вновь демонстрирует, как в области компьютерной инженерии повторное использование и модульность помогают нам эффективно достигать целей.

Второй пример сосредоточен на управлении неструктурированными текстовыми данными. В жизни и работе мы часто сталкиваемся с вызовами управления личными или корпоративными приватными данными, которые зачастую являются сложными. Мы применили методы сегментации, векторизации и обработки с помощью больших языковых моделей, чтобы быстро создать прототип персональной базы знаний. Этот прототип может эффективно управлять и организовывать личные данные знаний, создавая социальную ценность. Текущий прототип находится на базовом этапе, и вы можете развивать его дальше в соответствии с вашими потребностями, используя свое творчество для реализации дополнительных функций.

В конце этой главы я хотел бы поделиться с вами своим наблюдением. Я поддерживаю практический подход к обучению, который является продолжением моего размышления над известным в технологическом сообществе законом: «Мы склонны переоценивать эффект технологий в краткосрочной перспективе и недооценивать его – в долгосрочной». Эта фраза является эпитафией к главе 6. Я верю, что, только глубоко наблюдая и лично переживая изменения, которые приносит развитие технологий, мы сможем более объективно оценивать технологические тренды, сохранять хладнокровные суждения в краткосрочной перспективе, понимать извилистую эволюцию в среднесрочной перспективе и твердо следовать долгосрочным целям, чтобы уверенно завершить наше технологическое путешествие.

Глава 8

Перспективы развития

Ваше время ограничено, поэтому не тратьте его, проживая чужую жизнь.

Стив Джобс

В 2023 году векторные базы данных стали объектом пристального внимания, поскольку они способны решать некоторые проблемы, с которыми большие языковые модели не могут справиться. В эпоху больших моделей существует два основных способа обработки данных: первый – это обновление знаний в больших моделях через предобучение или тонкую настройку, второй – это подключение к большим моделям векторных баз данных. Предобучение или тонкая настройка сталкиваются с проблемами низкой эффективности обновления и высоких затрат – это называют «ограничением по времени» при обновлении знаний больших моделей. Это затрудняет понимание большими моделями новейших данных (на уровне часов или дней). Кроме того, большие модели сами по себе не обладают эффективным механизмом изоляции данных, интегрируя все знания в плоской форме внутри модели, что создает сложности в области конфиденциальности данных, – это называют «ограничением по пространству». Ограничение по времени и ограничение по пространству делают интеграцию частных данных предприятий во внешние большие модели крайне сложным, даже в случае локально развернутых моделей, поскольку обучение на данных из различных областей предприятия слишком дорого.

Эти два ограничения означают, что в эпоху больших языковых моделей их взаимодействие с данными требует более специализированных баз данных, а именно векторных баз данных, предназначенных для хранения и обработки векторных данных. Развитие технологий больших моделей происходит стремительно. Какие возможные направления развития могут быть у векторных баз данных, чтобы они могли эффективно обслуживать эпоху больших моделей?

В этой главе мы рассмотрим возможные направления развития векторных баз данных с двух различных точек зрения. Во-первых, мы проанализируем эволюцию способов распределения вычислительных мощностей и данных за последние десятилетия, чтобы предсказать, могут ли векторные базы данных в эпоху больших моделей взять на себя более широкую роль платформизации. Этот подход сосредоточен на прогнозировании будущего с исторической точки зрения. Во-вторых, мы сосредоточимся на текущих сценариях генерации, дополненной поиском (RAG), которые уже обслуживаются векторными базами данных. Мы проанализируем с точки зрения приложений, в каких аспектах

векторные базы данных можно усилить и оптимизировать для обслуживания этих сценариев. Этот подход больше ориентирован на выявление и расширение возможностей векторных баз данных на базе существующих приложений.

Надеемся, что через обсуждение этих двух аспектов мы сможем более полно понять направление развития векторных баз данных, а также их важность и потенциальные новые возможности в эпоху больших языковых моделей.

8.1. С точки зрения эволюции отрасли

В компьютерной индустрии за последние десятилетия произошло несколько волн развития: от эпохи мейнфреймов до эпохи ПК и интернета, затем до эпохи мобильного интернета. А сейчас мы вступаем в эпоху больших языковых моделей. Со сменой компьютерной эпохи способы, которыми человечество управляет вычислительными мощностями и данными, также постепенно развиваются: от использования вычислительных мощностей и данных в лабораторных условиях до использования их в центрах обработки данных и, наконец, в облачных вычислениях. Для эффективного управления этими вычислительными мощностями и данными необходима кропотливая работа программистов. На основе огромного количества написанного кода были созданы различные продукты, такие как платформы виртуализации вычислительных мощностей, платформы больших данных для управления офлайн-данными и распределенные базы данных для управления онлайн-данными. Платформизация имеет два значительных преимущества: во-первых, она помогает реализовать временное совместное использование ресурсов, снижая тем самым стоимость их использования для каждого отдельного индивида. Во-вторых, она снижает порог использования, позволяя не задумываться о деталях реализации нижнего уровня и использовать ресурсы сразу. Платформизация возможностей продукта является естественной моделью развития в компьютерной индустрии. Мы можем пойти дальше и обсудить следующий вопрос: как будет развиваться способ управления вычислительными мощностями и данными в эпоху больших языковых моделей с широким применением ИИ? Появятся ли аналогичные возможности для платформизации?

8.1.1. Новая парадигма управления данными человеком

На ранних этапах развития компьютерной индустрии программисты вводили программные инструкции и данные в компьютер, пробивая отверстия на перфокартах в определенном формате, что было крайне неэффективным методом программирования. Затем с изобретением ассемблера и высокоуровневых языков программирования программирование стало более продвинутым, гибким и эффективным. В процессе эволюции компьютерной индустрии производительность человека значительно возросла. Однако с точки зрения управления вычислениями, независимо от эпохи и вычислительных мощностей, программисты всегда играли незаменимую роль в управлении вычислительными мощностями. Они выполняют эту роль, используя языки программирования, и мы можем назвать эту парадигму «управление вычислительными мощностями человеком через язык программирования», как показано на рис. 8.1.

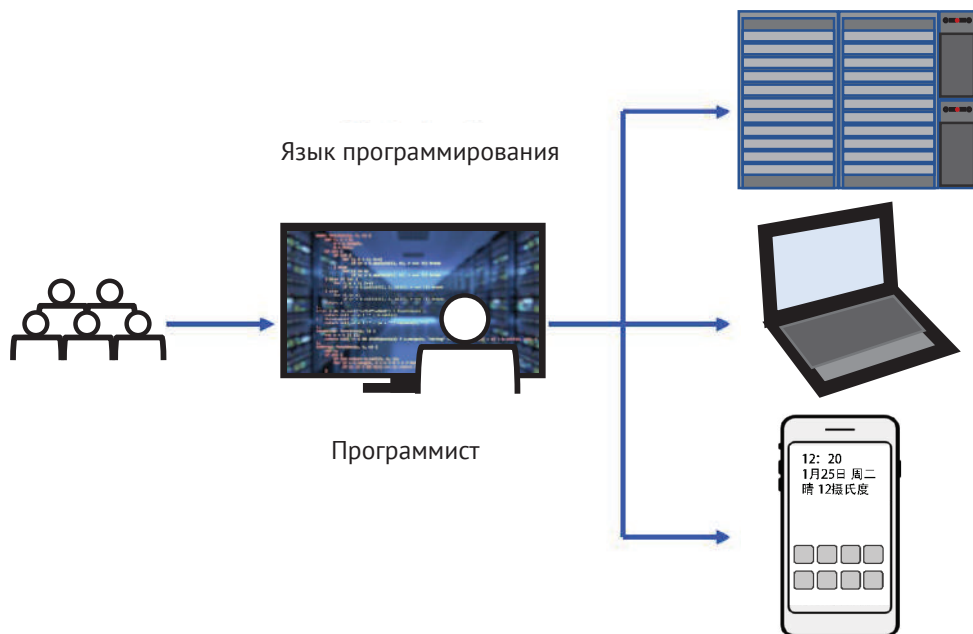


Рис. 8.1. Управление вычислительными мощностями человеком через язык программирования

Например, большинство используемых нами веб-приложений и мобильных приложений создаются на основе кода, вручную написанного программистами. В этом процессе программист, по сути, выполняет роль «посредника», переводя человеческие потребности в компьютерные программы, которые может распознать машина. Для выполнения этого перевода программист должен в совершенстве владеть языком программирования, чтобы обеспечить общение человека и машины. Этот метод использовался на протяжении десятилетий, вплоть до недавнего появления больших языковых моделей. В эту эпоху способ управления вычислительными мощностями человеком может претерпеть значительные изменения.

Благодаря предобучению большие языковые модели впитали в себя огромное количество знаний человеческого общества, что позволило им обрести способность понимать естественный язык, то есть то, что мы часто называем «способностью к рассуждению». Люди могут начать использовать естественный язык, чтобы поручить большим языковым моделям выполнение важных задач, таких как написание программ, составление документов, интерпретация статей и т. д. Мы можем назвать эту парадигму «управление вычислительными мощностями человеком через естественный язык», как показано на рис. 8.2.

По своей сути этот сдвиг парадигмы имеет глубокое значение: задачи посредничества, которые ранее требовали участия программистов, теперь могут быть выполнены большими языковыми моделями с более высокой способностью к обобщению. В данном контексте «обобщение» означает, что большая языковая модель обладает большей универсальностью и способна выполнять

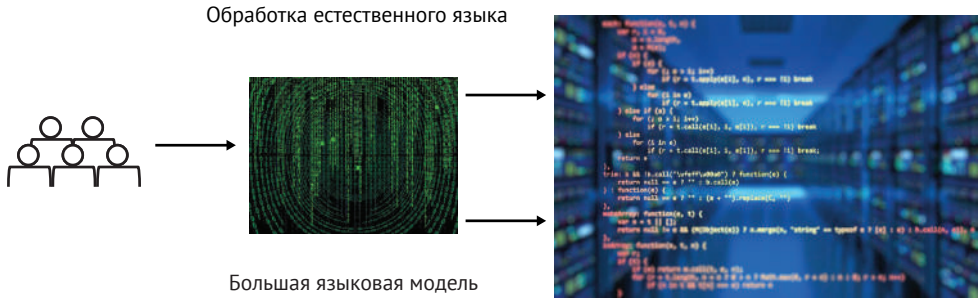


Рис. 8.2. Управление вычислительными мощностями человеком через естественный язык

вычислительные задачи, охватывающие все аспекты человеческих потребностей. В отличие от нее отдельный программист может быть компетентен лишь в ограниченном или даже специфическом наборе задач. Вычислительные платформы, основанные на больших языковых моделях, имеют потенциал стать новым поколением интеллектуальных вычислительных платформ, что может значительно ускорить процесс цифровизации человеческого общества.

Тем не менее для завершения цифровизации реального мира необходимо учитывать не только цифровизацию на уровне вычислений, но и на уровне данных. Развитие последних десятилетий показывает, что по мере изменения потребностей человечества в вычислительных мощностях способы управления данными с помощью компьютеров также становятся более разнообразными. До эпохи больших языковых моделей человечество управляло данными в основном через файловые системы на жестких дисках, реляционные и нереляционные базы данных. Мы можем назвать эту парадигму «управление данными через язык программирования», как показано на рис. 8.3.

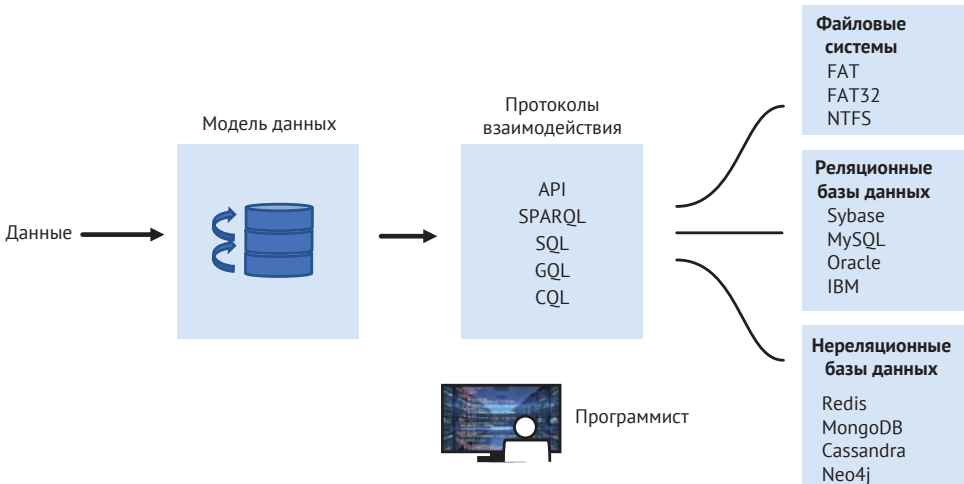


Рис. 8.3. Управление данными через язык программирования

Мы отмечаем, что эволюция способов управления данными и вычислительными мощностями имеет синергетическую связь, и эти способы также зависят от того, как программисты используют языки программирования для взаимодействия с соответствующими протоколами. Таким образом, программисты продолжают играть роль «посредника». Однако из рис. 8.2 мы видим, что, как только мы сможем управлять вычислительными мощностями через естественный язык, мы постепенно уменьшим зависимость от программиста как «посредника». По мере того как задачи, выполняемые посредником, становятся менее значительными, возникает необходимость управлять данными через естественный язык, если мы хотим продолжить координировать управление вычислительными мощностями и данными. Мы можем назвать эту парадигму «управление данными через естественный язык».

8.1.2. Векторные данные устраняют различия в форматах данных

Ключ к реализации управления данными через естественный язык заключается в векторных данных, которые могут стать важным промежуточным форматом. Способ реализации такого подхода показан на рис. 8.4.

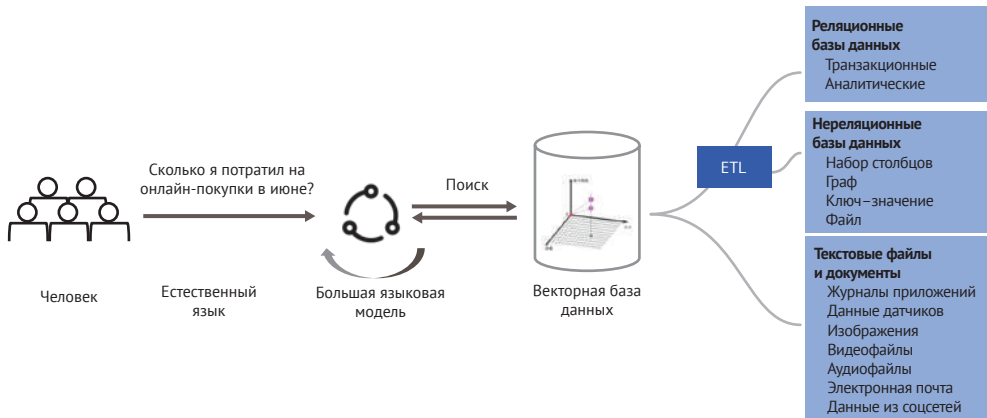


Рис. 8.4. Векторные данные как промежуточный формат данных

За последние десятилетия быстрого развития в человеческом мире накопилось огромное количество данных, из которых лишь малая часть структурирована, а большая часть – не структурирована. Основным вызов, с которым мы сталкиваемся при работе с этими данными, заключается в достижении унификации форматов. Векторные данные обладают естественной способностью к этому. Через векторизацию мы можем привести данные различных форматов к единому векторному формату, с которым мы можем взаимодействовать.

Унификация формата данных важна не только для удобства их централизованного хранения, но и для того, чтобы естественный язык человека также можно было векторизовать. Таким образом, команды по управлению данными по своей сути также становятся векторными данными. Векторизованные команды могут естественно взаимодействовать с данными в век-

торной базе данных, что позволяет реализовать управление данными через естественный язык.

Как только формат векторных данных станет ключом к унификации различных форматов накопленных данных, объем векторных данных, с которыми нам предстоит работать, начнет стремительно расти. Это приведет к тому, что только на основе более мощной векторной базы данных мы сможем эффективно управлять огромными объемами векторных данных. Поэтому платформизация векторных баз данных становится нашим приоритетным направлением. Платформизация означает снижение затрат и порога использования. В течение последних десятилетий в процессе развития компьютерной отрасли эффективность создания продуктовых возможностей на основе платформенного подхода была подтверждена многократно.

8.1.3. Ключевые аспекты платформизации векторной базы данных

Платформизация векторной базы данных направлена на решение проблем управления векторными данными, вызванных их стремительным ростом. С одной стороны, необходимо развивать основные возможности базы данных, так как только векторная база данных с корпоративными возможностями сможет лучше обслуживать эпоху больших языковых моделей. Мы можем назвать это направление «БД для ИИ». С другой стороны, с развитием ИИ-технологий для дальнейшего снижения затрат и порога использования для разработчиков необходимо интегрировать возможности ИИ (то есть интеллектуализацию) в векторные базы данных. Это направление мы можем назвать «ИИ для БД», как показано на рис. 8.5.

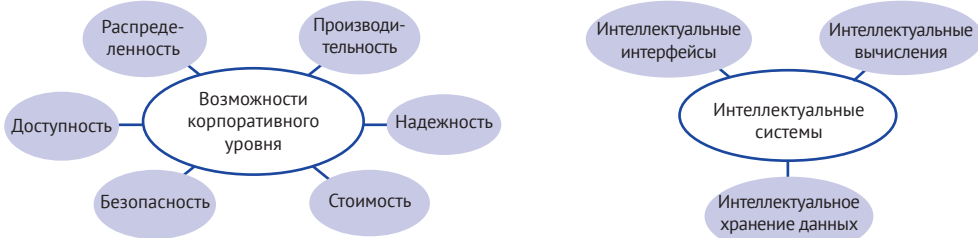


Рис. 8.5. Ключевые аспекты платформизации векторной базы данных

Векторная база данных должна взять на себя задачу управления огромными объемами векторных данных в эпоху больших языковых моделей, и корпоративные возможности системы базы данных станут ее основой. Компании должны достичь совершенства в следующих ключевых аспектах.

- **Распределенность:** использование совершенной распределенной системы для хранения векторных данных большего масштаба в условиях одного экземпляра для объемов уровня сотен миллиардов или даже триллионов строк.
- **Производительность:** сочетание программных и аппаратных технологий для снижения задержки доступа к данным, что увеличивает пропускную способность всей системы, достигая задержки на уровне миллисекунд.

- Надежность: посредством избыточности реплик и модели согласованности быстро восстанавливать данные при отказе или аномалии узла, предотвращая потерю данных.
- Затраты: применение эффективных алгоритмов и аппаратных средств для повышения экономической эффективности хранения и доступа к данным, что позволит разработчикам хранить больше данных и извлекать больше коммерческой ценности.
- Безопасность: обеспечение безопасности пользовательских данных через аутентификацию доступа, аудит поведения и другие меры безопасности, предотвращая утечки данных и другие проблемы безопасности.
- Доступность: за счет избыточности узлов усиление способности системы предоставлять услуги при отказе одного узла, обеспечение непрерывности бизнеса, что позволяет предоставлять услуги в непрерывном режиме.

Эти корпоративные возможности накапливались в области технологий баз данных на протяжении многих лет. Для векторных баз данных мы можем опираться на накопленный опыт традиционных баз данных, чтобы эффективно обслуживать огромные объемы векторных данных. Конечно, помимо существующих технологий, развитие ИИ-технологий также принесло некоторые средства для дальнейшего улучшения. Опираясь на ИИ, мы можем осуществить интеллектуализацию векторных баз данных на следующих трех уровнях.

- Интеллектуализация интерфейса: поскольку векторная база данных должна позволять человеку управлять данными через естественный язык, ее внешний интерфейс должен быть основан на естественном языке. Этот подход значительно отличается от языка запросов традиционных баз данных. Только те векторные базы данных, которые смогут реализовать управление данными на основе естественного языка, смогут лучше справляться с вызовами эпохи больших языковых моделей.
- Интеллектуализация вычислений: в эпоху больших языковых моделей ядро векторной базы данных также должно применять технологии ИИ. Например, в традиционных реляционных базах данных для сортировки результатов запроса используется функция `sort`, и обычно она основывается на математическом сравнении чисел (например, больше или меньше). В то время как векторная база данных может использовать возможности ИИ для улучшения таких функций сортировки, осуществляя повторное ранжирование результатов по семантическому порядку, например по положительной или отрицательной окраске. Интеграция возможностей ИИ в ядро векторной базы данных может повысить ее конкурентоспособность на уровне вычислений.
- Интеллектуализация хранения: на уровне хранения традиционные базы данных могут использовать общие алгоритмы сжатия для повышения эффективности хранения и снижения затрат. Подобно уровню вычислений, в контексте векторных баз данных, используя технологии ИИ для кластеризации данных и интеллектуального распределения, можно дополнительно повысить эффективность хранения. Векторная база данных также может интегрировать ИИ на уровне хранения для усиления своей конкурентоспособности.

Необходимо вновь подчеркнуть, что корпоративные возможности векторной базы данных фактически отражают концепцию «БД для ИИ», то есть улучшение обслуживания ИИ за счет интеллектуализации хранения данных, в то время как повышение интеллектуализации векторной базы данных отражает концепцию «ИИ для БД», то есть использование ИИ для дальнейшего улучшения возможностей обслуживания базы данных. Возможности ИИ, векторные данные и векторная база данных образуют эффект маховика, взаимно способствуя быстрому развитию всей отрасли в эпоху больших языковых моделей.

8.2. С ТОЧКИ ЗРЕНИЯ ПРИМЕНЕНИЯ В ОТРАСЛИ

Чтобы решить проблему ограничений больших языковых моделей во «времени» и «пространстве», разработчики ИИ-приложений часто прибегают к использованию внешних баз знаний с данными ограниченного доступа. На текущем этапе общепринятым решением для устранения этих ограничений больших языковых моделей является подход RAG. Векторная база данных, как основной компонент технологии RAG, обслуживает этап извлечения. Эффективность извлечения (точность, производительность, затраты и т. д.) напрямую определяет эффективность последующего этапа генерации. Поэтому в ближайшем будущем успешное обслуживание приложений в ИИ-индустрии будет одним из приоритетных направлений развития векторных баз данных.

8.2.1. Введение в RAG

RAG (генерация, дополненная поиском) – это технология обработки естественного языка, сочетающая извлечение информации и генерацию. Она обычно используется для усиления возможностей языковых моделей, особенно в сценариях, где необходимо извлечь информацию из большого объема данных и на основе этой информации сгенерировать текст. RAG определяет модель использования внешних данных большой языковой моделью, включая производство знаний, хранение, извлечение и интеграцию с большой моделью. Для удобства понимания мы поясним этапы технологии RAG на примере процесса построения базы знаний на основе RAG, схема которого изображена на рис. 8.6.

Этот процесс можно разделить на генерацию и использование базы знаний. На этапе генерации мы сосредоточены на том, как эффективно организовать и сохранить знания в векторной базе данных, что включает следующие подэтапы.

1. Предобработка знаний: разделение имеющихся у разработчика файлов знаний на несколько независимых фрагментов, которые представляют собой минимальные единицы знаний для последующего извлечения.
2. Векторизация знаний: на основе минимальных единиц знаний выбирается подходящая модель векторизации для преобразования знаний в векторные данные.
3. Хранение векторов: на основе результатов векторизации выбирается подходящий тип индекса для хранения векторов в векторной базе данных.

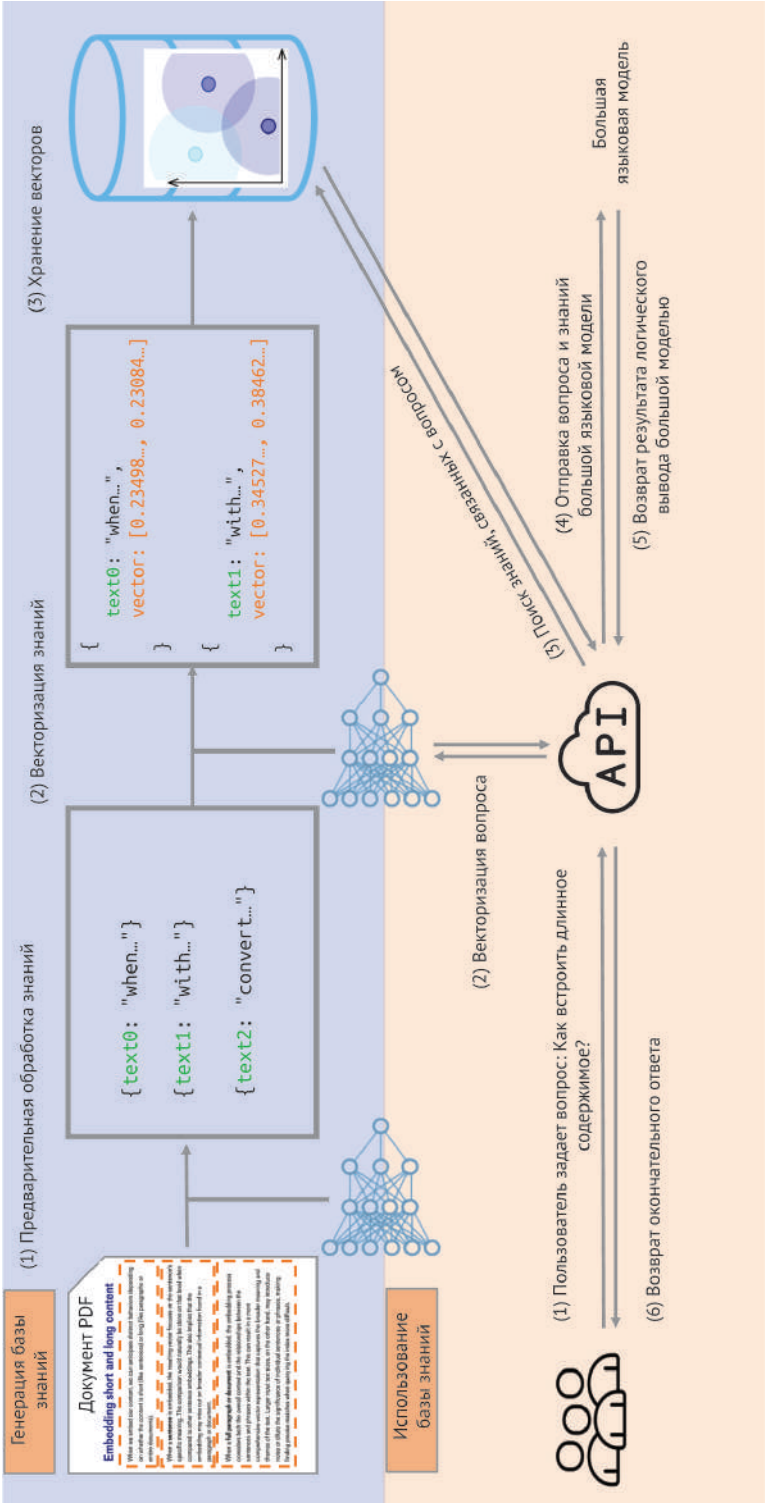


Рис. 8.6. Процесс построения базы знаний на основе технологии RAG

На этапе использования мы сосредоточены на извлечении знаний и взаимодействии с большой языковой моделью, что включает следующие подэтапы.

1. Вопрос пользователя: пользователь через интерфейс приложения задает вопрос, например: «Как векторизовать длинное содержимое?»
2. Векторизация вопроса: с помощью модели векторизации, связанной с базой знаний, выполняется векторизация вопроса пользователя, создающая векторное представление вопроса.
3. Извлечение знаний, связанных с вопросом: извлечение из базы знаний знаний, связанных с вопросом, для последующего использования большой языковой моделью.
4. Передача вопроса и знаний большой языковой модели: передача вопроса и извлеченных связанных знаний большой языковой модели.
5. Возврат результатов рассуждений большой языковой моделью: большая языковая модель, получив вопрос и связанные знания, проводит рассуждения и возвращает результаты.
6. Возврат окончательного ответа: на основе результатов рассуждений большой языковой модели пользователю возвращается окончательный ответ.

Объединяя производственную и пользовательскую стороны базы знаний, мы можем легко интегрировать данные предприятий или частных лиц с большой языковой моделью, создавая персональную базу знаний. Вы могли заметить, что этот подход фактически совпадает с обсуждаемой в главе 7 моделью программирования, где мы уже незаметно применили технологию RAG. Технология RAG – это техническое решение, выработанное и обобщенное многими разработчиками ИИ-приложений на практике, и она получила широкое признание среди специалистов индустрии.

8.2.2. Снижение порога использования RAG

В процессе построения базы знаний на основе технологии RAG можно заметить, что векторная база данных играет ключевую роль. В текущем широко используемом процессе векторная база данных отвечает только за хранение и извлечение векторных данных. Однако с развитием индустрии векторные базы данных расширяются на этапы, предшествующие и следующие за хранением и извлечением. Особенно на этапе производства данных многие процессы будут постепенно встраиваться в векторные базы данных в виде новых модулей.

Например, мы можем добавить новый модуль соединителя данных, который предоставит более широкий спектр соединителей в векторной базе данных, снижая порог получения данных для разработчиков. Мы также можем добавить модуль предобработки данных, встроив часть предобработки данных в векторную базу данных, чтобы помочь разработчикам автоматически выполнять такие задачи, как сегментация и обработка данных. Мы также можем продолжать добавлять в векторную базу данных модули векторизации, автоматического построения индексов и сортировки результатов, чтобы упростить разработку приложений на основе технологии RAG, еще больше снижая порог использования для разработчиков. Модули расширения векторной базы данных показаны на рис. 8.7.

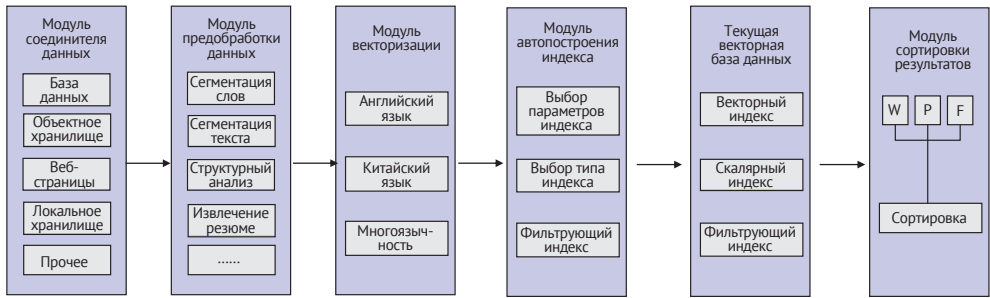


Рис. 8.7. Модули расширения векторной базы данных

Ниже представлено подробное описание возможностей этих модулей.

Модуль соединителя данных: учитывая разнообразие накопленных данных в человеческом обществе, встроенные в векторную базу данных соединители могут быстро импортировать данные из других систем данных, значительно расширяя границы векторной базы данных и позволяя внешним данным быстро поступать в нее.

Модуль предобработки данных: для использования RAG, поскольку исходные данные обычно имеют большой объем, разработчикам необходимо заранее выполнять сегментацию. Встроенный векторной базой данных стандартный предобработчик данных значительно упростит процесс обработки для разработчиков, избавляя их от необходимости заботиться о преобразовании данных. Процесс предобработки аналогичен процессу преобразования данных ETL (extract, transform, load – извлечение, преобразование, загрузка) в традиционных базах данных.

Модуль векторизации: с помощью встроенных соединителей и модуля предобработки пользовательские данные поступают в систему векторной базы данных. Однако эти исходные данные нельзя сохранить в векторной базе данных напрямую. Поэтому векторная база данных должна предоставить встроенную модель для выполнения векторизации данных.

Модуль автоматического построения индекса: в традиционной векторной базе данных разработчикам необходимо заранее указывать типы и параметры индексов. Векторная база данных может автоматизировать эту настройку, реализуя возможность автоматического построения индекса. Она может значительно снизить затраты разработчиков и повысить эффективность извлечения.

Модуль ранжирования результатов: в эпоху больших языковых моделей извлечение информации из пользовательских знаний возвращает k результатов, основанных на сходстве векторов. Эти результаты можно подвергнуть вторичной сортировке с помощью моделей, что позволяет более точно соответствовать запросам пользователя и повышает точность ответов больших моделей на вопросы пользователей.

Как видно, с широким применением технологии RAG в индустрии ИИ векторные базы данных будут иметь множество направлений для эволюции и улучшения. Границы векторных баз данных будут постоянно расширяться, что позволит лучше обслуживать потребности разработчиков ИИ-приложений

и создавать больше приложений. Это создаст эффект маховика между возможностями ИИ, векторными данными и векторными базами данных, способствуя развитию всей отрасли.

8.3. РЕЗЮМЕ

Будучи «ветераном» в области облачных вычислений, я стал свидетелем того, как они прошли путь от сомнений до формирования социальной инфраструктуры. Я прекрасно понимаю трудности и вызовы, с которыми сталкивается продвижение новых технологий. Они часто проходят десятилетние или даже более длительные циклы, и облачные вычисления не исключение. Технологии искусственного интеллекта сейчас переживают аналогичный процесс. В 2023 году они привлекли огромное внимание по всему миру и повысили ожидания людей. Однако я предполагаю, что этот всплеск энтузиазма вскоре перейдет в стадию стабильности, за которой последует более осторожное суждение осуществимости интеллектуальных технологий.

Я настоятельно рекомендую техническим специалистам сохранять рациональный ум и углубленно изучать технологии. Только углубленное понимание технических деталей позволит точно оценить соответствие технологий реальным потребностям. Постоянно применяя технологии для удовлетворения потребностей пользователей и активно реагируя на их (положительную или отрицательную) обратную связь, мы можем добиться быстрого итерационного развития небольшими шагами, что является прагматичной и эффективной стратегией коммерциализации технологий.

Векторные данные имеют потенциал стать важной структурой данных в эпоху ИИ, а специализированные векторные базы данных для хранения и извлечения векторных данных, вероятно, сыграют в этой сфере значительную роль. Глубокое изучение технологической эволюции векторных баз данных поможет нам глубже понять технологический прогресс ИИ. Однако в процессе развития векторные базы данных неизбежно столкнутся со множеством вызовов и могут в конечном итоге стать совершенно другими, даже перестав называться векторными базами данных. Но если мы будем активно участвовать в этом процессе эволюции, сохраняя любопытство и активно практикуясь, мы сможем построить лучшую версию себя и создать более качественные продукты.

Все только начинается, впереди еще много возможностей и мечтаний, которые ждут, чтобы мы их реализовали!

Предметный указатель

Б

База данных 36
 автономная 57
 векторная 42
 динамическая схема 75
 документ 67
 интерфейс 67
 коллекция 66
 модульная 56
 нереляционная 38
 поле 67
 распределенная с интеграцией хранения и вычислений 58
 распределенная с разделением хранения и вычислений 59
 реляционная 37
 экземпляр (инстанс) 64

Библиотека

 FAISS 83
 HNSWLib 99
 NuRaft 148

Большая языковая модель 43

В

Вектор

 данные 40
 евклидово расстояние 29
 косинусное 28
 скалярное произведение 29
 сходство 28

Векторизация данных 77

Г

Гибридный запрос 78

Глубокая нейронная сеть
 AlexNet 47

И

Индекс 69

 HNSW 72
 IVF 72
 векторный 70
 векторный плоский 72
 первичный ключ 69

 перестроение 72
 фильтрующий 72

К

Класс

 HttpServer 94
 MasterServer 162
 ProxyServer 162
 VectorDatabase 115

Кластер 145

М

Моментальный снимок 135

П

Персистентность 128

Показатель производительности 73
 p99 74

 задержка доступа
 запросов в секунду (QPS) 73
 полнота результатов запроса 74
 пропускная способность экземпляра 73

Предзапись (WAL) 129

Псевдоним 76

Т

Теорема CAP 145

Э

Эталонное тестирование (бенчмарк) 207

Е

ETL (extract, transform, load) 265

І

ImageNet конкурс по масштабному распознаванию изображений (ILSVRC) 48

Р

RAG (генерация, дополненная поиском) 262
RocksDB 112

С

SDK 218

Книги издательства «ДМК Пресс» можно купить оптом и в розницу
на складе издательства по адресу:
Москва, ул. Электродная, д. 2, стр. 12, офис 7,
тел. +7 (499) 322-19-38,
а также заказать на сайте www.dmkpress.com
с доставкой в любой регион РФ.

Ло Юнь

Векторные базы данных

Главный редактор *Яценков В. С.*
editor@dmkpress.com
Перевод *Шевкун И. А.*
Корректор *Абросимова Л. А.*
Верстка *Луценко С. В.*
Дизайн обложки *Трофимова С. В.*

Формат 70×100 1/16.

Гарнитура «PT Serif». Печать цифровая.

Усл. печ. л. 21,78. Тираж 200 экз.

Веб-сайт издательства: www.dmkpress.com

Популярность векторных баз данных резко выросла в последние годы как прямое следствие перехода индустрии к моделям представления знаний через эмбединги и задачам, где классические реляционные или даже NoSQL-подходы оказываются неэффективными. Данная книга является практическим руководством по созданию векторных баз данных с нуля, а также раскрывает базовые принципы векторного представления данных.

Благодаря систематическому изложению читатели могут полностью освоить процесс построения векторной базы данных, включая импорт данных, построение индексов, оптимизацию запросов и настройку производительности.

Основные темы книги:

- понятие векторного представления данных;
- особые возможности векторных баз данных;
- реализация автономной базы данных;
- реализация распределенной базы данных;
- оптимизация векторных баз данных;
- практические примеры применения.

Издание предназначено всем, кто интересуется передовой технологией работы с данными и хочет научиться применять ее на практике. Для эффективного чтения необходимо знать основы программирования.



www.dmk.pf



ISBN 978-5-93700-422-2



9 785937 004222 >